Handwriting robot

Final Report

D.R. Braune
23539195

Submitted as partial fulfilment of the requirements of Project EPR400
in the Department of Electrical, Electronic and Computer Engineering
University of Pretoria

November 2026

Study leader: Prof. H.C. Myburgh

Part 1. Preamble

This report describes work I completed in my final year project, designing and implementing a handwriting robot that recognises handwritten text and its writer and reproduces text on paper in a chosen writer's handwriting style.

1. Project proposal and technical documentation
This main report contains an unaltered copy of the approved Project Proposal (as Part 2 of the report).

All the code that I developed appears as a separate submission on the AMS.

2. Project history
This project does not continue the work of a previous student project, and no part of a previous project's design or code was reused. The following parts of the work are not my own and are identified as such wherever they appear in the report. The physical components of the gantry and the capture rig (the two stepper motors, the two driver modules, the camera, the single-board computer, the switches, the pen, the power supply and the standard belts, pulleys, rods, bearings and fasteners) were bought, as the project proposal stated they would be. The layer arrangement of the text recogniser follows a published network described in the literature study, and is cited where it is used in Section 3.4; the implementation of that arrangement is my own. The public handwriting database used for training, and its transcriptions, are the work of its compilers and are cited. The software libraries listed under declaration 6 below are the work of their authors. Everything else reported on here, including all of the image processing, the training and inference code, the style extraction and synthesis, the G-code generation and the motion control, I designed and implemented myself. Section 6.3 states this in full.

MUST STILL BE COMPLETED BY ME

Confirm the first sentence above before submission. Appendix 4 requires this paragraph to state explicitly whether the project builds on an earlier project, and to cite that project by author and year if it does. I could not establish this from the project files, so I have written the statement that matches my own understanding — if any part of this project continues a previous student's work, name that work here by author and year and say exactly what was taken from it.

3. Language editing
This document has been language edited by a knowledgeable person. By submitting this document in its present form, I declare that this is the written material that I wish to be examined on.

My language editor was _____.

_____ _____

Language editor signature *Date*

MUST STILL BE COMPLETED BY ME

Fill in the language editor's name and obtain their signature and date, or attach their written confirmation at the end of the report and write "proof of language editing is attached" in place of the signature. The report is rejected without this.

4. Plagiarism declaration

I, D.R. Braune, understand what plagiarism is and have carefully studied the plagiarism policy of the University. I hereby declare that all the work described in this report is my own, except where explicitly indicated otherwise. Although I may have discussed the design and investigation with my study leader, fellow students or consulted various books, articles or the internet, the design/investigative work is my own. I have mastered the design and I have made all the required calculations in my lab book (and/or they are reflected in this report) to authenticate this. I am not presenting a complete solution of someone else or that I found somewhere.

Wherever I have used information from other sources, I have given credit by proper and complete referencing of the source material so that it can be clearly discerned what is my own work and what was quoted from other sources. I acknowledge that failure to comply with the instructions regarding referencing will be regarded as plagiarism. If there is any doubt about the authenticity of my work, I understand that I may have to attend an oral ancillary examination/evaluation about the work.

5. AI usage declaration

Artificial intelligence tools, including large language models and code-generation tools, were used during this project. They were used to draft and edit the text of this report, to assist with writing, refactoring and reviewing source code, and to help build scripts that audited the code and checked the reported measurements. They were not used to produce any of the measured results reported in Section 4: every number in this report comes from running the project's own code on the project's own data, and the raw output files are part of the code submission. The choice of methods, the design of the system, the decisions about what to build and what to abandon, and the interpretation of the results in Section 5 are my own.

MUST STILL BE COMPLETED BY ME

Read declaration 5 above and make it exactly true before submission, adding or removing items as needed. The template's default wording, which denies any AI use, was deleted because it is not true of this project. A declaration that does not match the facts is itself an integrity violation, independently of the quality of the work, so this paragraph must be checked word by word and signed off by me.

6. Software library usage declaration

Unless explicitly stated otherwise in the next paragraph, the author confirms that no software library functions, packages, or toolboxes were used in the design, implementation, or analysis presented in this report. All code and computations were developed entirely by the author.

The exceptions to the above are the following. NumPy (`numpy.org`) was used for array storage and elementary array arithmetic, and Pillow (`python-pillow.org`) for reading and writing image files and for drawing text labels and boundary lines onto diagnostic images that no network reads. No image-processing, machine-learning or scientific routine from either was used in the signal path. Every operation that changes a pixel value the networks are judged on — scaling a crop, halving and rotating a page, warping a training image, blurring a stroke edge and drawing the synthesised ink — is performed by one module written for this project, and each of those operations was verified against the library call it replaced. The convolution, normalisation, pooling, recurrent, loss and optimisation routines were likewise written from first principles, as Section 3.4 describes. SciPy (`scipy.org`) was used in two places that are not part of the deployed system: `scipy.ndimage.gaussian_filter` and `scipy.ndimage.map_coordinates` in the training-time image augmentation, and `scipy.signal.find_peaks` in the script that cuts the public database pages into lines. Flask and Flask-CORS (`flask.palletsprojects.com`) provide the web server. OpenCV (`opencv.org`) is used only to read frames from the USB camera, and libgpiod (`libgpiod` Python bindings) only to drive and read the general-purpose input and output pins of the single-board computer; both are hardware-interfacing libraries and are imported only inside the single function that needs each. The Hugging Face `datasets` package (`huggingface.co/docs/datasets`) was used once, in a standalone script, to download the public line-level handwriting database; nothing in the running system imports it. Finally, the weights of the writer-identification network were trained with PyTorch (`pytorch.org`) and then converted into this project's own weight format; the network that runs on the machine, and the from-scratch training program described in Section 3.4, contain no PyTorch code and do not import it.

MUST STILL BE COMPLETED BY ME

Two things to settle in declaration 6. First, confirm whether the deployed text recogniser's weights (`paper_cnn_bilstm_ctc_scratch_best.npz`) were trained from random initialisation by my own training program, or were warm-started from the PyTorch-trained checkpoint using the `--warm-start` option. I could not determine this from the saved file, and the answer changes what Section 3.4 and this declaration may claim. Second, check this list against the code submission and add anything I have missed.

7. Unaltered Project Proposal

I certify that the Project Proposal appearing as the Introduction section of the report is an unaltered copy of the approved Project Proposal.

Signature

With my signature below, I hereby certify that all statements, declarations, and information presented above are complete.

D.R. Braune Date

MUST STILL BE COMPLETED BY ME

Sign and date the report here. The report is rejected without this signature.

TABLE OF CONTENTS

Part 2. Project definition: approved Project Proposal

1. Project description
2. Technical challenges in this project
3. Functional analysis
4. System requirements and specifications
5. Field conditions
6. Student tasks

Part 3. Main Report

1. Literature study 1
    1.1. Existing concepts 1
        1.1.1. A need exists to be able to decode the writer's style and written text from an image 1
        1.1.2. A need exists to reproduce text on paper accurately and fast in any writer's style 5
    1.2. Intended contribution and discussion 6

2. Approach 8
    2.1. From functional analysis to concept 8
    2.2. Order in which the subsystems were built 8
    2.3. Design alternatives and trade-offs 9

3. Design and implementation 10
    3.1. Design summary 10
    3.2. System block diagram 10
        3.2.1. Hardware block diagram 13
        3.2.2. Software block diagram and systems-level description 13
    3.3. Hardware design 14
        3.3.1. Plotter design and mechanics 15
        3.3.2. Stepper motor and driver 17
        3.3.3. Interface circuits, supply and pin assignment 22
        3.3.4. Step resolution, microstepping and calibration 24
        3.3.5. End-stops, debounce and the pen-lift mechanism 27
        3.3.6. G-code generation and the motion mapping 30
    3.4. Page conditioning and line segmentation 33
        3.4.1. Design goals and the vocabulary of a text line 33
        3.4.2. Array primitives: summed-area tables and run tests 34
        3.4.3. Lighting normalisation and binarisation 35
        3.4.4. Deskew by searching rotations 36
        3.4.5. Line pitch: the scale unit the rest of the block uses 38
        3.4.6. Removing what is not handwriting 39
        3.4.7. Line centres on the cleaned mask 40
        3.4.8. Seam cut, pixel ownership and the final crop 41

3.4.9. Summary of the block's constants 43
3.4.10. Testing the segmentation block 43
3.4.11. Design alternatives and limitations 45
3.5. Text recognition and writer identification 46
3.5.1. The shared convolutional backbone 46
3.5.2. Sequence modelling and text recognition by CTC 49
3.5.3. Writer-identification head and stroke normalisation 52
3.5.4. The data sets and their leakage assessment 56
3.5.5. Training methodology, the optimisers and the from-scratch gradient check 57
3.5.6. Edit distance: how accuracy is measured 60
3.5.7. Alternatives considered 60
3.5.8. Known faults and limitations 61
3.6. Style extraction and handwriting synthesis 61
3.6.1. Forced alignment: cutting a joined line into characters 62
3.6.2. From ink to ordered pen paths 64
3.6.3. Glyph quality gates and the cross-writer letter prior 66
3.6.4. Style statistics and permanent storage 68
3.6.5. Synthesis: choosing and placing a glyph 69
3.6.6. Best-of-*N* search with joint scoring 70
3.6.7. Alternatives considered 73
3.6.8. Known faults and limitations 74
3.7. Design reconsiderations 75
3.7.1. A second writer classifier, judging shape instead of ink 75
3.7.2. End-to-end training in place of a frozen backbone 76
3.7.3. Selecting a candidate on both criteria instead of legibility alone 77
3.7.4. Line-level style calibration 78
3.7.5. Naming a photographed page by shape rather than by ink 78
3.7.6. A reconsideration not yet carried out: detecting the need for an anchor automatically 80
3.8. Software design and system integration 81
3.8.1. Process flow and start-up 81
3.8.2. Timing chains of the two modes 82

4. Results 85
4.1. Summary of results achieved 85
4.2. Qualification tests 85
4.2.1. Test material and the software written to perform the tests 87
Qualification test 1: Reproduction of text in a writer's style (R1)
Objectives of the test or experiment 87
Equipment used 87
Test setup and experimental parameters 87
Steps followed in the test or experiment 88
Results or measurements 88
Observations 90

Statistical analysis 93
Qualification test 2: Reading the text and naming the writer of unseen handwriting (R2)
Objectives of the test or experiment 93
Equipment used 93
Test setup and experimental parameters 94
Steps followed in the test or experiment 94
Results or measurements 94
Observations 95
Statistical analysis 96
Qualification test 3: Time taken to reproduce text (R3)
Objectives of the test or experiment 97
Equipment used 97
Test setup and experimental parameters 97
Steps followed in the test or experiment 97
Results or measurements 98
Observations 98
Statistical analysis 98
Qualification test 4: Time taken to identify and decode a page (R4)
Objectives of the test or experiment 99
Equipment used 99
Test setup and experimental parameters 99
Steps followed in the test or experiment 99
Results or measurements 100
Observations 100
Statistical analysis 101
Qualification test 5: Absolute positional accuracy of the gantry (R5)
Objectives of the test or experiment 101
Equipment used 101
Test setup and experimental parameters 101
Steps followed in the test or experiment 102
Results or measurements 102
Observations 102
Statistical analysis 102
Qualification test 6: Number of writing styles trained and stored (R6)
Objectives of the test or experiment 103
Equipment used 103
Test setup and experimental parameters 103
Steps followed in the test or experiment 104
Results or measurements 104
Observations 104
Statistical analysis 104

5. Discussion 105
5.1. Critical evaluation of the design 105
5.1.1. Interpretation of results 105
5.1.2. Critical evaluation 109

5.1.3. Unsolved problems 111
5.1.4. Strong points of the design 114
5.1.5. Expected failure conditions 115
5.2. Considerations in the design 116
5.2.1. Ergonomics 116
5.2.2. Health and safety 117
5.2.3. Environmental impact 117
5.2.4. Social and legal impact 118
5.2.5. Ethics clearance 118

6. Conclusion 119
6.1. Summary of the work completed 119
6.2. Summary of observations and findings 119
6.3. Contribution 120
6.4. Suggestions for future work 121

Appendix

Metric definitions of the qualification tests 123
Confidence intervals and resampling 123
Character accuracy and its three averages 123
Word accuracy is a recall, not one minus the word error rate 124
Writer accuracy is reported per line, not per page 124
Decoding 124
Inter-rater agreement of the panel that was not convened 124
Style correlation on geometric features 124
Overlay distances in the pixel domain 125
Positional error and its uncertainty budget 126
Additional results and prepared measurement sheets 126
Provenance of the data sets and checkpoints 126
Per-feature and per-judge breakdown of qualification test 1 127
Per-writer breakdowns 127
Storage check of qualification test 6 128
Prepared sheets for the sub-tests not carried out 128
Features prescribed for the style correlation 129
Distance distributions behind the overlay test 130

LIST OF ABBREVIATIONS

AdamW	Adam optimiser with decoupled weight decay
AMS	assignment management system (code submission)
API	application programming interface
ARM	processor architecture of the ODROID N2+ (Advanced RISC Machine)
AUC	area under the receiver operating characteristic curve
BiLSTM	bidirectional long short-term memory
BN	batch normalisation
CAD	computer-aided design
CER	character error rate
CNN	convolutional neural network
CRNN	convolutional recurrent neural network
CTC	connectionist temporal classification
EMNIST	Extended Modified National Institute of Standards and Technology (data set)
FC	field condition (FC1, FC2); also fully connected layer where stated
FU	functional unit
G-code	line-oriented machine-control instruction language
GPIO	general-purpose input/output
HSV	hue-saturation-value
HTR	handwritten text recognition
HTTP	hypertext transfer protocol
HWR	handwriting recognition
IAM	Institute of Computer Science and Applied Mathematics (handwriting database)
ID	identification (writer identification)
IoU	intersection over union
JPEG	Joint Photographic Experts Group (image format)
JSON	JavaScript Object Notation
LCS	longest common subsequence
LSTM	long short-term memory
MAC	multiply-accumulate operation
PC	personal computer
POPIA	Protection of Personal Information Act (South Africa)
PSU	power supply unit
R1–R6	requirements 1 to 6 of the project proposal
ReLU	rectified linear unit
RMSprop	root-mean-square propagation (optimiser)
RNN	recurrent neural network
SBC	single-board computer
STEP/DIR	step and direction control signals of a stepper-motor driver
USB	Universal Serial Bus
WER	word error rate

NOTATION

x_{h}	x-height of the handwriting (height of lower-case letters without ascenders or descenders)
x	number of characters of a sentence or page (in the time requirements R3 and R4)
N	number of candidate syntheses tried in the best-of-N search (“tries”)
$A_{\mathrm{c}}, A_{\mathrm{w}}$	character accuracy and writer accuracy

Part 2. Project definition: approved Project Proposal

This section contains the problem identification in the form of the complete approved Project Proposal, unaltered from the final approved version that appears on the AMS.

[MUST STILL BE COMPLETED BY ME: check these eight pages against the approved proposal on the AMS and confirm that every answer, the title block and the Feedback block now appear. The included file is a flattened copy; the original is kept unaltered as `project_proposal.pdf`.]

For use by the Project lecturer	Approved		Revision required	

Feedback

For use by the Project module lecturer only

To be completed by the student

PROJECT PROPOSAL 2026

Project no	HCM4	Revision no	1

Title	Surname	Initials	Student no	Study leader (title, initials, surname)
Mr	**Braune**	**D.R**	23539195	Prof. H.C. Myburgh

Project title (this is the title on the project concept note)

Handwriting robot

Language editor details	Language editor signature
MTBraune	
Student declaration I understand what plagiarism is and that I have to complete my project on my own.	**Study leader declaration** This is a clear and unambiguous description of what is required in this project. Approved for submission (Yes/No)
Student signature	Study leader signature and date 20 April 2026

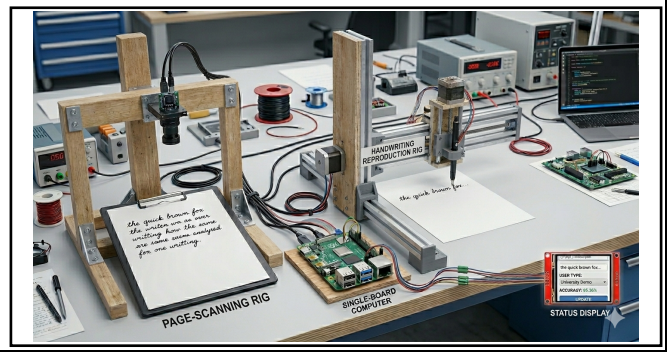
1. Project description.

What is the problem to be solved with your project? Describe in non-technical language what your project is about. What does your system have to do? Refer explictly to Figure 1 that you inserted on the right.

The need for handwriting still exists for authenticity, ease of communication and personal touches. The problem is that many individuals do not possess the time, skills or ability to write neatly and legibly. This leads to the solution presented in this proposal: a handwriting robot capable of recognizing a user's handwriting style and being able to replicate it.

The proposed solution is a handwriting robot capable of training on multiple different user's handwritten text (using a page-scanning rig to capture the handwritten page like the one seen in the image) and being able to produce any desired piece of text in any of these user's writing styles (using a text reproduction rig like the one seen in the image). The system will be able to recognize which user authored a piece of text and identify what text was written (shown to the user through a display as seen in the image). A gantry with 2-dimensional movement will perform actions with a pen to automatically reproduce the desired text.

Figure 1. Insert an image below that captures the idea with your project. Explain in the text on the left.



2. Technical challenges in this project

Describe the technical challenges that are *beyond* those encountered up to the end of third year and in other final year modules.

2.1 Primary *design* challenges

What is the intended core technical challenge in the project? Which aspect or aspects of the design of the system do you expect to be the most challenging?

1. Text image processing will be performed from first principles, including the isolation of letters from an image of various fonts and sizes of handwritten text.
2. A neural network will be designed and trained from first principles to be able to categorize text to different user's styles based on certain style elements, and identify the text written.
3. A gantry rig will be designed to be capable of performing the necessary precise 2-dimensional movements using a pen, whilst holding the pen and paper securely in place.
4. Pen path planning will be implemented, which involves the conversion of written characters to a 2D motion the gantry needs to perform, and then mapping the movement to stepper motor controls.

2.2 Primary *implementation* challenges

Which aspect or aspects of the implementation do you expect to be the most challenging?

1. Image processing and deep learning will be performed on a single board computer (instead of on a personal computer), which has limited memory, storage and power.
2. Smooth motor control will be needed to produce precise and consistent pen movement to create accurate text.
3. Near real-time classification of handwritten text and reproduction of digital text on paper will need to be achieved.

All processing will be done on an embedded platform, and not on a PC platform. Confirmed by having ticked this box. ✔

Library functions will be used for hardware interfacing only. Confirmed by having ticked this box. ✔

3. Functional analysis

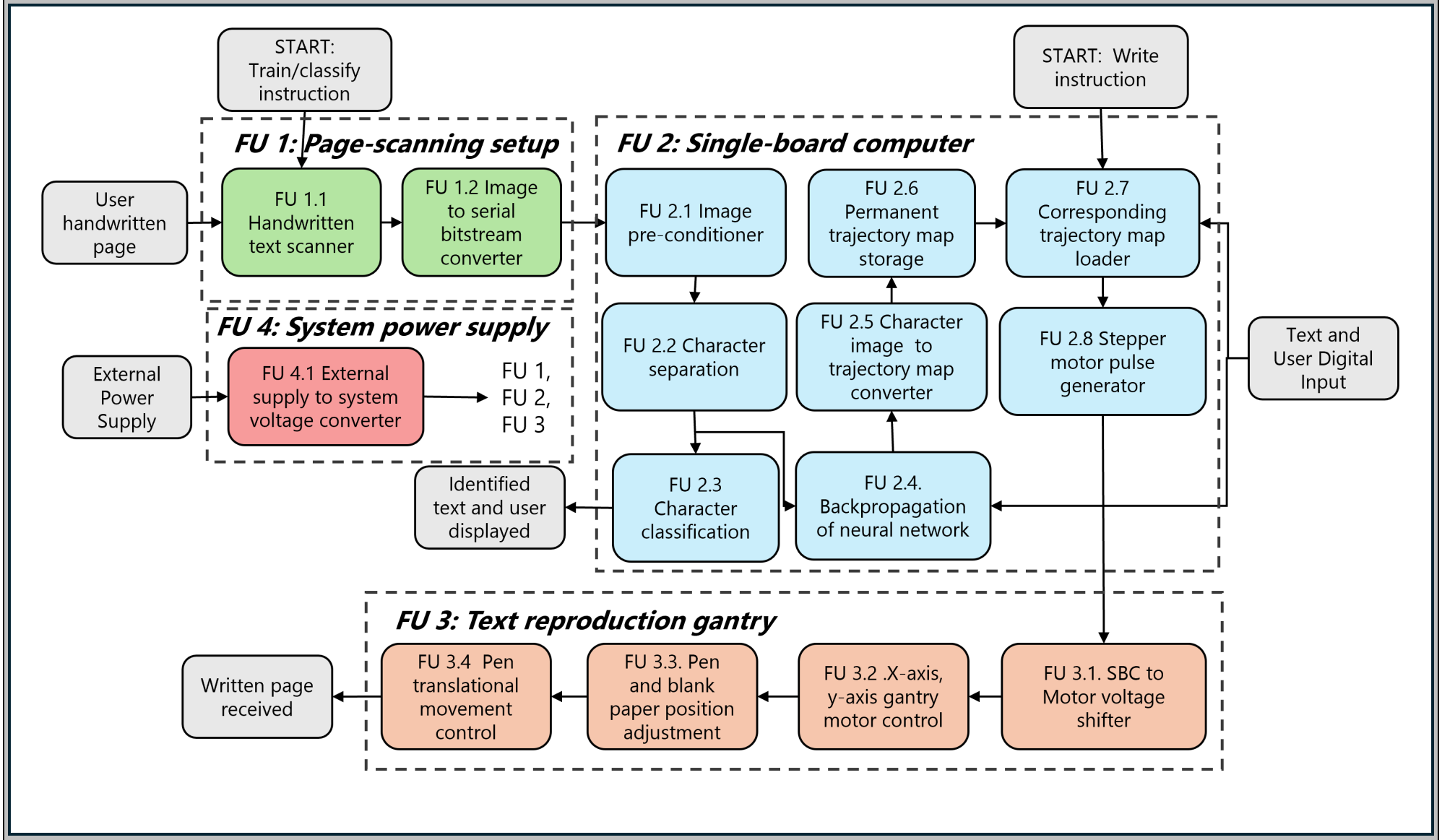
3.1 Functional description

Describe the design in terms of system functions as shown on the functional block diagram in section 3.2. This description should be in *narrative format*. **DO NOT** use a bullet list.

There are three main modes of operation for the handwriting robot: training, classification and reproduction
Training/Classification: First, the user will input their page of handwritten text to the scanner (FU 1.1). The user will select to either train or classify, and will input the intended digital text alongside this through an input interface. Once the page is scanned, a camera USB driver will be used to transfer the image to the single board computer (FU 1.2). The image will be conditioned first to fix warping or noise in the image (FU 2.1), and then the character separation will occur. This will separate the page characters (or groups of characters that are connected) to match the desired neural network input size (FU 2.2). The difference between training and classification occurs here: if the 'training' option is selected, the separated characters will be used to adjust the neural network neuron weightings against the desired digital text that was input (FU 2.4). Each separated character or group of characters will also be converted into a trajectory map (coordinate map of required pen motion) (FU 2.5). This trajectory map will be stored permanently within the system (FU 2.6). If the 'classify' option is selected, the separated characters or group of characters will be passed into the trained neural network classifier (FU 2.3), and the corresponding digital text and detected user will be outputted to the display interface.

Reproduction: If the user sends a 'write' instruction, the digital text and user style desired to be reproduced will be provided. The corresponding trajectory for each character/group will be retrieved from storage (FU 2.7), and will be converted into stepper motor controls required to produce the desired pen movements (FU 2.8). These stepper motor controls will be sent to the corresponding motors via a stepper motor driver to allow for forward and reverse movements (FU 3.1). The x-motor, y-motor and pen lift mechanism will be controlled through the stepper motor driver (FU 3.2/3.4). After the pen has been adjusted and is held securely by the pen holder in the robot (FU 3.3), precise handwritten imitations of each character will be written.

3.2 Functional block diagram (this should not be a flow diagram)



4. System requirements and specifications

These are the core requirements of the system or product (the mission-critical requirements) in table format **IN ORDER OF IMPORTANCE**. Requirement 1 is the most fundamental requirement.

	Requirement 1: the fundamental functional and performance requirement of your project	**Requirement 2** (Second in order of importance)	**Requirement 3** (Third in order of importance)
1. Core mission requirements of the system or product. Focus on requirements that are **core** to solving the engineering problem. These will reflect the solution to the problem.	The system must be able to accurately reproduce any desired digital text on paper in the writing style of any writer from the training set.	The system must be able to accurately decode the content of the handwritten text, and identify which writer from the training set wrote the handwritten text.	The system must reproduce text on paper at a near real time speed, at a rate that can be considered practical and useful.
2. What is the target specification (this must be in *measurable* terms) to be met in order to achieve the requirement in 1. above?	The system should reproduce desired text with a minimum character accuracy of 85% while having a high stylistic correlation of 85% to the chosen writer's style.	The system must be able to identify the writer of the handwritten text from the training set and the content of the written text with an overall accuracy of 95%.	The system must be capable of reproducing the desired text in the writing style of any writer in the training set at a rate of no slower than 4 seconds per character.
3. Explain where this specification originated : i.e., **explain** the value that you selected. I.e., *how* will meeting the specification given in point 2 above *solve the problem*?	An accuracy of 85% (averaged from Babushkin et al. results [1, p8]) is high enough to represent the input text, but practical to account for the build up of precision loss in the system.	An overall classification accuracy of 95% (similar to existing classifiers mentioned in Acta Informatica Pragensia [2, p6]) ensures practical usability and consistency.	Averaging the time required to produce text between 0% and 1% error (Ossi Teikari [3, p35]) gives approximately 4 seconds per character, allowing the system to reproduce text at a practical rate with minimal error.
4. How will you demonstrate at the examination that this requirement and specification (points 1 and 2 above) have been met? Be explicit about how you will ***prove*** these were met.	A sentence of text will be reproduced in a specific user style, and the amount of characters that can be visibly seen to match the input text (in the style of the chosen user) must be >= to 85% of the total characters produced.	A predefined paragraph of text must be handwritten by a writer in the training set and classified by the system. The classified text and writer must match the input text and writer with an overall accuracy of >= 95% .	A sentence of x characters long will be sent to the system to be reproduced, the system must be able to reproduce it within 4*x seconds.
5. Your own design contribution: what are the aspects that *you will design and implement yourself* to meet the requirement in point 2? If none, *remove this requirement.*	Accurate conversion from character image to pen trajectory map, precise motor control from pen trajectory map, and a structurally sound gantry design for accurate drawing will be designed and implemented by the student.	A rig to capture an image of written text, image conditioning and noise removal, character separation and processing, classification with a neural network, and communicating necessary data to a display will be designed by the student.	Techniques to optimize pen movements for quicker drawing, smooth motor control and robust design for efficient gantry movements will be designed and implemented by the student.
6. What are the aspects to be taken off the shelf to meet this requirement? If none, indicate "none". Clearly specify for what tasks **library functions** will be used (if relevant to the project).	Physical motors, pen, single board computer, stepper motor driver, and separate components for gantry design e.g. screws or belts will be taken off the shelf.	A camera, camera driver (library function), single board computer, separate components for image-capturing rig, and a physical device where data is displayed will be taken off the shelf.	Efficient physical motors, single board computer, stepper motor driver, and separate components for gantry design e.g. smooth ball bearings will be taken off the shelf.

System requirements and specifications page 2

	Requirement 4 (Fourth in order of importance)	**Requirement 5**	**Requirement 6**
1. Core mission requirements of the system or product. Focus on requirements that are core to solving the engineering problem. These will reflect the solution to the problem.	The system must be able to identify the writer of the handwritten text, and decode the content of the handwritten text in a short, practical amount of time.	The gantry of the system must have precise pen movements and accurately draw the desired characters with minimal positional error.	The system should be able to be trained on multiple different writing styles and be capable of storing all the writers' necessary character trajectory maps within the system permanently.
2. What is the target specification (this must be in *measurable* terms) to be met in order to achieve the requirement in 1. above?	The system must identify the writer of the written text from the training set and decode the content of the text at a rate of no slower than 25ms per character with an extra 2 seconds for image capture/conditioning.	The gantry of the system must be capable of drawing the desired character trajectory map while maintaining an absolute positional error that does not exceed 0.5mm.	The system should be capable of training on a minimum of 10 different users' writing styles, and be capable of storing the necessary character mappings for each of these 10 user's permanently.
3. Explain where this specification originated: i.e., explain the value that you selected. I.e., *how* will meeting the specification given in point 2 above solve the problem?	2 seconds was given to image capturing and conditioning due to transmission times and complex conditioning and noise removal algorithms, and 25ms per character classification due to simple forward propagation calculations.	According to research by T. Huang and R. Xiong [4] an absolute positional error of 0.3mm with a gantry is achievable and indicates sufficient accuracy, therefore a maximum absolute error of 0.5mm was chosen for slightly more leeway.	Having 10 trained user styles stored at the same time proves multi-user neural network recognition and multi-user storage. More than 10 is not required due to neural network complexity and storage limitations.
4. How will you demonstrate at the examination that this requirement and specification (points 1 and 2 above) have been met? Be explicit about how you will *prove* these were met.	A page with a sentence of x handwritten characters will be given to the system. The time from the initial instruction to the received classified text and user should be within 2 + x*0.025 seconds.	The system should be instructed to recreate the same word twice without varying the dataset characters, and the two words should appear identical on-top of each other, and not exceed an absolute positional error of 0.5mm.	The system should display the current amount of users in the system, and any random user should be selected out of the 10 and have text reproduced in their style to prove the system can be trained with 10 different users.
5. Your own design contribution: what are the aspects that *you will design and implement yourself* to meet the requirement in point 2? If none, *remove this requirement.*	A rig to capture an image of written text, image conditioning,optimized and efficient character separation, neural network character classifier, and calculation and display of time taken will be designed and implemented by the student.	Accurate conversion from pen trajectory map to precise stepper motor control, robust gantry system to write characters, secure pen adjuster and holder, and a secure paper holder will be designed and implemented by the student.	A rig with a camera to capture new text, sufficiently complex neural network to be able to classify minimum 10 user's styles, and a storage strategy to store 10 user's character trajectory mappings will be designed by the student.
6. What are the aspects to be taken off the shelf to meet this requirement? If none, indicate "none". Clearly specify for what tasks **library functions** will be used (if relevant to the project).	A camera, camera driver, single board computer, separate components for image-capturing rig, and a physical device where time is displayed will be taken off the shelf.	Precise stepper motors, separate components needed for gantry, pen and paper holders, and a pen will be taken off the shelf.	A camera, camera driver, single board computer, components for image-capturing rig, and permanent storage device will be taken off the shelf.

System requirements and specifications page 3

	Requirement 7 (Seventh in the order of importance)	**Requirement 8**	**Requirement 9**
1. Core mission requirements of the system or product. Focus on requirements that are core to solving the engineering problem. These will reflect the solution to the problem.			
2. What is the target specification (this must be in *measurable* terms) to be met in order to achieve the requirement in 1. above?			
3. **Explain where this specification originated**: i.e., explain the value that you selected. I.e., *how* will meeting the specification given in point 2 above solve the problem?			
4. **How will you demonstrate at the examination** that this requirement and specification (points 1 and 2 above) have been met? Be explicit about how you will *prove* these were met.			
5. Your own design contribution: what are the aspects that *you will design and implement yourself* to meet the requirement in point 2? If none, *remove this requirement.*			
6. What are the aspects to be taken off the shelf to meet this requirement? If none, indicate "none". Clearly specify for what tasks l**ibrary functions** will be used (if relevant to the project).			

5. Real world field conditions

These are the ACTUAL, REAL WORLD CONDITIONS under which your project has to work *and will be demonstrated.*

	Real world field condition 1 (required to include)	**Real world field condition 2** (optional to include)	**Real world field condition 3** (optional to include)
Field condition requirement. In which real world conditions does the system have to operate?E.g., "lab conditions" or "outside in uncontrolled conditions". Describe the one, two or three most important field conditions.	The system is expected to perform in well-lit areas like a lab or office environment, at an operating range of 400-1000 lux, to ensure minimal noise in image capturing.	The system is expected to run on a stable, level and vibration free surface, to ensure the page scanning is accurate and the pen in the gantry can write precisely without fluctuations.	

6. Student tasks

6.1 Design and implementation tasks of the student

List your primary design and implementation tasks in bullet list format (5-10 bullets). These are *NOT* product requirements, but *YOUR* tasks.

- The student will research types of neural networks, their structure, the effect of neurons and layers, and how to extract features of text from different layers.
- The student will research image conditioning, noise removal, character separation strategies, and pen trajectory mapping.
- The student will design and implement the convolution neural network, image conditioning, and character separation algorithms.
- The student will simulate the classifier on existing handwritten text datasets with adjusted character separation strategies and neuron/layer numbers to meet requirements.
- The student will construct a rig to capture images of handwritten text on an A4 page, and implement the neural network and algorithms on a chosen single board computer.
- The student will test the new classifier implementations on the single board computer with the captured images to ensure requirements are met.
- The student will design the gantry system in CAD software and construct it with the necessary store-bought individual components.
- The student will design and implement the pen trajectory mapping algorithm and the stepper motor control.
- The student will test the constructed gantry with the implemented classifier to ensure the remainder of the requirements are met.

6.2 New knowledge to be acquired by the student

Describe what the theoretical foundation to the project is, and which *new* knowledge you will acquire (*beyond* that covered in any other undergraduate modules).

- The student will acquire new knowledge of training and classification of character images using a neural network.
- The student will acquire new knowledge involving the extraction of text features from neural network layers.
- The student will acquire new knowledge about the process of image processing with character isolation.
- The student will acquire new knowledge about pen trajectory map creation.
- The student will acquire new knowledge on 2D gantry design and implementation with precise motor control.
- The student will acquire new knowledge about the implementation of neural networks on a single board computer.

7. References

Format your reference list either in IEEE reference format or in APA reference format. Check what your study leader prefers and use consistently.

[1] V. Babushkin, H. Alsuradi, M. O. Al-Khalil, and M. Eid, “Analyzing handwriting legibility through hand kinematics,” Frontiers in Artificial Intelligence, vol. 8, Mar. 2025, doi: https://doi.org/10.3389/frai.2025.1426455.

[2] Sidar Agduk and Emrah Aydemir, “Classification of Handwritten Text Signatures by Person and Gender: A Comparative Study of Transfer Learning Methods,” Acta Informatica Pragensia, vol. 11, no. 3, pp. 324–347, Dec. 2022, doi: https://doi.org/10.18267/j.aip.197.

[3] O. Teikari, "Reproducing Handwriting with Position-Controlled Robots," Master's thesis, Dept. Pervasive Systems, Tampere University of Technology, Tampere, Finland, 2016.

[4] T. Huang and R. Xiong, "Cost-Effective Robotic Handwriting System with AI Integration," in Proc. 2024 IEEE Long Island Systems, Applications and Technology Conference (LISAT), 2024.

Part 3. Main Report

1. Literature study

Text in red is not the author's own wording: it marks the pen-mapping description, the intended contribution, and the individual sentences that were corrected for factual accuracy. All remaining text is the author's own.

1.1 Existing concepts

1.1.1 A need exists to be able to decode the writer's style and written text from an image

The ability of a computer or system to identify written text after being trained on large amounts of data is known as HWR (Handwriting Recognition) or HTR (Handwritten Text Recognition) [**?**, p. 39]. One type of handwritten text recognition is offline recognition, which means the data being identified by the system is static and captured post-completion of the page of handwritten text. This is ideal for identifying the text from a fixed image of handwritten content by utilizing image processing techniques and algorithms to identify the text that was written [**?**]. The other type is online recognition, which is a more dynamic process that identifies text during the creation of the handwriting, adding the variable of time to the classification of text. The addition of a time variable includes data about the order of strokes to create text as well as pen-up or pen-down movements, which can influence the type and ability of reproduction and handwriting synthesis methods.

Every stage of the process to achieve accurate handwritten text recognition requires a deep understanding of the different techniques available:

Datasets and resolution: Multiple datasets have typically been used for training handwritten text recognition modules, for example, the EMNIST, MNIST and IAM datasets [**?**, p. 2]. The EMNIST (Extended Modified National Institute of Standards and Technology) dataset consists of approximately 814 255 handwritten characters and numbers, each being 28 by 28 pixels in size [**?**, p. 40]. There is only a single, disconnected character per image, which is used in many existing HTR models, but these models cannot effectively decipher connected handwritten words, like cursive words. The IAM handwriting dataset provides much more variety, as it consists of 5 685 complete sentences, with a total of 115 320 words, gathered from 657 different authors, consisting of 26 letters and 27 punctuation marks [**?**]; the paper that introduced the database reports an earlier state of it, 1 066 forms by about 400 writers [**?**, p. 39]. The images from the IAM dataset were scanned at 300 dpi with 8 bits per pixel in grayscale [**?**]. The dataset used influences the accuracy for the classification of text with higher variation, as well as defines the resolution of the camera required to capture the handwritten text to classify. The IAM distribution of cropped lines is not organised by author, so the writer is not evident from the line images alone, while the distribution of full pages is arranged so that each page belongs to a known writer; the database does record the writer of every form in its metadata, so the two can be joined [**?**].

Image-conditioning: The image-conditioning of handwriting pages includes multiple techniques to be performed on the handwritten text before being passed into the classification and feature recognition algorithms. This is used to account for dimensional adjustments, removal of inconsistencies, and normalization of the data, to make the images more appropriate for text separation

and classification [**?**]. Normalization is used to increase the accuracy of HTR models by reducing the effect that unwanted features of the data could have on the performance of the model. There are multiple different normalization techniques such as min-max scaling, z-score normalization, and log-scaling. Encoding techniques are also used in HTR model pre-processing to get rid of unintentional biases [**?**, pp. 40–41]. Binarization is used to differentiate the background page and obscurities from the actual ink of the written text, computed by pixel. One method of binarization is done from a global ink threshold taken from the grey-level histogram, which does not require any parameters like a localized shadow level radius, and is used for ideal, evenly lit page scans. The other method involves a localized threshold computed per pixel out of the statistics of a window around it, which costs more in terms of computation time and power but can tolerate a lighting gradient or shadow. Deskewing is a technique used to fix the rotation of scanned pages, by rotating the page through certain angle steps until the horizontal projection of the ink is most sharply peaked, which occurs when the lines of text all mostly lie horizontally. Deslanting is used to fix the lean of the individual characters by applying a horizontal shear that shifts each row sideways in proportion to its height, forcing more pixels to be aligned within the same column rather than slanted over multiple columns.

Text Separation: Text separation is the process of breaking up a page of handwritten text into smaller segments or blocks that can be used in feature extraction and neural network classifiers. There are three methods that existing models employ to separate text: line separation, individual character separation or word separation. All three methods involve breaking down paragraphs of handwritten text into smaller segments. The accuracy of the extraction of smaller segments heavily influences the effectiveness of the classifier. Some existing techniques for this are classic dissection techniques, recognition-based segmentation, hysteresis thresholding, and bounding box techniques [**?**, p. 41]. Line separation can be done by converting images to grayscale, obtaining inverted binary images, and then drawing bounding boxes over each line [**?**]. Firstly, when lines are mostly straight and separated by a blank row, line separation can be performed by reading the minimums of the row ink profile (the sum of ink in each row) to form boundaries. When descenders of lines begin to overlap or merge, the straight row boundary separator is replaced with a boundary that bends around the descenders by performing a least-cost path traced between the ink lines, and around the descenders. To achieve either word or character separation, the page must still be broken up into individual lines and then into separate words. Character separation just goes one step further and splits the word into individual characters.

Types of Neural Networks: Many different types of neural network models have been implemented in previous papers, including ANNs (artificial neural networks), CNNs (convolutional neural networks), RNNs (recurrent neural networks), and various combinations of them to achieve the best accuracy.

ANNs are created from the standard neural network architecture consisting of multiple neurons, weights, hidden layers, biases, etc., and can accurately perform pattern recognition for handwritten text. CNNs are built on the architecture of ANNs but also facilitate feature extraction and identifying smaller, more important information in text. This leads to a reduction of necessary learnable parameters and streamlines model training [**?**]. RNNs contain the same fundamental structure as CNNs but also have the capability to consider sequential and dependent data. The ability to capture time-dependent patterns is useful for deciphering various handwritten fonts, and because RNNs can consider dependencies in the data, this can improve the accuracy of handwritten text recognition when considering common linguistic dependencies in language [**?**]. A combination of both CNNs and RNNs can significantly enhance feature extraction by using

the CNN's pattern recognition and ability to identify diverse handwriting styles, and the RNN's ability to use textual dependencies for improved contextual understanding [**?**]. This combination for improved context awareness allows an entire line of text to be read without word or character separation. The convolutional layers are used to convert a cropped line of text into different feature columns, and the bidirectional recurrent layer reads the sequence of features in each direction, providing context of what comes before and after each character or feature. This implementation is used for unconstrained offline handwriting [**?**, p. 855], meaning natural handwriting that is not restricted to isolated characters or to a fixed vocabulary, and the full line crop must exist before classification can begin because the recurrent layer reads it from both ends. In an RNN, the gradient is multiplied by the same weight matrix at every time step as it propagates backwards, so over many steps it shrinks and loses its effect (this is the vanishing gradient problem). Forget gates are added to bypass the vanishing effect and allow gradient errors to still retain an effect after many time steps. Putting convolutional layers before the recurrent stack allows the feature extraction and sequence reading to be trained as one network [**?**]. The columns produced from the convolutional layers and max pooling tend to outnumber the number of characters to be classified, therefore the CTC (connectionist temporal classification) loss [**?**, p. 370] is used, which adds a blank symbol and sums the probability of every column-to-character alignment that collapses to the transcription, the collapse being to squash repeated symbols first and then remove the blanks. The arrangement is shown in Figure 1.

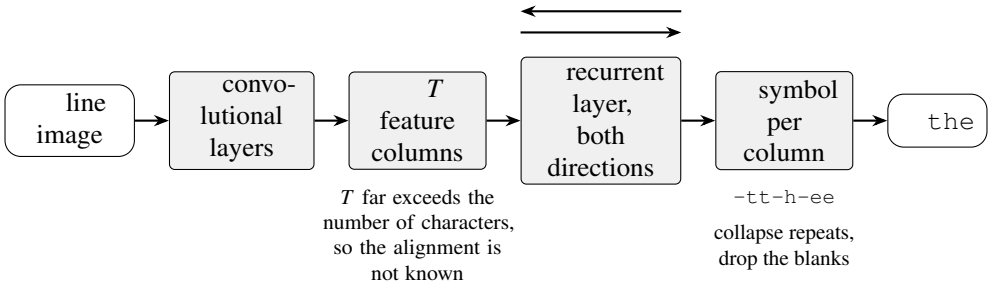


Figure 1. Reading a whole text line without cutting out characters, after the arrangement of Kizilirmak and Yanikoglu [?, pp. 4–5]. The convolutional layers produce T feature columns, a recurrent layer reads them in both directions, and a symbol including a blank is emitted per column; collapsing repeats and removing the blanks gives the transcription.

Existing Models: Two baseline models have been primarily considered in existing research papers: the LeNet-5 CNN model and the ResNet-34 CNN model. LeNet-5 is the most common of the two and was created by Yann LeCun in 1998 [**?**, Fig. 2]. It is a CNN structure that consists of seven layers, including 3 convolutional layers, 2 pooling layers, and lastly, 2 fully connected layers. A fully connected layer refers to a portion of a typical ANN model, while the convolutional layers and pooling layers will be discussed later in this literature study. The input size to the model is 32 by 32 pixels, and the model uses a hyperbolic tangent activation function in the fully connected layers and a softmax activation function in the output layer. This model is popular for use in object detection and image classification [**?**]. ResNet-34 is also a popular CNN model known for its residual architecture. The residual architecture adds skip connections that let the gradient bypass layers, which is what allows the network to be trained at that depth; separately, its weights from training on a large image set are commonly reused by transfer learning, which forms the foundation of the feature extraction in this model and allows for fast and efficient training from limited datasets [**?**].

Feature extraction: The convolutional layers of a CNN are used to perform feature extraction,

which means they can capture subtle patterns in different areas of a segment of text by sliding a convolutional kernel over the character or word image. Translation invariance is created by sliding the convolution kernel, which allows consistent patterns to be extracted from images [**?**]. An N by N input layer is convolved with an f by f kernel and returns an $(N-f+1)$ by $(N-f+1)$ output. Feature maps are created by applying convolutions to different sections of the image being classified. By identifying common features, the kernel-sliding convolutional technique can produce more accurate classification. Each extracted feature is multiplied with a corresponding kernel value to produce a weighting, and then a bias and activation function are applied until an output feature map is obtained [**?**].

An extended version of the standard CNN is the deep CNN, which refers to a CNN that can extract higher-complexity features like handwritten strokes. The problem is that a larger, deeper CNN creates a 'vanishing gradient' problem where there are so many layers that during back-propagation, some layers never get their weightings changed. Thus, there is a limit to the number of layers a standard CNN can have, where adding more layers yields no further benefit. This is why a residual network is used to implement a deep CNN by using residual blocks and skip connections across various layers so that all layers will still update their weightings during backpropagation regardless of the number of layers [**?**].

After a singular round of feature selection, many parameters are still required to classify text with these high-level features. Max-pooling is used to downsample these feature maps until only the essential features remain, minimizing the number of parameters needed to classify handwritten characters and preventing overfitting [**?**]. Figure 2 shows one convolutional layer followed by one max-pooling layer, which is the unit the models are stacked from.

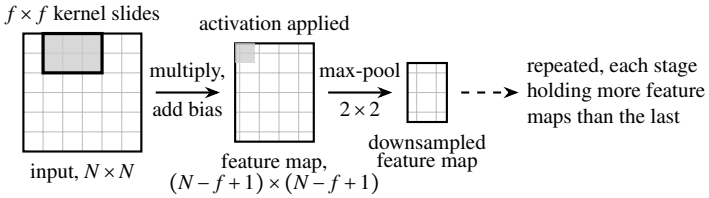


Figure 2. One convolutional layer and one max-pooling layer. The kernel slides over every position of the input, so an $N \times N$ input gives an $(N-f+1) \times (N-f+1)$ feature map, which pooling then downsamples.

Training: The way networks are trained corresponds to the architecture they are built from. The weights are initially randomized with a variance determined by the size of each layer so the signal remains constant as it passes through more layers [**?**]. Batch normalization is the process of taking the outputs of the current batch of training data and subtracting the average and dividing by their standard deviation [**?**, Alg. 1]. Having more sensibly ranged values allows a higher learning rate to be used safely. Dropout is a technique used to reduce overfitting which removes a random subset of a layer's outputs on each training step [**?**, p. 1930]. This introduces variation and prevents the network from depending on any single feature. Another method to increase variation is dataset enlargement which includes applying shears, small rotations, slants and random smooth warps etc. to simulate more variation in training text [**?**]. Learning rate reduction is a technique that is used to adjust the learning rate over time to allow for large changes initially to speed up training, and fine learning later on to prevent overshooting. Final accuracy can be calculated as the character error rate (which refers to the number of mismatches in characters compared to the ideal result) or the word error rate, which is calculated from word mismatches.

This is usually significantly higher than the character error rate as one character mismatch ruins the whole word [**?**, p. 1].

Techniques for writer identification: There are multiple approaches to identifying the author of a written piece of text. The first way is through feature extraction, which measures text-independent features like slant, height, stroke width, word and character spacing, and connectivity between characters. The author is determined by finding which writer possesses the feature metrics closest to that of the text measured. Another feature that can be extracted is the distribution of edge directions in the ink, which gives the 'texture' of the written sentence [**?**]. The other method of writer identification is using a neural network trained off the author's handwriting, which can use a feature extractor used for text recognition [**?**] instead of the hardcoded methods and metrics for quantifying features. Much more content and pages are required to train the author classifiers in this case compared to creating specific feature extracting methods.

1.1.2 A need exists to reproduce text on paper accurately and fast in any writer's style

There are various existing text reproduction methods and linear actuation techniques which can influence the accuracy and speed of text reproduction.

Handwriting synthesis in a writer's style: Before being able to reproduce handwriting through hardware, the desired text to be reproduced has to still be created in the particular author's style. Four methods will be discussed on how to achieve this:

Recurrent stroke level synthesis: A recurrent neural network is used to predict not the characters to be written, but rather the pen movement that must be taken next, with small offset predictions at a time [**?**, p. 1]. It creates a spread of possible answers as no letter is written identically. The network is trained off an online recorded sample of someone's handwriting. Since it predicts pen movements the outputs are already presented as movements which a plotter requires. Training a network to produce accurate outputs requires hundreds of lines of online data.

Adversarial image synthesis: This method creates an image of text using a GAN (generative adversarial network), which trains a text generator and discriminator to identify fake generated text from real handwritten text [**?**, p. 273]. Both the generator and discriminator improve per round until the generator can produce well-reproduced text and the discriminator can no longer tell the difference. It is called 'adversarial' as two networks are trained simultaneously, one to generate text and the other to judge the text. This requires a significant amount of time and compute to train, but the final system produces high quality recognisable text in a single pass once trained. A text and writer classifier are still required to verify the generated text along the way as well. This method produces a raster image which must be converted to pen paths for a plotter to take.

Diffusion-based synthesis: This method generates a fake text raster image by reversing a gradual noise-generating process [**?**, Sec. 1]. The system trains by slowly adding noise to handwritten text, remembering the way noise was added, until the final image is pure random noise. Then to generate a new line of text, the system starts with complete random noise and denoises it until an accurate line of text is reproduced. Only one reference image of the target writer is needed at generation time, although training the model still requires a large set of labelled handwriting images, yet it requires many steps and time to denoise the random noise image. This method also produces a raster image which must still be converted to pen stroke mappings.

Glyph-library based synthesis: This method does not use any generators to create text, but rather assembles words from shapes or glyphs taken out of existing handwritten pages. Haines *et al.* [**?**, p. 26:1] describe a technique that stores a library of labelled glyphs per each user, and then creates the specified sentence by choosing the specific glyphs and then assembling them according to which variant of the glyph the writer usually used in the requested position, and the level of connectedness of the text, the texture of the line of ink, and the spacing of the writing. It requires a small number of handwritten pages per author to create decent glyph libraries, and the main complexities of the method only involve connecting one glyph to another and selecting the most appropriate glyphs.

Assessing generated handwritten text: Accuracy of generated handwriting can be measured through performing image-level distances and correlation to real handwritten text, for example an IoU test (Intersection over Union) which measures the overlap between a predicted region and the true region. Another method is by passing it through an independent text and author classifier and reporting the accuracy of the reproduced text. A survey can also be conducted for human validation of what text appears real and what appears reproduced.

Method of reproduction: There are three main methods to reproduce handwritten text on paper: Cartesian systems, serial-chain robots, and parallel-chain robots. A Cartesian system consists of a gantry with x – y movement. It has restricted movement in only two directions using linear actuation. Serial-chain robots are the most common in industrial settings for automated movement. They consist of multiple linkages chained together, one component connected to the next in a single chain, like a typical robot arm. The third method is parallel-chain robots, which consist of multiple serial-chain linkages that can precisely control automated movement [**?**].

Pen mapping techniques: A pen path is stored as a list of points, and a drawn stroke usually has far more points than a plotter needs. Pen mapping is the step that reduces this list while keeping the shape of the stroke. The Douglas-Peucker algorithm is the common choice [**?**]: it draws a straight line between the two ends of a stroke, finds the point that sits furthest from that line, and keeps it only if it is further away than a chosen tolerance, then repeats on each half [**?**]. A small tolerance keeps many points and follows the stroke closely, while a large tolerance keeps few points and cuts corners, so the tolerance sets the trade-off between the file size and the accuracy of the drawn shape. The alternatives are fitting a smooth curve such as a spline through the points, which needs a fit to be solved, and resampling the stroke at a fixed spacing, which spaces the points evenly but ignores where the curves are. When the handwriting only exists as an image, the path has to be recovered first by thinning the ink down to a one-pixel centre line and then joining those pixels into strokes, which means guessing the drawing order wherever two strokes touch or cross.

Linear actuation types: There are multiple methods of linear actuation that could be considered for a Cartesian plotting system. Some popular examples of linear actuation methods are electric, pneumatic, belt-drive, rack-and-pinion, and ball-screw systems. Electric and pneumatic methods are often used for industrial cases, but belt-drive, rack-and-pinion, and ball-screw linear actuators have been used in multiple handwriting reproduction systems due to their durability, reliability, and cost-effectiveness [**?**].

1.2 Intended contribution and discussion

The literature above covers each stage of the problem on its own, but it does not cover them

as one machine. Recognition is reported on evenly lit scans or on isolated characters, writer identification is reported over sets of hundreds of writers rather than a small closed group, and handwriting synthesis is judged as an image rather than as something that has to be drawn with one pen. Almost all of this work is also built on libraries such as OpenCV [**?**, p. 41], which hide what each stage actually costs in operations and memory. That cost matters here, because every stage of this project must run on a single-board computer.

The main contribution of this project is therefore the full integration of the system, from a photographed page to ink on paper, with every stage implemented from first principles and no machine-learning or image-processing libraries used anywhere in the pipeline. This covers the creation of the dataset, the conditioning and line separation of photographed pages, a line-level text recogniser, the identification of the author, the synthesis of new text in that author's style, and the generation of the pen paths and G-code that drive the gantry. Writing each stage by hand means the forward and backward passes of every layer, the loss functions and the optimisers are known quantities whose cost can be measured and reduced to fit the available hardware.

Two things set the approach apart from the work reviewed. The first is that the system classifies both the text that was written and the author who wrote it, using the same feature extractor for both, so that one page yields a transcription and an identity. The second is that the synthesised handwriting is produced as geometry rather than as an image, so it can be drawn directly without having to recover a pen path from pixels, and the recogniser and the author classifier are then reused to judge the synthesised output before it is committed to paper. The reasoning behind each of these choices, and the alternatives that were rejected, is given in Section 3.

2. Approach

This section explains how the proposal's functional analysis became a first concept, why the subsystems were then built in the order they were, and which alternatives were weighed. Mathematics, component values and experiments belong to Section 3.

2.1 From functional analysis to concept

The functional analysis describes a chain rather than a collection of parts (Figure 3): each block consumes the output of the one before it, so the accuracy of the whole is the product of the accuracies of its parts rather than their average.

The first concept kept the proposal's four functional units, namely page scanning (FU 1), the single-board computer (SBC) and its eight processing steps (FU 2), the gantry (FU 3) and the power supply (FU 4), and made one decision per block; Table 4 in Section 3.2 maps each unit onto the block that implements it. One decision applies to all of them: no computer-vision or machine-learning library was used, so every block was implemented from first principles with only numpy for array arithmetic. That is why Section 3 derives methods a library call would hide, and it narrowed the field to those that could be written and verified in the time available.

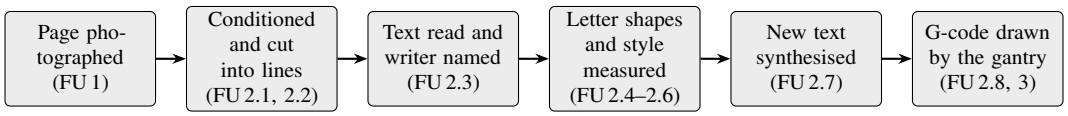


Figure 3. The chosen concept as a chain, with the proposal's functional-unit numbers. Training builds the style store from the first four blocks and runs offline, classification stops after the third, and reproduction runs the last two. The as-built hardware and software block diagrams are Figures 4 and 5.

Seven departures from that concept are declared with their reasons in Table 4; the three that shaped it are the separation of text lines rather than characters, since joined letters have no reliable boundaries, the split of FU 2.3 into two networks instead of one character classifier, and training off the SBC, which needs far more memory than inference, with all inference left on the SBC as required. **[MUST STILL BE COMPLETED BY ME: state on which computer the networks were trained and the profiles built; the proposal requires embedded-only processing.] [MUST STILL BE COMPLETED BY ME: reason for the pen-lift change.] [MUST STILL BE COMPLETED BY ME: supply voltages, current ratings and distribution for FU 4.]**

2.2 Order in which the subsystems were built

Figure 3 is a data path, and the subsystems were deliberately not built along it. The design divides into a motion subsystem and a software subsystem meeting at one interface, a file of G-code; fixing it at the outset decoupled them, since the gantry draws any valid G-code whatever produced it. The gantry was therefore built first and on its own (Section 3.3): the 0.5 mm limit of R5 is set by its mechanics, transmission and step resolution alone, so nothing in it depends on which recogniser or synthesis route was later chosen, and it carries the only schedule risk outside

the work itself, since parts take time to arrive and may arrive faulty. Hand-written G-code was enough to commission and measure it before any software existed.

The software allowed no such freedom, because an error in one block propagates and compounds and is hardest to see at its origin: a slightly mis-cut text line is still a plausible image while it silently bounds the accuracy of everything downstream. Each block was therefore brought to a measured standard before the next was begun, in pipeline order — conditioning and line segmentation (Section 3.4), without whose clean labelled line images no network can be trained; the recogniser and then the writer classifier (Sections 3.5 and 3.5), in that order because the second reuses the first's feature extractor; glyph and style extraction (Section 3.6), which needs the recogniser to locate the letter shapes; synthesis (Section 3.6); and G-code generation (Section 3.3.6). The page-scanning rig was fixed during the first of these, once freehand photographs rather than the algorithm proved to be what limited the result, since a camera held at one height looking straight down removes scale, tilt and framing variation at source; a flatbed scanner would have lit it more evenly but is larger and slower.

2.3 Design alternatives and trade-offs

Each decision was weighed against the same criteria: accuracy (R1, R2), processing time on the SBC (R3, R4), the data available for thirteen writers (R6), implementation from first principles, and mechanical and electrical simplicity and safety. Table 1 gives the alternatives and the reason for each preference.

Decision	Alternatives considered and how they compare	Preferred and reason
Hardware		
Robot type	(a) Cartesian gantry: identity kinematics, axis errors do not add up. (b) Serial arm: inverse kinematics, joint errors add up. (c) Parallel robot: stiff, but a small workspace and complex kinematics.	(a): position follows from step counts, so the 0.5 mm limit of R5 is simplest to meet and verify [**?**].
Interface to the gantry	(a) A custom protocol fits the synthesiser exactly, but only the synthesiser can then exercise the machine. (b) G-code is a standard shared with commercial equipment, readable and writable by hand.	(b): it let the gantry be commissioned and measured before any software block existed, and keeps the two subsystems independently testable.
Software		
Unit of recognition	(a) Cut characters and classify each: needs character-level labels, which are absent, and cannot repair a wrong cut. (b) Cut lines and read with a sequence recogniser: no boundaries below the line, context used, larger network.	(b): cursive cannot be segmented reliably, labels exist per line, and segmenter errors would bound system accuracy.
Synthesis of writing	(a) A neural generator needs recorded pen trajectories, which are unavailable. (b) Pretrained image generators give raster output, no control of size or slant, and are slow without a graphics processor. (c) A library of the writer's own letter shapes needs only nine pages and gives controllable strokes, but loses some cursive links.	(c): a from-scratch generator learned slant and roundness but not legible letters, and image generators took tens of seconds per line on a processor-only computer.

Table 1. Design decisions, alternatives considered and the reason for each preferred solution.

Each block was held to a measurable criterion from the requirements — characters read correctly, writers named, lines cut correctly, pen position in millimetres — and measured alone before the chain was measured end to end; where no known truth exists, as for the drawn page, a round trip was used. That measurement later replaced four designs, collected in Section 3.7. Only the camera, SBC, motors, drivers, switches, pen, standard gantry parts and power supply were bought. **[MUST STILL BE COMPLETED BY ME: list the mechanical parts I designed and made.]** Section 3 takes each block in build order, so each is met with its input explained.

3. Design and implementation

3.1 Design summary

This section summarises the project design tasks and how they were implemented (see Tables 2 and 3). The rows are the deliverables of line 5 of the table in Section 4 of the approved project proposal and the nine tasks of its Section 6.1. The summary is split over two tables only so that neither runs across a page break; the headings are the same in both. A task is marked *Completed* only when the deliverable itself works. Where any part of it is missing, unbuilt or unmeasured it is marked *Incomplete*, and the second column says what is missing. Only work designed and implemented by the student is claimed. The parts that were bought as finished items, namely the camera, the single-board computer, the motors and their driver modules, the pen, the switches, the power supply and the standard mechanical parts, carry no design credit. Every image-processing, machine-learning, trajectory and motion-control algorithm in the running system was written by the student in Python on top of NumPy array arithmetic. No machine-learning framework supplies any layer, gradient or optimiser. The exceptions are all on the offline training path or at the interfaces, and are listed in declaration 6 of the preamble. SciPy is used in exactly two files, for peak finding when the public database pages are cut into lines and for the Gaussian filtering and coordinate mapping of the elastic augmentation; the remaining libraries interface with the camera, the general-purpose pins, the image files, the database download and the network.

Two deviations from the proposal shape several rows. First, the proposal planned to separate the page into characters (functional unit 2.2) before classification. The implemented design separates the page into text lines only and reads each line with a sequence recogniser, because the characters of joined handwriting have no reliable boundaries and a separation error would pass straight into a wrong reading (Sections 3.4 and 3.5). Second, the proposal placed training and the building of trajectory maps on the single-board computer (functional units 2.4 to 2.6). In the implemented system the networks and the per-writer style profiles are produced by offline programs on a general-purpose computer, and only inference, synthesis and motion control run on the board. The interface therefore offers no training mode. **[MUST STILL BE COMPLETED BY ME: state the computer used for network training (the project notes mention a laptop processor and a cloud notebook) and confirm that the single-board computer was not used for training.]**

3.2 System block diagram

Two diagrams describe the implemented design: Figure 4 gives the hardware blocks and how they connect to the single-board computer, and Figure 5 gives the software data flow of the three modes of operation, which are reproduction, classification and offline training. Both follow the functional block diagram of the approved proposal, which divided the system into the page-scanning set-up (FU 1), the single-board computer (FU 2), the gantry (FU 3) and the power supply (FU 4). The four physical units were kept, but the software units FU 2.1 to FU 2.8 were reorganised; Table 4 relates the implemented blocks to the proposed units and gives the reason for every departure.

Deliverable or task	Implementation	Completion of deliverable or task, and section in the report
Research of network types, structure, layers and feature extraction (task 6.1.1)	A literature study of convolutional, recurrent and sequence-to-sequence recognisers, of writer-identification methods and of the alternatives to each, used to choose the methods of Sections 3.5 and 3.5.	Completed. Section 1.
Research of conditioning, noise removal, separation and trajectory mapping (task 6.1.2)	A literature study of thresholding, illumination correction, line and character separation, thinning and exemplar-based handwriting synthesis, used to choose the methods of Sections 3.4 and 3.6.	Completed. Section 1.
Image conditioning and noise removal (FU 2.1; R2, R4)	Implemented from first principles using only array arithmetic: size normalisation, page localisation, flat-field illumination correction with summed-area tables, local adaptive thresholding, skew correction and removal of printed rules, faint print and neighbouring-page ink. The individual operations are standard and are cited in Section 3.4; the stages chosen, their order and every threshold were settled for this project.	Completed. Section 3.4.
Separation into characters (FU 2.2; R2, R4)	Implemented as separation into text lines and diagram regions rather than into characters (connected components, glyph-scale estimation, chaining, baseline merging, text versus non-text scoring, one deskewed crop per line). Character territories are derived only offline, by forced alignment, when style profiles are built. The reason for the change from the proposal is given above and in Section 3.5.	Completed. Sections 3.4, 3.5.
Optimised and efficient character separation (R4; task 6.1.3)	The segmentation was profiled and its two dominant routines, component labelling and the box sums, were rewritten in C by the student at 84 to 88% faster each, 25 to 32% faster per page. The C routines are not called by the deployed program, and segmentation still exceeds the R4 allowance by over an order of magnitude.	Incomplete. Sections 3.4, 4.
Neural network that decodes the text (FU 2.3; R2)	Convolutional and bidirectional recurrent network with connectionist temporal classification (CTC) output, 3 358 763 trainable parameters. The layer pattern follows a published design [**?**, p. 9]; the student chose the input geometry, data pipeline, label cleaning, augmentation and staged training, and wrote the from-scratch module (forward and backward passes, optimisers, no machine-learning library) used by training, validation and deployment alike.	Completed. Section 3.5; measured accuracy 98.61% character and 94.21% word on 669 real line crops (Section 4).
Neural network that identifies the writer (FU 2.3; R2)	Two thirteen-class convolutional classifiers sharing the recogniser's backbone and layer module: an ink-based one for photographs, which reached 99.78% over 1358 real lines, and one whose input is re-drawn at a constant pen width by the student's stroke normalisation, with page-level aggregation.	Completed. Section 3.5; measured accuracy 93.67% for the normalised classifier after a training fault was corrected (Section 4).
Training on datasets and choice of layer numbers (FU 2.4; task 6.1.4)	Training programs for the recogniser and both classifiers on a public database, a public line-level set and the photographed personal pages, all built on the from-scratch module of Section 3.5.5, whose every layer gradient was checked against numerical differentiation in 78 configurations. A sweep over widths, depths and learning rates was programmed but no result was kept, so the layer numbers are the published design's and are not justified by this project's own measurement. Training is offline.	Incomplete. Sections 3.5, 3.5.
Trajectory maps and permanent storage (FU 2.5 to 2.7; R1, R6)	Letter shapes are harvested by forced alignment with the recogniser, thinned to centre-line pen paths, normalised and stored as one JavaScript Object Notation (JSON) file per writer with measured style statistics; thirteen styles exist, for eight database writers and five personal writers. The builder is an offline program.	Completed. Section 3.6.
Pen trajectory mapping and G-code (FU 2.5 to 2.8; R1, R3)	A synthesiser assembles the writer's own letter paths into lines (slant, spacing, joins, wobble) with a best-of-N search judged by the recogniser and the writer classifier; a generator turns the trajectory into G-code, the numerical-control command language of plotters, with placement, stroke ordering and pen-state tracking, and a simulator replays it.	Completed. Sections 3.6, 3.3.6.
Optimisation of pen movement (R3)	Greedy nearest-endpoint stroke ordering reduces pen-up travel. The saving it achieves was not measured.	Completed. Section 3.3.6.

Table 2. Design summary, part 1 of 2: the research, image-processing, network and trajectory tasks.

Deliverable or task	Implementation	Completion of deliverable or task, and section in the report
Capture rig and deployment on the single-board computer (FU 1; R2, R4; task 6.1.5)	A generic USB camera in a printed housing that clamps the page flat, read with a live preview and a five-frame warm-up; the recognition, synthesis and motion programs run on the ODROID N2+ behind a web server. The rig geometry, camera resolution and lighting are not documented. **[MUST STILL BE COMPLETED BY ME: camera model, resolution, mounting height, field of view, lamp, lux readings, photograph of the rig.]**	Incomplete. Sections 3.2, 3.8.
Testing the classifier on the single-board computer with captured images (task 6.1.6)	The deployed programs were written to run on the board and do run on it, but no accuracy or timing figure in this report was obtained on the board; every measurement reported in Section 4 was taken on a laptop.	Incomplete. Section 4.
Gantry design and construction in CAD (FU 3.3, 3.4; R5; task 6.1.7)	A Cartesian X/Y gantry built by the student from bought parts: one motor turning an 8 mm cross-shaft that drives both ends of the moving bridge, and a second motor on that bridge driving the carriage, both on GT2 belts over linear bearings and chromed rods, with four end-stops. Hand-measured travel 204 mm (X) and 262 mm (Y). The machine exists but no CAD drawing or photograph was kept. **[MUST STILL BE COMPLETED BY ME: CAD files and photographs.]**	Incomplete. Section 3.3.
Pen holder, pen adjuster and paper holder (R5)	The pen is carried on the moving carriage in a holder with the toggle cam of the lift mechanism above it, and the page is clamped flat in the camera housing for capture. Whether the holder provides a pen-height adjustment, and how the paper is held on the writing bed, are not recorded. **[MUST STILL BE COMPLETED BY ME: confirm how the pen height is adjusted and how the paper is held down on the writing bed, and supply a photograph of each; neither is documented in the project files.]**	Incomplete. Section 3.3.
Power supply (FU 4)	A 24 V supply feeds both stepper drivers through a 100 µF bulk capacitor. The conversion to the voltages the single-board computer and the pen motor require is not documented, no power budget was drawn up and the supply's rating is unknown.	Incomplete. Section 3.3.2.
Stepper-motor control and pen lift (FU 2.8, 3.1 to 3.4; R5; task 6.1.8)	A G-code interpreter generates step and direction pulses directly on the general-purpose pins, with Bresenham interpolation, absolute step bookkeeping, calibration between end-stops, debounced end-stop monitoring and software limits. The pen is lifted by a toggle gear motor confirmed by a position switch, switched by a transistor stage designed by the student, rather than through the stepper driver as the proposal planned. During bring-up the X-axis direction pin was not observed to change level on reversal and that fault was not resolved.	Incomplete. Section 3.3.
Smooth motor control for efficient gantry movement (R3)	The interpreter steps at a constant interval within a segment and executes no acceleration ramp, although the generator's time model assumes one. Abrupt starts at the feeds used may lose steps, and no torque margin was computed because the carriage masses were not measured.	Incomplete. Sections 3.3.1, 3.3.4.
Display and user interface (R4, R6)	A web server exposes classification, generation, calibration and sending of jobs; browser pages show the text, the writer with a confidence, the stored writers and a G-code preview. The synthesis time is returned, but the classification time that R4 asks to be displayed is not shown.	Incomplete. Section 3.8.
Full-system testing against the requirements (task 6.1.9)	The accuracy, writer-identification and timing evaluations of Section 4 were designed and run by the student. The physical writing time of R3, the on-board timing of R4, the positional error of R5 and the random-writer reproduction of R6 were not measured, because the gantry bring-up was not completed.	Incomplete. Section 4.

Table 3. Design summary, part 2 of 2: the capture rig, the gantry hardware, the motion control and the verification tasks.

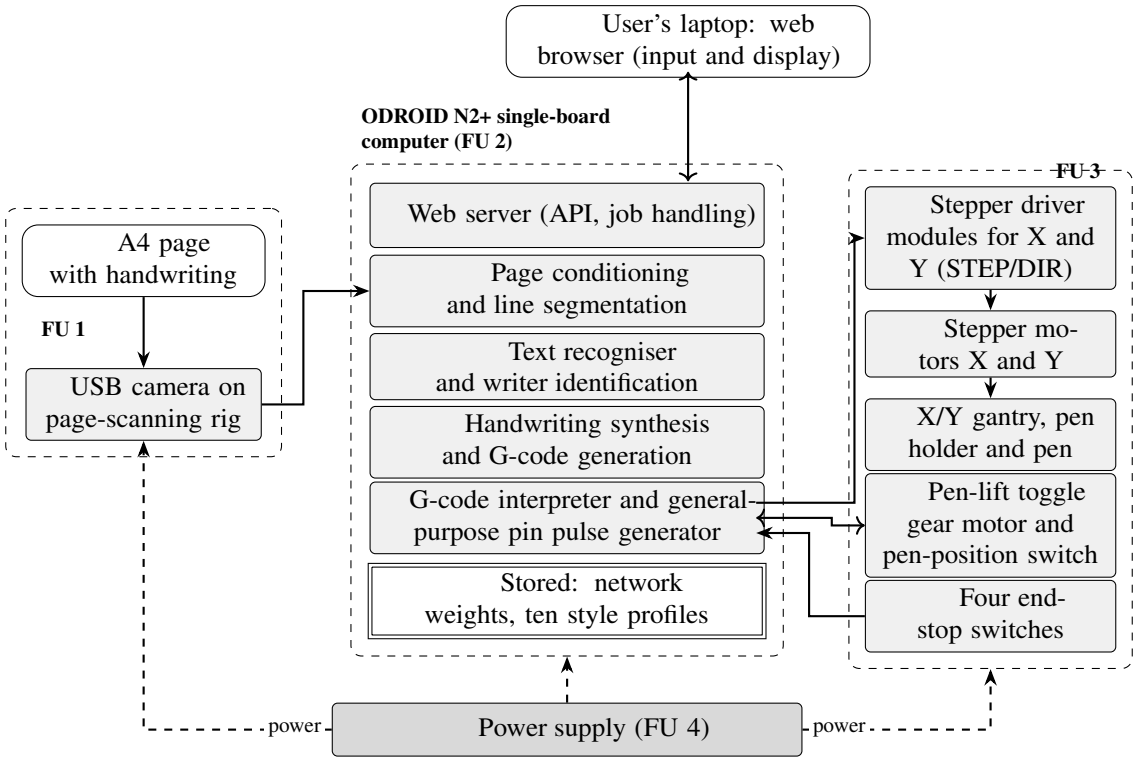


Figure 4. Hardware block diagram of the implemented system. Solid arrows show signals and data, dashed arrows show the power supply. FU numbers refer to the functional units of the project proposal.

3.2.1 Hardware block diagram

The hardware of Figure 4 is four groups. The page-scanning rig (FU 1) holds a generic USB camera above an A4 page, read through the operating system's own driver, so no camera driver was designed by the student. The single-board computer (FU 2) is an ODROID N2+ and runs every software block, including the web server through which a laptop controls the robot. The gantry (FU 3) moves each axis with one stepper motor and one driver module, whose STEP and DIR inputs go straight to general-purpose pins, so there is no separate microcontroller; four end-stop switches bound the travel, and a toggle gear motor with a position switch raises and lowers the pen. The power supply (FU 4) feeds all three. The circuit itself, with its component values and design calculations, is given in Section 3.3. **[MUST STILL BE COMPLETED BY ME: model numbers of the camera, stepper motors and driver modules, the supply voltages and current ratings of the power supply, the way in which the pen motor is switched, and the logic-level and grounding arrangement between the single-board computer and the drivers; these are not documented in the project files.]**

3.2.2 Software block diagram and systems-level description

The system learns how a person writes from photographs of that person's pages and later writes new text in the same style. Figure 5 shows the three chains of blocks that do this and the

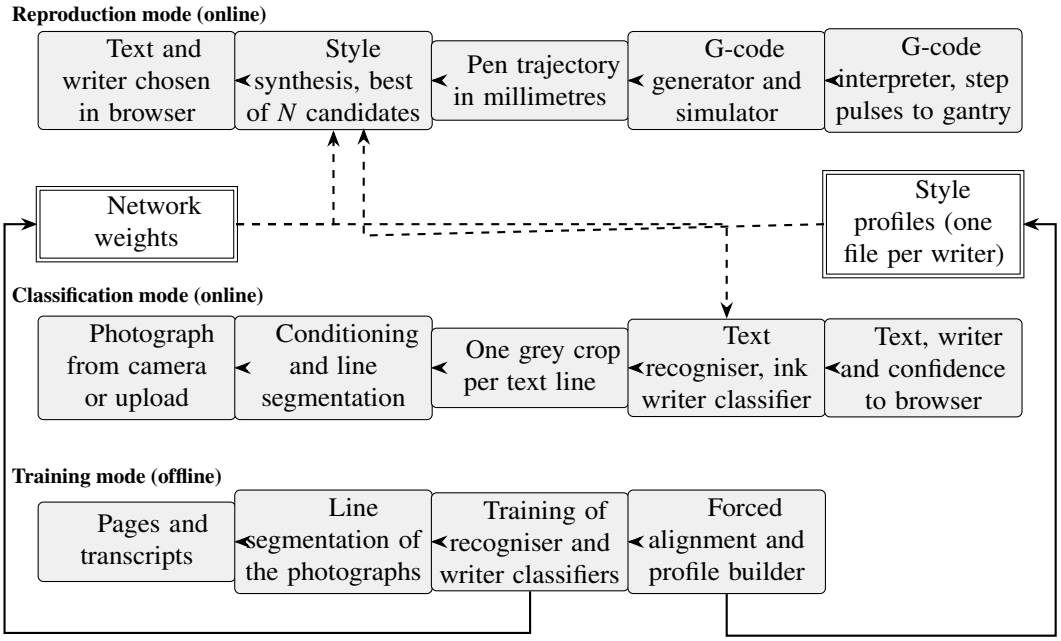


Figure 5. Software data flow of the three modes. Dashed arrows are reads from permanent storage, solid arrows between the training row and the two stores are writes. The same line-segmentation and recognition blocks serve classification, training and the judging of synthesised handwriting.

two permanent stores they share, the network weights and the per-writer style profiles. In the classification mode (R2, R4) the page is conditioned and separated into text lines (Section 3.4), each line is read by the text recogniser (Section 3.5) and classified by the ink-based writer classifier (Section 3.5), and the writer probabilities of the lines are averaged to a page decision. In the reproduction mode (R1, R3, R5, R6) the synthesiser (Section 3.6) assembles the writer's own stored letter paths (Section 3.6) into a line, repeats the random construction for N candidates and keeps the one that the recogniser and the stroke-normalised writer classifier jointly prefer; the trajectory is converted to G-code (Section 3.3.6) and executed by the interpreter after the gantry has been calibrated between its end-stops (Section 3.3). The training mode runs offline: the same segmentation block prepares the pages, the networks are trained and the profile builder writes the two stores, after which no image and no training code are needed to use a style.

3.3 Hardware design

This subsection gives, component by component, the theory behind the chosen mechanism, the calculations that fix every value in the circuit, the final design, and the tests that were run on it (functional units FU 2.8, FU 3.1 to FU 3.4; requirements R3 and R5). It is ordered as the machine was built: the mechanism that moves the pen, the motor chosen to drive it, the circuits that drive that motor, the resolution the whole transmission yields, the protection placed around it, and last the program format that commands it. The computer emits both drivers' STEP and DIR pulses itself, with no microcontroller between, so the end-stop read lies inside the loop that emits every step. All parts were bought; what is claimed is the selection, the four interface circuits, the calculations below, the motion-control program and the 800-line G-code interpreter [**?**].

Proposal unit	Implemented block	Difference and reason
FU 1.1, 1.2	USB camera over an A4 page, read by the operating-system driver.	As planned; the “serial bitstream converter” is the standard camera interface.
FU 2.1	Page conditioning (Section 3.4).	Much larger than planned: lighting, skew, ruled paper, neighbouring pages and diagrams are handled.
FU 2.2	Line segmentation (Section 3.4).	Lines instead of characters, because cursive characters have no reliable boundaries.
FU 2.3	Text recogniser and writer classifier (Sections 3.5 and 3.5).	Two separate networks instead of one character classifier.
FU 2.4	Offline training programs.	Not on the single-board computer, because training needs far more memory and time than inference.
FU 2.5, 2.6	Style-profile builder and permanent profile files (Section 3.6).	Offline; stores letter paths per writer instead of one trajectory per recognised character.
FU 2.7	Synthesiser (Section 3.6).	Constructs new text from stored letter variants and adds a best-of-N search.
FU 2.8	G-code generator and interpreter with pulse generation (Sections 3.3.6 and 3.3).	Split into two stages with a standard interchange format, which allows the trajectory to be checked without the gantry.
FU 3.1	Stepper driver modules on 3.3 V pins.	No separate voltage-shifter block is documented. **[MUST STILL BE COMPLETED BY ME: confirm.]**
FU 3.2	Two stepper motors on one gantry.	As planned.
FU 3.3, 3.4	Pen holder and toggle pen-lift motor with position switch.	Pen lift is not driven through the stepper driver, as the proposal assumed, but by a one-direction gear motor with feedback switch.
FU 4	Power supply.	Not documented in the project files.
(new)	End-stop switches and web server.	Added for protection of the mechanism and to replace the proposed display and input interface.

Table 4. Relation of the implemented blocks to the functional units of the project proposal.

3.3.1 Plotter design and mechanics

The chosen method of text plotting is a Cartesian gantry system with two independent axes of movement and a pen up/down mechanism. Complex kinematics equations and error dependencies are avoided by using two isolated x and y movement methods for reproduction [**?**], so a page target needs no inverse kinematics and an error in one axis does not rotate the other. Linear GT2 timing belts and pulleys are used to convert rotary motion from stepper motors and axles to the desired Cartesian movements. Timing belts are used for their reliability, cost effectiveness, and higher speed travel [**?**].

A toothed belt over two pulleys converts a motor’s rotation into a linear carriage position with no backlash of the kind a lead screw’s nut would show, and its tension carries the load so that a jammed carriage makes the belt slip rather than stalling the motor, protecting the driver and the gearing. Against a ball screw, rack-and-pinion, pneumatic cylinder or direct linear motor, a GT2 belt drive was preferred on durability, reliability and cost for this long, light travel, which needs speed rather than force [**?**].

A Cartesian gantry consists of a stationary rail that controls movement in one axis, and a movable rail that controls the other axis of movement. For this system and future explanations, the stationary rails control y-axis movement, while the x-axis rail moves along the y-axis. Figure 6 explains the axes of the gantry.

FIGURE MUST STILL BE INSERTED BY ME

Labelled diagram of the Cartesian gantry axes and the pen up/down mechanism: the two fixed *y*-axis rails, the *x*-axis rail that rides on them, the cross-shaft and both *y* pulleys, and the pen carriage.

Figure 6. Cartesian gantry axes and pen up/down mechanism.

The *x*-axis rail needs to move with stability and consistency, which is why it is supported on both ends by dual rails on the *y*-axis. A single stepper motor was chosen to drive both pulleys on both *y*-axis rails to ensure synchronicity and save costs. Two independent stepper motors driving one axis creates synchronicity issues, because any difference in timing, missed steps or load torques could offset the movement between the two motors and rotate the bridge out of square; synchronising two open-loop steppers to the same step count is itself a design problem rather than a given. The challenge with using one motor to drive two timing pulleys is finding a method to extend the shaft of the motor to connect to both timing pulleys without overloading the motor and applying too much torsion or twist to the new axle. A rigid coupler was used to extend the axle of the stepper motor to a chromed steel rod, so the bridge cannot go out of square provided the shaft itself does not twist.

To reduce strain on the motors, and allow for increased accuracy and precision, linear bearings were fitted around the chromed rod rails to allow all parts to slide with very little friction. To verify the stepper motor could handle the weight of the *x*-axis rail further, the torque required to move the rail at the desired speed was calculated; that calculation is carried out in Section 3.3.2 together with the stepper motor type chosen and its product specifications.

Equation 1 gives the travel per revolution and the pitch radius for pitch p on N_t teeth,

$$\ell_{\text{rev}} = p N_t = 2 \cdot 20 = 40 \text{ mm}, \qquad r = \frac{\ell_{\text{rev}}}{2\pi} = \frac{40}{2\pi} = 6.37 \text{ mm}, \tag{1}$$

which is the single conversion between motor rotation and carriage travel for this machine: every resolution, speed and torque figure in the subsubsections that follow passes through it, there being no gear reduction anywhere in the drive train.

Table 5 gives the drive-train components and the reason for each. 2 mm pitch on 20-tooth pulleys gives the 40 mm per revolution of Equation 1; measured travel is 204 mm (X) and 262 mm (Y). One motor turns one 8 mm cross-shaft through a 5-to-8 mm coupler, a pulley at each end, so the bridge stays square without a second motor and without a synchronisation scheme; a second motor on that bridge drives the carriage along it, so the lower axis also accelerates that mass.

MUST STILL BE COMPLETED BY ME

Add to Table 5 the part numbers and supplier of the motors, driver carriers and supply, and state how 24 V is converted to the voltages the ODROID N2+ and pen motor need: no converter is documented.

Component	Quantity, rating	Reason for the choice, and content to be supplied
ODROID N2+ board	1	The proposal requires all processing on an embedded platform; one board runs everything off its 3.3 V pins.
NEMA 17 stepper	2, 1.68 A/phase, 0.44 N m holding [?]	Open-loop position from a step count [?], so R5 needs no encoder. **[MUST STILL BE COMPLETED BY ME: model number and datasheet revision of the motors fitted.]**
A4988 driver carrier [?]	2	One STEP and one DIR line per axis is the smallest pin count; the chopper runs the 1.68 A winding from 24 V, and 1/16 microstepping [?], [?, p. 6] is set by jumpers. **[MUST STILL BE COMPLETED BY ME: part number, datasheet revision and sense-resistor value of the carriers fitted; the microstepping jumpers, the minimum STEP pulse width, and whether a level shifter is fitted.]**
Power supply	1, 24 V	A higher rail forces current into the winding inductance faster, allowing the step rates of Section 3.3.4; inside the A4988's 8 to 35 V range [?, p. 4]. **[MUST STILL BE COMPLETED BY ME: voltage and current ratings of the motor, computer and pen supplies, the grounding scheme and the camera supply.]**
Bulk capacitor	1, 100 µF	Supplies the chopping current locally (Equation 9).
GT2 belt, 2 mm pitch, 20-tooth pulleys, idlers	2 axes	Durability, reliability and cost over ball-screw, rack-and-pinion, pneumatic and direct electric drive [?]; idler tensioning limits backlash. Measured travel 204 mm (X), 262 mm (Y). **[MUST STILL BE COMPLETED BY ME: the calibrated steps per millimetre printed for both axes.]**
Chromed rods, linear bearings, 5 to 8 mm rigid coupler	as required	Carry the carriage so the belt only pulls it; the coupler extends the motor shaft to the cross-shaft that drives both *y* pulleys.

Table 5. Drive-train components, with the reason for each choice. Every part was bought; the interface circuits, the calculations and the control program are the student's own.

The mechanical parts that are not bought stock were modelled and printed for this machine. Figure 7 shows the CAD models and Table 6 gives their dimensions and the function of each.

MUST STILL BE COMPLETED BY ME

Measure and fill in the Dimension column of Table 6 from the CAD models, and export the eight renderings of Figure 7 to `Figures/cad-component-01.png` through `cad-component-08.png`. Export the axis diagram to `Figures/gantry-axis-diagram.png`.

Mechanical testing that was run is reported qualitatively: sliding the printed carriages with bearings over the rod railings to check friction showed an initial mislabelling between rod diameter and linear-bearing inner diameter, resolved with a vernier calliper, and several bearings found misaligned in their prints were removed to reduce friction.

3.3.2 Stepper motor and driver

A stepper motor does not continuously turn with a PWM input, but rather turns at specific fixed angles each time there is a switch in the windings (the coils of the stepper motor). It does this through a process called *chopping*, which switches the current supply on the winding until the current rises to a certain point, and then switches off at the target current. This occurs at high frequencies and effectively keeps the current going to the coils within a specific range so as to not overshoot, even when the motor is not actively stepping. A step is created by slightly adjusting the current value between two coils just enough to allow the rotor's teeth to make a small jump or 'step'. The stepper motor is perfect for the precise movements required for handwriting reproduction with the gantry, as it produces steps at exact, known intervals, with no encoders or feedback loops required.

FIGURE MUST STILL BE INSERTED BY ME
CAD model: rail platform

FIGURE MUST STILL BE INSERTED BY ME
CAD model: rod/switch holder

FIGURE MUST STILL BE INSERTED BY ME
CAD model: pulley holder

FIGURE MUST STILL BE INSERTED BY ME
CAD model: pen holder

FIGURE MUST STILL BE INSERTED BY ME
CAD model: pen clicker housing

FIGURE MUST STILL BE INSERTED BY ME
CAD model: belt fastener

FIGURE MUST STILL BE INSERTED BY ME
CAD model: rotary cam

FIGURE MUST STILL BE INSERTED BY ME
CAD model: motor holder

Figure 7. CAD models of the gantry components designed and printed for this machine.

Component	Dimension	Description
Rail Platform	**[MUST STILL BE COMPLETED BY ME: dim.]**	Base plate that locates both *y*-axis rails and fixes their separation, so the bridge stays square.
Rod/Switch Holder	**[MUST STILL BE COMPLETED BY ME: dim.]**	Clamps the end of a chromed rod and carries the end-stop microswitch for that axis.
Pulley Holder	**[MUST STILL BE COMPLETED BY ME: dim.]**	Holds the idler pulley and provides the adjustment that tensions the belt.
Chromed steel rod *y*-axis	**[MUST STILL BE COMPLETED BY ME: dia. × len.]**	The two fixed rails the bridge slides along, on linear bearings.
Chromed steel rod *x*-axis	**[MUST STILL BE COMPLETED BY ME: dia. × len.]**	The moving rail that carries the pen carriage across the page.
Pen holder	**[MUST STILL BE COMPLETED BY ME: dim.]**	Grips the pen barrel on the carriage and sets the pen's height above the paper.
Pen clicker housing	**[MUST STILL BE COMPLETED BY ME: dim.]**	Houses the N20 gear motor and the microswitch of the pen up/down mechanism.
Belt Fastener	**[MUST STILL BE COMPLETED BY ME: dim.]**	Clamps both ends of the GT2 belt to the carriage so the belt pulls it directly.
Rotary cam	**[MUST STILL BE COMPLETED BY ME: dim.]**	Turned 90° per pulse by the gear motor; toggles the pen between up and down.
Motor Holder	**[MUST STILL BE COMPLETED BY ME: dim.]**	Mounts a NEMA 17 to the frame and aligns its shaft with the coupler and cross-shaft.

Table 6. Gantry component dimensions and descriptions.

Because the current is set by the chopper's target rather than by the winding resistance alone, a 24 V rail can be used on a motor rated for a lower voltage, and the driver draws its own supply current in short bursts rather than smoothly. The consequence for the circuit is a chopper current the leads cannot follow, which a local bulk capacitor must absorb; that capacitor is sized in Section 3.3.3.

Two stepper motors are used in the gantry plotter, one for the x and one for the y-axis direction. The type of stepper motor chosen was the NEMA 17 bipolar stepper motor, with a current rating of 1.68 A per phase and a holding torque of 0.44 N m [**?**]. The following calculations were done to determine if the NEMA 17 stepper motor would be strong enough for the use case of the gantry plotter, and if a single motor could control both y-axis belt drives safely.

[MUST STILL BE COMPLETED BY ME: confirm the motor's step angle against its datasheet. The draft hardware notes record 400 steps per revolution (0.9°), whereas the control program, the generated G-code headers and Equation 14 all assume 200 full steps per revolution (1.8°). The two differ by a factor of two in steps per millimetre; the homing calibration of Equation 18 measures the true figure and so the machine works either way, but the report must state one of them.]

Two torque figures, and why only one of them may be used. Two torque figures are quoted for a stepper and must not be confused. The *holding torque* is the torque the motor resists when stationary and energised, and it is the figure datasheets lead with — 0.44 N m, or 44 N cm, for this motor. The *pull-out torque* is the torque the motor can deliver while actually stepping at a given rate, and it falls as the rate rises, because the winding inductance limits how fast current is established. Only the pull-out torque at the rates actually used is a safe basis for a design, because a motor that loses synchronisation under load does not stall gracefully: it skips steps, and in open-loop control a skipped step is an unrecoverable position error for the rest of the job.

For a winding of inductance L and resistance R driven from a rail V, the current reaches its regulated value with time constant L/R but is forced by the chopper at a rate V/L, so the fraction of rated current reached within one step interval τ of Equation 15 falls as τ shortens, and with it the available torque:

$$\frac{T(\tau)}{T_{\text{hold}}} \approx 1 - \exp\!\left(-\frac{V\,\tau}{L I_{\text{coil}}}\right), \tag{2}$$

which is why the 24 V rail was chosen over a 12 V one for the same motor: at fixed τ, doubling V in Equation 2 moves the knee of the torque–speed curve to roughly twice the step rate. Equation 2 is an approximation that ignores back-electromotive force and the chopper's own regulation, and it is given here to show the shape of the dependence rather than to predict a value; the manufacturer's measured pull-out torque curve is the quantity the design is checked against below.

Speed-to-torque margin check. The commanded linear feed is converted to a motor speed, read against the manufacturer's measured pull-out torque curve, and compared with the torque the moving mass actually demands.

$$n = \frac{v}{\ell_{\text{rev}}} \times 60 = \frac{4000\ \text{mm/min}}{40\ \text{mm/rev}} = 100\ \text{RPM} \tag{3}$$

where n is the motor speed in revolutions per minute, $v = 4000$ mm/min is the commanded feed (60 mm/s rounded up for margin), and $\ell_{\text{rev}} = 40$ mm is the belt travel per revolution already established in Equation 1 from the GT2 pulley's pitch ($p = 2$ mm, $N_t = 20$ teeth).

$$T_{\text{po}}(n) = T_1 + \frac{n - n_1}{n_2 - n_1}\left(T_2 - T_1\right) = 7 + \frac{100 - 60}{120 - 60}(21 - 7) \approx 16.3\ \text{N cm} \tag{4}$$

where $T_{\text{po}}(n)$ is the pull-out torque available at speed n, found by linear interpolation between the two bracketing points read off the manufacturer's pull-out torque curve [**?**]: $(n_1, T_1) = (60\ \text{RPM}, 7\ \text{N cm})$ and $(n_2, T_2) = (120\ \text{RPM}, 21\ \text{N cm})$. Note that 16.3 N cm is already well below the 44 N cm holding figure, which is exactly the distinction drawn above. **[MUST STILL BE COMPLETED BY ME: confirm `stepperds` in the bibliography is this specific pull-out torque curve (24 V, 1.68 A); add a separate citation if it is a different datasheet from the one already cited for the motor's basic ratings.]**

$$T_{\text{req}} = r F_f = r\,\mu\,m\,g = 0.00637 \times 0.004 \times 0.950 \times 9.81 \approx 0.024\ \text{N cm} \tag{5}$$

where T_{req} is the torque needed to move the carriage at constant velocity, $r = 6.37$ mm is the pulley pitch radius of Equation 1 (the belt pulls on the pulley's pitch circle, so this is the radius that converts carriage force to motor torque), $\mu \approx 0.004$ is an assumed linear-ball-bearing rolling-friction coefficient, $m = 0.950$ kg is the moving mass (0.400 kg motor + 0.400 kg chromed steel rods + remainder in 3D-printed carriage parts), and $g = 9.81$ m/s^2. **[MUST STILL BE**

COMPLETED BY ME: cite a bearing manufacturer's catalogue value for the rolling-friction coefficient μ; it is currently an assumed, not a sourced, figure. The conclusion is not sensitive to it: at $\mu = 0.005$ the margin of Equation 6 is 560 instead of 680, and the design is not torque-limited at either value.]

$$\frac{T_{\text{po}}}{T_{\text{req}}} = \frac{16.3}{0.024} \approx 680 \tag{6}$$

The pull-out torque available at the highest commanded speed exceeds the torque the carriage actually demands by a factor of about 680, so the drive is not torque-limited at any feed used by this machine, and a single motor driving both y-axis belts through the cross-shaft is comfortably within its capability.

FIGURE MUST STILL BE INSERTED BY ME
Photograph of the NEMA 17 stepper motor fitted

FIGURE MUST STILL BE INSERTED BY ME
Photograph of the A4988 stepper driver carrier fitted

Figure 8. NEMA 17 stepper motor (left) and A4988 stepper motor driver (right).

What the constant-velocity check leaves out. Equation 5 is a constant-velocity figure: it balances friction only, and says nothing about the torque needed to *accelerate* the carriage. The full requirement is

$$T = r\left(m_{\text{eff}}\, a + F_f\right) + J_{\text{rot}}\frac{a}{r}, \tag{7}$$

in which J_{rot} is the rotational inertia of the pulleys, the cross-shaft and the rotor. That second term is not negligible for this machine: the lower axis turns a cross-shaft with a pulley at each end, so its rotating inertia is carried at the same acceleration as the carriage, and referring it to the carriage gives an equivalent mass J_{rot}/r^2. With $r = 6.37$ mm the division by r^2 is a multiplication by about 2.5×10^4 m^{-2}, so even a small rotor inertia contributes an equivalent mass of the same order as the carriage itself; a design that ignored it would understate the torque required. The effective mass of the lower axis is therefore

$$m_{\text{eff}} = m_{\text{carriage}} + m_{\text{upper axis}} + m_{\text{pen}} + \frac{J_{\text{rot}}}{r^2}, \tag{8}$$

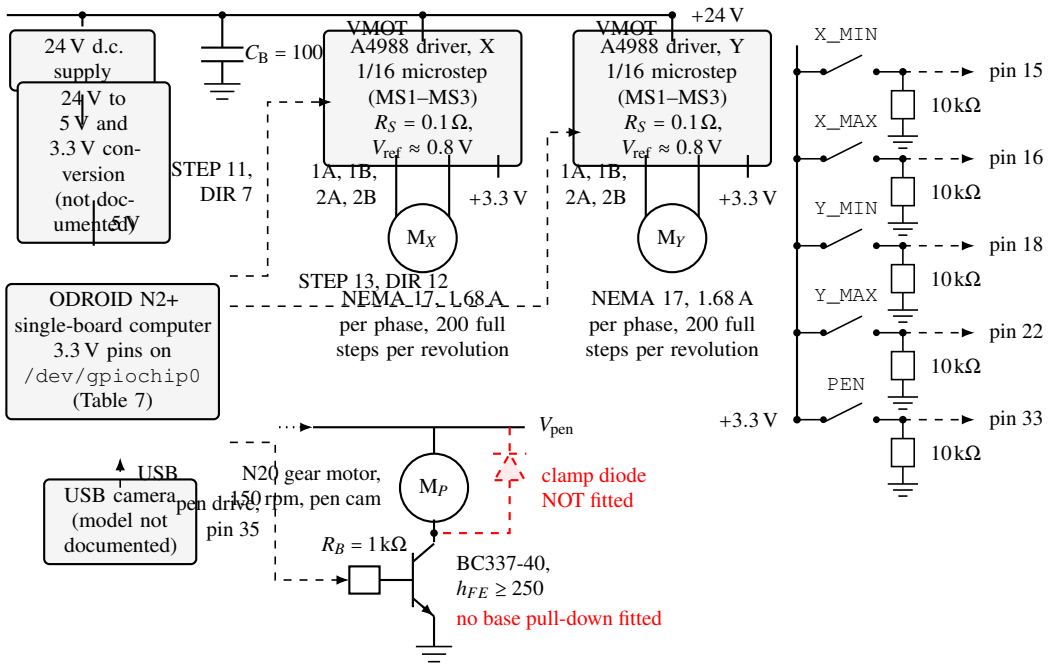
the upper axis riding on the lower one: the two axes do not have the same load, so a torque margin established on the upper axis does not transfer to the lower one. Because the coil current is deliberately derated to about 1 A per phase for thermal reasons (Section 3.3.3), the torque available is itself below the datasheet figure in roughly the same proportion, and that derating must be applied before the margin is taken.

Given the factor of 680 in Equation 6, the acceleration and inertia terms of Equation 7 would have to raise the demand by nearly three orders of magnitude before the drive became torque-limited, which the masses involved cannot do; the constant-velocity check is therefore taken as sufficient for the feeds this machine uses. It is for the same reason that the design carries no acceleration ramp. **[MUST STILL BE COMPLETED BY ME: weigh the carriages and measure the**

friction force directly, then evaluate Equation 7 at the 400 mm/s^2 of Equation 23 using the effective mass of Equation 8, to replace the assumed μ and m of Equation 5 with measured ones.]

3.3.3 Interface circuits, supply and pin assignment

The pen-lift mechanism is a different problem from the two axes: a gear motor cannot be switched directly from a logic pin, because the pin cannot supply the motor's current. A bipolar transistor is used as a switch, with the pin driving its base through a current-limiting resistor and the motor's current flowing through the collector–emitter path. Figure 9 is the complete circuit, with every component value on it and an undocumented stage making the 5 V and 3.3 V the board and pen motor need. Three of those values follow from calculation rather than from a datasheet recommendation, and each is derived below.



Solid: power. Dashed: header signals. Dotted: undocumented. One shared ground.

Figure 9. Complete circuit with all component values. The clamp diode is dashed because it is *not* fitted, and no base pull-down is fitted either; the conversion stage, the pen rail and the supply's current rating are not documented.

Bulk capacitor. The chopper draws current in bursts that the supply leads cannot follow, so a capacitor sits at the drivers; the rail sag over one 1.68 A burst of 10 µs is

$$\Delta V = \frac{I\,\Delta t}{C} = \frac{1.68 \cdot 10 \times 10^{-6}}{100 \times 10^{-6}} = 0.17\ \text{V}, \tag{9}$$

which is small against 24 V and clear of the driver's undervoltage threshold [**?**]. Three gaps remain in this part of the design: one capacitor is shared where one belongs at each driver, no ceramic logic-supply capacitor is documented, and the supply current was never budgeted. **[MUST STILL BE COMPLETED BY ME: budget four phases at the derated coil current plus pen motor, board and camera against the supply rating, and confirm whether each**

driver has its own bulk and logic capacitor.]

Coil current and its derating. The datasheet relation [**?**, p. 9] between the trimmer reference, the carrier's two sense resistors and the regulated current gives

$$I_{\text{coil}} = \frac{V_{\text{ref}}}{8\,R_S} \quad\Longrightarrow\quad V_{\text{ref}} = 8\,R_S\,I_{\text{coil}} = 8\cdot 0.1\cdot 1.68 = 1.34\ \text{V} \tag{10}$$

for the 1.68 A rating [**?**] and the 0.1 Ω resistors usually fitted. That rating is deliberately not used. Each driver dissipates about $2I_{\text{coil}}^2 R_{DS(\text{on})}$, which has only the small carrier board to leave through, so an unheatsinked carrier is held near 1 A per phase and V_{ref} of Equation 10 is derated from 1.34 V to about 0.8 V, trading holding torque for a driver that will not shut down thermally in the middle of a page. The margin of Equation 6 is large enough to absorb that derating several hundred times over. **[MUST STILL BE COMPLETED BY ME: confirm the sense resistor fitted (0.05, 0.068 and 0.1 Ω are all sold) and the measured reference voltage, give the coil current Equation 10 then returns as a fraction of 1.68 A, and say whether the drivers have heatsinks.]**

Switch pull-downs. With one switch closed, and with all four end-stops closed, the pull-down current is

$$I_{\text{pull}} = \frac{V_{DD}}{R} = \frac{3.3}{10000} = 0.33\ \text{mA}, \qquad 4I_{\text{pull}} = 1.3\ \text{mA}, \tag{11}$$

negligible on this board. The 10 kΩ of Equation 11 is bounded in both directions: a smaller value wastes current, and a larger one lets pin leakage lift the open-switch voltage, so $R \le V_{IL}/I_{\text{leak}}$ [**?**]. Its RC time constant is microseconds against the 6 ms re-read of Section 3.3.5, so the debounce belongs in the program and not in the network. **[MUST STILL BE COMPLETED BY ME: take V_{IL} and the input leakage from the ODROID N2+ documentation [?] and state the upper bound they put on the pull-down; give the board revision and date of that document, as the bibliography entry has neither; and confirm each switch is wired to 3.3 V, not 5 V, giving the header pin that rail comes from, since Figure 9 infers it from program comments.]**

Pen-motor switching stage. With the pin at 3.3 V and $V_{BE(\text{sat})}$ taken as 0.7 V [**?**], the base current through the 1 kΩ resistor is

$$I_B = \frac{V_{\text{pin}} - V_{BE(\text{sat})}}{R_B} = \frac{3.3 - 0.7}{1000} = 2.6\ \text{mA}, \tag{12}$$

and with $h_{FE(\text{min})} = 250$ for the −40 gain group [**?**] and an overdrive factor k_{od} of three to five for hard saturation, the switchable collector current is

$$I_{C(\text{max})} = \frac{h_{FE(\text{min})}\,I_B}{k_{od}} = \frac{250\cdot 2.6\ \text{mA}}{k_{od}} = \frac{650\ \text{mA}}{k_{od}}, \tag{13}$$

which is 217 mA at $k_{od} = 3$ and 130 mA at $k_{od} = 5$. Equation 13 is comfortable for a motor drawing up to about 200 mA, and the saturated dissipation $P = V_{CE(\text{sat})}I_C \approx 0.7\cdot 0.2 = 0.14$ W is well inside the unheatsinked TO-92 package [**?**]. The overdrive factor is the design decision here: dividing by k_{od} rather than using the full $h_{FE}I_B$ keeps the transistor in hard saturation at the minimum gain of the group, so the collector–emitter drop stays at $V_{CE(\text{sat})}$ and the dissipation stays at the figure above even for the worst-case device. **[MUST STILL BE COMPLETED BY ME: give the manufacturer (onsemi and Nexperia both supply this part), part number and datasheet revision of the transistor fitted, and check the gain group, $V_{BE(\text{sat})}$, $V_{CE(\text{sat})}$ and TO-92 dissipation against it; then measure the pen motor's free-running and stall current against Equation 13, since above about 200 mA the stage is not hard saturated and R_B must be reduced.]**

Two faults in this stage, and the pin levels. Two items in the pen switching stage are design *faults*, not untested ones. *No clamp diode is fitted*: at turn-off the energy $\frac{1}{2}LI^2$ stored in the motor's inductance appears as a reverse voltage that stresses the transistor and injects noise into the shared ground, a diode across the motor, cathode to supply, having simply been omitted. *No base pull-down is fitted* either, so the base floats in whatever input state the pin holds at power-up. The missing diode is also the mechanism behind the end-stop behaviour of Section 3.3.5: with no clamp and a shared ground, a coil current I lifts the local ground by IR_{gnd}, and the switch inputs are referred to that ground. The fix is a clamp diode across each motor and a switch return to a single ground point, not a longer debounce.

STEP, DIR and enable are driven from 3.3 V pins with no level shifting, sound only if $3.3\,\mathrm{V} > V_{IH}$ with margin, the datasheet putting the logic high threshold at $0.7V_{DD}$ [**?**, p. 4]. During commissioning the X axis drove to its first end-stop but the direction pin was not observed to change level on reversal, and the fault was not resolved; a level incompatibility, a wiring or configuration fault, a damaged pin and a program fault are equally consistent with it, so no cause is claimed.

MUST STILL BE COMPLETED BY ME

Resolve the X-axis direction fault, which blocks R5 entirely. Compare V_{IH} with 3.3 V, then meter DIR at the pin and at the driver while commanding a reversal.

Pin assignment. Twelve lines on one chip carry the whole interface, and all twelve are requested in a single call so that a second instance of the program cannot claim them during a job.

Function	Offset / header pin	Direction, initial level
X motor DIR, STEP	64/7, 68/11	output, high / low
Y motor DIR, STEP	81/12, 69/13	output, high / low
X_MIN, X_MAX, Y_MIN, Y_MAX	72/15, 65/16, 66/18, 67/22	input
Pen switch, pen motor	70/33, 71/35	input; output, low

Table 7. Pin assignment on `/dev/gpiochip0`; all twelve are requested in one call, so a second instance cannot claim them during a job. Header numbers are unchecked against the board pinout.

3.3.4 Step resolution, microstepping and calibration

Three properties of a stepper drive fix the resolution of this machine. The motors used here take 200 *full steps* per revolution, so one full step is 1.8°; because position is simply the number of steps commanded, no encoder reports where the carriage actually is, which is what *open-loop* control means, and the design simply trusts the step count. That trust is the assumption on which requirement R5 rests, and it fails silently if a step is ever missed. The driver subdivides each full step by *microstepping*: rather than switching the two windings fully on or off, it holds them at intermediate current ratios, placing the rotor at intermediate angles. At the 1/16 setting used here, one full step becomes 16 microsteps, so a revolution is 3200 microsteps; microstepping buys resolution and smoother motion but not proportionally more accuracy, because the intermediate positions are held by the ratio of two winding currents and are therefore more sensitive to load torque than a full step is.

Equation 14 gives the step resolution from 200 full steps [**?**], 16 microsteps [**?**, p. 6] and $\ell_{\text{rev}} = 40\,\text{mm}$,

$$\kappa = \frac{200 \cdot 16}{\ell_{\text{rev}}} = \frac{3200}{40} = 80\ \text{steps/mm}, \qquad \Delta = \frac{1}{\kappa} = 12.5\ \mu\text{m}, \tag{14}$$

which calibration replaces by each axis's measured value. Targets are *absolute* step counts, $\text{round}\big((x+5)\kappa\big)$ with a 5 mm offset, differenced against the tracked position, so rounding cannot accumulate. Quantisation error is at most $\Delta/2 = 6.25\ \mu$m per axis, 8.8 μm combined, 57 times finer than the 0.5 mm of R5 — but that is the *digital* resolution only, and belt stretch, backlash, missed steps, switch repeatability and pen compliance were not measured. Equation 15 gives the delay of every pulse for a move of length d at feed v_f over $N_{\text{maj}} = \max(|n_x|, |n_y|, 1)$ major-axis steps,

$$\tau = \max\Big(\frac{d/v_f}{N_{\text{maj}}},\ 2\,\mu\text{s}\Big), \qquad v_f = \max(F/60,\ 0.1), \tag{15}$$

with F in mm/min: 900 gives $\tau = 833\ \mu$s (1200 steps/s), 2400 gives 312.5 µs (3200 steps/s, 60 rpm), homing 625 steps/s or 7.8 mm/s. Because *every* pulse of both axes sleeps for τ, a segment takes $(n_{\text{maj}} + n_{\text{min}})\tau$ and Equation 16 gives the effective feed,

$$v_{\text{eff}} = v_f \frac{n_{\text{maj}}}{n_{\text{maj}} + n_{\text{min}}} \in \Big[\frac{v_f}{2}, v_f\Big], \tag{16}$$

unity axis-parallel, $\frac{1}{2}$ at 45°. There are *no acceleration ramps*, which the margin of Equation 6 justifies but does not prove safe at every load [**?**].

Interpolating a straight line. Stepping the two axes independently would draw an L-shaped path, so the program interpolates with Bresenham's integer line algorithm [**?**, p. 28]: an error term e is carried along the major axis of $a_x = |\Delta x|$ and $a_y = |\Delta y|$ steps, the major axis is pulsed every iteration and the minor axis whenever e would go negative, and the end-stops are read before every major step. Keeping e in $[0, a_x)$ gives the minor-axis count after k major steps as

$$m_k = \Big\lceil \frac{k\,a_y - \lfloor a_x/2 \rfloor}{a_x} \Big\rceil, \tag{17}$$

which is within one step of the ideal $k\,a_y/a_x$, about 6 µm, using integer arithmetic only. Equation 17 is the reason no floating-point division appears in the inner loop: the deviation from the true line is bounded by construction rather than by rounding behaviour. Algorithm 1 states the loop.

Algorithm 1 Interpolated segment with in-loop end-stop check

Require: dx, dy in steps (x taken as the major axis, $a_x = |dx| \geq |dy| = a_y$), feed v_f

```
delay ← max((distance/v_f)/a_x, 2 µs) {τ of Equation 15}
e ← ⌊a_x/2⌋ {Bresenham error term of Equation 17}
for k = 1 to a_x do
    if Algorithm 3 returns a triggered end-stop then
        stop the segment at once {latches as in Section 3.3.5}
    end if
    pulse the major-axis STEP pin; e ← e − a_y
    if e < 0 then
        pulse the minor-axis STEP pin; e ← e + a_x
    end if
    sleep for delay
end for
```

Direction-agnostic calibration. Equation 14 rests on assumed transmission values and the position after power-up is unknown, so each session begins with a direction-agnostic calibration, written so that it never needs to know which direction leads to which switch. It drives one way at 1.6 ms per step until *either* switch of that axis triggers, reverses, and counts the steps N to the *other*. With the hand-measured travel L, the resolution, the initial step counter and the position in millimetres are

$$\kappa_{\text{axis}} = \frac{N}{L}, \qquad n_0 = N \text{ if the second switch was MAX else } 0, \qquad x_{\text{mm}} = \frac{n}{\kappa} - 5, \tag{18}$$

so the zero of Equation 18 lies 5 mm inside the minimum switch, the usable area is 194×252 mm, and each axis retracts 5 mm off its last switch. The two traverses need $(204 + 262)/7.8 \approx 60$ s, derived and not measured. The result is deliberately not stored between sessions, since a bump between sessions would silently invalidate it, and an abort refuses all jobs. Algorithm 2 states the procedure.

Algorithm 2 Direction-agnostic calibration of one axis

drive one way at 1.6 ms per step until *either* switch of this axis triggers
reverse; count steps N until the *other* switch triggers
$\kappa_{\text{axis}} \leftarrow N/L$ {L the hand-measured travel, Equation 18}
if the second switch to trigger was the maximum-side one **then**
    $n_0 \leftarrow N$
else
    $n_0 \leftarrow 0$
end if
retract 5 mm off the switch that is still closed
if an already-triggered switch, an unexpected switch on the other axis, or 200 000 steps pass with no trigger **then**
    abort calibration; refuse all jobs until it is re-run
end if

The tightest tolerance in the whole hardware design falls on that hand measurement of travel. An error δL scales every commanded position by $x\,\delta L/L$, so to keep that scale error below half of the 0.5 mm allowed by R5 over the 194 mm of usable X travel,

$$\delta L \le \frac{0.25 \text{ mm}}{194 \text{ mm}} L = \frac{0.25}{194} \cdot 204 \text{ mm} = 0.26 \text{ mm} \quad (0.13\ \%), \tag{19}$$

and switch-trigger repeatability adds directly to the zero error on top of that. Equation 19 is the tightest tolerance anywhere in the hardware design, and it falls on a measurement taken with a hand-held rule.

Measured dimensional accuracy. Equation 14 and the calibration of Equation 18 are claims about the machine, and both were checked by drawing a 100×100 mm square of literal absolute coordinates (`results/gantry_square/`) and measuring the ink. Nothing in that file passes through the trajectory placement of Section 3.3.6, so a discrepancy is the machine's and not the software's. Table 8 gives both passes.

Side lengths alone cannot distinguish a square from a parallelogram, so they do not test the recorded 1° axis skew (`SQUARENESS_ERROR_DEG`) at all: a skewed drive leaves all four sides equal and moves only the diagonals. For a side s and an included angle $90° - \phi$,

$$d_{\mp} = s\sqrt{2\big(1 \mp \cos(90° - \phi)\big)}, \qquad d_{+} - d_{-}\Big|_{s=100,\ \phi=1°} = 142.65 - 140.18 = \mathbf{2.47} \text{ mm}, \tag{20}$$

Pass	Travel constants used	Square measured	Scale error	Travel implied, $L\times$drawn$/100$
1	204.0×262.0 mm	97.6×99.4 mm	$-2.4\,\%$, $-0.6\,\%$	199.2×260.3 mm
2	199.2×260.3 mm	100.69×100.37 mm	$+0.69\,\%$, $+0.37\,\%$	$\mathbf{200.6} \times \mathbf{261.3}$ mm
3	200.6×261.3 mm	**[MUST STILL BE COMPLETED BY ME: redraw and measure]**	target $< \pm 0.1\,\%$	—

Table 8. Dimensional calibration of the gantry, measured on a drawn 100×100 mm square. Pass 1 corrected the whole of a 2.4 % error from a single measurement and overshot; pass 2 is a 0.69 % correction off a much smaller error and should therefore be the last. Within ± 0.1 mm on a 100 mm side is as well as an ink line can be read with callipers, and is the stopping condition. The two sides of each pair agreed to 0.06 mm in X and 0.10 mm in Y, so the 0.32 mm difference *between* the axes is real and the two are corrected apart.

so by Equation 20 the skew is unmissable on the diagonals and invisible on the sides. A skew is not a scale and no constant can correct it; it is a rail-alignment fault. **[MUST STILL BE COMPLETED BY ME: measure both diagonals of the square already drawn — Equation 20 predicts 141.42 mm each if the axes are square and a 2.47 mm split if the recorded 1° skew is still present — and enter them in Table 8.]**

FIGURE MUST STILL BE INSERTED BY ME

The drawn 100 mm square, photographed or scanned flat with a rule in frame, annotated with the four measured side lengths and the two measured diagonals against the commanded 100.00 mm and 141.42 mm.

Figure 10. The drawn 100×100 mm calibration square with the measured sides and diagonals annotated against their commanded values, which is the evidence for Table 8 and Equation 20.

A model of the interpreter for the test word of Section 3.3.6 gives 6.8 s drawing, 10.8 s travel and 9.1 s dwell, 26.7 s in all, 2.4 s per character, being a model and not a measurement. **[MUST STILL BE COMPLETED BY ME: state the method and uncertainty of the L_X and L_Y measurement, which Equation 19 requires better than 0.26 mm; measure the positional error for R5 (the same word drawn twice and overlaid) and the return-to-zero repeatability; measure the highest feed without lost steps.]**

3.3.5 End-stops, debounce and the pen-lift mechanism

A mechanical switch does not change state cleanly: its contacts strike, separate and strike again over a few milliseconds, so a single read taken during that interval may return either level. Treating one high read as a trip would stop the machine at random, and treating the first read

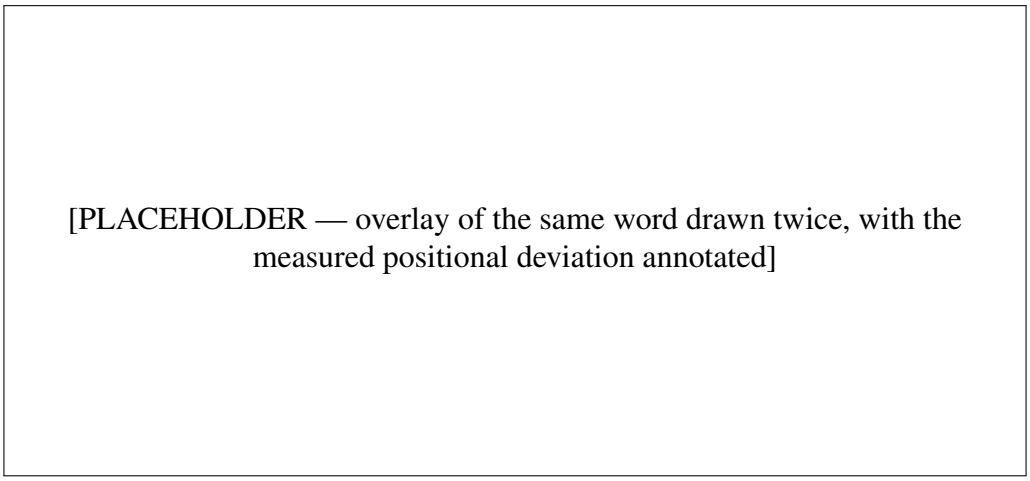


Figure 11. Placeholder: the repeatability test of requirement R5 — the same word written twice by the gantry, scanned and overlaid, with the maximum and mean deviation between the two traces marked, to show whether the digital resolution of Equation 14 is actually realised by the physical machine.

as final would miss a genuine trip. The standard remedy is to *debounce* the input, requiring the same level on several reads spaced far enough apart to outlast the bouncing.

The pen-lift mechanism itself is a design choice justified against the alternative of a third stepper or a servo motor. A servo motor expects a continuous pulse-width-modulated position command and typically needs its own timer peripheral or a precisely timed software loop running concurrently with the gantry's own step timing, competing for the same processor cycles that generate STEP pulses. An N20 geared d.c. motor run in one direction only, with a cam that toggles the pen up or down every 90° of rotation, needs only an on/off drive pulse of roughly known duration and a microswitch to confirm the result; it is cheap, widely available, has high torque at low speed through its internal gearbox for its size, and has a smaller profile than a comparable servo, at the cost of giving no absolute position, only a toggle.

All four end-stop switches and the pen switch are read before every major-axis step and every G-code line, and count as triggered only on three consecutive high reads 6 ms apart (Algorithm 3). Three reads rather than two were needed because, with no flyback diode on a shared ground (Section 3.3.3), switching noise held an input high for longer than a single re-check, so the debounce is compensating for a hardware omission as well as for contact bounce. Equation 21 gives the resulting trip-to-stop latency,

$$t_{\text{stop}} \approx \tau + (3-1)\cdot 6\ \text{ms} \approx 13\ \text{ms}, \tag{21}$$

during which the carriage moves 0.2 mm at 15 mm/s and 0.5 mm at 40 mm/s, inside the 5 mm edge tolerance.

On a trip the program latches a stop, sets both STEP pins low, retracts off every triggered switch and aborts the job; targets are clamped to the calibrated area and the latch clears only by explicit command, refused while a switch is still triggered. Protection is *software only*: no hardware stop that cuts motor power is documented, the switches are polled only during a job, no endpoint aborts a running job, and no lock prevents two simultaneous jobs.

The implemented lift, planned in the proposal as a stepper, gives only pulses, so the pen state must be tracked in software, and the pen switch is feedback only (high = up). The pen-drive

Algorithm 3 Debounced end-stop read, called before every major-axis pulse

Require: set of end-stops to ignore (already triggered on this axis)
for each end-stop not in the ignore set **do**
  if it does not read closed **then**
    continue to the next end-stop
  end if
  re-read it twice more, 6 ms apart {three reads in total, debounce of Equation 21}
  if it read closed on all three reads **then**
    return that end-stop as triggered
  end if
end for
return none triggered

routine of Algorithm 4 must therefore test the switch *before* energising the motor, since the cam turns one way only and an unnecessary pulse would toggle the pen to the wrong state rather than leave it alone. The pen-lift motor's drive pulse width was found by direct testing to be 260 ms of rotation plus 90 ms of settling, 350 ms in total, rather than by calculation, since the cam geometry and the gearbox reduction were not modelled.

Algorithm 4 Pen toggle

Require: desired state wantUp
if any end-stop is currently triggered **then**
  refuse the request
else if the pen switch already reads wantUp **then**
  return {the motor must *not* run; the cam turns one way only}
else
  energise the pen motor
  while before the deadline and the pen switch does not yet read wantUp **do**
    wait
  end while
  if the switch now reads up **then**
    wait 40 ms more to seat the cam
  end if
  de-energise the pen motor
end if

During tuning the clicker sometimes over-rotated and the pen did not push down fully before the switch read as triggered, requiring the pulse width to be tuned empirically to the 350 ms figure above; differing pen pressure on a book or an exam pad, rather than a flat board, also introduced inconsistent ink, so a rigid clipboard with a single page clamped flat was adopted. One pen toggle was never timed with an independent stopwatch, only inferred from the tuning above, and the generator's 0.35 s assumption is the largest single contributor to the predicted writing time of Section 3.3.4. Neither the DIR-to-STEP timing nor the driver's minimum STEP width is modelled, and missed steps cannot be detected without feedback, so the true positional accuracy must come from the repeatability test of Figure 11, not from Equation 14 alone.

Simple-shape tests (repeating a line over itself, drawing a rectangle of known dimensions, and sending G-code produced by third-party software) were used during bring-up to verify G-code

following, calibration and end-stop detection, and surfaced three problems resolved before the circuit above was finalised: end-stop priority clashing between calibration and run-time checks, generated text initially drawing backwards, and flex in the pen-lift mechanism skewing writing slightly along one axis.

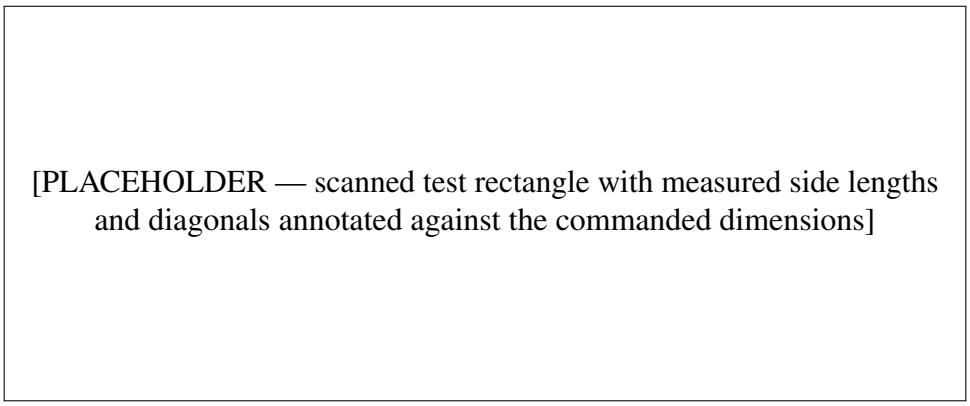


Figure 12. Placeholder: a simple-shape test — a rectangle of known commanded dimensions, drawn and scanned, with the measured sides and diagonals marked beside the intended values, to show the gantry's dimensional accuracy independent of handwriting.

3.3.6 G-code generation and the motion mapping

A pen trajectory in millimetres (Section 3.6) has to become a machine program before the interpreter of the sections above can execute it. The format used is G-code, the command language of plotters and numerically controlled machines standardised as ISO 6983-1 [**?**]: a list of text lines, each naming a command and its arguments. Only five commands are used: `G21` and `G90` set the units to millimetres and the coordinates to absolute, `G0` is a move with the pen up and `G1` a move with the pen down, `M3` and `M5` lower and raise the pen, `G4` pauses, and `M2` ends the program. Coordinates are *absolute*, so a rounding error in one move cannot accumulate into the next, and settings such as the feed rate are *modal*, staying in force until another line changes them. The format splits the motion chain in two: the generator needs to know nothing about the gantry electronics, and the interpreter needs to know nothing about handwriting, with a plain-text file between them that can be inspected and replayed without the machine present.

The trajectory and the machine frame both have X right and Y up. For a trajectory of bounding box (x_0, y_0, x_1, y_1), width w and height h, the block is shrunk by Equation 22, never enlarged, if it would leave the 200×200 mm work area:

$$s = \min\left(1,\ \frac{X_{\max} - X_{\mathrm{o}}}{w},\ \frac{Y_{\mathrm{o}} - Y_{\min}}{h}\right), \qquad X = X_{\mathrm{o}} + (x - x_0)\,s, \quad Y = Y_{\mathrm{o}} - (y_1 - y)\,s, \tag{22}$$

with the origin $(X_{\mathrm{o}}, Y_{\mathrm{o}}) = (10, 180)$ mm. Each pen lift is followed by a *dwell* of 350 ms (Section 3.3.5), so a stroke costs 0.7 s of dwell, which is why dwell dominates the predicted job time. The generator can also write a step schedule with a time estimate, assuming each move starts and ends at rest under the standard trapezoidal velocity profile used for point-to-point moves on automatic machines [**?**], with acceleration $a = 400$ mm/s^2, 15 mm/s drawing and 40 mm/s

travelling (Equation 23):

$$T(d) = \begin{cases} \dfrac{d}{v} + \dfrac{v}{a}, & d \geq 2d_a, \\ 2\sqrt{\dfrac{d}{a}}, & d < 2d_a, \end{cases} \qquad d_a = \frac{v^2}{2a}. \tag{23}$$

For drawing $d_a = 0.28$ mm, so a typical 0.35 mm segment is triangular and never reaches 15 mm/s. For the test word of Figure 13, a draw path of 83.2 mm in 239 moves and a pen-up path of 250.3 mm in 14 moves with 26 pen pulses, the model gives 14.1 s drawing, 7.5 s travel and 9.1 s dwell, 30.7 s in all, or 2.8 s per character for an assumed 11 characters.

Every constant of the generator is listed in Table 9 with its role. The motor, driver and transmission values are assumptions recorded in the program comments, not measurements; the calibrated steps per millimetre of Section 3.3.4 replace the derived ones in the interpreter, so a scale error in the generator's assumption does not reach the machine.

Field	Value	Meaning
Steps, microstepping, travel per revolution; steps per millimetre	200, 16, 40.0 mm; 80.000	Assumed 1.8° stepper, driver setting and 2 mm-pitch belt on a 20-tooth pulley, giving $200 \cdot 16/40$.
Work area; origin and park	0 to 200 mm in X and Y; (10, 180) mm	Limits of the clamp; top-left corner of the text block.
Draw and travel feed	900, 2400 mm/min	Pen down (15 mm/s), pen up (40 mm/s, emitted once).
Acceleration; minimum segment	400 mm/s^2; 0.12 mm	Used only by the time model; shorter moves are collapsed.
Pen pulse and settling; pen codes and power-on state	260 and 90 ms; `M5`, `M3`, up	Assumed time for a 90° rotation and the dwell after a toggle; pen up and pen down, the up state at power-on being assumed.

Table 9. Configuration of the G-code generator. Only the work area and the two feeds are design choices of this project; the rest are assumptions about parts, which is why the interpreter re-measures the resolution rather than trusting the figure in the file header.

The strokes are ordered by a greedy nearest-endpoint rule, grouped into text rows, so only order and direction change and the ink is identical; this is the pen-movement optimisation of R3. A program is a comment header, `G21`/`G90`, then one block per stroke at the 900 mm/min draw feed, and finally pen up, parking and `M2`; Figure 13 is the start of a generated program.

```
; start of a generated program -- author yeukita
; steps/mm  X=80.000  Y=80.000 (3200 microsteps/rev)
G21   ; millimetres
G90   ; absolute
G0 F2400                    ; travel feed, emitted once
G0 X10.059 Y179.271         ; travel to the first stroke
M3   ; pen DOWN (one 90deg pulse of the one-way gear motor)
G4 P0.350   ; wait for the 90deg rotation to finish
G1 F900                     ; draw feed
G1 X10.084 Y178.920         ; one line per vertex (0.35 mm apart)
```

Figure 13. Start of a generated program for the test word of writer yeukita (14 strokes, 239 draw moves, vertices 0.348 mm apart on average). The trailing comments on the motion lines are annotations added here.

The generator is the one stage whose output can be checked exactly, because both its input and its correct output are geometry. An independent second parser reads the finished G-code file back as though it were the machine, following X and Y, starting a polyline at `M3` and closing it at `M5`.

The trajectory that went in and the polylines that come back out are compared by Equation 24:

$$\mathrm{IoU} = \frac{|M_\mathrm{t} \cap M_\mathrm{g}|}{|M_\mathrm{t} \cup M_\mathrm{g}|}, \qquad e_\mathrm{mean} = \frac{1}{J}\sum_j \min_i \|g_j - s_i\|, \qquad e_\mathrm{max} = \max_j \min_i \|g_j - s_i\|, \tag{24}$$

drawn onto a common grid of 8 pixels per millimetre for the area ratio IoU, and sub-sampled vertex-to-vertex distances for the mean and worst case. This measures only the loss between the trajectory and the file — rounding to three decimals, stroke reordering and work-area clamping — and says nothing about the mechanical error of Section 3.3.4, which is a separate matter. The project notes quote IoU ≈ 0.99 and a text-accuracy change of at most 0.3 percentage points between direct and read-back rendering, for an earlier system state (Table 26 shows later pairs); e_mean and e_max were never logged. **[MUST STILL BE COMPLETED BY ME: run the comparison for a set of generated programs and report** IoU, e_mean **and** e_max**; the saving produced by the stroke-ordering optimisation of R3 was not measured.]**

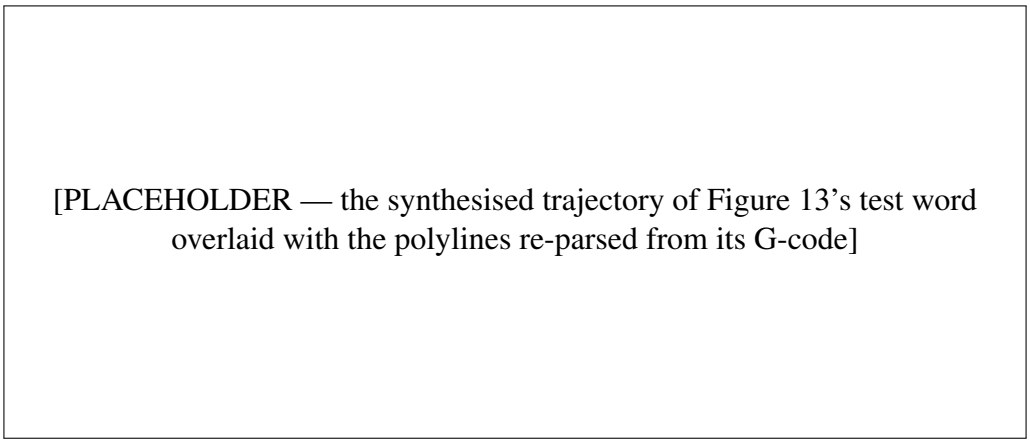


Figure 14. Placeholder: the G-code verification of Equation 24 — the original trajectory and the re-parsed polylines overlaid, with the measured IoU, e_mean **and** e_max **printed beside the figure, showing the generator introduces negligible geometric loss.**

Three faults in the generator are worth stating plainly rather than left for a reader to find. The stroke rows are sorted by ascending Y, so a multi-line job starts with the *bottom* line, not in reading order. The interpreter has one *modal* feed register and no rapid mode, but the generator emits the rapid feed once at the start and the draw feed before every stroke, so pen-up travel after the first stroke runs slower than the time model assumes. Lines wrap at a width that reaches the driver's clamp boundary, which would distort the last millimetre of a full-width line.

[MUST STILL BE COMPLETED BY ME: page locators outstanding in this subsection. The sources `acarnley2002`, `bc337ds`, `biagiotti2008`, `iqs2024`, `iso6983`, `odroidn2ds`, `stepperds`, `teikari2016` are each cited here without a page or numbered element. Open each one, find the claim it is cited for, and add the locator in the form `\cite[p. N]{key}`, following the rule of `REFERENCE_AUDIT.md`: a locator is written only after the source has been read and the claim found, never inferred from a page range or a search snippet.]

3.4 Page conditioning and line segmentation

This subsection turns a photograph of a handwritten page into clean, deskewed, white-background line images for the text recogniser of Section 3.5 and the writer classifier of Section 3.5 (functional units FU 2.1 and FU 2.2; requirement R2). It is ordered as the data flows: the array primitives the rest of the block is built from, lighting and binarisation, deskew, the measurement of the scale unit, removal of what is not handwriting, the line centres, and finally the seam cut and crop.

The deployed implementation is `SegmentLean.py`, eleven functions in about 570 lines behind a single entry point, `SegmentLines(path)`. The web server, the style-profile builder and the writer-classifier trainer all call that same entry point, so training pages are cut by exactly the code that cuts a page during normal use. Published methods are cited where they are used, and the rules around them are plain array arithmetic written for this project. No part of this block resamples through an imaging library: every operation that changes a pixel value is performed by the project's own module, implementing Equation 25 directly; the library is used only to read the file from disk and to write the optional debug overlay.

3.4.1 Design goals and the vocabulary of a text line

A flat-bed scan can be conditioned by grey conversion, binarisation and bounding-box line separation, which is what published pipelines do [**?**, **?**]. A photograph is harder: it adds uneven illumination, the surroundings of the sheet, skew, printed rules, edge slivers, creases, and lines that touch each other. The block therefore has three design goals:

1. It must work at any camera resolution and any handwriting size **without manual tuning**.
2. It must **never delete handwriting** in order to remove a nuisance.
3. It must keep **as few fixed constants as possible**.

All three are met by one decision: letting the page set its own scale. The photograph is reduced 2:1 per axis by box averaging, and the *line pitch P* is measured on that working image (Section 3.4.5), after which almost every threshold in the block is a multiple of P rather than an absolute pixel count. Goal 3 is therefore testable rather than aspirational, and Section 3.4.9 discharges it: of the thirty-odd constants in the block, only four are absolute — the blur radius, the floor of 60 ink pixels per emitted line, the 64×640 canvas size and the dynamic-programme weights.

Before any threshold can be fixed, the line itself needs a shared vocabulary. Every subsystem from here onwards measures handwriting against five quantities shown in Figure 15: the *baseline*, the *x-height* x_{h} (the height of a lower-case letter with no tall part and no tail), an *ascender* and *descender* reaching above and below that, and the *line pitch P*, the baseline-to-baseline distance. The pitch P is the unit of this block, because segmentation separates one line from the next; the x-height x_{h} is the unit of everything downstream (Sections 3.5.3, 3.6, 3.6), because measuring in x-heights removes both the camera's distance and the writer's absolute size as variables, so one stored style profile can be drawn at any pen size.

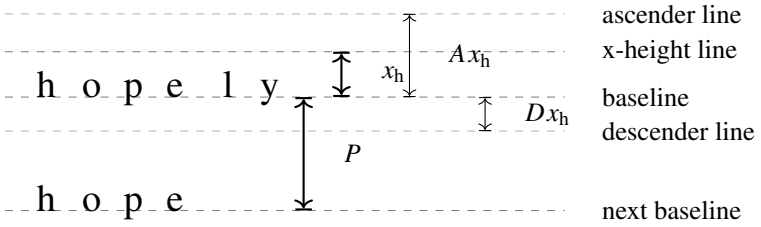


Figure 15. The five quantities every subsystem measures handwriting against. $A > 1$ **because an ascender reaches above the x-height line, and** $D < 0$ **because a descender falls below the baseline.** P **is the unit used by this block;** x_h **is the unit used by every block after it.**

3.4.2 Array primitives: summed-area tables and run tests

Almost every operation in this block is a sum, a maximum, or a test of whether anything is set, taken over a rectangular window. Computed directly, a window of radius r costs $O(r)$ per pixel per axis, too slow here because the illumination blur below uses a radius of up to 40 pixels. The module therefore builds a summed-area table [**?**], also called an integral image, in one $O(HW)$ pass of two cumulative sums, after which the sum over a window of *any* size costs four look-ups instead of $(2r+1)^2$ additions. Three stages of this block resample an image, and each does so by bilinear interpolation at the source position given by Equation 25, the one exception being an ownership map resampled by nearest neighbour, because its values are integer line indices that interpolation would invent new values for:

$$y_s = \frac{(i+0.5)\,h}{h'} - 0.5, \quad x_s = \frac{(j+0.5)\,w}{w'} - 0.5, \quad I'(i,j) = \sum_{a,b\in\{0,1\}} \omega_{ab}\, I(\lfloor y_s \rfloor + a, \lfloor x_s \rfloor + b). \tag{25}$$

The table itself is the running two-dimensional sum of the image,

$$S(y,x) = \sum_{i<y} \sum_{j<x} I(i,j), \tag{26}$$

where S is the summed-area table and I the image being summed. Equation 26 is built once per image. Any window sum is then read from four of its entries,

$$\mathrm{BoxSum} = S(y_2,x_2) - S(y_1,x_2) - S(y_2,x_1) + S(y_1,x_1), \tag{27}$$

where $y_2 = y_1 + 2r + 1$ and $x_2 = x_1 + 2r + 1$ are the far corners of a window of radius r, the array having been padded by r with edge replication beforehand. The cost of Equation 27 does not depend on r, which is what makes the radius-40 illumination blur of Section 3.4.3 affordable.

The same cumulative trick gives the two binary operations the block needs. A dilation along one axis is

$$\mathrm{AnyWithinRadius}(m,r)[i] = \big[\, c[i+2r+1] - c[i] > 0 \,\big] = \Big[\bigvee_{k=-r}^{r} m[i+k] \Big], \tag{28}$$

where m is the binary mask, c its cumulative sum and r the radius. Equation 28 is a dilation because the bracket is true exactly when at least one pixel within r is set. A square dilation separates into two applications of it, one per axis, so it costs the same at any radius. An erosion is the complemented dilation of the complement, and a closing is a dilation followed by an erosion,

so no other structuring element has to be implemented. The second operation marks long runs,

$$\mathrm{LongRun}(m,n)[j] = \big[\,\mathrm{ch}[h_j+1]-\mathrm{ch}[l_j]>0\,\big], \qquad l_j=\max(0,j-n+1), \quad h_j=\min(j,L-n), \tag{29}$$

where n is the minimum run length, ch the cumulative count of positions at which a run of n consecutive set pixels begins, L the length of the axis, and l_j, h_j the first and last such start positions that could cover index j. Equation 29 is true at every pixel lying on a run of at least n set pixels, and is used by Section 3.4.6 to find printed rules. Only one true Gaussian filter is used anywhere in the block, a 25-tap kernel smoothing the row profile of Section 3.4.5 with $\sigma = 4$ rows, about a tenth of a typical pitch.

Algorithm 5 states all three primitives. None contains a loop over pixels: each is two cumulative sums and a slice difference, which is why a window of any size costs the same.

Algorithm 5 The three array primitives, each independent of window size

BoxMean(image, radius r) {mean over a $(2r+1)\times(2r+1)$ window}
  pad the image by r, replicating the edge
  integral ← cumulative sum down, then cumulative sum across {Equation 26}
  return (four corner look-ups into integral) $/\,(2r+1)^2$ {Equation 27}

AnyWithinRadius(mask, radius r, axis) {one-axis dilation, Equation 28}
  c ← cumulative sum of the mask along the axis
  return $c[i+2r+1]-c[i]>0$ for every position i

LongRunMask(mask, length n, axis) {pixels lying on a run of n or more, Equation 29}
  c ← cumulative sum of the mask along the axis
  hit ← $\big(c[i+n]-c[i]=n\big)$ {a run of n starts at i}
  ch ← cumulative sum of hit
  return $\mathrm{ch}[h_j]-\mathrm{ch}[l_j]>0$ {some such run covers j}

3.4.3 Lighting normalisation and binarisation

The intensity measured by a camera is approximately the reflectance of the paper multiplied by the illumination falling on it. Because the nuisance term is a multiplication and not an addition, it is divided out rather than subtracted: the working grey image G is divided by a single box mean of itself, giving a ratio image ρ close to 1 on paper whatever the lighting, and on ink the ink-to-paper reflectance ratio of the pen itself. Once the shading is divided out, the ink decision is a single global threshold, chosen by Otsu's method [**?**, p. 63], which is possible only because of the division — this is the step that removed the long chain of clean-up filters the project's first segmenter needed.

The ratio image is formed by dividing the grey image by a box mean of itself,

$$\rho(p) = \frac{G(p)}{\max\big(\mathrm{BoxMean}_r(G)(p),\,1\big)}, \tag{30}$$

where $\rho(p)$ is the ratio at pixel p, G the working grey image and $\mathrm{BoxMean}_r$ the mean over a window of radius r, evaluated through Equation 27. The denominator is floored at 1 so a black border cannot divide by zero. The radius is tied to the image height,

$$r = \max\big(8,\,\min(40,\,\mathrm{round}(H/41))\big), \tag{31}$$

where H is the height of the working image in rows. Equation 31 makes the background estimate cover the same fraction of the page at any camera resolution. The radius must be much larger than a letter, or letters would darken their own background estimate and be divided away; the bounds of 8 and 40 rows keep it so at the resolutions used here.

The threshold separating ink from paper is then the one that best splits the histogram of ρ into two groups, by Otsu's criterion [**?**, p. 63],

$$t^{*} = \arg\max_{t} \; \omega_B(t)\,\omega_F(t)\left[\mu_B(t) - \mu_F(t)\right]^2, \tag{32}$$

where t^{*} is the chosen threshold, $\omega_B(t)$ and $\omega_F(t)$ are the fractions of pixels below and above a candidate t, and $\mu_B(t)$ and $\mu_F(t)$ are their means. The quantity maximised in Equation 32 is the between-class variance, and it is evaluated for all 256 histogram bins at once. A pixel is marked as ink when $\rho < t^{*}$. Applying Equation 32 globally is defensible only because Equation 30 has already removed the shading that would otherwise force a local threshold.

One extra test is needed beyond the threshold: a solid-area veto removes any window entirely ink, since no pen stroke fills a 9×9 window, which catches a binder shadow or the dark gutter of a bound book. The whole stage is the four array statements of Algorithm 6, and holds no tunable grey level — which is the result of dividing the illumination out before thresholding rather than after.

Algorithm 6 Lighting normalisation and the ink mask

```
1:  OtsuThreshold(values) {the cumulative form of Equation 32, all 256 bins at once}
2:      histogram ← 256 bins of the values clipped to [0, 1.2]
3:      w, m ← cumulative class weight and cumulative class mean
4:      variance ← (total mean · w − m)^2 / (w (1 − w))
5:      return the bin centre maximising variance
6:
7:  InkMask(grey)
8:      r ← max(8, min(40, round(H/41))) {Equation 31}
9:      ratio ← grey / max(BoxMean(grey, r), 1) {divide, not subtract: Equation 30}
10:     mask ← ratio < OtsuThreshold(ratio)
11:     solid ← BoxMean(mask, 4) > 0.999 {a 9 × 9 window wholly ink}
12:     return mask ∧ ¬(dilate solid by 3), and ratio
```

The threshold 0.999 rather than 1 absorbs floating-point error in the integral image; the fringe is removed with the blob because a shadow's edge is as unwanted as its interior.

3.4.4 Deskew by searching rotations

When text lines are horizontal, the row-sum profile of the ink has sharp peaks at the line bodies and deep troughs between them; any tilt smears those peaks, since a single row then cuts across several lines. This projection-profile approach to skew detection [**?**] is a standard alternative to a Hough transform over ink edges [**?**], chosen here because a page of handwriting and printed rules gives the Hough transform many competing line candidates, whereas the row profile already favours the direction most of the ink actually runs in. The sharpness of the profile is therefore a measure of how level the page is, and can be maximised directly.

FIGURE MUST STILL BE INSERTED BY ME

Four-panel figure on one real page: the photograph, its grey conversion, the box-mean background estimate, and the resulting ratio image ρ, showing the shading removed before Otsu's threshold is applied. Exportable from `InkMask`.

Figure 16. The four stages of Equation 30 on one real page — the photograph, its grey conversion, the box-mean background estimate, and the resulting ratio image ρ, showing the shading removed before Otsu's threshold is applied.

The row profile at a trial angle is the ink count per row,

$$P_\theta(y) = \sum_x R_\theta(y,x), \tag{33}$$

where $P_\theta(y)$ is the number of ink pixels in row y and R_θ is the ink mask notionally rotated by θ. The sharpness of Equation 33 is scored by its sum of squares, and the angle maximising that score is taken,

$$J(\theta) = \sum_y P_\theta(y)^2, \qquad \theta^* = \arg\max_\theta J(\theta), \tag{34}$$

where $J(\theta)$ is the objective and θ^* the chosen skew angle. Squaring before summing is what makes Equation 34 work: the total amount of ink does not change under rotation, so $\sum_y P_\theta(y)$ is fixed, whereas the sum of squares is largest when the same ink is concentrated into the fewest rows, which happens when the lines are level.

The rotation is never performed to score a candidate. Each ink pixel is mapped straight to the row it would land in,

$$k = (y - c_y)\cos\theta - (x - c_x)\sin\theta + c_y, \tag{35}$$

where k is that row index, (y,x) the pixel's coordinates and (c_y,c_x) the image centre. Counting rows through Equation 35 was measured 31 times faster than rotating the mask seventeen times.

The search sweeps 17 candidate angles from −4° to +4° in steps of 0.5° on the stage-2 mask reduced by a further factor of two, with no refinement pass, so the best accuracy reachable is half a step, 0.25°. Ink that would fall outside the row range at a trial angle is allowed to extend the profile rather than being clipped, since clipping piles out-of-range pixels into one row whose artificial spike scores higher than any real text line; an earlier version with this clipping fault chose an angle four degrees wrong on a tilted page. Only once an angle is chosen is the page itself rotated, bilinearly, and the ink mask and ratio image recomputed, because Equation 30 is not rotation-equivariant.

The stage was measured against injected ground truth, and the result closes the loop on Equation 34: seven angles lying on the search grid were recovered *exactly* in 35 of 35 trials, and four angles deliberately placed between grid points were recovered to 0.25° — exactly half the search step — in 20 of 20 trials (Table 13). The objective therefore picks the correct grid point every time, and the residual error is entirely the coarseness of the grid rather than any weakness in Equation 34. Halving the step would halve the residual at twice the cost.

3.4.5 Line pitch: the scale unit the rest of the block uses

Every threshold from here to the end of the block is expressed in multiples of the line pitch P, so P must be measured before any of them can be applied. It is obtained from the row-ink profile, the mask pixel count per row over a central window, smoothed by the Gaussian kernel of Section 3.4.2:

$$\mathrm{prof}(y) = \sum_{x=\lfloor 0.05W \rfloor}^{\lfloor 0.97W \rfloor - 1} \mathrm{ink}(y,x), \qquad \mathrm{sm} = \mathrm{prof} * k, \tag{36}$$

where $\mathrm{prof}(y)$ is the ink count in row y, W is the image width, k is the 25-tap Gaussian kernel and sm the smoothed profile. The window in Equation 36 excludes the outer five per cent on the left and three per cent on the right, where a binding shadow or a torn edge would otherwise contribute a constant to every row.

A peak of sm is a row satisfying three conditions together,

$$\mathrm{sm}(y) \geq \mathrm{sm}(y-1), \qquad \mathrm{sm}(y) > \mathrm{sm}(y+1), \qquad \mathrm{sm}(y) > 0.06 \max_{u} \mathrm{sm}(u), \tag{37}$$

where y is a candidate row. The comparisons in Equation 37 are deliberately asymmetric, one non-strict and one strict, so the last row of a flat-topped peak is taken exactly once. The relative floor of 0.06 scales with the amount of ink on the page rather than being an absolute count, so a faint page is not rejected wholesale.

The pitch is the *median* of the gaps between consecutive peaks, used whenever at least two peaks exist; a median rather than a mean so one unusually large gap, such as a paragraph break, cannot shift it. Three fallbacks follow in order when fewer than two peaks are found, so the stage degrades rather than failing (Algorithm 7). Measured over the 30 photographed pages, P ranged from 47 to 58 pixels — a 23 % spread across five writers at one camera distance, which is the variation goal 1 of Section 3.4.1 exists to absorb and the reason no threshold in this block is an absolute pixel count.

Algorithm 7 Measuring the line pitch, with three fallbacks

```
profile ← ink pixels per row, over the central 5 % to 97 % of the width {Equation 36}
smoothed ← profile convolved with a 25-tap Gaussian, σ = 4 rows
peaks ← interior rows satisfying all three tests of Equation 37
if at least two peaks then
    return the median gap between consecutive peaks {the normal path}
else
    autocorrelate the mean-removed profile; take the first lag in 15 … min(200, L/3) reaching
    0.7 of the segment maximum
end if
if no such lag then
    P ← max(15, round(2.2 · median letter height))
end if
if no usable components either then
    P ← 48 rows {last resort; never reached on the 30 pages tested}
end if
```

The autocorrelation is a fallback only, and deliberately so: it can lock onto a multiple of the true spacing, which would halve the line count without any other symptom. The smoothed profile is

returned alongside P and reused for the line centres of Section 3.4.7, so the scale unit and the line positions cost one pass over the mask between them.

3.4.6 Removing what is not handwriting

Three kinds of nuisance are removed in turn, each identified by a property that handwriting does not share. The area outside the sheet is removed because handwriting disappears under a coarse enough average while the sheet itself does not. Printed structure — ruled paper, underlines, a bound book's spine shadow — is removed because it runs straight for longer than a line spacing, which handwriting does not, a long directional opening rather than a fixed-size filter. Edge slivers, corner shadows and creases are removed because, although not straight, they are judged as whole connected pieces rather than pixel by pixel: each fragment is labelled by the same connected-component logic used throughout this block, and the whole piece is then scored on being long and thin or sparse inside its own bounding box.

The long-run primitive of Equation 29, bracketed by a dilation and a re-intersection with the real ink, gives Equation 38 for flat (horizontal) and upright (vertical) printed structure, where $\mathcal{D}_r^y$ and $\mathcal{D}_r^x$ are one-axis dilations:

$$\begin{aligned} \text{flat} &= \mathcal{D}_1^y\Big(\text{LongRun}^x\big(\mathcal{D}_1^y(\text{ink}),\ \lfloor 1.25P \rfloor\big)\Big) \wedge \text{ink}, \\ \text{upright} &= \mathcal{D}_3^x\Big(\text{LongRun}^y\big(\mathcal{D}_3^x(\text{ink}),\ \lfloor 1.4P \rfloor\big)\Big) \wedge \text{ink}. \end{aligned} \tag{38}$$

The two run lengths, $1.25P$ horizontally and $1.4P$ vertically, are longer than any single letter can be, so no letter is mistaken for a rule; the dilation before the run test tolerates a slight tilt, and the re-intersection means only real ink is ever removed. Both are stated as multiples of the P measured in Section 3.4.5, which is why that stage has to come first.

Separating ink into pieces for the third kind of nuisance needs connected-component labelling: every set pixel is given an integer label, two pixels sharing a label when a path of set pixels joins them. The mask is closed first so pen lifts inside a word are bridged, and the labelling is a run-length union-find pass, each row reduced to its horizontal runs before a union-find structure records which runs belong together, so the whole page is labelled in one pass over the rows. Every discard statistic is accumulated over the *original* ink pixels, not the closed mask, so closing can join pieces but never add area to a decision.

Each closed piece is then judged as a whole and discarded if any one of the eight predicates of Table 10 holds, for a piece of area A, height h and width w, with $e = \max(3, \lfloor 0.5P \rfloor)$ counting as touching the frame, x_L the fifth percentile of the left edges of the letter-like pieces ($A \geq 0.065P^2$, $h \leq 2.2P$) and faintness defined as $\bar{\rho}_c > 0.85t^*$, relative to the page's own threshold so that one constant serves pencil and fibre tip.

That first predicate is the one place where goal 2 of Section 3.4.1 — never delete handwriting to remove a nuisance — is knowingly traded away, and the trade is recorded rather than hidden: it is accepted because a photograph of an open book is a more common failure than a page written with no left margin at all.

Discard condition	What it is meant to catch
$x_{\min} \le e$	A piece touching the left frame edge: the facing page, or the desk.
$x_{\max} \ge W - 1 - e$, $w \le 0.5P$	A narrow sliver at the right edge; a real line there is wider.
$y_{\min} \le e$ or $y_{\max} \ge H - 1 - e$, $h \le 0.5P$, $w \le 0.5P$	A small fleck at the top or bottom frame edge.
$h > 2.2P$, $w \le 0.5P$	A tall thin streak over two line spacings: a binder edge or a fold.
$w \le 0.12P$, $h \ge 1.2P$	A bar thinner and taller than any letter.
faint, $w \le 0.2P$, $h \ge 0.6P$	A printed margin rule or a crease.
($y_{\min} \le e$ or $y_{\max} \ge H - 1 - e$), $h \le 1.2P$, $A/(hw) < 0.15$, faint, $w > P$	An edge shadow or fold: wide, faint, at an edge, too sparse inside its own box to be writing.
$x_{\max} < x_L - 1.2P$	A hole punch, a staple shadow or a page number in the gutter.

Table 10. The eight discard predicates applied to each closed piece of ink; the background label is exempt from all of them. The first row is the most aggressive rule in the module: any piece whose leftmost ink lies within e of the left frame edge is discarded whatever its size or darkness, which removes the facing page of an open book but also costs the first letters of a page photographed with no left margin.

FIGURE MUST STILL BE INSERTED BY ME

A page of ruled paper before and after Equation 38 removes the printed rules, showing that handwriting crossing a rule survives while the rule itself is removed.

Figure 17. A page of ruled paper before and after Equation 38 removes the printed rules, confirming that handwriting crossing a rule survives while the rule itself is removed.

3.4.7 Line centres on the cleaned mask

The pitch of Section 3.4.5 was measured on the raw ink mask, because the clean-up of Section 3.4.6 needed it. Now that the nuisances are gone, the profile and the pitch are *recomputed* on the cleaned mask by the same Equation 36, and the peaks of Equation 37 become the line centres. Recomputing matters because a printed rule contributes a full-width row of ink, which is the strongest peak a profile can have; measuring the centres before clean-up on ruled paper would place a line on every rule.

The peaks are reduced to centres by greedy non-maximum suppression: a peak is accepted only if it lies at least $0.6P$ from every centre already accepted. The separation is more than half a pitch, so two genuine lines can never be merged into one.

3.4.8 Seam cut, pixel ownership and the final crop

A straight cut halfway between two line centres would sever every descender and ascender that crosses it, so the boundary between two lines must bend around the ink. The two layers of ink recovery — a strict page-wide mask and a looser threshold trusted only next to ink a line already owns — exist because a threshold loose enough to catch a faint stroke is also loose enough to catch a shadow, and *hysteresis*, using two thresholds instead of one, is the standard remedy, the same principle used to link edges in edge detection applied here to ink.

The boundary itself is found the way a seam is found in content-aware image resizing: the cheapest vertical path across a cost image, found column by column with dynamic programming rather than guessed as a straight line [**?**, p. 10:3]. Between consecutive centres $c_0 < c_1$ the cost field of Equation 39 and the dynamic-programme recursion of Equation 40 give the cheapest path across it:

$$C(y,x) = 4.0\,\tilde{m}(y,x) + 0.012\left(\frac{y-\mu}{\max(1,(c_1-c_0)/2)}\right)^2, \qquad \mu = c_0 + \tfrac{1}{2}(c_1-c_0), \tag{39}$$

where $C(y,x)$ is the cost of placing the boundary at row y of column x, $\tilde{m}$ is the dilated ink mask, and μ is the midline between the two centres c_0 and c_1. The two weights in Equation 39 are three hundred to one: crossing a single ink pixel costs 4.0 against a maximum of 0.012 for sitting a half-gap away from the midline, so the midline term only breaks ties across blank gaps between words and never persuades the seam to cut a letter.

The cheapest path is accumulated column by column,

$$A(y,x) = C(y,x) + \min_{|d|\le s}\big[A(y+d,x-1) + 0.02\,|d|\big], \qquad A(y,0) = C(y,0), \tag{40}$$

where $A(y,x)$ is the cost of the cheapest path reaching (y,x), d is the row step taken between adjacent columns and $s = 7$ bounds it. The penalty $0.02\,|d|$ in Equation 40 prices vertical movement, so the seam prefers to stay level unless ink gives it a reason to climb. Algorithm 8 states the minimisation, which is exact: a path cost is a sum over columns, and every admissible predecessor is considered.

Algorithm 8 The least-ink seam between two line centres

Require: consecutive line centres $c_0 < c_1$, step bound $s = 7$

cost $\leftarrow 4.0 \cdot$ closedInk $+ 0.012\big((\text{row} - \mu)/\text{halfGap}\big)^2$ {Equation 39}
acc$[:,1] \leftarrow$ cost$[:,1]$
for $x = 2$ **to** W **do**
  for each displacement $d \in -s \ldots +s$ **do**
    candidate$[d] \leftarrow$ acc$[\text{row}+d,\ x-1] + 0.02\,|d|$
  end for
  acc$[:,x] \leftarrow$ cost$[:,x] + \min_d$ candidate$[d]$ {Equation 40}
  back$[:,x] \leftarrow$ the d that achieved the minimum
end for
backtrack from the cheapest entry of the last column

All rows of a column are evaluated at once as $2s+1$ shifted candidate arrays, so only the displacements and the backtrack are loops. The two outermost boundaries are fixed at $\lfloor 0.75P \rfloor$

beyond the first and last centre, so L centres give exactly L bands; the whole computation is $O(15HW)$ at worst, the most expensive single step in the block.

A pixel belongs by default to the band it lies in. Six correction rules are then applied on top of that default, each of which may move a piece to an adjacent band or drop it but never further. They rest on each band's share of a piece's ink, with 0.15 counting as a genuine presence, and on the *major* band, defined as the topmost band reaching the piece, since a piece spanning two lines is the upper one's descender.

Rule	Applies to	Action, and the reason for it
1	A remnant in a band adjacent to the major band: under $0.12P^2$ there, under 0.35 of the piece's ink, no wider than $0.55P$	Returned to the major band. The width test separates a cut-off tail, which is narrow, from a genuine short word, which is wide.
2	A piece under $0.06P^2$; a *speck* is under $0.01P^2$	Dropped if no *big* piece ($\geq 0.05P^2$) of its own line lies within $6P$ horizontally, else moved one band towards the body it belongs to, asymmetrically ($0.5P$ below, $0.7P$ above), because a mark below a line is usually its own comma or descender while one well above it is more often an accent of the line above. A speck out of band drops rather than moves.
3	A piece of $0.06P^2$ to $0.15P^2$	Given to whichever of the three nearest bands has its body closest in y.
4	A *dot*: area under $0.1P^2$, extent at most $0.45P$, aspect ratio under 2.2 so that a short dash does not qualify	Followed downwards up to $\lfloor 0.4P \rfloor$ rows to the first *stem* ($\geq 0.03P^2$) and moved to that stem's band only if that band's body also lies $0.25P$ to $0.7P$ below it, where an *i* dot sits relative to its own x-height band. Together the two conditions are much stronger than either alone, since the scan can meet the wrong stem on a crowded page and the band test can accept a mark belonging to no stem.
5	The same search on the raw, unclosed labelling, for pieces of at least $0.015P^2$	Catches a dot the clean-up closing has welded to a descender of the line above. A ring-footed stroke the same welding makes — a blob of width $0.2P$ to $0.45P$, aspect ratio 0.6 to 1.6 — also follows the stem below, but only if it encloses a hole, since a dot ring does and a hook does not. Neither rule reassigns a whole word ($\geq 0.2P^2$).
6	A fragment of at least $0.006P^2$ with no ink within $0.28P$ of its own band's centre	Offered to the line whose ink it touches, or failing that to the nearer neighbouring centre if the fragment sits within $0.2P$ of it. Fragments placed by rule 5 are locked, since a dot deliberately lifted into a band would otherwise be pushed straight back.

Table 11. The six pixel-ownership correction rules. The size ladder they share runs $0.006P^2$, $0.01P^2$, $0.015P^2$, $0.03P^2$, $0.05P^2$, $0.065P^2$, $0.1P^2$, $0.12P^2$ and $0.2P^2$, every entry a multiple of the measured pitch squared, so no rule is tied to a pixel count.

Each crop is rendered from the full-resolution page and composited from three layers: the band's own ink mask, a recovery mask from re-applying Equations 30 and 32 inside the crop alone, and the halo of the band's own strong ink. The three are combined by the hysteresis rule

$$\alpha = \max\Big(\alpha_{\text{strong}},\ \mathbb{1}[\rho_{\text{local}} < t^{*}_{\text{local}}] \cdot \alpha_{\text{halo}} \cdot \mathbb{1}[\text{not owned by another band}]\Big), \tag{41}$$

where α is the opacity written into the crop, α_{strong} comes from the page-wide mask, α_{halo} from the looser local threshold t^{*}_{local}, and $\mathbb{1}[\cdot]$ is 1 when its condition holds and 0 otherwise. The second term of Equation 41 is the loose layer, admitted only next to ink the band already owns, which is the hysteresis: a faint stroke touching known ink is kept, while an equally faint shadow elsewhere is not.

Finally the crop's contrast is stretched,

$$\ell = \text{clip}\left(\frac{255\,(v-b)}{\max(w-b,1)}, 0, 255\right), \qquad \text{out} = 255\left(1 - \left(\tfrac{255-\ell}{255}\right)^{0.55}\right), \tag{42}$$

where v is the input grey value, b and w are the black and white points taken from the crop's own histogram, ℓ is the linearly stretched value and out the value written. The exponent 0.55 in Equation 42 thickens thin strokes slightly, which matters because the recogniser was trained on ink of a similar weight. Outside the mask the crop is pure white, and it is shifted so the first

inked column sits at a fixed offset, since the recogniser's input is left-flush (Equation 43):

$$s = \min\Big(\frac{64}{h}, \frac{640}{w}\Big), \qquad h' = \max(1, \mathrm{round}(hs)), \qquad w' = \max(1, \mathrm{round}(ws)). \tag{43}$$

3.4.9 Summary of the block's constants

Goal 3 of Section 3.4.1 was to keep as few fixed constants as possible, and Table 12 is the evidence for or against it. It collects every constant of the minor operations of all stages, with the reason for each; the constants of the core operations are already in the equations above. Only four entries are not expressed in units of the measured pitch P — the Otsu range, the $0.2\,\%$ and 0.45 paper-region tests and the floor of 60 ink pixels — so every other threshold in the block scales with the page automatically, which is what makes the block work at any camera resolution without tuning.

St.	Quantity and value	Reason
2	Otsu histogram over $[0, 1.2]$, 256 bins	paper sits at $\rho \approx 1$ and nothing above 1.2 carries ink information, so a highlight cannot drag the cut
2	solid-area veto $\mathrm{BoxMean}_4(\mathrm{ink}) > 0.999$, cleared over 7×7	no pen stroke fills a 9×9 window; the fringe goes with the blob, and 0.999 absorbs integral-image rounding
3	17 angles, $-4.0°$ to $+4.0°$ in $0.5°$	one sweep, no refinement, over a sixteenth of the photograph's area
3	rotation fill 0 for the candidate masks, 255 for the page	outside the sheet must add no ink to the profile; corners brought in must be white paper
4	paper region: block-average by $s = 4$, blur $\max(1, \lfloor 0.15P/s \rfloor)$, dark below $0.45\times$ median	a ratio and not a grey level, so it survives a change of exposure
4	whole frame accepted as paper below $0.2\,\%$ dark cells; sheet eroded $\lfloor 0.4P \rfloor$ per axis	correct for a photograph filled by the sheet; the erosion keeps the sheet's darkened rim out of the ink mask
4	piece closing $\max(1, \lfloor 0.1P \rfloor)$	bridges the pen lifts inside a word, so a word is one piece
5	centre separation $0.6P$	more than half a pitch, so two genuine lines can never be merged
6	seam closing $\max(2, \lfloor 0.07P \rfloor)$	fills only the gap beside a letter stem; a dilation would make a blob the cut would trace
6	outer boundaries $\lfloor 0.75P \rfloor$ beyond the first and last centre	covers the first line's ascenders and the last line's descenders
6	band emitted above $\max(60, 0.12\,\mathrm{median}_n N_n)$ owned pixels and $\max(6, 0.2P)$ rows	keeps a short line carrying an eighth of the typical ink; the absolute 60 catches a tiny-median page
6	vertical gap trimmed only if it removes more than $0.8P$	spares the ordinary gap between a line's body and its own ascenders
6	crop padding $\lfloor 0.15P \rfloor$ per side, halo $\lfloor 0.20P \rfloor$ per axis, local recovery radius $\max(3, \min(20, \mathrm{round}(h_{\mathrm{crop}}/8)))$	the halo anchors recovery to ink the line already owns; the local radius judges a pale line against its own contrast
6	stretch of Equation 42: b the 40th percentile under the mask, w the 75th of the crop	the 40th percentile is the typical ink darkness, so one dark speck cannot flatten the line

Table 12. Constants of the minor operations of each stage, with the reason for each. Every entry in units of the measured pitch P scales with the page; the four that do not are the Otsu range, the $0.2\,\%$ and 0.45 paper-region tests and the floor of 60 ink pixels.

3.4.10 Testing the segmentation block

Two properties of this block were tested against a known answer on the 30 photographed pages of the five personal writers, 2448×3264 pixels each. Table 13 gives the results: the block found the correct number of lines on all 30 pages, 669 of 669 lines, with no page over- or under-segmented, against the hand-typed transcription; and the deskew of Equation 34 recovered seven grid-aligned injected angles exactly in 35 of 35 trials and four off-grid angles to $0.25°$ in 20 of 20 trials.

The 30 pages are the pages the system was built on, so the line-count result is not an unseen test;

Measurement	Result	What it shows
Line count against the hand-typed transcription, 30 pages	**30 of 30** pages exact; 669 of 669 lines	The line separation is correct on every page tested, and the relative floor of Equation 37 lost no line.
Deskew, 7 angles on the search grid, 5 pages	**35 of 35** exact	The objective of Equation 34 picks the correct grid point every time.
Deskew, 4 angles between grid points, 5 pages	**20 of 20** at 0.25°	The residual error is exactly half the search step, so the stage is quantisation-limited, not objective-limited.
Time per page, 2448 × 3264	**4.9 to 9.7 s**, mean 7.9 s	The cost of the deployed module. Measured with the project's own resampling throughout.
Measured line pitch P	47 to 58 pixels	The scale unit varies by 23 % across these writers, which is what the block is designed to absorb.
Chosen skew angle on the unrotated pages	−1.5° to +3.0°	Real photographs are tilted by more than a degree, so the stage is doing necessary work.

Table 13. Measured performance of the deployed segmentation block over the 30 photographed pages of the five personal writers. These are the pages the system was built on, not an unseen test set, so the line-count result shows that the block handles the material it was designed for and not that it generalises to new conditions.

the deskew result is stronger, since the injected angle is ground truth no part of the design could have been fitted to. The timing figure also corrects an attribution: the 24.8 to 69.1 s per page reported in Section 4 is the retired segmenter, `SegmentPage.ProcessPage`; the deployed module is roughly five times faster at 4.9 to 9.7 s, because it makes one pass over a half-resolution image where the retired module made four passes over a larger one. The mean of 7.9 s is still about four times the two seconds requirement R4 allows for capture and conditioning. Using the project's own resampling rather than the imaging library's compiled resampling costs about 40 % of the block's run time, the measured price of the library-free claim. **[MUST STILL BE COMPLETED BY ME: update the segmentation figures of Section 4 and the R4 budget to 4.9–9.7 s; time `SegmentLean.SegmentLines` on the ODROID N2+; export the figures placeholdered above, plus the paper-region mask and the `_cuts_overlay.jpg` that `SegmentLean.py` already writes.]**

FIGURE MUST STILL BE INSERTED BY ME

One full photographed page with the final seam cuts overlaid in colour — this is the `_cuts_overlay.jpg` that `SegmentLean.py` already writes, so it needs exporting rather than producing.

Figure 18. The seam-cut overlay on one real, photographed page, showing each line's final boundary bending around ascenders and descenders rather than cutting straight through them.

3.4.11 Design alternatives and limitations

Table 14 sets the main design choices against the alternatives considered. The project's first segmenter, `SegmentPage.py`, was about 2200 lines and sixty functions against the eleven here; it resized every photograph to a fixed long side, thresholded locally, removed printed structure in four passes, took a median glyph height as its scale unit, scored components for being diagrams, chained them into lines and deskewed each line separately. No part of the deployed pipeline calls it any more, and three of its capabilities were given up with it: diagram tagging, per-line deskew, and a wider ±8° search at 0.1°. The replacement is about a quarter of the size and roughly five times faster, at the cost of those three capabilities.

Decision	Alternatives, and why the deployed choice was made	Measured evidence
Lighting	Subtracting a blurred background leaves contrast dependent on brightness. The nuisance is multiplicative, so dividing gives an ink-to-paper ratio independent of the lamp.	Removed the clean-up chain the first segmenter needed: **570 lines against 2200**.
Ink threshold	A local mean less an offset marks something even in an empty region and needs a tuned offset. After the division, one global Otsu threshold is enough.	**Zero** tunable grey levels remain in the stage (Algorithm 6).
Scale unit	A median glyph height is upset by a heading in another size; a fixed-size page throws away resolution. The pitch P is measured on the page itself.	P spans **47 to 58 px** over five writers, a 23 % range absorbed automatically.
Lines	Chaining components along a baseline curve needs a correct set of components first, propagating every clean-up error. The row profile yields the pitch and centres together in one $O(N_p)$ pass.	**30 of 30** pages, **669 of 669** lines exact (Table 13).
Line boundaries	A straight cut at the midline severs descenders and ascenders. The least-ink seam bends around them.	**[MUST STILL BE COMPLETED BY ME: count the descenders a midline cut would sever on one page, against the seam.]**
Skew	A Hough transform of ink edges [**?**] is dominated by ruled lines and diagram strokes. The projection objective is also the quantity the pitch stage goes on to use, so it costs nothing extra.	**35/35** grid angles exact, **20/20** off-grid to 0.25°.
Rule removal	A long morphological opening erases underlines and cannot follow a slope. The dilated long-run test needs two constants where chaining fragments across gaps needs five.	**2 constants** against 5 (Equation 38).
Not implemented	Text-versus-diagram tagging, per-line deskew and a four-point perspective warp are all absent; the first is accepted for plain prose pages, the residual drift is tolerated downstream, and the camera looks straight down.	Not built. Cost of the retirement: **4.9–9.7 s** against 24.8–69.1 s.

Table 14. Design alternatives of the deployed segmentation block. The last row records capabilities of the first segmenter that were deliberately given up. Only the deployed choice was built in each row, so the evidence column measures the choice made and not the alternative rejected; where no such measurement exists the cell says so.

The block assumes left-to-right writing in a single column of roughly horizontal lines, and the following limitations follow from that design. Skew beyond ±4° is not corrected at all, and up to 0.25° always remains. Two columns of text, or a diagram beside the text, produce profile peaks that belong to no single line, and an untagged diagram is cut into line-shaped crops and read as nonsense. A line peaking below six per cent of the strongest is lost silently, although no such line occurred in the 30 pages tested. A blank page raises an exception rather than returning no lines. Illumination varying faster than the radius of Equation 30 leaves one side of a hard shadow edge mis-thresholded, page curl and perspective are untreated, and colour is treated like black ink because the module is monochrome throughout.

[MUST STILL BE COMPLETED BY ME: page locators outstanding in this subsection.

The sources `crow1984, duda1972, postl1986` are each cited here without a page or numbered element. Open each one, find the claim it is cited for, and add the locator in the form `\cite[p. N]{key}`, following the rule of `REFERENCE_AUDIT.md`: a locator is written only after the source has been read and the claim found, never inferred from a page range or a search snippet.]

3.5 Text recognition and writer identification

This subsection describes the two neural networks that read a handwritten line into text and decide which of the thirteen stored writers wrote it (functional unit FU 2.3; requirements R1, R2), component by component: the convolutional backbone the two share, the sequence model and alignment-free loss that make text recognition possible at all, the writer-identification head and the stroke normalisation it depends on, and the from-scratch training and verification of every layer. Every layer, gradient and optimiser update described here is implemented in one module written for this project, `NumpyNN.py` (Section 3.5.5), and no machine-learning library supplies any of them. All inference runs through that module, and so does the training of the text recogniser whose results are reported in Section 4. The writer classifier was trained by both routes, through `NumpyNN.py` and through an equivalent library implementation, and the checkpoint used for the measurements is the library-trained one; Section 5.1.2 gives the comparison that decided this and the cost it carries. The distinction is kept explicit here rather than claimed away, because a library-trained checkpoint is not evidence of a from-first-principles implementation however the inference is executed.

Both networks follow the layer pattern of Kizilirmak and Yanikoglu [**?**, p. 9], chosen because it was evaluated on the public handwriting database used here [**?**] and reports accuracies in the range R2 needs. That paper describes an approach and a layer pattern rather than publishing code, so the network was built from its description: the layer pattern, filter counts and recurrent width are taken from it, while the input geometry, data pipeline, label cleaning, augmentation and staged training recipe were decided for this project. That hybrid of convolution and recurrence is the combination reported to improve feature extraction and contextual understanding together [**?**, **?**].

3.5.1 The shared convolutional backbone

A convolutional neural network (CNN) is a stack of layers each of which applies the same small learned filter at every position of its input [**?**, Sec. II-A], [**?**]. Applying one filter everywhere, rather than learning a separate weight for every pixel, is what makes the layer useful for handwriting: a stroke junction means the same thing wherever on the line it appears, so the filter that detects it should not have to be learned again for each position. A single convolutional layer only sees a small local neighbourhood, but stacking several lets later layers combine the simple strokes earlier layers detect into the more complex shapes of whole letters, which is why depth rather than one wide layer is used. Three further layer types complete the pattern used here, each solving a specific problem of training a deep stack: *batch normalisation*, the *rectifier* nonlinearity, and *pooling*.

A convolution with kernel size k, stride s and p rows and columns of zero padding on each side

of input X_{pad} is

$$Y[o,i,j] = b_o + \sum_{c=1}^{C_{\text{in}}} \sum_{u=0}^{k-1} \sum_{v=0}^{k-1} W[o,c,u,v]\, X_{\text{pad}}[c, s\,i+u, s\,j+v], \tag{44}$$

$$Y = \tilde{W}\,\text{cols} + b\,\mathbf{1}^{\top}. \tag{45}$$

Equation 44 is six nested summations, too slow in array arithmetic, so the implementation uses the *im2col* rearrangement of Equation 45: every $k \times k$ patch the filter would see is copied out as one column, the filter weights reshaped into rows, and the whole convolution becomes a single matrix product, agreeing with the direct form to about 10^{-13} and about ten times faster. With $k = 3$, $p = 1$, $s = 1$ the output keeps the input's height and width, so only pooling layers change the tensor size. Batch normalisation [**?**, Alg. 1] (Equation 46) rescales each channel to zero mean and unit variance over the current training batch and then applies a learned scale γ and shift β; at inference, with no batch to average over, running statistics collected during training replace μ_B, σ_B^2 and the whole operation folds into the single affine map of Equation 47:

$$\hat{x} = \frac{x - \mu_B}{\sqrt{\sigma_B^2 + \varepsilon}}, \qquad y = \gamma\hat{x} + \beta, \qquad \varepsilon = 10^{-5}, \tag{46}$$

$$a = \frac{\gamma}{\sqrt{\sigma_{\text{run}}^2 + \varepsilon}}, \qquad b = \beta - a\,\mu_{\text{run}}, \qquad y = a\,x + b. \tag{47}$$

The rectifier is $\max(0,x)$ [**?**, p. 807]: some nonlinearity is essential, because a stack of purely linear layers collapses into one linear layer however deep it is, and the rectifier's derivative of 0 or 1 neither saturates nor shrinks the gradient across twelve layers. Max-pooling takes the largest value in each 2×2 window, stride 2, halving both axes with no learned parameters [**?**]; it discards three quarters of the positions, keeping the network small enough for the target hardware, and tolerates a one-pixel shift, since moving a stroke slightly usually does not change which value in its window is largest.

Figure 19 gives the three convolutional stages (32, 64 and 128 channels, 2, 4 and 6 layers respectively) with two intervening max-pools, each verified for this project against the published pattern; what follows the backbone differs between the two networks and is given in Sections 3.5.2 and 3.5.3. The full recogniser holds 3 358 763 trainable parameters, of which the recurrent layers hold 70.5 % and the third convolutional stage 24.2 % (Table 15).

Table 15 resolves those 3.36 million parameters into the layer that holds each one, which shows where the capacity of the network actually sits.

The arithmetic cost follows from the same shapes. Summing the multiply–accumulates of every convolution, of the two recurrent layers over all T steps and of the output projection gives

$$N_{\text{MAC}} = \sum_{\ell} 9\,C_{\text{in},\ell} C_{\text{out},\ell} H_\ell W_\ell + 2T\,[4n(128+n) + 4n(512+n)] + T \cdot 512 \cdot 75 \approx 4.17 \times 10^9 \tag{48}$$

per line, of which 3.79×10^9 are convolutional. Equation 48 is the reason the convolutions and not the recurrent layers dominate the measured inference time, even though Table 15 shows the recurrent layers holding 70.5 % of the parameters: a convolutional filter is applied at every one of tens of thousands of positions, whereas a recurrent weight is used once per time step.

Both networks share one input convention rather than each defining its own. Every crop, for either network, is resized uniformly to fit a 64×640 canvas, pasted left-aligned and vertically

Layer	Output shape	Trainable parameters
Input	$1 \times 64 \times 640$	0
Conv 3×3 (1→32) and (32→32), BN, ReLU	$32 \times 64 \times 640$	9 696
Max-pool 2×2	$32 \times 32 \times 320$	0
Conv 3×3 (32→64) and 3× (64→64), BN, ReLU	$64 \times 32 \times 320$	129 792
Max-pool 2×2	$64 \times 16 \times 160$	0
Conv 3×3 (64→128) and 5× (128→128), BN, ReLU	$128 \times 16 \times 160$	813 312
Height max-pool	$128 \times 1 \times 160$	0
BiLSTM layer 1 (128→256, 2 directions)	160×512	790 528
BiLSTM layer 2 (512→256, 2 directions)	160×512	1 576 960
Linear (512→75) and log-softmax	160×75	38 475
Total		**3 358 763**

Table 15. Layer shapes and trainable parameters of the text recogniser (batch dimension omitted). A convolution with batch normalisation holds $C_{\text{out}}(9C_{\text{in}}+1)+2C_{\text{out}}$ parameters and one LSTM direction $4n(I+n)+8n$ with $n=256$, which is where every entry comes from. Of the total the LSTM layers hold 70.5 %, stage 3 24.2 %, stage 2 3.9 %, the output layer 1.1 % and stage 1 0.3 %. Batch normalisation also stores 2 176 non-trainable running statistics, so a checkpoint holds $3\,358\,763+2\,176=3\,360\,939$ numbers, 13.4 MB as 32-bit floating point.

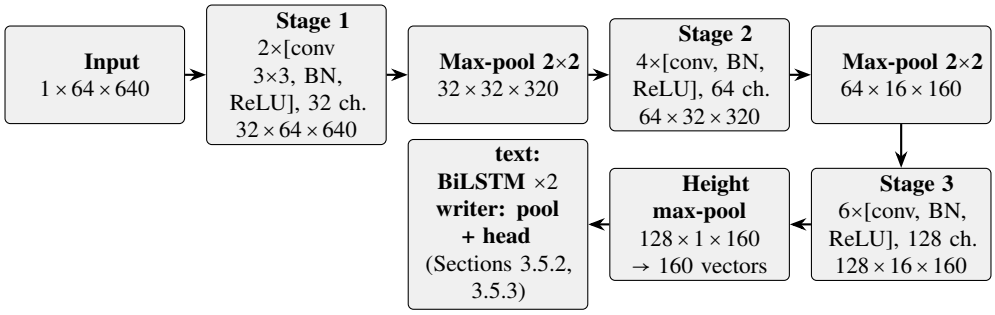


Figure 19. The shared convolutional backbone, the layer pattern being that of Kizilirmak and Yanikoglu [?, p. 9]. Tensor shapes are channels × height × width; BN denotes batch normalisation. The two height-collapsed feature sequences feed the text recogniser's recurrent stage and the writer classifier's pooling head respectively.

centred, inverted and normalised so paper is −1 and ink +1 (Equations 49, 50):

$$s=\min\!\Big(\frac{64}{h},\frac{640}{w}\Big),\qquad w'=\max\big(1,\operatorname{round}(ws)\big),\qquad h'=\max\big(1,\operatorname{round}(hs)\big),\tag{49}$$

$$x=\frac{(1-p/255)-0.5}{0.5}=1-\frac{2p}{255}.\tag{50}$$

A single scale factor on both axes avoids distorting letter proportions, a plain stretch having first been tried and rejected for exactly that reason; the left flush makes the map from a network column back to a crop pixel the exact inverse $x_{\text{orig}}=x_{\text{canvas}}/s$, which Section 3.6 depends on.

Before the eval-mode form was trusted on the single-board computer it was cross-checked against the training-mode forward pass on the same weights with statistics frozen: over 1358 real lines the top-1 agreement was 100.00 % with a maximum absolute logit difference of 7.6×10^{-6}, confirming Equation 47's folded form reproduces Equation 46 to within numerical noise.

3.5.2 Sequence modelling and text recognition by CTC

The input is one grey-scale line image; the output is the string that was written. Three properties of handwriting fix the design: cursive characters are joined, so their boundaries cannot be located reliably and cutting the line into characters, as the proposal planned, would pass every cutting error into a wrong reading; the number of characters is unknown and has no fixed relation to image width; and the available training material gives line images with transcriptions, but not the position of every character. The convolutional backbone gives one feature vector per narrow vertical slice of the line, via the height max-pool $z_t[c] = \max_i F[c,i,t]$, $t = 1,\dots,T = 160$, a maximum because the question is only whether a feature occurs anywhere in that column, which should answer the same way for an ascender and a short letter. A plain recurrent network multiplies by the same weight matrix every step, so a gradient over a long sequence either shrinks away or grows without bound; a long short-term memory cell [**?**] avoids this by carrying a cell state forward by addition, $c_t = f_t \odot c_{t-1} + i_t \odot g_t$, through three learned gates, so a gradient can cross many steps without vanishing. A *bidirectional* layer [**?**] runs a second pass over the reversed sequence and concatenates both outputs, necessary for joined handwriting where strokes such as those of u, n and m are nearly identical locally and can only be told apart from what follows.

This bidirectional-recurrent, alignment-free combination is the one that established accurate unconstrained handwriting recognition from whole lines rather than cut characters [**?**, p. 855]. Connectionist temporal classification [**?**, p. 370] is what makes training possible from line transcriptions alone, with no character-level labels: rather than scoring the single alignment that happens to be correct, it sums the probability of *every* alignment (*path*) that would produce the right transcription, using a 75th *blank* symbol so that a genuine doubled letter is not merged away by the path-collapse rule.

The recurrent output at each step is turned into a distribution over the symbol set by a linear layer and a log-softmax,

$$\ell_t = W_o h_t + b_o, \qquad \log y_t^k = \ell_t^k - \log \sum_{j=0}^{74} e^{\ell_t^j}, \tag{51}$$

where h_t is the recurrent output at step t, W_o and b_o the output weights and bias, ℓ_t the logits, and y_t^k the probability of symbol k at step t. Equation 51 is evaluated in log form, subtracting the log-sum-exp rather than dividing by the sum, because the probabilities are multiplied over 160 steps and would underflow in a linear representation.

The probability of a transcription is then the summed probability of every path that collapses onto it,

$$P(\pi \mid x) = \prod_{t=1}^{T} y_t^{\pi_t}, \qquad p(y \mid x) = \sum_{\pi:\mathcal{B}(\pi)=y} P(\pi \mid x), \qquad \mathcal{L}_{\mathrm{CTC}} = -\ln p(y \mid x), \tag{52}$$

where π is one path through the lattice, x the input line, y the transcription, $\mathcal{B}$ the collapse map that merges repeats and deletes blanks so that $\mathcal{B}(\text{–hh-e-ll-ll-o}) = \text{hello}$, and $\mathcal{L}_{\mathrm{CTC}}$ the training loss. The number of paths grows exponentially with T, so the sum in Equation 52 is evaluated by a forward–backward dynamic programme [**?**, p. 372] costing $O(TU)$ for a transcription of U characters, which also yields the training gradient in closed form as the difference between what the network predicted and what the alignment lattice expects.

The recursion runs on the blank-augmented label y' of length $S = 2U + 1$. The forward variable

$\alpha_t(s)$ is the total probability of consistent prefixes of length t ending at position s and obeys

$$\alpha_t(s) = \Big[\alpha_{t-1}(s) + \alpha_{t-1}(s-1) + \mathbb{1}\!\left[y'_s \neq \varnothing \wedge y'_s \neq y'_{s-2}\right]\alpha_{t-1}(s-2)\Big]\, y_t^{y'_s}, \tag{53}$$

whose three terms are the only transitions into s — stay, advance, or skip a blank — the skip being forbidden when y'_s is blank or equals y'_{s-2}, which is what allows a genuine double letter to survive the merge. Carried to the last step, Equation 53 gives $p(y \mid x) = \alpha_T(S) + \alpha_T(S-1)$ at $O(TS)$ cost. The backward variable $\beta_t(s)$ is its mirror image and excludes the emission at step t, so that the product $\alpha_t(s)\beta_t(s)$ counts that emission exactly once; the occupancy and the closed-form gradient with respect to the pre-softmax scores are then

$$\gamma_t(s) = \frac{\alpha_t(s)\,\beta_t(s)}{p(y \mid x)}, \qquad \mathrm{occ}_t^k = \sum_{s:\,y'_s = k} \gamma_t(s), \qquad \frac{\partial \mathcal{L}}{\partial \ell_t^k} = y_t^k - \mathrm{occ}_t^k. \tag{54}$$

Equation 54 is what makes CTC cheap to train: the gradient is the difference between what the network predicted and what the alignment lattice expects, so no explicit differentiation of the recursion is needed.

Both recursions run in the logarithmic domain, because a product of up to 160 probabilities underflows 64-bit arithmetic. Additions therefore become

$$\mathrm{LSE}(a,b) = m + \ln\!\left(e^{a-m} + e^{b-m}\right), \qquad m = \max(a,b), \qquad \mathrm{LSE}(-\infty,-\infty) = -\infty, \tag{55}$$

with a forbidden transition carried as $-\infty$; the last case of Equation 55 must be special-cased, since the general expression would evaluate $\infty - \infty$. Algorithm 9 states the recursion and the gradient together.

Algorithm 9 The CTC forward recursion and the loss gradient

Require: log-probabilities $\mathrm{lp}[t,k]$ for T steps and 75 symbols; label y of length U
1: $y' \leftarrow$ blank, y_1, blank, $y_2, \ldots, y_U$, blank $\{S = 2U+1\}$
2: $\log\alpha[1,\cdot] \leftarrow -\infty$ everywhere except the first two positions
3: **for** $t = 2$ **to** T **do**
4: **for** every position s of y' simultaneously **do**
5: stay $\leftarrow \log\alpha[t-1,s]$; advance $\leftarrow \log\alpha[t-1,s-1]$; skip $\leftarrow \log\alpha[t-1,s-2]$
6: **if** y'_s is blank **or** $y'_s = y'_{s-2}$ **then**
7: skip $\leftarrow -\infty$ {the forbidden transition of Equation 53}
8: **end if**
9: $\log\alpha[t,s] \leftarrow \mathrm{LSE}(\text{stay}, \text{advance}, \text{skip}) + \mathrm{lp}[t, y'_s]$
10: **end for**
11: **end for**
12: $\log\beta \leftarrow$ the same recursion run backwards from $t = T$, excluding the emission at t
13: $\log P \leftarrow \mathrm{LSE}\big(\log\alpha[T,S],\ \log\alpha[T,S-1]\big)$
14: $\gamma \leftarrow \exp(\log\alpha + \log\beta - \log P)$
15: $\mathrm{occ}[t,k] \leftarrow \sum_{s:\,y'_s=k} \gamma[t,s]$ {Equation 54}
16: **return** loss $-\log P$ and gradient probabilities $-$ occ

The three shifted reads are evaluated for all S positions at once rather than in a loop over s. Three conventions of the implementation matter for the losses quoted below: each line contributes its whole negative log-likelihood and the loss routine divides only by the batch size, so a quoted loss is a mean *per line* and not per character; lines with no valid path are given zero loss; and every line uses the full $T = 160$ steps including the padding.

At inference the alignment no longer matters and the most probable symbol per step is taken directly,

$$\hat{\pi}_t = \arg\max_k y_t^k, \qquad \hat{y} = \mathcal{B}(\hat{\pi}), \tag{56}$$

where $\hat{\pi}$ is the best path and $\hat{y}$ the string read back. Equation 56 is best-path, or *greedy*, decoding, used here with no lexicon, language model or beam search, so nothing in the system corrects a misspelling; every error reported in Section 4 is the network's own.

Algorithm 10 Greedy decoding: the collapse map $\mathcal{B}$ of Equation 56

```
best ← arg max over the 75 symbols at each of the 160 time steps
previous ← none;  text ← empty
for each token in best do
    if token is not the blank and token ≠ previous then
        append the symbol of token to text
    end if
    previous ← token
end for
return text
```

The 74 symbols are the 26 lower-case and 26 upper-case letters, the ten digits and twelve symbols (space and `. , ; : ' " ! ? ( ) −`), with index 0 reserved for the CTC blank, which is why the output layer has 75 classes. Characters outside the set are removed from every label, lines that become empty are dropped, and lines over 120 characters, which indicate two merged lines, are dropped.

The 74 symbols are the 26 lower- and upper-case letters, the ten digits and twelve punctuation symbols; class 0 is the blank. Training minimises the CTC loss by mini-batch RMSprop [**?**, Lec. 6e] with weight decay, gradient-norm clipping at 5 [**?**, Alg. 1] against the exploding gradients a long recurrent unroll can produce [**?**], and a plateau learning-rate schedule, in the four stages of Table 16, each motivated by a limitation of the previous one: the base stage reproduced the published recipe on lines cut by this project's own tools; the clean-label stage warm-started from it on the public set; fine-tuning on the personal lines alone was rejected because it cut general accuracy by about ten points, catastrophic forgetting barely reduced by freezing the convolutions; and the joint stage therefore continues on both sets at once with a weighted sampler giving each personal line weight f/n_P and each public line $(1-f)/n_T$, $f = 0.20$.

Augmentation, distorting each training image slightly and differently every time it is used so the network cannot memorise it, applies at most one of shear, rotation, elastic distortion or perspective distortion per training image, never to validation or test images.

Table 17 gives the measured accuracy. On the public test partition (2 915 lines, used for no decision) the recogniser reaches 91.82 % character accuracy; on 669 correctly cut personal crops, 98.61 % character and 94.21 % word accuracy; through the complete page-to-text pipeline including segmentation, 61.21 %, the largest discrepancy in this report and only partly attributable to recognition itself, since the two figures used different recogniser versions (Section 3.5.2). Recognition speed measured 8.86 ms per character over 669 real crops, inside the 25 ms per character of R4 by about 2.8 times on the development laptop; timing on the ODROID N2+ was not measured. **[MUST STILL BE COMPLETED BY ME: measure recognition time per line on the ODROID N2+.]**

The recognition head carries two limitations that bear directly on how its fig-

Stage	Data	Schedule and selection	Checkpoint and character accuracy
1 Base	own lines	$\eta_0 = 10^{-3}$, batch 16, halve after 5 stalls; best validation loss	best validation loss 1.06 per line
2 Clean labels	public set	as above, up to 8 restarts; best public validation loss	`hf_best`: 93.87 / 91.60 % public validation / test
3 Personal (rejected)	personal lines	$\eta_0 = 3 \times 10^{-4}$, 60 epochs; best personal validation loss	`personal_best`: 84.26 % public, 90.2 % personal
4 Joint	both, $f = 0.20$	$\eta_0 = 5 \times 10^{-4}$, 30 epochs, clip 5; best min of both accuracies	`joint_best`: 94.03 / 91.82 % public, 89.64 % personal

Table 16. The four training stages, with the character accuracies measured on each stage's own checkpoint. All stages use RMSprop with weight decay 10^{-5}. None of these figures was measured on `paper_cnn_bilstm_ctc_scratch_best.npz`, the file the deployed loader actually prefers (Section 3.5.2).

Data set	Result	Status of the figure
Public set, test (2 915 lines)	91.82 % character; WER 26.7 %	Used for no decision; the cleanest general-benchmark figure.
Personal crops, 669 correctly cut lines	98.61 % character, 94.21 % word	Writers appear in fine-tuning data; the operating figure of Section 4.
Full page pipeline, ten personal pages	61.21 % (CER 38.79 %)	Only figure exercising the complete page-to-text chain; earlier recogniser version.
Recognition speed, 669 crops	8.86 ms/character, 353 ms/line	Margin of about 2.8× against R4 on the laptop; not measured on the board.

Table 17. Accuracy and speed of the text recogniser. Appendix 6.4 gives the full per-writer breakdown.

ures should be read. Four text checkpoints exist; the deployed loader prefers `paper_cnn_bilstm_ctc_scratch_best.npz`, the output of the from-scratch trainer of Section 3.5.5, whose accuracy was never logged, so no figure in this report is measured on the exact file the deployed system uses — the figures above describe the joint checkpoint. **[MUST STILL BE COMPLETED BY ME: evaluate the scratch checkpoint and quote its figures beside the joint ones, or name the single checkpoint being submitted.]** Correcting the evaluation labels alone raised an early ten-author hold-out accuracy from 42.7 % to 89.4 %, showing label quality rather than the network limited the earliest measurements. The CTC routine does not survive an empty label sequence, a confined indexing fault rather than an algorithmic one.

3.5.3 Writer-identification head and stroke normalisation

The task is closed-set: choose one of $K = 13$ known writers from a line image. A classifier trained on raw grey-scale crops read real held-out lines at 99.78 % over 1358 lines but identified the writer of *synthesised* output in only about 28 % of trials, because the plotter draws every stroke at one width and one pressure while real handwriting carries a thickness and ink density a convolutional network readily exploits when training lines are few, and such a judge predicts neither what the machine can draw nor how a reader recognises a hand, which is by geometry. A second classifier was therefore trained on *stroke-normalised* crops, in which thickness is removed by construction. Both are retained, and which one serves which input is itself a design

decision that was revised once a page was photographed under conditions unlike the training pages: the shape classifier now names *both* real photographed pages and the best-of-N search of Section 3.6, the ink classifier being kept because it remains the more accurate of the two on in-domain photographs (Table 30). Section 3.7.5 gives the measurement that forced the revision and the confidence it costs.

The writer-identification head differs from the recogniser's in one deliberate way: the recurrent layers are not used, because a writer's slant and stroke habits do not depend on the order the letters come in, so adding sequence information would only cost parameters.

The head pools the backbone's feature map twice, deliberately with two different operations,

$$m_{c,t} = \max_{r=1,\dots,16} F_{c,r,t}, \qquad f_c = \frac{1}{160}\sum_{t=1}^{160} m_{c,t}, \qquad f \in \mathbb{R}^{128}, \tag{57}$$

where $F_{c,r,t}$ is the backbone feature map at channel c, row r and column t, $m_{c,t}$ the height-pooled map and f_c the final descriptor for channel c. The two operations in Equation 57 answer two different questions. The height *maximum* asks whether a stroke pattern occurs anywhere in a column, as in the recogniser. The width *mean* asks how *often* it occurs along the whole line, which is the right question for a habit rather than a one-off accident; a maximum there would answer only whether the writer ever made the stroke once.

The descriptor is then mapped to one score per writer by two linear layers,

$$h = \mathrm{ReLU}(W_1 f + b_1) \in \mathbb{R}^{96}, \qquad z = W_2\,\mathrm{Drop}_{0.2}(h) + b_2 \in \mathbb{R}^{K}, \tag{58}$$

where W_1, b_1, W_2, b_2 are the learned weights and biases, h the hidden activation, $\mathrm{Drop}_{0.2}$ dropout at rate 0.2 and z the K logits. The dropout in Equation 58 [**?**, p. 1930] is active only during training, and stops the small head memorising individual lines when only 63 to 184 lines exist per writer. Training uses the cross-entropy loss, Equation 59, whose gradient with respect to the logits is the simple form of Equation 60:

$$p_k = \frac{e^{z_k}}{\sum_{j=1}^{K} e^{z_j}}, \qquad \mathcal{L} = \frac{1}{B}\sum_{n=1}^{B}\Big[\log\sum_{j} e^{z_j^{(n)}} - z_{y_n}^{(n)}\Big], \qquad \hat{k} = \arg\max_k p_k, \tag{59}$$

$$\frac{\partial \mathcal{L}}{\partial z_k^{(n)}} = \frac{1}{B}\big(p_k^{(n)} - \mathbb{1}[k = y_n]\big). \tag{60}$$

Stroke normalisation re-draws a line at a constant pen width proportional to its own letter height, by Otsu binarisation [**?**, p. 63], rule removal, the core-band x-height estimate of Equation 61, closing of hairline breaks, Zhang–Suen thinning [**?**], and re-inking at a width $w = \mathrm{clip}(\mathrm{round}(0.14\,x_h), 2, 9)$ independent of scan resolution:

$$H_{\mathrm{run}} \geq \max(24, \lfloor 1.1h \rfloor) \;\wedge\; V_{\mathrm{run}} \leq \max(3, \lfloor 0.12h \rfloor), \qquad x_h = \mathrm{base} - \mathrm{top}. \tag{61}$$

`Skeletonize` is the Zhang–Suen thinning [**?**] and `Dilate` the re-inking at the constant width. Because the width is a fixed fraction of the *measured* x-height, the result is independent of scan resolution: an x-height of 20 pixels gives a three-pixel pen whatever the camera.

The two classifiers hold 966 445 parameters each, the ink classifier trained end to end from the recogniser backbone, the shape classifier also trained end to end after an earlier frozen-backbone configuration proved a training fault (Section 3.7.2). On a real photograph the ink classifier's line probabilities are averaged across the page, Equation 62, with no weighting by length or

Algorithm 11 Stroke normalisation: removing the pen-width cue by construction

Require: grey crop; pen width as a fraction of x-height, 0.14

ink ← Otsu binarisation, ruled lines removed, hairlines closed, components under 6 pixels dropped
band ← first to last row whose ink count reaches 0.45 of the maximum {the core band of Equation 61}
x_h ← height of that band
if $x_h < 4$ **then**
    $x_h \leftarrow \max(8,\ \text{crop height}/3)$ {fallback}
end if
$w \leftarrow \text{round}(0.14\,x_h)$, clipped to $[2, 9]$; $k \leftarrow w$ made odd
return $\text{Dilate}(\text{Skeletonize}(\text{ink}), k)$ {a sample with no ink left is dropped, not classified}

confidence:

$$\bar{p}_k = \frac{1}{N}\sum_{n=1}^{N} p_k^{(n)}, \qquad \hat{k} = \arg\max_k \bar{p}_k, \qquad \text{confidence} = 100\,\bar{p}_{\hat{k}}\ \%. \tag{62}$$

For synthesised output the shape classifier's probability for the target writer becomes one term of the best-of-N score of Section 3.6.6 instead.

Table 18 shows how little of a classifier is new: both reuse the recogniser's convolutional backbone and replace its recurrent stack and output head with two pools and two fully connected layers, so almost all of the capacity is shared rather than learned afresh.

Block	Output shape	Trainable parameters
Convolutional backbone, 12 layers as Table 15	$128 \times 16 \times 160$	952 800
Height max-pool, width mean-pool (Eq. 57)	128	0
FC 128→96, ReLU, dropout 0.2 (Eq. 58)	96	12 384
FC 96→13	13	1 261
Total (head 13 645)		**966 445**

Table 18. What the writer classifiers add to the recogniser's backbone, FC denoting a fully connected layer. Every backbone shape and parameter count is that of Table 15, the recurrent stack and the CTC head being replaced by the two pools and the two layers above; the shapes are identical but none of the 84 backbone tensors hold the same values. Batch normalisation adds 2 176 non-trainable running statistics, so the network holds 968 621 numbers. A forward pass needs about 3.79×10^9 multiply–accumulate operations and the parameters occupy about 3.9 MB as 32-bit floating-point numbers.

The ratio in Table 18 is what makes the training fault of Section 3.7.2 consequential: fitting the head alone fits 13 645 of 966 445 parameters, that is **1.4 % of the model**, on features that a backbone trained for a different task produced. Algorithm 12 is the deployed forward pass, which is the same code the trainer runs for validation, so no separate inference implementation had to be verified.

The recurrent layers of the recogniser are absent, because writer identity does not depend on the letter sequence. Taking a maximum over height and a mean over width makes the feature a position-independent bag of local stroke patterns. The weights come from a plain array archive written by the trainer, with no third-party model format parsed.

Table 19 gives the measurements that motivated the design; chance level for thirteen writers

Algorithm 12 Deployed forward pass of both writer classifiers

Require: a batch of line images

1: $x \leftarrow$ convolutional stage 1, then 2×2 max-pool
2: $x \leftarrow$ convolutional stage 2, then 2×2 max-pool
3: $x \leftarrow$ convolutional stage 3 $\{(B, 128, 16, 160)\}$
4: $x \leftarrow$ maximum over the height axis $\{(B, 128, 160)$, Equation 57$\}$
5: $f \leftarrow$ mean over the width axis $\{(B, 128)\}$
6: $h \leftarrow \mathrm{ReLU}(\mathrm{linear}(f))$ $\{(B, 96)$; the dropout layer is inactive here$\}$
7: **return** $\mathrm{linear}(h)$ $\{(B, 13)$ raw scores, Equation 58$\}$

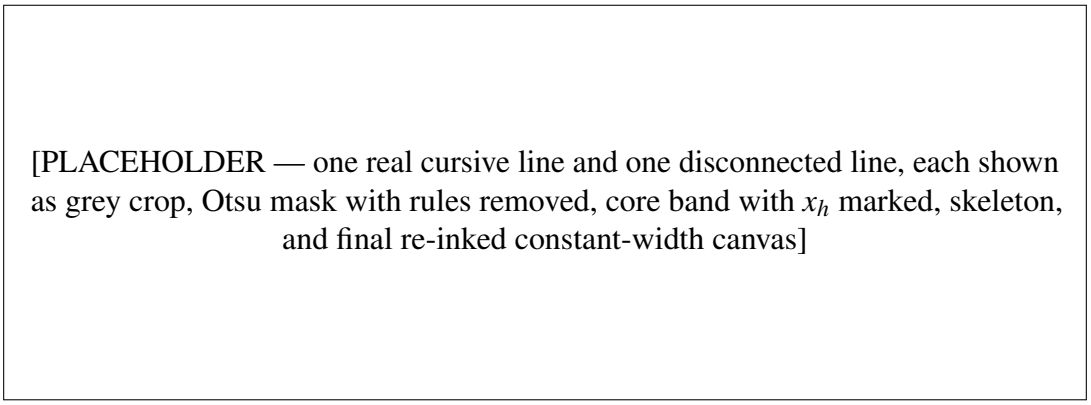


Figure 20. Placeholder: the six steps of stroke normalisation (`StrokeNormalize`) applied to one database line and one personal line, showing slant, proportions and connection habits surviving while pen thickness is removed, for a disconnected and a strongly cursive hand side by side.

is 7.7 %. The ink classifier reaches 99.78 % over 1358 real lines; the shape classifier, after the end-to-end correction of Section 3.7.2, 93.67 % on the same lines, up from 83.36 % under the superseded frozen-backbone recipe, where 118 of one writer's 148 lines had gone to a single neighbour.

Measurement	Result	Status of the figure
Ink classifier, 1358 real lines	99.78 %	The live server's classifier for photographs.
Shape classifier, same lines, end-to-end	93.67 %	Corrects the frozen-backbone fault of Section 3.7.2.
Ink classifier on synthesised output	28 % over 39 lines	Cannot judge constant-width output; the reason the shape classifier exists.
Shape classifier on synthesised output	89.7 % at best-of-8 through G-code	Circular: the same classifier family selects the candidates (Section 3.6.8).

Table 19. Evidence for the writer classifiers. Chance level for thirteen writers is 7.7 %, and none of these are held-out test accuracies in the strict sense of Section 3.5.8.

A single network with a shared text head and writer head was also tried: writer accuracy reached 98.9 to 100 % by epoch 20 while text accuracy stalled at 28 to 29 %, so separate single-task networks were adopted instead.

3.5.4 The data sets and their leakage assessment

Every accuracy quoted above and in Section 4 belongs to one of four data sets, each with a different split rule and a different reason why its figure may be optimistic. This sub-subsection comes before the training recipe deliberately: a training decision made on a leaky validation set is a different decision, and Table 20 is what bounds how far any figure in this report may be read.

Data set	Split rule	Caveat on the validation figure
Lines segmented by this project's own block from scanned database pages, labels generated by alignment (first stage only)	The last labelled page of every writer is held out.	Same writers in training and hold-out, so the figure is writer-dependent, and the labels are project-generated and noisy.
Public line-level IAM transcription set: 6 482 training, 976 validation, 2 915 test lines (6 481, 976, 2 915 after label cleaning)	Official training, validation and test partitions.	The validation set drives learning-rate decay and checkpoint choice, so its accuracy is mildly optimistic; the test set is used for no decision. The partitions are believed, but not verified, to be writer-independent, and the warm start may have seen some of the same forms.
13 photographed pages of the two personal writers (8 and 5 pages), 355 transcription lines of which one is a diagram marker. **[MUST STILL BE COMPLETED BY ME: confirm which personal set the deployed recogniser was fine-tuned on — this row's two writers, 13 pages and 355 lines, or the five writers, 30 pages and 669 lines that Section 4 reports on — and correct this row and the one below to match; the saved checkpoints do not say.]**	Per page, 15 % of the lines (at least one) drawn at random with a fixed seed; about 52 validation lines in total.	Validation lines come from the same pages, pen, paper, lighting and vocabulary as the training lines, so the figure measures same writer, same notebook, new lines, and is optimistic. It also takes part in checkpoint selection.
Ten-author hold-out (eight database writers, two personal)	Evaluation set only.	The held-out database lines may have been seen in the line-level training data and their labels are project-generated, so this is not a writer-independent measure.

Table 20. Data sets of the text recogniser and their leakage assessment. No figure in the report is measured on a split that is both writer-independent and labelled by hand; the public test partition is the closest, and it is the only set used for no decision.

The labels of the database pages were generated by aligning the printed prompt words with the detected ink word runs by dynamic programming. An earlier proportional labelling assigned wrong text to lines, and correcting the evaluation labels alone raised the measured ten-author hold-out accuracy from **42.7 % to 89.4 %**, which establishes that label quality, not the network, had limited the earliest measurements; that is the reason the recogniser was later trained on a public line-level set with clean labels [**?**].

Training images only are augmented after the resize with at most one of four geometric transform-

ations, filled with white: a horizontal shear with $k \sim U(-0.6, 0.6)$, a rotation in $U(-2.5°, 2.5°)$, an elastic distortion with displacement fields smoothed at $\sigma \in \{3, 4\}$ and scaled by $\alpha \in \{15, 20\}$, and a perspective distortion moving each corner by up to $dW/2$ and $dH/2$ with $d \sim U(0.03, 0.08)$. Each is enabled independently with probability 0.5 and one of the enabled ones is then chosen uniformly, so that $P(\text{no augmentation}) = 1/16 = 6.25\,\%$. No colour, contrast, noise or blur augmentation and no synthetic pre-training data are used.

3.5.5 Training methodology, the optimisers and the from-scratch gradient check

With no automatic-differentiation library available, every layer's backward pass is the standard published gradient result for that layer type, but each had to be written out and evaluated by code written for this project rather than supplied by a framework. Two places in that implementation carry real risk independent of the algebra: the col2im scatter-add of the convolution backward pass, which must accumulate because one input pixel appears in up to nine overlapping patches, and the reversed direction of the bidirectional recurrent layer, whose incoming gradient must be reversed and whose returned gradient reversed back. Both would be wrong silently and would not disturb a falling training loss.

Every backward pass was verified against a central-difference numerical estimate of the same gradient, Equation 63, accepted only when the relative difference stayed below 10^{-6}:

$$\frac{\partial f}{\partial \theta_i} \approx \frac{f(\theta + \varepsilon e_i) - f(\theta - \varepsilon e_i)}{2\varepsilon}, \qquad \text{reldiff} = \frac{|g_{\text{analytic}} - g_{\text{numeric}}|}{\max(|g_{\text{analytic}}|, |g_{\text{numeric}}|, 10^{-8})}. \tag{63}$$

Of 78 configurations of layer variant and tensor, **77 passed**, relative differences between 4.9×10^{-12} and 1.4×10^{-10}, four to six orders of magnitude inside the tolerance, over 2044 individual tensor elements probed in double precision. The one failure is the CTC empty-label fault of Section 3.5.2, confined to lattice indexing rather than the transition rule or the gradient.

The gradients themselves. With $\mathrm{d}A$ denoting $\partial \mathcal{L}/\partial A$, the gradients are the convolution's (Algorithm 13), batch normalisation's [**?**, Alg. 1], the rectifier's [**?**, p. 807], dropout's [**?**, p. 1930] and the max-pool routing's, Equation 64, the adaptive pools', Equation 65, the linear layer's, Equation 66, the bidirectional LSTM's [**?**] and the two losses', Equation 54 for CTC [**?**, p. 372] and Equation 60 for cross-entropy.

$$\mathrm{d}x = \mathrm{d}Y \odot \mathbb{1}[x > 0] \;\; \text{(rectifier)}, \qquad \mathrm{d}x = \mathrm{d}Y \odot M, \quad M_k = \frac{\mathbb{1}[\xi_k \geq p]}{1 - p} \;\; \text{(dropout)}, \tag{64}$$

$$\mathrm{d}F[n, c, i, t] = \mathbb{1}\big[\, i = r^\star(n, c, t) \,\big]\, \mathrm{d}z_t[n, c], \qquad \mathrm{d}m[n, c, t] = \frac{1}{T}\, \mathrm{d}f[n, c], \tag{65}$$

$$\mathrm{d}W = \mathrm{d}Y^\top X, \qquad \mathrm{d}b = \sum_{m=1}^{M} \mathrm{d}Y[m, \cdot], \qquad \mathrm{d}X = \mathrm{d}Y\, W. \tag{66}$$

In Equation 65 $r^\star$ is the winning row cached by the height maximum, so a feature that is never the column maximum receives nothing, and batch normalisation's input gradient couples all $N = BHW$ elements of a channel. Each of these is an exact derivative of a forward equation above, which makes the check that follows a check of the *algebra* rather than a measurement of an error.

Two places where the implementation, not the algebra, is the risk. Both would fail silently. The col2im step must accumulate, because for a 3×3 kernel one input pixel appears in up to

Algorithm 13 The convolution and its gradient; the col2im step must accumulate

forward:
  cols ← every 3×3 patch of the zero-padded input, as a column
  out ← reshape$(W \cdot \text{cols}) + \text{bias}$ {one product: Equation 45}
backward, given dY:
  dW ← d$Y \cdot \text{cols}^{\top}$, summed over the batch
  dBias ← dY, summed over batch, height and width
  dCols ← $W^{\top} \cdot \text{d}Y$
  dInput ← $\mathbf{0}$
for $i = 1$ **to** 3 **do**
  for $j = 1$ **to** 3 **do**
    dInput$[:,:,i..i{+}H,\ j..j{+}W]$ += dCols$[:,:,i,j,:,:]$ {col2im **must** accumulate}
  end for
end for

nine columns, so it is nine slice additions into a zero array; a reshape alone would keep one contribution in nine and is the likeliest place for a convolution backward pass to be quietly wrong. In the bidirectional layer the backward direction runs on the reversed sequence, so its recursion must be driven by the reversed incoming gradient and its returned input gradient reversed back; omitting either yields a gradient wrong only in time ordering, which a falling loss curve would hide. Algorithm 14 shows where the two sit in one training step, and the commented line is the single structural decision of the architecture: the point at which an image problem becomes a sequence problem.

Algorithm 14 One training step and the forward pass it calls

for each shuffled mini-batch of line images and target strings **do**
  logits, cache ← **Forward**(images)
  loss, dLogits ← **CtcLoss**(logits, targets) {Algorithm 9}
  gradients ← **Backward**(dLogits, cache)
  Optimiser.Step(gradients)
end for

Forward(images):
  features ← convolutional backbone $\{(B, 128, 16, 160)\}$
  pooled ← maximum over the height axis $\{(B, 128, 1, 160)\}$
  sequence ← move the width axis to the front $\{(160, B, 128)$ — the image becomes a sequence here$\}$
  sequence ← two stacked bidirectional LSTM layers
  return linear projection to 75 classes, then log-softmax

Every routine named in Algorithm 14 is one written for this project; nothing in the loop comes from a machine-learning framework. A recurrent layer is evaluated one step at a time, and its four gates are read as four blocks of rows out of a *single* matrix product rather than computed separately, which halves the number of products per step; the tuple cached at each step is exactly what the backward pass needs, so no activation is recomputed when the gradient is propagated back through time.

The optimisers. The recogniser uses RMSprop [**?**, Lec. 6e]; both writer classifiers use AdamW, the decoupled-decay form [**?**, Alg. 2] of Adam [**?**, Alg. 1], with cosine annealing. The moment estimates and their bias corrections are

$$m_s = \beta_1 m_{s-1} + (1-\beta_1) g_s, \qquad \hat{m} = \frac{m_s}{1-\beta_1^{\,s}},$$
$$v_s = \beta_2 v_{s-1} + (1-\beta_2) g_s^2, \qquad \hat{v} = \frac{v_s}{1-\beta_2^{\,s}}, \tag{67}$$

and the parameter update with its schedule is

$$\theta \leftarrow \theta - \eta\Big(\frac{\hat{m}}{\sqrt{\hat{v}}+\varepsilon} + \lambda_{\mathrm{d}}\theta\Big), \qquad \eta_t = \frac{\eta_0}{2}\Big(1+\cos\frac{\pi t}{T_{\max}}\Big), \tag{68}$$

with g_s the gradient at step s, $\beta_1 = 0.9$, $\beta_2 = 0.999$, $\varepsilon = 10^{-8}$, decay $\lambda_{\mathrm{d}} = 10^{-4}$ and the rate falling from η_0 to zero over $T_{\max}$ epochs. The bias correction of Equation 67 matters most in the first few dozen steps, where m_s and v_s are still dominated by their zero initial values and an uncorrected update would be far too small; the decoupled decay of Equation 68 is what distinguishes AdamW from Adam with an L2 penalty, since adding the penalty to g_s instead would let the adaptive denominator scale it differently for every parameter. Weight decay therefore sits in the update rather than the gradient, and the gradient-norm clip is applied *before* the moment update so one clipped batch cannot contaminate running averages that decay over many subsequent steps. The ink classifier ran at $\eta_0 = 10^{-3}$ for 20 epochs on batches of 16 images with the clip at 5; the superseded frozen-head run used $\eta_0 = 3\times10^{-3}$, 500 epochs, batches of 64 feature vectors, no clipping and only the head parameters on 3 680 stored vectors, and is the configuration that reached 83.36 %.

The verification, and its one failure. Each backward pass was verified against a central-difference estimate,

$$\frac{\partial \mathcal{L}}{\partial \theta_i} \approx \frac{\mathcal{L}(\theta+\varepsilon e_i) - \mathcal{L}(\theta-\varepsilon e_i)}{2\varepsilon}, \qquad \varepsilon = 10^{-5}, \tag{69}$$

$$\frac{\max_i |g_i^{\mathrm{a}} - g_i^{\mathrm{n}}|}{\max\big(1,\ \max_i|g_i^{\mathrm{a}}| + \max_i|g_i^{\mathrm{n}}|\big)} < 10^{-6}, \tag{70}$$

a configuration being accepted when the relative difference of Equation 70 stays below 10^{-6} for the analytic gradient g^{a} against the numerical one g^{n}. A configuration is one pair of layer variant and tensor, and the scalar loss is $\mathcal{L} = \sum_j o_j R_j$ for fixed random R, so that $\partial\mathcal{L}/\partial o = R$ exactly. All arithmetic is in double precision and up to 40 random elements of each tensor are probed, 2044 in total; on the stricter per-element measure the worst case over all 2044 is 2.0×10^{-7}, on a recurrent LSTM weight whose gradient is itself near zero, where the central difference has no significant digits left. Max-pooling and the rectifier were given inputs from a permuted linear spacing, so that no element lies within ε of the kink; dropout was checked with the mask fixed; batch normalisation was checked in its training form, so the folded inference map is not relied upon; and the check runs against the batched form the trainer uses.

The one configuration that failed is a genuine defect, reported here rather than removed. The CTC loss does not survive an empty label sequence: for $U = 0$ the blank-interleaved target has $S = 1$ entry, but two of the shifted arrays carrying the recursion are built with two entries regardless of S, so the routine writes a two-element array into a one-element row and raises a shape error instead of returning the loss. Those arrays implement the $s-1$ and $s-2$ terms of Equation 53, so the fault is confined to the lattice indexing and touches neither the transition rule nor the gradient of Equation 54; a single-character target and a repeated-character target, which exercises the

forbidden skip, both pass. Its practical effect is small, because a line with no transcription never reaches the trainer, but the function is not safe against one. The check also confirmed that with no possible alignment, $2U + 1 > T$, the loss is infinite with undefined gradients, so such samples must be removed before training.

One shared training module builds either the recogniser or a writer classifier from the same layer functions, and both trainers write every tensor into one plain array archive keyed by layer name.

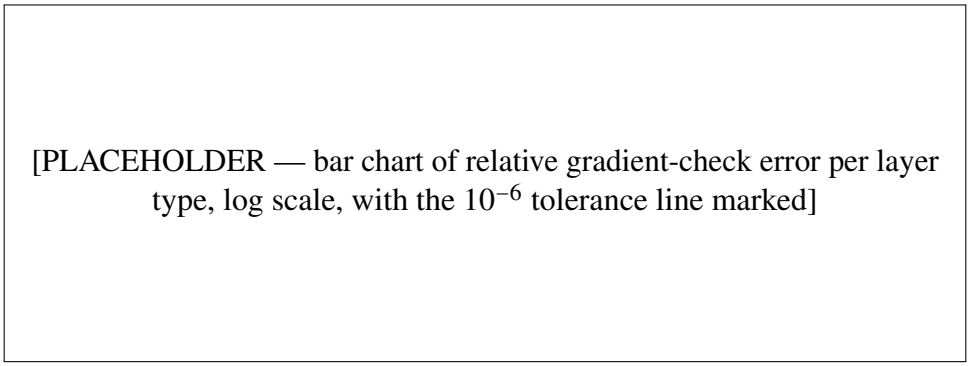


Figure 21. Placeholder: the 78 gradient-check configurations of Equation 63, relative error on a log scale per layer type, with the acceptance threshold marked, to show the margin by which each layer's backward pass was verified.

3.5.6 Edit distance: how accuracy is measured

Every character accuracy in this subsection is one minus a micro-averaged character error rate, and the error rate is built on the Levenshtein edit distance computed by the dynamic programme

$$D[i][j] = \min\big(D[i-1][j]+1,\; D[i][j-1]+1,\; D[i-1][j-1]+\mathbb{1}[a_i \neq b_j]\big), \tag{71}$$

with $d_{\text{lev}}(a,b) = D[|a|][|b|]$, $D[i][0] = i$ and $D[0][j] = j$. The three branches of Equation 71 are a deletion, an insertion and a match or substitution, so the distance is the smallest number of single-character edits between the two strings. The error rate and the word measure are then

$$\text{CER} = \frac{\sum_n d_{\text{lev}}(\hat{y}_n, y_n)}{\sum_n |y_n|}, \qquad \text{WordAcc} = \frac{\text{LCS}(\text{reference words}, \text{predicted words})}{\#\text{reference words}}, \tag{72}$$

so the character error rate of Equation 72 is the total edit distance over the total reference length, and the measure is case-sensitive. The word measure is a longest-common-subsequence recall and *not* $1 - \text{WER}$, as Appendix 6.4 sets out at length.

One inconsistency must be stated rather than left to be discovered: Section 4 averages instead the floored per-line accuracy $\max(0, 1 - d_{\text{lev}}/|y|)$ after lower-casing, so the subsystem figures of this subsection and the qualification figures of Section 4 are *not computed identically and should not be differenced*.

3.5.7 Alternatives considered

Table 21 compares the chosen line-level CNN-BiLSTM-CTC design against the alternatives weighed for text recognition; only the chosen design was built and measured, so the statements

about the others are arguments rather than results. For writer identification, hand-crafted features such as directional ink-trace width, text-independent allograph statistics [**?**], open-set metric learning, and pen-width augmentation of the ink classifier were weighed and none built, for want of enough writers and lines to justify them over the two from-scratch classifiers actually trained.

Alternative	Argument for, and against
Character segmentation and a per-character classifier	Simple network and the proposal's plan, but needs a segmenter that works on joined letters, errors cannot be repaired by context, and needs character-level labels the available data lack.
Line-level CNN-BiLSTM-CTC (chosen) [**?**, p. 2298]	No segmentation below the line, alignment-free training from line transcriptions, context from both sides, 3.36 million parameters a from-scratch implementation can run. Against: greedy decoding leaves spelling errors uncorrected.
Lexicon or language-model decoding	Repairs near-miss words and gives the lowest error in the published design, but needs a corpus and a decoder, and this project's technical vocabulary is largely out of vocabulary. Future work.
Multidimensional recurrence or deeper backbones with skip connections	A multidimensional recurrent layer is not obviously needed once height is already pooled away by the backbone; the 12-layer network with batch normalisation trained well and larger models cost memory and time on the target hardware.
Attention encoder–decoder or transformer	Strong accuracy on large corpora, but far more parameters, large pre-training data and slow autoregressive decoding, hard to implement from scratch for the board.
Word-level recognition	Uses the lexicon directly, but needs word segmentation and a closed vocabulary, and is poor for names, numbers and symbols.

Table 21. Alternatives to the line-level recogniser.

3.5.8 Known faults and limitations

1. Neither classifier has an untouched test set: the best-validation checkpoint is also the reported one, and for personal writers training and validation lines can share a page, pen and session, which a line classifier can exploit.
2. The shape classifier's verdict on synthesised output is not independent, since the same classifier family also selects the candidates it is then used to judge (Section 3.6.8).
3. The inherited augmentation was never retuned for writer identification and distorts the slant cue it also measures.
4. Writer owen remains a systematic error for the shape classifier, 48 lost lines belonging to him alone, unsolved.
5. A sweep over width, depth, learning rate and optimiser (14 configurations) was programmed for the recogniser but no result file kept, so neither network's size is shown to be an optimum found by this project rather than the published design's own choice.

[MUST STILL BE COMPLETED BY ME: page locators outstanding in this subsection. The sources `bulacu2007`, `hochreiter1997`, `ji2021`, `mamun2024`, `marti2002`, `schuster1997`, `yahya2021`, `zhang1984` are each cited here without a page or numbered element. Open each one, find the claim it is cited for, and add the locator in the form `\cite[p. N]{key}`, following the rule of `REFERENCE_AUDIT.md`: a locator is written only after the source has been read and the claim found, never inferred from a page range or a search snippet.]

3.6 Style extraction and handwriting synthesis

This subsection describes how a writer's style is learned from photographs of that writer's pages, stored permanently as a per-writer glyph library and set of style statistics, and later

re-assembled into new text (functional units FU 2.5 to FU 2.7; requirements R1, R3, R6), component by component: cutting a joined line into characters, recovering an ordered pen path for each, filtering and summarising the result into a style profile, and finally the synthesiser that re-assembles profiles into new handwriting with a best-of-N search.

Training a writer trains no network: a *style profile* is one JavaScript Object Notation (JSON) file built once per writer, holding a library of that writer's own letter shapes (*glyphs*, each a thin centre-line pen path) plus measured slant, letter height, spacing and joining statistics, after which no page images and no networks are needed to write in that style. The builder is an offline program using the recogniser of Section 3.5 for inference only, run on ten pages each of the eight database writers (last labelled page held out) and the photographed pages of the five personal writers, 15 % of lines held out.

3.6.1 Forced alignment: cutting a joined line into characters

In joined handwriting a character has no visible boundary, and no per-character labels exist, so the position of each letter has to be inferred from the line's own transcript and a recogniser that reads lines. *Forced alignment* is the name for using a trained recogniser backwards: normally it is asked what the line says; here the answer is already known, and it is instead asked *where* it saw each character of that known answer. Where the CTC training loss *sums* the probability over every possible alignment, because for training it does not matter which was right, forced alignment takes the *maximum* instead and keeps only the single most probable path, because here the alignment is precisely what is wanted. This is the Viterbi algorithm [**?**].

With the transcript's blank-augmented form y' of length $S = 2U + 1$ and $\ell_t(k) = \ln y_t^k$ the recogniser's log-probabilities, $\delta_t(s)$ is the log-probability of the best path reaching position s at step t:

$$\begin{aligned} &\delta_1(1) = \ell_1(\varnothing), \quad \delta_1(2) = \ell_1(y'_2), \quad \delta_1(s) = -\infty \ (s > 2), \\ &\delta_t(s) = \ell_t(y'_s) + \max\left\{\delta_{t-1}(s),\ \delta_{t-1}(s-1),\ \delta_{t-1}^{\text{skip}}(s)\right\}. \end{aligned} \tag{73}$$

where $\delta_t(s)$ is the log-probability of the best path reaching position s of y' at step t, $\varnothing$ is the blank symbol and $\delta_{t-1}^{\text{skip}}(s)$ is the transition that jumps a blank between two distinct characters. Equation 73 differs from the CTC sum of Equation 52 in one respect only: it maximises where the loss sums, because the alignment itself is wanted here rather than the total probability of the transcription. A back-pointer $\psi_t(s)$ is stored alongside every maximisation so the best path can be recovered once the recursion reaches $t = T$. Algorithm 15 is this recursion stated as a procedure.

Back-pointers recovered from the maximisations give the whole best path at cost $O(TS)$. A CTC network emits a character at any single step inside its true extent, so the recovered span's *centre* c_k is reliable but its edges are not; the provisional boundary between two characters is the midpoint $\hat{x}_k = \frac{1}{2}(c_{k-1} + c_k)$, refined to the cheapest nearby column by Equation 74, using that the only place a joined letter honestly ends is a ligature, which is both *thin* (little ink in its column) and *low* (near the baseline):

$$\begin{aligned} &\text{cost}(x) = \text{thin}(x) + 1.15\,\text{top}(x), \qquad r = \max\bigl(3, \text{round}(0.5 x_h)\bigr), \\ &x^* = \arg\min_{|x - \hat{x}_k| \le r} \left[\text{cost}(x) + 0.6\,\frac{|x - \hat{x}_k|}{r}\right]. \end{aligned} \tag{74}$$

Algorithm 15 Forced alignment by Viterbi recursion (per time step t)

Require: blank-augmented transcript y' of length S; log-probabilities $\ell_t(k)$, $t = 1, \dots, T$
if $t = 1$ **then**
  $\delta_1(1) \leftarrow \ell_1(\varnothing)$; $\delta_1(2) \leftarrow \ell_1(y'_2)$; $\delta_1(s) \leftarrow -\infty$ for $s > 2$
else
  for $s = 1$ **to** S **do**
    candidates $\leftarrow \{\delta_{t-1}(s),\ \delta_{t-1}(s-1),\ \delta^{\text{skip}}_{t-1}(s)\}$ {the skip term fires only where y' allows it}
    $\delta_t(s) \leftarrow \ell_t(y'_s) + \max(\text{candidates})$; $\psi_t(s) \leftarrow \arg\max(\text{candidates})$
  end for
end if
if $t = T$ **then**
  follow ψ back from $\arg\max_s \delta_T(s)$ to recover the single best path {cost $O(TS)$ in total}
end if

A line is dropped before its letters enter the library when its mean alignment log-probability falls below the 25th percentile of that same writer's own lines, a per-writer rather than absolute threshold, since the most difficult hands would otherwise lose almost everything; this discards 22 of 88 lines for writer 153. Cuts are forced strictly increasing within one line, since snapping each boundary independently can collapse two neighbours onto the same column.

The cuts are vertical rather than along the writer's slant, a measured design decision: glyphs cut along the slant carry a sheared edge that only matches their original neighbour and do not retile cleanly at new spacings, which lowered writer identification of synthesised output from 81.8 % to 68.6 % in a design-stage probe, so vertical cuts were kept, although no raw log of that probe survives.

[PLACEHOLDER — one disconnected personal line and one strongly cursive line, with the aligned character centres and final ligature cuts marked as vertical lines on the ink]

Figure 22. Placeholder: forced-alignment cutting on a lightly connected hand and a strongly cursive hand side by side, showing the cuts land cleanly between letters on the first and are forced through genuine strokes on the second, which is why cursive hands are reproduced least legibly (Section 3.6.8).

Algorithm 16 states the two routines that locate each character. The first is Equation 73: it differs from the CTC loss of Equation 53 in taking the *maximum* where the loss takes a sum, which is what turns a probability into a single best alignment, and it keeps back-pointers so that the path itself can be recovered. The second is the cut cost of Equation 74 evaluated for every column, which is minimised near the boundary to place the cut at a ligature — that is, at a column that is

both thin and low.

Algorithm 16 Forced alignment, and the per-column cut cost that places the boundary

ForcedAlign(log-probabilities, transcript):
  build y' ← blank-interleaved transcript $\{S = 2U + 1\}$
  $\delta[1,\cdot] \leftarrow -\infty$ except the first two positions
for $t = 2$ **to** T **do**
  for every position s simultaneously **do**
    stay ← $\delta[t-1,s]$; advance ← $\delta[t-1,s-1]$; skip ← $\delta[t-1,s-2]$
    if y'_s is blank **or** $y'_s = y'_{s-2}$ **then**
      skip ← $-\infty$
    end if
    best ← the largest of the three {**maximum**, not a sum: this is the one change from Algorithm 9}
    backpointer$[t,s]$ ← which one won
    $\delta[t,s]$ ← best + log Prob$[t, y'_s]$
  end for
end for
  back-trace from the better of the last two positions
  character i owns the frames whose winning position is $2i$

CutCostPerColumn(column ink profile, baseline, x-height):
  smoothed ← profile averaged over $\max(3, \text{round}(0.12\,x_h))$ columns, odd
  thin ← smoothed / 60th percentile of the non-empty columns
  topAbove ← (baseline − highest ink row)$/x_h$, clipped to $[0,3]$
  cost ← thin + 1.15 topAbove
  cost ← 0 wherever the column holds no ink {a true gap is free}

3.6.2 From ink to ordered pen paths

A thinned image is a picture of the pen's path, not yet an *ordered* path, and four geometric operations recover the ordering, each resting on the 8-neighbourhood degree of a skeleton pixel: degree one is a stroke end, two an interior point, three or more a junction. The skeleton is *traced* into chains grown from each unvisited end or junction, stopping at a junction rather than choosing a branch there, which is what makes the decomposition unique — a junction being precisely the point where the true pen order cannot be recovered from the image alone. Short false branches left by thinning are *pruned* as spurs; because a self-crossing stroke such as the loop of an e breaks into several chains at the crossing although the pen never lifted, nearby chain ends with nearly opposed directions are *re-joined*; and each chain is finally *simplified* by the Douglas–Peucker recursion [**?**] at $\varepsilon = 0.6$ pixels.

The line is binarised and thinned by the same functions as Section 3.5.3 (Otsu, rule removal, closing, Zhang–Suen [**?**]), with the *slant* scored on the ink's column histogram sheared by a trial angle θ:

$$x'_i(\theta) = \left\lfloor x_i - (y_i - \bar{y}) \tan\theta \right\rceil, \qquad H_\theta(c) = \#\{i : x'_i(\theta) = c\}, \qquad \hat{\theta} = -\arg\max_\theta \sum_c H_\theta(c)^2. \quad (75)$$

where (x_i, y_i) are the ink coordinates, $\bar{y}$ the mean ink row, $x'_i(\theta)$ the column each pixel falls

into after shearing by a trial angle θ, H_θ the resulting column histogram and $\hat{\theta}$ the estimated slant. Equation 75 maximises the sum of squared bin counts for the same reason as the deskew objective of Equation 34: shearing conserves the total ink, so the sum of squares is largest when vertical strokes stack into the fewest columns, which happens when the shear has cancelled the writer's slant.

Each simplified point of a chain in territory $[L, R)$ is then mapped by Equation 76 into a writer-independent frame, baseline 0, unit x-height, slant removed, the same normalisation goal pursued for cursive word images in the literature [**?**]:

$$y_n = \frac{y_b - y}{x_h}, \qquad x_n = \frac{x - L}{x_h} - \sigma\, y_n, \qquad \sigma = \tan\hat{\theta}. \tag{76}$$

The whole line is thinned *once*, before territory cuts are applied to the traced centre lines, because thinning each slice separately would create a spurious spine along the cut boundary. On a side confirmed connected, the outward-running low connector is trimmed when enough of the glyph width survives, and the entry and exit heights are recorded for the synthesiser's own connector (Section 3.6.5).

Thinning leaves a one-pixel-wide set of ink pixels, which is a picture of the pen path and not yet an ordered path. Four operations recover the ordering, and each rests on the 8-neighbourhood degree of a skeleton pixel p,

$$\deg(p) = \#\big\{q \in \mathcal{N}_8(p) \;:\; q \text{ is a skeleton pixel}\big\}, \tag{77}$$

which classifies every pixel by Equation 77: an *end* has deg = 1, an *interior* pixel deg = 2, and a *junction* deg ≥ 3.

Tracing. A chain is grown from each unvisited end or junction and extended through interior pixels only, stopping at the next end or junction, so that the chain set covers every skeleton pixel exactly once and each chain is an ordered pixel sequence. Stopping at junctions rather than choosing a branch there is what makes the decomposition unique: a junction is a point at which the true pen order is not recoverable from the image, so the decision is deferred to the re-joining rule below rather than taken arbitrarily.

Spur pruning. Thinning a stroke of finite width leaves short false branches at the ends of real strokes. A chain is discarded as a spur when

$$\text{len}(C) < 2.5 \text{ pixels} \quad \wedge \quad \exists\, p \in \{p_1, p_{|C|}\} \;:\; \deg(p) = 1, \tag{78}$$

that is when it is shorter than 2.5 pixels *and* has at least one free end. The conjunction in Equation 78 matters: a short chain between two junctions is the crossbar of a letter such as A or the waist of an e and must be kept, whereas a short chain hanging off a junction carries no pen motion.

Junction re-joining. A self-crossing stroke, such as the loop of an e or the ascender of an l, breaks into several chains at the junction where it crosses itself, although the pen never left the paper. Two chain ends i and j are re-joined into one chain when they are close and nearly collinear,

$$\|p_i - p_j\| \le 1.5 \text{ pixels} \quad \wedge \quad \arccos\big(-\mathbf{d}_i \cdot \mathbf{d}_j\big) < 75^\circ, \tag{79}$$

where $\mathbf{d}_i$ and $\mathbf{d}_j$ are the unit outward directions of the two chains at those ends. The negative sign in Equation 79 is the continuation test rather than a parallelism test: two chains continue each other when one runs *into* the junction along the direction the other runs *out* of it, so their outward directions are nearly opposed and $-\mathbf{d}_i \cdot \mathbf{d}_j$ is near +1. A threshold of 75° admits the curvature a pen shows through a crossing while rejecting a genuine corner, at which the two strokes meet at

nearer a right angle.

Curve simplification. A traced chain holds one point per pixel, far more than a pen path needs, and its points lie on a lattice. Each chain is therefore simplified by the Douglas–Peucker recursion [**?**]: for a chain from p_1 to p_n, the vertex of greatest perpendicular distance from the straight line joining its ends,

$$d_{\max} = \max_{1<i<n} \frac{\left|(p_n - p_1) \times (p_i - p_1)\right|}{\|p_n - p_1\|}, \tag{80}$$

is found; if $d_{\max} \le \varepsilon$ the whole chain is replaced by its two end points, and otherwise it is split at that vertex and both halves simplified in the same way. With $\varepsilon = 0.6$ pixels, which is 0.05 to 0.12 mm at 4 mm per x-height, the deviation of the simplified path from the traced one is bounded by ε everywhere by construction. Equation 80 measures distance to the infinite line through the two ends, which fails on a closed chain: there the two ends coincide, the denominator vanishes and the whole loop would collapse to two points. A chain whose ends lie within $\max\big(1.5, 0.25 \max_i \|p_i - p_0\|\big)$ pixels of each other is therefore split at its farthest vertex first and simplified in two open halves, which is what keeps the loops of e, l and o.

Resampling and smoothing. Thinning leaves a staircase on a diagonal stroke, whose median turn angle is 8.9° with a 90th percentile of 59.5°; drawn directly, that staircase appears as tremor. Kept variants are therefore resampled at 0.055 x-heights of arc length and smoothed by two passes of the three-point average

$$q_i \leftarrow \tfrac{1}{4} q_{i-1} + \tfrac{1}{2} q_i + \tfrac{1}{4} q_{i+1}, \tag{81}$$

with the end points held fixed. Equation 81 is a binomial kernel, so it attenuates the highest representable spatial frequency — the alternating lattice staircase — completely while leaving a smooth curve almost unchanged, and fixing the ends preserves the entry and exit points that the synthesiser's connectors attach to.

3.6.3 Glyph quality gates and the cross-writer letter prior

Of about 2 122 to 4 031 raw instances per writer, roughly 10 % survive five gates, ordered cheapest-first and most-local-first: geometry, outliers, a cross-writer prior, consensus against the same writer's other instances of that letter, and typicality. The hardest to design is the cross-writer prior, because deciding whether a letter is malformed needs more examples of well-formed letters than one writer's ten pages contain; thirteen writers together do, so whatever all thirteen hands agree on is taken to be the letter itself rather than any one style.

Each glyph is reduced to a 12×12-grid shape descriptor v_g (144 numbers), damped, blurred across neighbouring cells and scaled to unit length so pen thickness cancels. The reference shape $\bar{v}_c$ for character c is the element-wise median over all thirteen writers, also unit length, and the prior distance is their cosine distance:

$$\mathrm{priorD}(g, c) = 1 - v_g \cdot \bar{v}_c \in [0, 1], \tag{82}$$

with the 70 % of each writer's variants having the smallest priorD kept. Equation 82 only ever *rejects* one of the writer's own variants; it never substitutes a shape from another writer, so no other person's handwriting can enter a profile through this gate. What it measures is *typicality*, not legibility, and a writer whose letters are consistently unusual scores badly on a prior built from everyone else.

Two further glyphs are derived per character: the *medoid*, the one real variant with the smallest

total L_1 distance to the character's other variants, used as the *anchor* fallback of Section 3.6.5 because it is a real pen stroke rather than an average of several; and the *denoised prototype*, the per-stroke median curve of the largest group of same-stroke-count variants, prepended only when clearly cleaner than the variants it summarises.

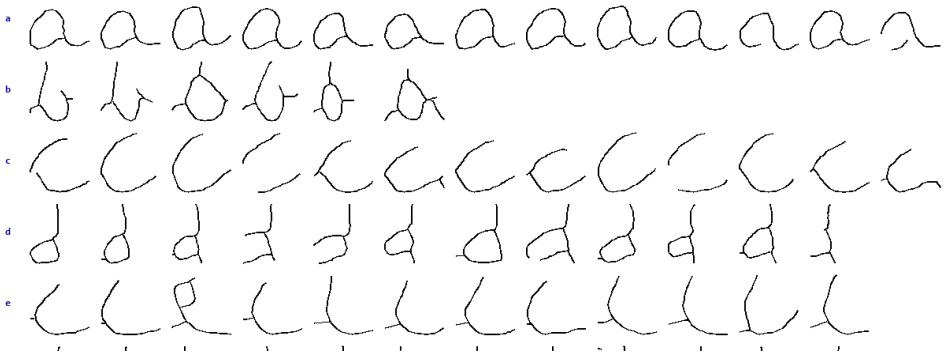


Figure 23. Stored variants of the letters a, b, c, d and e of writer 153, a strongly joined hand; each row is one character with up to 12 variants, the short tails at the sides being remains of ligatures.

About 2 122 to 4 031 raw instances are extracted per writer and roughly **10 %** survive. Five gates are applied per character, in the order of Table 22. The cascade is ordered cheapest-first and most-local-first: the geometric tests reject on properties of *one* glyph, the prior on agreement with *all thirteen* writers, and the last two on agreement with *that writer's own* other instances of the same letter, so a glyph is only compared with its peers once it is known to be a plausible letter at all.

#	Gate	Test	Compared against	Survivors
1	Geometry	Size limits, at most six strokes, a class prior on height, the absence of a swallowed neighbour, and a pen length above the adaptive floor $\max(0.30, 0.5\,\tilde{p}_c)$ on that character's own median pen length $\tilde{p}_c$. The floor is adaptive because one fixed value either kept neighbour bleed or discarded genuinely short letters.	the glyph itself	**[MUST STILL BE COMPLETED BY ME: %]**
2	Outliers	Width and height must lie within fixed ratios of the 40th-percentile width and the median height.	that letter's own distribution	**[MUST STILL BE COMPLETED BY ME: %]**
3	Cross-writer prior	Keeps the **70 %** nearest the pooled shape, by Equation 82.	all thirteen writers	70 %
4	Consensus	Keeps the nearest **75 %** to the L1-medoid of the kept descriptors, which removes glyphs contaminated by a neighbour.	that writer's own instances	75 %
5	Typicality	Keeps the best **12** by closeness to the reference width and height.	that writer's own instances	≤ 12
			end to end, measured	**about 10 %**

Table 22. The five quality gates, in the order applied. Only the last three have a stated keep-fraction by construction; the first two reject a variable number, which is why the end-to-end survival rate is measured rather than predicted from the product of the gates.

[MUST STILL BE COMPLETED BY ME: record how many instances each of gates 1 and 2 rejects, over all thirteen writers, so that the 10 % end-to-end figure can be resolved into the five gates rather than only measured at the end.]

FIGURE MUST STILL BE INSERTED BY ME

A funnel or bar chart of instance counts surviving each of the five gates of Table 22, for one writer, from the raw extraction to the final twelve variants per character; and beside it a strip of eight instances of one letter, those rejected marked with which gate rejected them.

Figure 24. Instances surviving each of the five quality gates for one writer, with examples of what each gate rejects.

3.6.4 Style statistics and permanent storage

Besides glyphs, the profile holds measured numbers: slant, pixel x-height, ascender and descender reach, word gap, and *connectedness*, the fraction of adjacent letter pairs whose connection flag was set during tracing. Table 23 shows the diversity this is built to reproduce — slant −10.5° to 42.0°, connectedness 0.195 to 0.968, word gap 0.94 to 2.68 x-heights across the thirteen writers — which is requirement R1's diversity target, and the thirteen files (about 4.8 MB total) load when the server starts.

Writer	Lines	Slant (°)	x_h (px)	A	D	Gap	Conn.
150	61	−4.5	47	1.58	−0.72	1.33	0.343
151	58	33.0	22	2.69	−1.09	1.59	0.968
153	66	30.0	33	1.59	−1.10	1.59	0.961
384	54	4.5	43	1.93	−0.71	1.37	0.500
552	47	−10.5	32	1.62	−0.56	1.53	0.609
dylan	79	−6.0	26	1.68	−0.62	1.79	0.195
thiya	48	30.75	15	2.24	−1.00	1.65	0.962
yeukita	74	0.0	27	1.45	−0.58	0.94	0.417

Table 23. Representative rows of the thirteen stored style profiles (schema in Table 24): kept lines, slant, pixel x-height, ascender A, descender D, word gap in x-heights, and connectedness.

[MUST STILL BE COMPLETED BY ME: *The builder's* `ExtractLineGlyphs` *currently raises an import error* (a stale reference to a function moved during the restructuring of `TrainText.py`), so no new writer can presently be added and none of the thirteen stored profiles can be regenerated from the page images; the stored profiles themselves are unaffected and still load correctly. `legibilityLambda` is also absent from all thirteen stored files, so the system synthesises every writer at $\lambda = 0$ until the tuner is re-run (Section 3.6.8).]

A profile is one text file per writer, and Table 24 lists the keys that carry design meaning, with the values of one writer as an example.

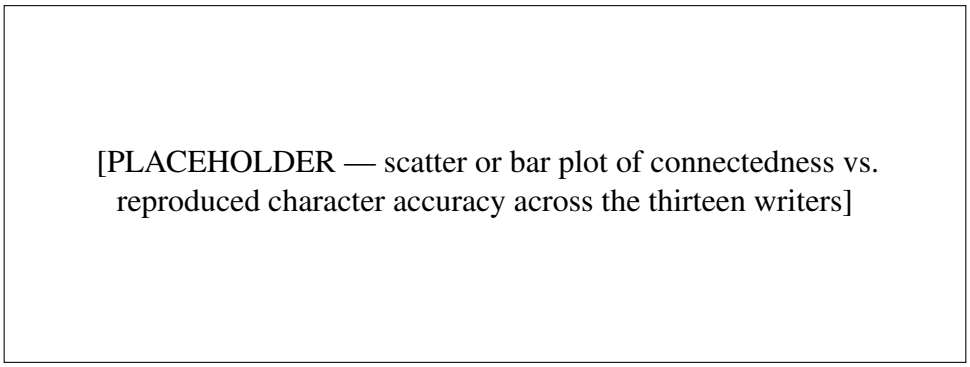


Figure 25. Placeholder: connectedness (Table 23) plotted against each writer's reproduced character accuracy from Table 26, to show quantitatively that the most joined hands are the hardest to reproduce.

Key	Meaning	Example (153)
`slantDeg`, `xHeightPx`, `strokeWidthXh`	Median slant (Equation 75), median pixel x-height and ink area divided by skeleton length in x-heights.	30.0, 33.0, 0.189
`ascender`, `descender`	Typical reach of tall and descending lower-case glyphs, in x-heights (defaults 1.7 and −0.6).	1.59, −1.10
`wordSpaceXh`, `connectedness`	Word gap measured on the real ink, in x-heights; fraction of adjacent in-word letter pairs that the pen joins.	1.59, 0.961
`letterAdvance`	Median advance of each character, in x-heights (45 keys).	`T`: 1.438, `e`: 1.147
`slantMeasRef`, `ascMeasRef`, `descMeasRef`	Slant and reach of the real full lines, measured with the estimators later used on synthesised renders; input of the style calibration (Equation 86).	30.0, 1.933, −1.468
`glyphs`, `idealGlyphs`, `idealMedoid`	Variant library: per character up to 12 kept variants and a prepended prototype, each holding its `strokes` as $[x, y]$ points in x-heights (baseline $y = 0$, upright) with the width, advance, lead, reach, entry and exit coordinates, connection flags and priorD the synthesiser needs; then the per-stroke median prototype and the medoid, which is the anchor.	37 characters, 283 variants; 23 and 29 prototypes
`legibilityLambda`	Legibility parameter λ of Equations 83 and 84; written by the tuner after the build, *not* by the builder. *Absent from all thirteen stored profiles*, so the deployed code falls back to $\lambda = 0$.	not present

Table 24. Keys of a style-profile file (JavaScript Object Notation) that carry design meaning; bookkeeping keys and the grey-level reference statistics are omitted. The example column was re-read from the stored file `StyleProfiles10/153.json`. The whole library is geometry in x-height units, which is why one profile serves any pen size.

3.6.5 Synthesis: choosing and placing a glyph

The synthesiser takes text, a style profile, a pen size x_{mm} in millimetres and a random seed, and returns a *trajectory*, pen-down polylines in millimetres; geometry stays in x-height units until one multiplication at the end, so one profile serves any pen size. Requirement R1 pulls in two directions at once: handwriting that resembles the writer argues for always using the writer's own letters, while handwriting that can be read argues for replacing the writer's worst letters with something clearer. The parameter $\lambda \in [0, 1]$ is the single control that sets where a writer sits between those two. The obvious way to trade them off, blending the writer's glyph and a clear *anchor* glyph geometrically, is *not* used, because an average of two different pen strokes is usually not a valid letter at all; instead λ scales the *probability* that a chosen variant is replaced by the anchor outright, so every letter drawn is always a real letterform.

Global shear, join probability and line pitch follow from the profile (Equation 83); among a

writer's own variants for one character the choice is weighted by rank and, when joining, by how well the variant's entry height matches where the pen already is; the anchor-substitution probability is Equation 84; and placement is Equation 85:

$$\theta_{\text{cap}} = 45^\circ - 19^\circ\lambda, \quad \sigma = \tan\big(\text{clip}(\theta_{\text{p}}, \pm\theta_{\text{cap}})\big) + \Delta_\sigma, \quad p_{\text{j}} = \min(c, 1), \tag{83}$$

$$P_{\text{swap}}(\lambda, d) = \begin{cases} 1 & d \geq 0.55, \\ 0 & \lambda \leq 10^{-3}, \\ 0.06\,\lambda & d \leq 0.12, \\ \text{clip}\big(\lambda\,(0.55 + 0.9\,b), 0, 1\big),\ b = \text{clip}\big(\frac{d-0.12}{0.43}, 0, 1\big) & \text{otherwise}, \end{cases} \tag{84}$$

$$X = P_x + (\ell + x + \sigma_g\, y)\, x_{\text{mm}}, \qquad Y = P_y + (y + \delta_g)\, x_{\text{mm}}. \tag{85}$$

A variant beyond prior distance $d = 0.55$ is replaced *unconditionally*, whatever λ; one closer than $d = 0.12$ is kept almost always; between those the probability rises with both d and λ. Below $\lambda = 10^{-3}$ substitution switches off entirely, so a writer tuned to zero keeps even their marginal letters. When a writer's medoid itself is too malformed to serve as the anchor, a hand-designed single-stroke skeleton (joined cursive or upright print, re-proportioned to that writer's ascender and descender reach) is used instead, reading back at about 96 % character and 84 % word accuracy, which is what makes it a usable floor.

Treating connectedness c as a hard threshold, joining every pair above it, was tried first and fused whole words into unbroken blobs, losing the irregularity that makes a hand look human; it is used as a per-pair *probability* p_{j} instead. Measured over a design-stage probe, the anchor share this produces ran from 0 % for writer 552 to 76 % for writer 151, the most joined hand of the set. One pseudo-random stream seeded by the seed argument supplies every draw, so the arguments fix the output completely; at the jitter amplitude the search uses, a letter is perturbed by about a 1.1° rotation, a 2.5 % scale change and under 0.05 mm translation. *Style calibration*: because glyphs are placed one instance at a time, an assembled line does not automatically reproduce the writer's own line-level slant and ascender reach (writer 150's ascenders came out about 7 % too tall, writer 551's slant about 9° too steep against real lines), so a fixed probe sentence is synthesised once per writer and three fixed-point iterations of Equation 86 adjust the shear shift Δ_σ and the scale factors K_A, K_D until the probe matches the stored real-line reference values:

$$\Delta_\sigma \leftarrow \Delta_\sigma + \tan\theta_{\text{ref}} - \tan\theta_{\text{got}}, \quad K_A \leftarrow \text{clip}\Big(K_A \frac{A_{\text{ref}} - 1}{A_{\text{got}} - 1}, 0.5, 1.6\Big), \quad K_D \leftarrow \text{clip}\Big(K_D \frac{D_{\text{ref}}}{D_{\text{got}}}, 0.5, 1.6\Big), \tag{86}$$

with "ref" the stored real-line value and "got" the probe's measured value by the estimators of Section 3.6.2, each correction clipped to bound its effect.

Every constant of the glyph choice, the placement, the joins and the randomness is listed in Table 25 with the design reason for its value, so that Section 3.6 can describe the construction without interrupting it to justify a number.

3.6.6 Best-of-N search with joint scoring

Single draws vary in quality, so a line is drawn N times and the best kept; the score must use both judges together, because selecting by text accuracy alone raised legibility but lowered writer identification for some writers in a design-stage probe, since the most legible of several random draws need not be the most characteristic.

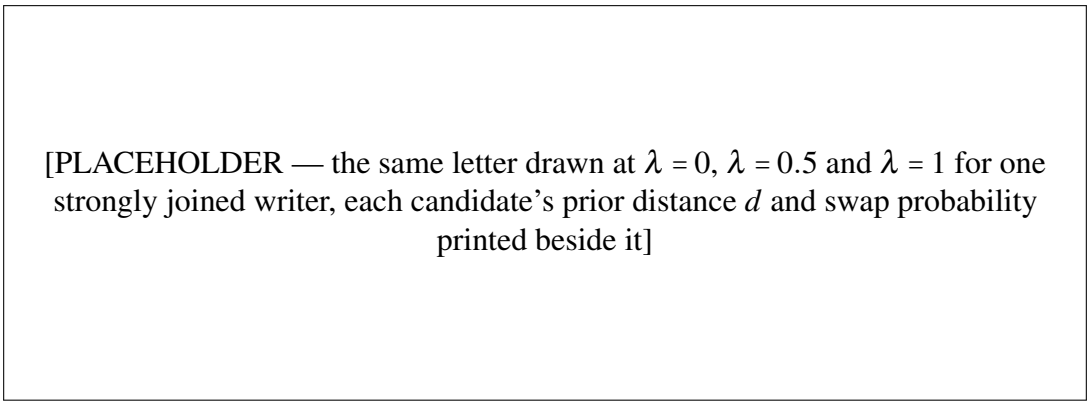


Figure 26. Placeholder: the legibility sweep of Equation 84 on one writer's worst-scoring letter, from the writer's own glyph through partial anchor substitution to the full anchor, with P_{swap} plotted against prior distance d for several λ as an inset curve.

Constant	Value	Reason for the value
Eligible variants per letter	5 best by priorD	80.5 against 75.5 % writer identification over all twelve
Anchor taken from the medoid	priorD ≤ 0.30	above it, not clean enough; skeleton used instead
Cursive anchor threshold	$c \ge 0.5$, off above 1	0.3 lowered the evaluation score
Anchor re-proportioning	$y \mapsto 1 + (y-1)\max(0.35, \frac{A-1}{0.75})$, $y \mapsto y\max(0.35, \frac{\lvert D\rvert}{0.55})$	skeleton takes the writer's reach; floor 0.35 keeps it from flattening
Pen advance	the glyph's own advance	never the character median
Anchor shear cap	$\pm\tan 30°$, $\pm\tan 22°$ print	40° collides even for a clean letter
Collision guard	$0.05\,x_{\text{mm}}$	shift right rather than let ink touch
Word gap	$g\,x_{\text{mm}}(0.85 + 0.3u)$, $u \sim U(0,1)$	so words do not fall on a grid
Line width; pitch	185 mm in the server; L (Eq. 83)	wrap inside the work area; writer's line spacing
Connector sag	$\min(0.11x_{\text{mm}}, 0.16\lvert\Delta x\rvert)$, plus noise	span-proportional: a fixed dip loops a short join
Polyline resampling	0.35 mm of arc length	even plotter motion
Jitter rotation; scale; translation	$2.2°J$; $0.05J$; $0.022J$ and $0.0132J$ x-heights	one amplitude sets how unsteady the hand looks
Jitter amplitude J	0.15 default, 0.5 search, 0.15 anchors	the search needs spread; anchors exist to stay legible
Baseline drift	walk of $0.05J \cdot 0.35$ per glyph, clipped ±0.25 x-heights	wanders slowly, clipped to stay on the line
Judges' render settings	0.085 x-height pen, blur 0.30 of it, 18 px/mm, pad $0.18\,x_{\text{mm}}$	identical ink for every candidate (Section 3.6.6)

Table 25. Constants of the synthesiser, with the design reason for each. Lengths in x_{mm} scale with the pen size; A, D, c and g are the profile's ascender, descender, connectedness and word gap. At the $J = 0.5$ the search uses, the jitter comes to a rotation of $1.1°$, a scale change of 2.5 % and a translation spread below 0.05 mm, which perturbs a letter without changing its identity.

For a rendered candidate I, with $\hat{y}(I)$ the recogniser's greedy reading and y the intended text, the text term $a(I)$ below is built from the Levenshtein edit distance [**?**, pp. 707–708], the smallest number of insertions, deletions and substitutions turning one string into the other,

$$a(I) = \max\Big(0, 1 - \frac{\text{Lev}(\hat{y}(I), y)}{|y|}\Big), \quad w(I) = \text{softmax}\big(f_{\text{wid}}(I)\big)_{a^\star}, \quad S(I) = \frac{2\,a(I)\,w(I)}{a(I) + w(I)}, \tag{87}$$

the harmonic mean of recogniser accuracy and shape-classifier confidence in the target writer $a^\star$,

dominated by whichever is smaller, so a candidate cannot win on one criterion while failing the other.

The search has two loops, given as Algorithm 17: the first draws up to N candidates and keeps the best by S; the second, a *repair* loop of at most three rounds, forced-aligns the intended text to the best image, pins the two worst-scoring characters to the anchor, re-synthesises the whole line, and accepts the result only if S strictly improves, so a repair that fixes a letter but makes the line resemble another writer is rejected. N is 6 as the function default, 14 for design-stage measurements, 20 as the web default and 30 in the evaluation script; the cost is entirely judge-bound, synthesising a candidate costing 11 to 16 ms against 0.5 to 4 s to score it, so N rather than the construction itself sets the reproduction time of requirement R3.

Algorithm 17 `SynthesizeJointBestOf`: candidate search with joint scoring and repair

Require: text, style profile, target writer $a^{\star}$, seed base, candidate budget N
best ← none; bestScore ← $-\infty$
for $k = 0$ **to** $N - 1$ **do**
  synthesise candidate I_k from seed base + k; compute $S(I_k)$ by Equation 87
  if $S(I_k) >$ bestScore **then**
    best ← I_k; bestScore ← $S(I_k)$
  end if
  if bestScore ≥ 0.99 **then**
    break {good enough; stop drawing further candidates}
  end if
end for
for $r = 1$ **to** 3 **do**
  repair round r: forced-align the intended text to best (Algorithm 15)
  pin the two worst-scoring characters to their anchor glyph; re-synthesise the line as I'
  if $S(I') >$ bestScore **then**
    best ← I'; bestScore ← $S(I')$
  else
    break {repair did not strictly improve S; reject it}
  end if
end for
return best

Table 26 lists the measurements that motivated the design, over six novel sentences, two seeds and ten writers. Joint best-of-N at $N = 14$ reached 97.5 / 95.8 % writer identification and 81.6 / 81.4 % character accuracy (direct / through G-code); on a matching comparison of identical words, synthesis costs 7.4 percentage points of character accuracy against the writers' own real handwriting (89.0 % vs. 81.6 %) and more than 1.5 times the word accuracy (70.5 % vs. 46.7 %), which locates the loss in spacing and word boundaries rather than in letterforms.

A candidate costs one synthesis, one rendering, one recogniser pass and one classifier pass, so the search time is

$$T \approx N\left(T_{\text{syn}} + T_{\text{ren}} + T_{\text{text}} + T_{\text{wid}}\right) + (\text{up to three repair candidates}), \tag{88}$$

each network pass being about 4×10^9 multiply–accumulate operations (Equation 48, Table 18). Table 27 resolves Equation 88 into its terms on the development laptop, and the result decides where any optimisation must go: synthesis costs 11 to 16 ms per candidate against 0.5 to 4 s for

Stage	Writer ID, direct / G-code (%)	Char. acc., direct / G-code (%)
Plain synthesis after label and glyph fixes	79.2 / 80.0	75.9 / 76.3
Text-only best-of-N with plain repair	76.7 / 73.3	83.8 / 82.7
Joint best-of-N, N = 14	97.5 / 95.8	81.6 / 81.4
Final system, 13 writers, 3 sentences, N = 8	92.3 / 89.7	79.2 / 79.2
Matching comparison: writers' own real handwriting	not applicable	89.0 / —
Matching comparison: synthesis of the same words	not applicable	81.6 / —

Table 26. Measurements that motivated the design (upper block) and the final system (lower blocks). Only the two matching-comparison rows are independent of the search and so the figures to trust most directly; the rest are qualified by Section 3.6.8.

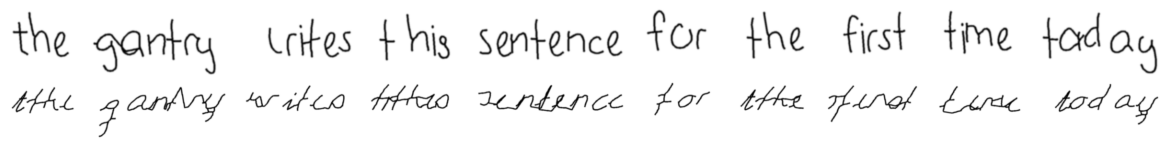


Figure 27. Synthesised line for writer 150 (upper, printed hand, read back with three small errors) and writer 153 (lower, joined hand, read back with several letter errors), one seed and candidate each, rendered from an end-to-end run.

the two judges, so the search is entirely judge-bound, and within the judges the dominant cost is thinning the full-resolution render rather than either network.

Quantity (56-character sentence)	Writer 150	Writer 153	Writer 551
Render size (pixels)	3585 × 225	5508 × 207	7876 × 277
Synthesis of one candidate (ms)	11	13	16
Rendering (ms)	20	27	56
Text recogniser (s)	0.50	0.49	0.74
Writer classifier incl. stroke normalisation (s)	1.03	1.65	3.74
Complete search, N = 14, with repair attempts (s)	23	37	137

Table 27. Indicative cost of the search on the development laptop (single profiling run, unknown background load, jitter 0.5); times on the single-board computer were not recorded. The spread across the three writers follows the render width, which follows the writer's own letter size and spacing, so the cost of a search is set by the hand being imitated and not only by N.

Were the candidate scores independent with distribution function F, then $\Pr[\max_k S_k \le \tau] = F(\tau)^N$ and the gain from raising N would grow roughly logarithmically, which is the usual order-statistics argument for a best-of-N scheme. The seeds share one profile, so independence is only approximate and no per-N curve was kept; the measured values at N = 6 and N = 14 in Table 26 are consistent with a diminishing return but do not establish its form.

3.6.7 Alternatives considered

Four families of handwriting synthesiser were weighed against the glyph library actually built. Table 28 sets them against the three quantities that decided the choice for this project: how much data per writer each needs, what it costs per line on hardware with no accelerator, and whether size, slant and spacing can be set as explicit numbers, which requirement R1 needs because the style profile is the deliverable.

Approach	Data needed per writer	Cost per line	Explicit style parameters?	Measured outcome, or why it was not built
Recurrent generator with attention and a mixture-density stroke output [?, p. 1]	hundreds to thousands of *on-line* pen lines	not measured	no	**Built and rejected.** A from-scratch version trained on skeleton-derived strokes learned slant, roundness and connectedness but *no legible letters*. Only 47 to 130 scanned lines exist per writer here, and none are on-line.
Adversarial or diffusion generator conditioned on a style exemplar [?, ?]	less, through pre-training	**35 to 45 s**	no	**Trialled and parked.** Emits a raster needing the same thinning to vectorise, and cannot control size, slant and spacing as numbers.
Exemplar-based synthesis from the writer's own captured letter-forms [?, p. 26:1] — **chosen**	about **nine pages**, no training	**3 to 8 s** per line; **15 ms** per candidate	**yes**	**Built and delivered.** Suits a single-board computer with no accelerator. Its one clear weakness is joined hands, whose joins are necessarily lost by the character-cutting of Section 3.6.1.
Font-style interpolation from a trained letter-form model	**[MUST STILL BE COMPLETED BY ME: not costed]**	**[MUST STILL BE COMPLETED BY ME: not costed]**	partly	**[MUST STILL BE COMPLETED BY ME: state whether this was considered, and if so on what grounds it was set aside.]**

Table 28. Alternatives to the glyph-library synthesiser. Two of the four were implemented far enough to measure; the per-line costs are from the same development laptop as Table 27 and are therefore comparable with each other, though not with the single-board computer. The decisive column for this project is the fourth: R1 requires a stored, inspectable style, which a learned generator does not provide.

FIGURE MUST STILL BE INSERTED BY ME

The same short sentence in the same writer's style, rendered four ways and stacked: the writer's real handwriting, the glyph-library synthesis delivered, the rejected recurrent generator's output, and one trialled diffusion sample. The point to show is that the recurrent output carries the slant and roundness but no readable letters, while the glyph library is legible but loses the joins.

Figure 28. The same sentence rendered by the writer's own hand and by three of the four approaches of Table 28, showing what each preserves and what each loses.

3.6.8 Known faults and limitations

1. *The legibility mechanism is inert as submitted*, because `legibilityLambda` is absent from all thirteen stored profiles and defaults to zero, so $\lambda = 0$ on the live path: only the unconditional substitutions at $d \geq 0.55$ fire, and every λ-dependent figure above describes a tuned configuration that is not what the delivered profiles currently run.
2. Every figure after the search is introduced is the opinion of the very networks that selected the output: the search loads the same weight files as the measurement, so the values show the pipeline can satisfy its two judges, not that an independent reader would agree.
3. λ, the anchor threshold and N were tuned on the same sentences Section 4 measures, and the style calibration of Section 3.6.5 measures both sides with the same estimator, so its agreement is partly self-fulfilling.

4. Writers 151 and 552, the two most joined hands, reach only 53 and 56 % character accuracy through G-code against about 84 % for the other eleven, the clearest limitation of the exemplar approach and a direct consequence of the cutting problem of Section 3.6.1.
5. Cutting a joined line destroys how a letter changes shape with its neighbours, which is the underlying reason writers 151 and 551 are reproduced least legibly.

[MUST STILL BE COMPLETED BY ME: page locators outstanding in this subsection. The sources `douglas1973`, `teikari2016`, `vinciarelli2001`, `viterbi1967`, `zhang1984` are each cited here without a page or numbered element. Open each one, find the claim it is cited for, and add the locator in the form `\cite[p. N]{key}`, following the rule of `REFERENCE_AUDIT.md`: a locator is written only after the source has been read and the claim found, never inferred from a page range or a search snippet.]

3.7 Design reconsiderations

The subsections above describe each subsystem as delivered, but five of those designs are not what was built first: each replaced an earlier version once the complete chain existed and the output could be measured end to end, which is the point at which faults invisible inside a single subsystem became apparent. They are collected here, separately from the component descriptions above, because each is a change of *approach* rather than a correction of code, and because the reason for each lies outside the subsystem it altered.

Table 29 is the map of this subsection: one row per reconsideration, the measurement that forced it, and the measurement that justifies the replacement. A change appears here only if both measurements exist; the one change with no clean before-and-after pair is marked as such rather than claimed.

Reconsideration	What forced it	Before	After
Shape classifier beside the ink one (Section 3.7.1)	The ink classifier cannot judge constant-width plotter output, so it could not score a synthesis candidate.	28 % writer identification on synthesised output	89.7 % at best-of-8 through G-code
End-to-end training (Section 3.7.2)	Fitting 1.4 % of the model on features extracted once, with no augmentation re-sampled between epochs.	83.36 % over 1358 real lines	**93.67 %** over the same lines
Joint candidate score (Section 3.7.3)	The most legible of N random draws has no reason also to be the most characteristic.	legibility only; writer identification fell for some writers	harmonic mean of both, Equation 87
Line-level style calibration (Section 3.7.4)	Glyphs are placed one at a time, so line-level slant and reach drift from the writer's own.	writer 150 ascenders +7 %; writer 551 slant +9°	*no clean pair measured* — see the marker in that section
Shape namer for photographed pages (Section 3.7.5)	The ink namer keys on ink statistics, which do not survive a change of pen or lighting.	30/30 dataset pages but **0/1** on a page photographed with a different pen	**31/31**, at a cost in pooled confidence

Table 29. The five design reconsiderations, each with the measurement that forced it and the measurement that justifies the replacement. Every figure in this table is repeated with its conditions in the subsection named in the first column.

3.7.1 A second writer classifier, judging shape instead of ink

The first writer classifier was trained on grey-scale crops and read real held-out lines at 99.78 % over 1358 lines, which satisfied requirement R2 on photographed pages. Measured on *synthesised*

output, however, the same classifier identified the intended writer in only about 28 % of trials. The cause is a property of the machine rather than of the network: the gantry draws every stroke with one pen at one width and one pressure, whereas real handwriting carries a thickness and an ink density that a convolutional network exploits readily when the training lines are few. A classifier that leans on those cues therefore predicts neither what the machine can draw nor how a reader recognises a hand, which is by geometry.

Table 30 sets the two classifiers against each other on both kinds of input, which is the comparison that justifies keeping both.

Input	Ink classifier	Shape classifier	Why the gap
1358 real photographed lines	**99.78 %**	93.67 %	Real ink carries a width and density cue the ink classifier may legitimately use.
Synthesised output through G-code	28 % (39 lines)	**89.7 %** at best-of-8	The plotter draws one width everywhere, so the ink cue is absent and the ink classifier has nothing left to read.

Table 30. Why two writer classifiers are kept rather than one. Chance level for thirteen writers is 7.7 %, so the ink classifier on synthesised output is close to four times chance and far below the 95 % of R2. The shape classifier's figure on synthesised output is not independent, since the same classifier family selects the candidates it then judges (Section 3.6.8).

The response was to train a second classifier of the same architecture on stroke-normalised crops, in which the ink is thinned to a centre line and redrawn at a constant width proportional to the letter height, so thickness is removed by construction (Algorithm 11). This is the reconsideration that matters most for requirement R1: style can be judged, and reproduced, on geometry alone once pen pressure and width are taken out of the comparison. Neither classifier replaced the other, so the writer-identification block carries two sets of weights rather than one, at the cost of double the training and storage for a capability the first classifier could not provide at all.

FIGURE MUST STILL BE INSERTED BY ME

Two panels on the same synthesised line: the raw render as the ink classifier sees it, and the stroke-normalised version as the shape classifier sees it, with the posterior over the thirteen writers plotted beneath each. The point to show is that the ink classifier's posterior is near-flat on constant-width output while the shape classifier's is peaked on the intended writer.

Figure 29. The same synthesised line as each classifier sees it, with the posterior over the thirteen writers beneath. Constant-width plotter output removes the cue the ink classifier depends on.

3.7.2 End-to-end training in place of a frozen backbone

The shape classifier was first trained by fitting only the 13 645 parameters of its head on feature vectors extracted once from a frozen backbone. That recipe is a training fault: because the

features were extracted once, no augmentation was re-sampled between epochs and every line contributed the same four fixed vectors for all 500 epochs. The parameter budget of Table 18 is what makes it consequential, and it is worth stating as a ratio,

$$\frac{\theta_{\text{fitted}}}{\theta_{\text{total}}} = \frac{13\,645}{966\,445} = 1.41\,\%, \tag{89}$$

so Equation 89 says the superseded recipe fitted one part in seventy of the model, on features a backbone trained for a *different task* had produced. Training all 966 445 parameters end to end instead raised the same measurement from 83.36 % to 93.67 % on the same 1358 real lines, a gain of 10.31 percentage points, and removed a failure in which 118 of one writer's 148 lines had been assigned to a single neighbour.

Every figure quoted for the shape classifier elsewhere in this report is from the end-to-end model; the frozen-head figures describe the superseded one, and are kept here only as the evidence for why the change was made.

FIGURE MUST STILL BE INSERTED BY ME

Two confusion matrices side by side over the same 1358 real lines: the frozen-head shape classifier at 83.36 % and the end-to-end one at 93.67 %. The cell to annotate is the one holding 118 of Owen's 148 lines in the left matrix, which the right matrix does not have.

Figure 30. Confusion matrices of the shape classifier before and after end-to-end training, over the same 1358 real lines. The systematic collapse of one writer into a single neighbour is the defect Equation 89 explains.

3.7.3 Selecting a candidate on both criteria instead of legibility alone

The candidate search first kept whichever of the N draws the text recogniser read most accurately. That raised legibility and, for some writers, lowered writer identification, because the most legible of several random draws has no reason also to be the most characteristic. The score was therefore changed to the harmonic mean of the recogniser's character accuracy a and the shape classifier's confidence w in the intended writer, Equation 87. The harmonic mean rather than the arithmetic one is the whole design decision, and the reason is visible in the limit,

$$\lim_{w \to 0} \frac{2aw}{a+w} = 0, \qquad \text{whereas} \qquad \lim_{w \to 0} \frac{a+w}{2} = \frac{a}{2}, \tag{90}$$

so by Equation 90 a candidate that is perfectly legible but in the wrong hand scores zero under the harmonic mean and still scores one half under the arithmetic one. The score is dominated by whichever criterion is weaker, which is exactly the behaviour wanted: a candidate cannot win on one criterion while failing the other.

The repair loop was changed for the same reason, accepting a repaired line only when the joint score improves, so a repair that fixes a letter but makes the line resemble another writer is rejected rather than taken. The evidence for both changes is Table 26.

3.7.4 Line-level style calibration

Glyphs are stored and placed one instance at a time, so a line assembled from them does not automatically reproduce the writer's line-level slant and ascender reach. The calibration of Section 3.6.5 was added to correct that drift, measuring a rendered probe with the same functions that measured the writer's real pages and returning a shear and reach correction. The drift it corrects was itself measured before the change, and Table 31 states it.

Writer	Quantity	Drift before calibration	After calibration
150	Ascender reach, against the writer's own real lines	about +7 % too tall	**[MUST STILL BE COMPLETED BY ME: measure]**
551	Line slant, against the writer's own real lines	about +9° too steep	**[MUST STILL BE COMPLETED BY ME: measure]**
all 13	Writer identification, calibration off versus on	**[MUST STILL BE COMPLETED BY ME: measure]**	**[MUST STILL BE COMPLETED BY ME: measure]**
all 13	Character accuracy, calibration off versus on	**[MUST STILL BE COMPLETED BY ME: measure]**	**[MUST STILL BE COMPLETED BY ME: measure]**

Table 31. The line-level drift the style calibration corrects. Unlike the other four reconsiderations this one has no clean before-and-after pair: the two drift figures were measured, but the effect of switching the calibration off and on over the same sentences and seeds was not.

Unlike the other reconsiderations, this one's benefit therefore rests on the probe agreeing with the stored reference rather than on a judge's score, and Section 3.6.8 records that this agreement is partly self-fulfilling, since it is measured by the same estimator on both sides. **[MUST STILL BE COMPLETED BY ME: fill Table 31: measure writer identification and character accuracy with the calibration of Section 3.6.5 disabled and enabled, over the same sentences and seeds as the other rows of Table 26, so that this change carries the same kind of evidence as the other four.]**

3.7.5 Naming a photographed page by shape rather than by ink

The division of labour set out in Section 3.7.1 — ink classifier for real pages, shape classifier for synthesised output — held until a page was photographed with a different pen under different light. On that page the ink classifier named the wrong writer with 84 % pooled confidence and *not one* of its thirteen lines named the true writer, while the same page's writer was correct on all thirty of the dataset pages. A single wrong page is not evidence of a systematic fault, so the failure mode was reproduced deliberately: each of 15 labelled pages was perturbed in five ways chosen to imitate a change of pen, ink and camera response, and all four classifier configurations were scored on all 75 resulting pages.

Table 32 identifies the mechanism: the ink namer is unharmed by a *thicker* pen and by a camera gamma shift, and loses exactly on the two perturbations that thin or lighten the ink. That is the signature of a classifier reading an ink statistic rather than a letter shape, and it is the same mechanism Section 3.7.1 found on plotter output, appearing this time on a real photograph.

Configuration	none	thicker pen	thinner pen	faint ink	gamma	overall
Ink namer, raw crops (as first deployed)	15/15	15/15	**12/15**	**12/15**	15/15	69/75 (92 %)
Ink namer, stroke-normalised	15/15	14/15	13/15	14/15	12/15	68/75 (91 %)
Shape namer, raw crops (deployed)	15/15	15/15	13/15	15/15	15/15	**73/75 (97 %)**
Shape namer, stroke-normalised	15/15	15/15	15/15	15/15	15/15	75/75 (100 %)

Table 32. Writer identification under a simulated change of pen, ink and camera response, over 15 labelled pages in five conditions each. The ink namer loses three pages to a thinner pen and three to faint ink — exactly the two perturbations that attack an ink statistic — while leaving a thicker pen and a gamma shift untouched. This is the mechanism behind the single real failure, reproduced 75 times.

The configuration actually deployed was then chosen over every page whose writer is known, Table 33.

Configuration	30 dataset pages	out-of-domain page	mean pooled confidence	Verdict
Ink namer, raw (first deployed)	30/30	**0/1**	99.3 %	Confident and wrong off-domain.
Shape namer, raw (deployed)	30/30	**1/1**	92.7 %	Chosen: correct everywhere, at a cost in confidence.
Ink raw + shape raw	30/30	**0/1**	—	Fails: the ink namer is too confident when wrong to be outvoted by one dissenter.
Ink normalised + shape raw	30/30	1/1	—	Also perfect, but needs two forward passes per line.

Table 33. Choosing the page namer over every page whose writer is known. The third row is the instructive one: averaging a confident wrong classifier with a correct one does not repair it, which is why the change is a replacement rather than an ensemble.

The change is one constant in the server and needs no retraining and no new data. Its cost is stated rather than hidden: over the 30 labelled pages the mean pooled confidence falls from 99.3 % to 92.7 % and the mean per-line agreement from 99.7 % to 96.0 %, while the per-line character accuracy is unchanged at 98.5 %. The old figure was partly false confidence — it is the same number that named the wrong writer at 84 %.

One caveat belongs with this result and is not resolved by the 75 synthetic trials: the decisive real-world evidence is a *single* photograph. The perturbations are a model of a pen-and-lighting change, not a sample of one, and the configuration that was perfect under them (shape namer on stroke-normalised crops, 75/75) is *not* the one that was correct on the real page, which shows the model does not capture the whole shift. **[MUST STILL BE COMPLETED BY ME: photograph three or four further pages of known writers with a pen and lighting deliberately unlike the training photographs, and re-run Table 33 over them, so that the out-of-domain column rests on more than one page.]**

FIGURE MUST STILL BE INSERTED BY ME
One line crop shown in all five conditions of Table 32 — unaltered, thicker pen, thinner pen, faint ink, gamma shift — with the two namers' posteriors for the true writer plotted beneath each. The point to show is that the ink namer's posterior collapses on the thinner-pen and faint-ink panels while the shape namer's does not.

Figure 31. One line crop under the five perturbations of Table 32, with each namer's posterior for the true writer beneath. The ink namer fails exactly where the ink statistic is attacked.

3.7.6 A reconsideration not yet carried out: detecting the need for an anchor automatically

One further change was identified during the glyph-library investigation but has not been implemented in the delivered system, and is recorded here as a design reconsideration still *open* rather than as a completed change, since the distinction between what was changed and what is merely proposed matters to how this subsection should be read.

The anchor-substitution mechanism of Section 3.6.5 is driven entirely by the per-glyph prior distance of Equation 82 and the single tuned parameter λ, applied uniformly within a writer. An examination of the glyph cutting and the cross-writer prior distribution for the most strongly joined personal writer showed that her low connectedness-adjusted legibility is not a scattered collection of individually poor letters but a structural property of her hand: her forced-alignment cuts fall through genuine strokes far more often than for a lightly joined writer, because her connectedness statistic of $c = 0.962$ (Table 23) leaves almost no low, thin ligature for Equation 74 to find.

The reconsideration this suggests is to replace the hand-tuned λ by a decision taken from the writer's *own* statistics, forcing anchor substitution on by default when

$$c > c^{*} \qquad \text{or} \qquad \mathbb{E}\big[\text{priorD}\big] > d^{*}, \tag{91}$$

where c is the profile's connectedness, priorD the per-glyph prior distance of Equation 82, and c^{*} and d^{*} two thresholds to be fitted across the thirteen writers. Equation 91 is a proposal and not a result: neither threshold has been fitted, and the evidence for the direction is one writer, Table 34.

[MUST STILL BE COMPLETED BY ME: implement and measure the connectedness-triggered default of Equation 91 across all thirteen writers, not only the one case it was explored on, and fit c^{*} and d^{*} from that sweep.]

[MUST STILL BE COMPLETED BY ME: confirm that these six are the complete set of reconsiderations made or identified during the project, and add any others that belong here — for example a change to the segmentation thresholds, to the number of writers, or to the requirement allocation — each with what was measured before and after.]

Measurement	Tuned λ	Anchors forced on	Reading
Shape classifier confidence in her identity	63–77 %	**94 %**	The direction is worth pursuing.
Character accuracy	baseline	modest further gain **[MUST STILL BE COMPLETED BY ME: quote the two figures]**	Consistent, but not quantified here.
Word accuracy	poor	**poor regardless of glyph source**	Points to a second, separate limitation in the joining geometry that this change would not fix.
All other twelve writers	—	**[MUST STILL BE COMPLETED BY ME: not measured]**	Equation 91 cannot be fitted from one writer.

Table 34. The substitution experiment on the most strongly joined writer's own held-out sentences. The last row is why this remains a proposal: a trigger with two thresholds cannot be fitted on a single data point.

3.8 Software design and system integration

This subsection describes how the blocks of Sections 3.3 to 3.7 form one application on the single-board computer, which runs a web server exposing them to a browser. All are Python programs written for this project; only the camera interface, the pin interface and the transport of web requests come from libraries, with the one exception below.

3.8.1 Process flow and start-up

Figure 32 is the process flow of the two run-time modes. At start-up the application loads the thirteen profiles and the three networks, each a from-scratch array-arithmetic evaluation of trained weights (Sections 3.5 and 3.5); the reproduction chain then calls the synthesiser once with both judges passed in and refuses to send unless calibration has succeeded in this session. Training is a third mode, the writers being pre-built files from offline programs. **[MUST STILL BE COMPLETED BY ME: the board's address and connection to the laptop, the operating system and Python version, how the server is started, and the memory the loaded models use.]**

The inference path is library-free; the sample-crop warm-up is not. A recorded cold-boot import closure confirms that `torch`, `scipy`, `cv2` and `tensorflow` are reached by none of the three networks and none of the classification, synthesis, G-code and motion paths: every weight comes from a plain array archive and every layer is evaluated by the project's own array arithmetic. One path breaks that. The start-up warm-up, given as pseudocode in Figure 34, calls `WRH.SampleCrops` for every writer, and on a cache miss that helper imports `BuildStyleProfile`, `TrainAuthor` and `TrainText`; `WebRenderHelpers.py` prepends `Software/CNN` and its `AuthorReproductionStuff` to `sys.path`, so those names resolve to the *retired* tree, whose `TrainText.py` and `TrainAuthor.py` import `torch`, `torchvision`, `pytesseract` and `scipy` at module level. `WebCache` as delivered has no glyph grid or crops for owen and thiya and none for abhinav, so the miss, and the import, happen on *every* start. **[MUST STILL BE COMPLETED BY ME: port the two**

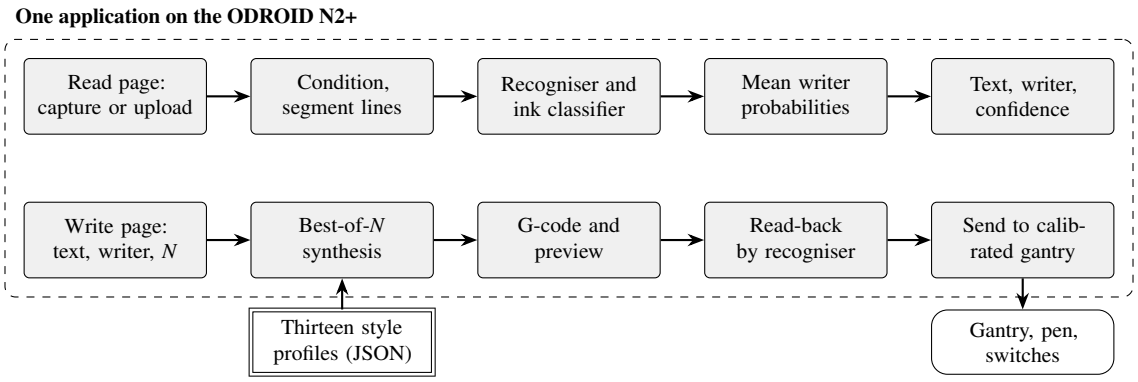


Figure 32. Process flow of the two run-time modes: classification above, reproduction below, the writer the upper chain identifies being what the lower chain is given. The profile builder of Section 3.6 is offline and appears in neither.

sample-crop helpers into `CleanPipeline`, or pre-populate `WebCache` for owen, thiya and abhinav, so that no import reaches `Software/CNN`; then repeat the cold-boot closure on the board.]

Known inconsistency: the stored accuracy figures. One route computes nothing: it returns figures hard-coded in `Server.py`, several no longer agreeing with Section 4 — the ink-based classifier at 100.0 %, the stroke-normalised one at 96.7 % on real ink and 99.2 % on synthesised output, the recogniser at 89.4 % on real ink, and the pipeline at 99.2 % writer identification with 85.8 % and 84.1 % character accuracy direct and through G-code. The code is stale here, not the report, and the page displaying those figures contradicts the Results chapter. **[MUST STILL BE COMPLETED BY ME: update the hard-coded accuracy block of `/api/model_info` to the final figures of Section 4, or have the route read them from the evaluation output.]**

3.8.2 Timing chains of the two modes

Requirement R4 limits the classification of x characters by Equation 92,

$$T_{\text{cls}} = T_{\text{cap}} + T_{\text{seg}} + \sum_{\text{lines}} \left(T_{\text{text}} + T_{\text{wid}} \right) \leq 2\ \text{s} + 0.025\, x\ \text{s}, \tag{92}$$

leaving both network passes 25 ms per character, and R3 limits reproduction by Equation 93,

$$T_{\text{rep}} = T_{\text{synth}} + T_{\text{gcode}} + T_{\text{motion}} \leq 4\, x\ \text{s}, \qquad T_{\text{synth}} \approx N C, \tag{93}$$

with C the cost of one candidate (Equation 88), T_{gcode} in milliseconds and T_{motion} the job time of Section 3.3.4; the calibration is excluded and T_{synth} included, the stricter reading of the proposal. A budget is only a design once it is apportioned, and Table 35 gives the allowance this design assigned to each stage of Equations 92 and 93 against the figure later measured for it.

Requirements R3 and R4 bound the *total* time of a job, and Equations 92 and 93 decompose each total into the stages that spend it. A budget is only useful once it is apportioned, and Table 35 states the allowance this design assigned to each stage, together with the laptop figure measured for it. The apportionment was made at design time from the structure of the chain, not fitted to the measurements: the recognition and classification terms were given the whole per-character allowance of R4 because they are the only terms that grow with x, leaving the fixed 2 s for capture,

conditioning and segmentation; and in R3 the physical writing was given three quarters of the per-character allowance, since it is the one term that cannot be made faster by a faster computer.

Stage	Design-stage allowance	Measured on the laptop, and the verdict
R4: capture, conditioning and segmentation	the whole fixed 2 s, being independent of x	24.8 to 69.1 s per page, 12 to 35 times the allowance: the term that fails R4, and it fails by an order of magnitude rather than marginally
R4: recognition and writer classification	the whole 25 ms per character, being the only terms growing with x	8.86 ms per character for recognition and 4.6 to 10.3 ms for classification, so about half the allowance together: met with margin
R3: synthesis	1 s per character, a quarter of the allowance	0.4 to 0.7 s per character at $N = 1$, rising to 1.8 to 3.4 s at $N = 30$: met only at small N, which is why N is the parameter that decides whether R3 can be met at all
R3: G-code generation	negligible by design, the generator doing no search	0.006 to 0.014 s per sentence: met by three orders of magnitude
R3: physical writing	3 s per character, three quarters of the allowance, being the irreducible term	2.7 to 4.0 s per character from the generator's model, no stopwatch reading: cannot be declared met, and the upper end of the model already exceeds the allowance

Table 35. Apportionment of the budgets of Equations 92 and 93 across the stages, with the laptop figures of Section 4 against each. No stage was timed on the single-board computer, so every verdict here is provisional on hardware that is faster than the target. The two stages that fail are the two that were never redesigned after first being written: page segmentation and the physical writing.

Read against Table 35, the failure of R4 is localised rather than general: four of the five stages are inside their allowance, two of them by a wide margin, and the whole shortfall sits in page segmentation. That is also the one stage for which a remedy was demonstrated but not deployed, the two routines reimplemented in C99 having reduced the retired segmenter by 25 to 32 % end to end (Section 3.4); a saving of that size would not by itself close a factor of twelve, so the conclusion the apportionment supports is that segmentation needs a different algorithm or a different language, not a faster implementation of the same one.

The emitting loop is short, and its one subtlety is the pen state: because the lift is a one-way toggle motor, a pulse emitted where none is needed inverts the pen for the remainder of the job, so the pen command is emitted only on a change of state.

```
emit the comment header, G21 (millimetres) and G90 (absolute)
emit the travel feed once

for each stroke of at least two points:
    SetPen( up )                  # emits M5 and a dwell ONLY on a change
    rapid move to the first point of the stroke
    SetPen( down )                # emits M3 and a dwell ONLY on a change
    emit the draw feed
    for every remaining point of the stroke:
        emit a linear move to that point, three decimals

SetPen( up ) ;  move to the park position ;  emit M2
```

Figure 33. The emitting loop of the G-code generator as pseudocode. `SetPen` returns nothing at all when the pen is already in the wanted state, which is what keeps the one-way toggle motor of Section 3.3.5 in step with the program's model of it; the dwell it emits on a change is the $260 + 90$ ms of Table 9. Fault 2 of Section 3.3.6 is visible here: the draw feed is emitted before every stroke but the travel feed only once, so every travel after the first runs at the draw feed.

The start-up warm-up is the one path in the application that can reach a machine-learning library,

as Section 3.8 records.

```
Prewarm( ):                  # a background thread, not the inference path
    for each writer, in order:
        try:
            build or fetch the cached glyph grid for that writer
            build or fetch two cached sample crops for that writer
        except any error:
            warn and continue              # a failure must not stop start-up
        mark that writer's cache as warm

start Prewarm in a background thread at start-up
any route needing a cached file answers "still preparing" until it is marked
```

Figure 34. The start-up warm-up as pseudocode. The two cache-building calls are the ones that, on a miss, reach into the retired source tree and import a machine-learning library; because three writers have no cached files as delivered, that miss happens on every start, which is the defect Section 3.8 records. Running it on a background thread is what keeps the delay off the first request, and marking each writer separately is what lets the routes that do not need a cache answer immediately.

The apportionment follows from the structure of the chain rather than from the measurements: the two network terms were given the whole per-character allowance of R4, being the only terms that grow with x, which leaves the fixed two seconds for capture, conditioning and segmentation; and in R3 the physical writing was given three quarters of the per-character allowance, being the one term that no faster computer can reduce. Read against that apportionment, four of the five stages are inside their allowance and the whole R4 shortfall sits in page segmentation, at twelve to thirty-five times what it was allowed. Table 36 lists the stages, and *no stage was timed on the single-board computer*: a 56-character sentence allows 224 s, while the search alone took 23 to 137 s at $N = 14$ on the laptop and the motion model gives 2.4 to 2.8 s per character, so R3 and R4 are not claimed to be met. **[MUST STILL BE COMPLETED BY ME: the timings of the stages on the single-board computer, listed in Table 36.]**

Stage	Known information and value to be supplied
Capture, conditioning and segmentation	Framing by the user, five discarded frames without preview; whole page on the board. **[MUST STILL BE COMPLETED BY ME: time from pressing capture to the stored image.]** **[MUST STILL BE COMPLETED BY ME: conditioning and segmentation time per page on the board.]**
Recognition and writer classification	Two passes of about 4×10^9 multiply–accumulate operations per line. **[MUST STILL BE COMPLETED BY ME: time per line on the board and the total for the R4 test paragraph.]**
Synthesis, G-code and preview	Synthesis time is returned in the server response (laptop values in Table 27); the generator and preview are milliseconds, not measured. **[MUST STILL BE COMPLETED BY ME: synthesis time on the board for the N used in the demonstration.]**
Gantry motion	Model 2.4 to 2.8 s per character (Section 3.3.4). **[MUST STILL BE COMPLETED BY ME: stopwatch time of a sentence of known length, from send to the final pen lift.]**

Table 36. Stages of the timing chains of the two modes and the measurements still missing.

Complete code has been submitted separately on the AMS. **[MUST STILL BE COMPLETED BY ME: confirm which archive and revision was submitted on the AMS.]**

4. Results

This section describes how the system was measured against the requirements and specifications of section 4 of the approved Project Proposal, and then gives every measurement that was taken, including those that fall short of a target.

Three principles govern what follows. The first is that the *classifier* and the *reproduction* are qualified on different material. Requirement R2 asks the system to read and name real handwriting, so it is measured on material held out from the network that performs the task. Requirement R1 asks the system to reproduce text in a writer's style, so it is measured on pages the robot itself drew; there the classifier is the measuring instrument and not the subject, and it is admissible as an instrument only because qualification test 2 establishes independently what it does on real handwriting.

The second principle is that a measurement is reported only against material whose provenance was checked, not material assumed to be unseen. The reader and the writer classifier were trained on different corpora, so what is held out differs between them, and qualification test 2 establishes by transcript lookup which lines are genuinely outside each network's training data before scoring any of them. Where that check disqualified material, the material is excluded and the exclusion is stated.

The third principle is that a line of handwriting is the unit of measurement. Each page carries between three and eighteen lines, each line is scored separately, and the number of lines behind every figure is stated.

Two machines were used, and which one produced a figure is stated with the figure. The timing measurements of qualification tests 3 and 4 were taken on the ODROID N2+ that carries the deployed system, driving the real gantry, so they describe the target platform. The accuracy measurements of qualification tests 1, 2 and 6 were taken on the development computer, an x86-64 machine running CPython 3.14; the platform affects how long those measurements took and not what they were, because the same weights and the same arithmetic produce the same answer on either machine. The geometry of qualification test 5 was measured by hand off drawn paper and belongs to neither.

4.1 Summary of results achieved

Table 37 compares the intended outcomes of the approved Project Proposal with the outcomes that were measured. The first column reproduces the requirement and its target specification as the proposal states it, the second gives the measured value, and the third points to the qualification test in which the measurement was made.

4.2 Qualification tests

The tests below appear in the order of importance of the requirements in section 4 of the approved Project Proposal. Each test states its objective, the equipment and the software written to perform it, the parameters and steps, the measurements, the observations that follow from them without interpretation, and, where the sample supports one, a statistical analysis.

Intended outcome	Actual outcome	Location in report
Core mission requirements and specifications		
R1: Reproduce text in the style of any writer of the training set, at least 85 % character accuracy and 85 % stylistic correlation.	**Partially met.** Character accuracy over 184 drawn lines of 14 pages was 84.41 % (853 of 5 471 characters wrong), 0.59 points below the target; word accuracy was 45.97 %. The stylistic half was measured two ways and they disagree. As the correlation the proposal specifies, it is **not met**: the pooled coefficient over nine geometric features is 0.714 against a target of 0.85. As writer identification from the robot's own output it is met: 159 of 184 lines, 86.4 % (95 % Wilson interval 80.7 to 90.6), and 14 of 14 pages when a page's lines are pooled.	Qualification test 1
R2: Decode the text and identify the writer with an overall accuracy of at least 95 %.	**Partially met.** The writer was named correctly on 153 of 156 lines held out from the writer classifier, 98.08 %, and on 11 of 11 writers when each writer's lines are pooled, which exceeds the target. The text of 1 076 lines held out from the reader was read at 94.39 % character accuracy and 73.47 % word accuracy, which is 0.61 points short of it.	Qualification test 2
R3: Reproduce text at no more than 4 s per character.	**Met.** Every complete reproduction recorded in the timing log met its allowance. The worst case over all 26 recorded runs was 3.207 s per character, 80.2 % of the allowance, on a 43-character block; over the nine attributable runs of the 378-character paragraph the range was 1.343 to 2.705 s per character, at most 67.6 % of the allowance.	Qualification test 3
R4: Identify the writer and decode the text within $2 + 0.025x$ s for x characters.	**Not met.** On the target platform a 500-character page took 46.26 s against an allowance of 14.50 s and a 982-character page 55.49 s against 26.55 s. The two network stages exceed the per-character term of the allowance by a factor of 1.6 to 2.4 and the four image stages exceed the fixed 2 s term by a factor of 8.1 to 8.4, so the requirement fails at every page length.	Qualification test 4
R5: Absolute gantry positional error of at most 0.5 mm.	**Met on the evidence available.** On a 100 mm calibration square the worst side was 0.30 mm from nominal and the mean absolute error was 0.175 mm. One square and one triangle were measured, which is not sufficient for a repeatability figure.	Qualification test 5
R6: At least 10 writing styles trained and stored permanently.	**Met.** 11 writer styles are trained and stored: six from the public database and five photographed for this project.	Qualification test 6
Field condition requirements and specifications		
FC1: Operate on pages photographed by the system's own camera under ordinary indoor lighting.	**Met.** All 14 reproduction pages and all camera captures were photographed indoors under room lighting with the system's own webcam at 3264×2448, and were processed without manual adjustment.	Qualification test 1, Qualification test 4
FC2: Operate on ordinary unruled A4 paper with a ballpoint pen.	**Met.** Every page reported here was drawn on unruled 80 g/m^2 A4 with the same ballpoint pen in the gantry pen holder.	Qualification test 1

Table 37. Summary of results achieved.

4.2.1 Test material and the software written to perform the tests

One body of test material serves the first two tests and is described once here. Fourteen pages were drawn by the robot, one or two for each of eleven writers, and each page carries up to three blocks of text chosen to separate three different questions. Block A is a 378-character paragraph, long enough to occupy eight to seventeen written lines, which exercises line segmentation and general text accuracy together. Block B is the 43-character pangram “The quick brown fox jumps over the lazy dog”, which contains every letter of the alphabet and occupies one or two lines, so it tests reading and writer identification on a single line with no multi-line segmentation to confound them. Block C is a passage taken from the writer’s own source material, which is the only text of the three that the writer themselves actually wrote.

Each drawn page was photographed, segmented into line crops, and every crop was read and classified separately, giving 184 line samples in total. A transcript was prepared for every page with one line of text for each written line, so that a read line and its intended text can be compared one to one; the line breaks of each transcript were taken from that page’s own segmentation overlay and not assumed.

Software written for the tests. Five programs were written for this chapter and no part of the measurement is done by hand. `blocks_master.py` segments each drawn page, reads and classifies every line, assigns each line to its block, and writes one record per line; `holdout_text.py` and the writer half of the same harness score the held-out real crops; `cursivity.py` measures how joined each writer’s genuine hand is; `style_distance.py` places the writers in a style space and measures how far each is from the others; `classify_gcode.py` renders a G-code job to an image and classifies it without the page ever being drawn. All figures in this chapter are produced by `make_results_figures.py` from a single colour and font definition, so that every figure is formatted alike.

Qualification test 1: Reproduction of text in a writer’s style (R1)

Objectives of the test or experiment

To determine whether text drawn by the robot in a stored writer’s style is legible as the intended text, and whether it still carries the identity of that writer. R1 sets two targets, 85 % character accuracy and 85 % stylistic correlation, and both are measured here.

Equipment used

The gantry described in Section 3.3, driven by an ODROID N2+; a Logitech C270 webcam at 3264×2448 mounted above the platen; unruled 80 g/m^2 A4 paper; a 0.7 mm ballpoint pen in the gantry pen holder; and a 150 mm vernier caliper reading to 0.01 mm for the page measurements. Scoring used the software named in Section 4.2.1.

Test setup and experimental parameters

Pages were drawn at a pen-down feed of 800 mm/min and a pen-up feed of 4 000 mm/min, with the best-of-N synthesis search at $N = 1$ except for the two most joined hands. The writer was identified by the stroke-normalised shape classifier `author_shape_11_pw170_weights.npz`,

which redraws every crop at a constant pen width of 0.17 x-heights before classifying it, and the text was read by `paper_cnn_bilstm_ctc_joint_best.npz` with greedy decoding, no beam search, no language model and no lexicon. Writers 151 and 551 were withdrawn from the trained set, so the classifier chooses between 11 writers and chance is 9.1 %.

Steps followed in the test or experiment

1. Synthesise each block of text in the stored style of the writer and emit G-code.
2. Draw the page on the gantry and photograph it with the system's own camera.
3. Segment the photograph into line crops and confirm that the number of crops equals the number of transcript lines.
4. Read every crop and score it against its transcript line.
5. Classify every crop and record the writer named, the confidence, and the probability given to the true writer.
6. Pool the lines of a page by ink to obtain one writer per page.

Results or measurements

Table 38 gives the result by block and Table 39 by writer. Character and word accuracy are reported as micro averages, that is one sum of edit distances over one sum of reference characters, so that a long line counts for more than a short one; the macro average over all 184 lines, which weighs every line alike, is 82.04 % character and 48.80 % word. Figure 35 shows the same block figures with their intervals, Figure 36 the per-writer figures, and Figure 37 the full confusion of writer identification over the 184 lines.

Block	Lines	Writer identified	Character	Word
A: paragraph, many lines	119	91.6 % [85.2, 95.4]	84.88 %	47.87 %
B: pangram, single line	21	76.2 % [54.9, 89.4]	78.40 %	41.41 %
C: writer's own source text	44	77.3 % [63.0, 87.2]	85.32 %	40.91 %
All three blocks	184	86.4 % [80.7, 90.6]	84.41 %	45.97 %

Table 38. Reproduction accuracy by block of text, over the 14 drawn pages. Bracketed figures are 95 % Wilson intervals on the proportion of lines whose writer was identified correctly. Character and word accuracy are micro averages.

Connectedness was measured on each writer's genuine pages as the number of connected ink components per character, over 45 sampled real line crops per writer. Against character accuracy the Pearson coefficient is $r = +0.855$ and the Spearman rank coefficient $\rho = +0.845$ over the 11 writers; against word accuracy $r = +0.771$ and $\rho = +0.782$; against the proportion of lines whose writer was identified, $r = +0.170$ and $\rho = -0.027$. Figure 38 plots the first and the last of these.

A writer may also be hard to identify because another writer's hand resembles theirs. To measure this, each writer was placed in a style space of eight standardised geometric descriptors taken from their own style profile — slant, x-height, ascender and descender extent, stroke width, ink fraction, aspect ratio and mid-band fraction — and the Euclidean distance to the nearest other

Writer	Lines	Writer identified	Character	Word	Comp./char
Y	13	92.3 % [66.7, 98.6]	97.01 %	74.16 %	0.739
150	13	53.8 % [29.1, 76.8]	96.75 %	76.40 %	0.766
O	16	100.0 % [80.6, 100]	94.25 %	58.51 %	0.951
B	16	93.8 % [71.7, 98.9]	93.11 %	68.24 %	0.980
384	16	87.5 % [64.0, 96.5]	86.93 %	43.18 %	0.681
552	15	80.0 % [54.8, 93.0]	86.17 %	48.39 %	0.609
152	15	80.0 % [54.8, 93.0]	85.74 %	49.44 %	0.512
A	27	100.0 % [87.5, 100]	77.29 %	18.09 %	0.527
153	19	57.9 % [36.3, 76.9]	75.72 %	31.46 %	0.255
588	18	94.4 % [74.2, 99.0]	73.19 %	22.99 %	0.547
T	16	100.0 % [80.6, 100]	60.97 %	13.41 %	0.208

Table 39. Reproduction accuracy by writer, ordered by character accuracy. A, B, O, T and Y are the five photographed contributors; the numbered writers are from the public database. The last column is connectedness, measured as connected ink components per character on the writer's own genuine pages, where a lower value is a more joined hand.

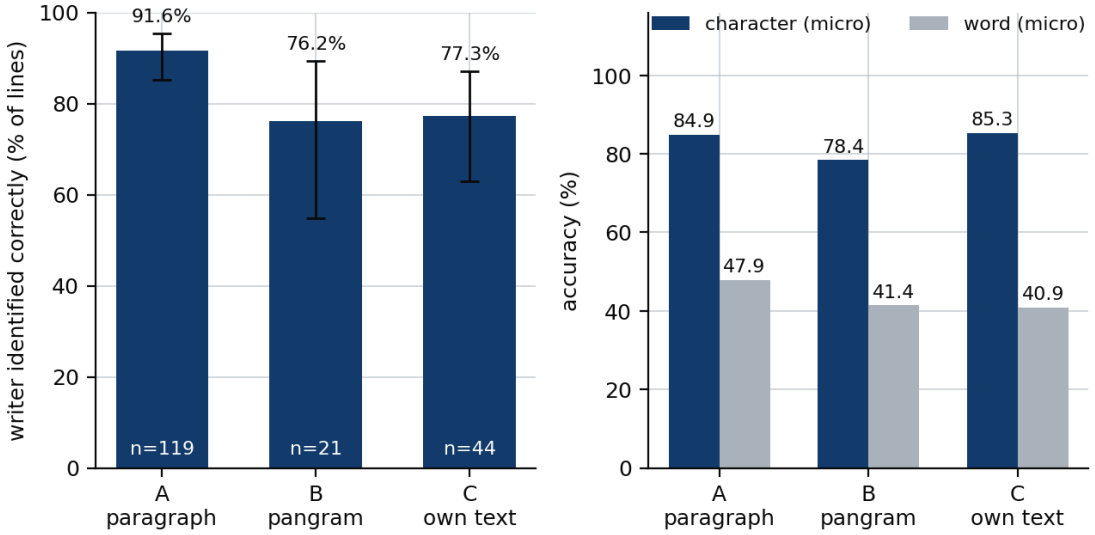


Figure 35. Reproduction accuracy by block. The left panel is the proportion of lines whose writer was identified correctly, with 95 % Wilson intervals and the number of lines behind each bar; the right panel is micro-averaged character and word accuracy.

writer was computed. Against the proportion of lines whose writer was identified, that distance gives $r = -0.244$ and $\rho = -0.100$. The three largest confusions of Figure 37 are 153 named as T on 6 lines, 150 named as Y on 5 lines and 552 named as A on 3 lines; in none of the three is the named writer the true writer's nearest neighbour in that style space, which are A, 152 and 588 respectively.

Six G-code jobs were rendered to images and classified without being drawn, which separates what the synthesis produces from what the gantry then does with it. Table 40 gives the result.

R1's stylistic specification is stated as a correlation, and it was measured as one. Nine geometric features were extracted from each writer's genuine lines and from the system's reproduction of the same text, and the Pearson coefficient between the two was taken feature by feature and

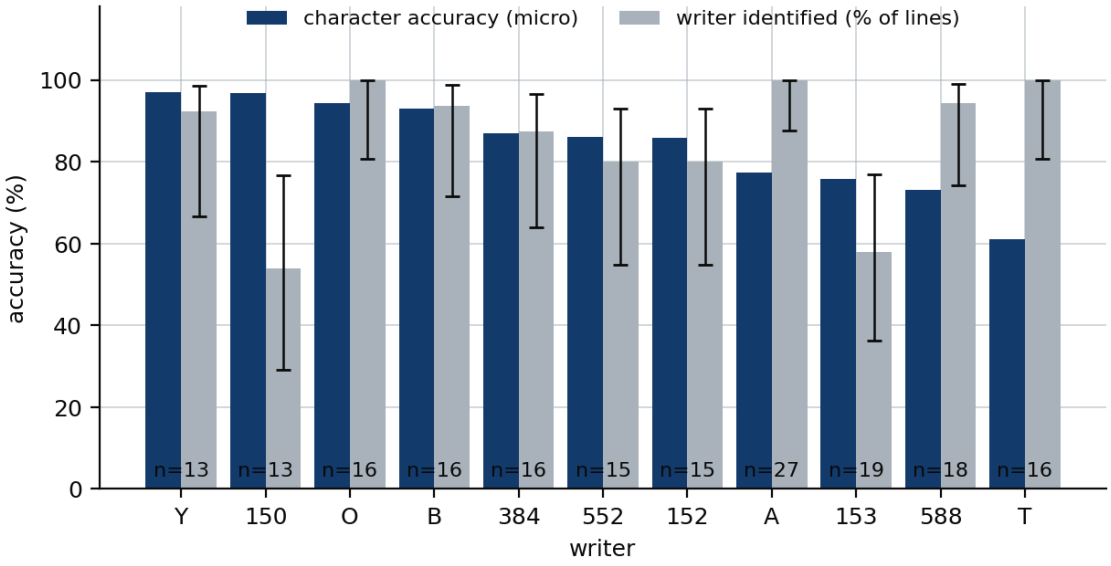


Figure 36. Reproduction accuracy by writer, ordered by character accuracy. Dark bars are micro-averaged character accuracy and silver bars the proportion of lines whose writer was identified, with 95 % Wilson intervals.

G-code job	Lines	Text lines	Named	Conf.	Lines agreeing
`2b7e03f1`	9 399	8	O	86.0 %	7/8
`99f136a0`	5 889	7	O	66.7 %	6/7
`bc8d7860`	13 747	17	A	75.6 %	15/17
`a6393598`	9 766	11	B	29.3 %	4/11
`16d4c05b`	1 255	2	B	72.4 %	2/2
`bfa7a5dc`	1 288	1	Y	79.2 %	1/1

Table 40. Writer identification from the G-code alone, with no page drawn. The job `2b7e03f1` has 9 399 G-code lines, which is the count the server log records for the run annotated `Owen_curated`.

pooled. A separate overlay test measured how far a reproduction sits from the hand it imitates against how far it sits from the other writers' hands, reduced to the ratio of the two mean distances and the area under the resulting curve. Table 41 gives both; Table 50 in the appendix gives the nine features separately and Figure 49 the two distance distributions.

A third measurement of style was taken that involves no classifier at all. The same 378-character paragraph was prepared for every one of the eleven writers, and the geometry of the resulting G-code was measured directly: the pen-down path length, which is the quantity of ink the machine would lay down, and the number of pen lifts, which is how often the hand leaves the paper. Both are properties of the emitted tool path and are read off it arithmetically, so neither can be influenced by the network that selects candidates. Figure 39 gives both for identical text.

Observations

Character accuracy over all 184 lines is 84.41 %, which is 0.59 points below the 85 % target; 853 of 5 471 characters were wrong. The proportion of lines whose writer was identified is 86.4 %,

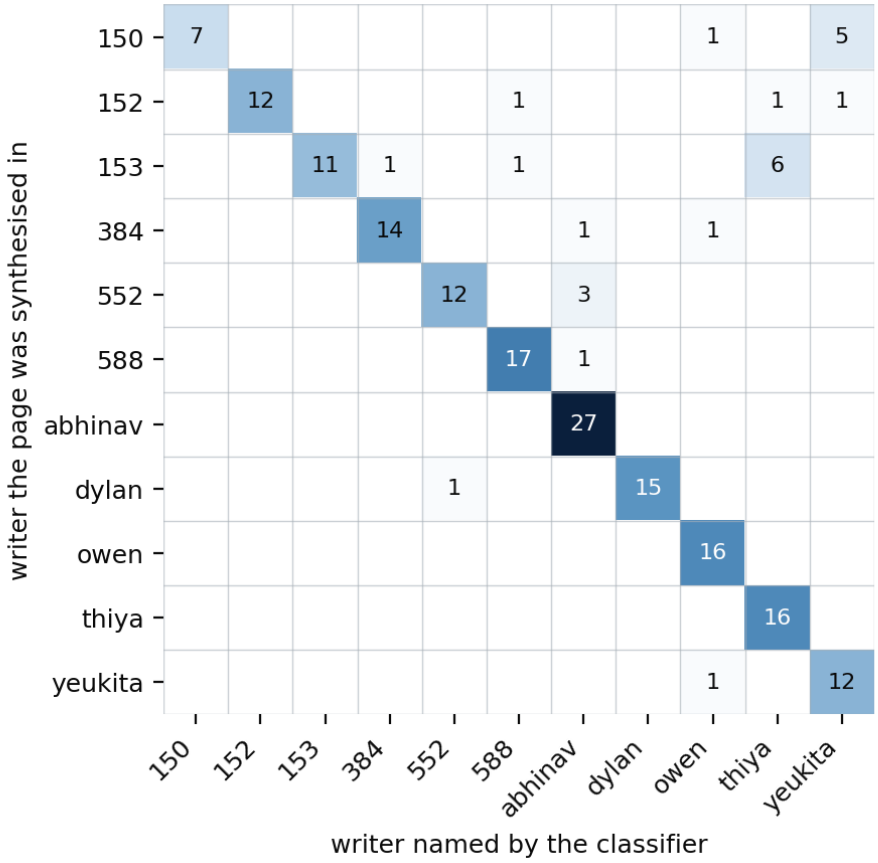


Figure 37. Writer identification over the 184 drawn lines. Rows are the writer the page was synthesised in and columns the writer the classifier named; the diagonal holds the 159 correct lines. Empty cells are zero.

Measure	Measured	Target	Verdict
Pooled style correlation over nine features	0.714	0.85	Not met
Best single feature (slant)	0.996	—	—
Worst single feature (descender extent)	0.148	—	—
Overlay distance ratio, mismatched to matched	2.2	—	—
Overlay area under the curve	0.71	—	—
Writer identified from the drawn output, by line	86.4 %	85 %	Met
Writer identified from the drawn output, by page	14/14	—	Met

Table 41. The two ways R1's stylistic half was measured. The correlation is the measure the proposal states and it falls 0.136 below its target; writer identification from the machine's own output exceeds 85 %. An area under the curve of 0.5 would mean the reproduction carries no writer information at all.

above the 85 % figure, and all 14 pages were attributed to the correct writer when their lines were pooled.

Of the three blocks, block A has the highest proportion of lines identified at 91.6 % and block B the lowest at 76.2 %. Block C has the highest character accuracy at 85.32 % and block B the lowest at 78.40 %. The intervals of blocks B and C overlap over most of their range.

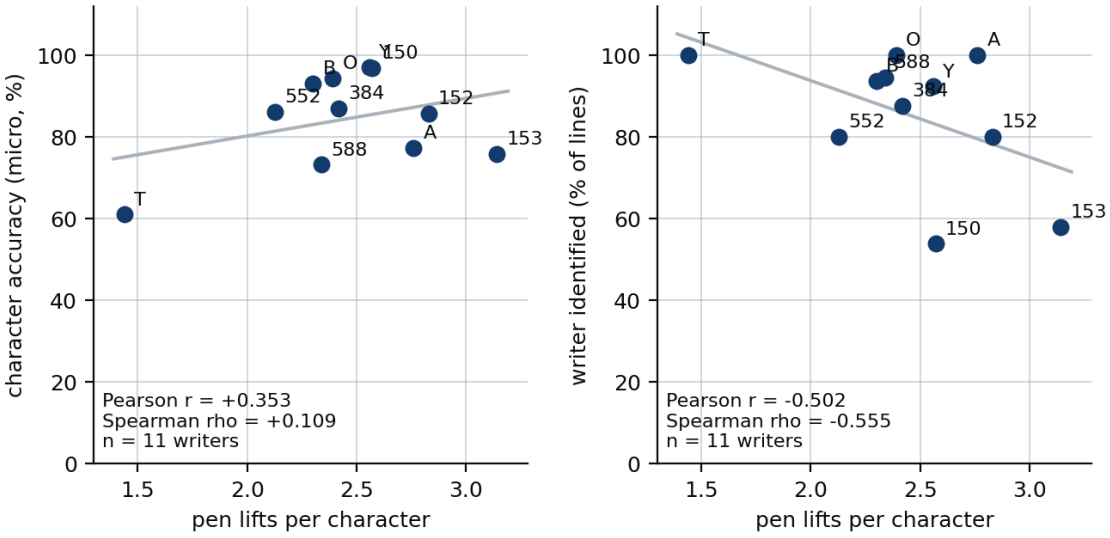


Figure 38. Accuracy against connectedness. The horizontal axis is connected ink components per character on the writer's own handwriting, where a lower value is a more joined hand. The grey line is a least-squares fit and the coefficients are printed on each panel.

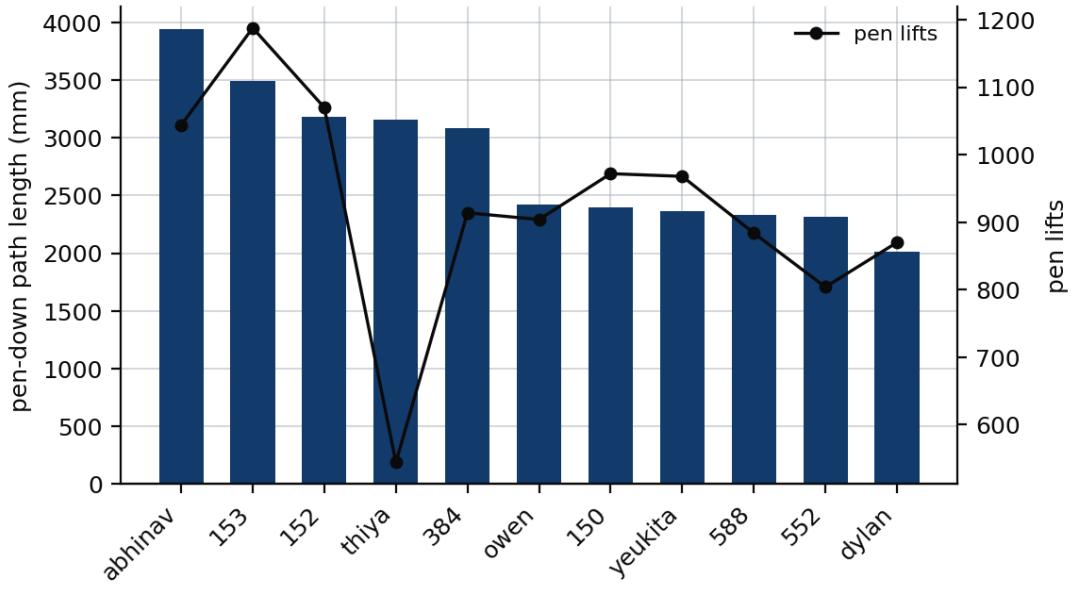


Figure 39. Pen-down path length and pen lifts for the same 378-character paragraph prepared in each of the eleven writers' styles, measured from the G-code before any page was drawn. Bars are the path length against the left axis and the line is the pen-lift count against the right axis; writers are ordered by path length.

Character accuracy by writer spans 60.97 % to 97.01 %, a range of 36.0 points. Four writers exceed 93 %: Y, 150, O and B. The four are also the four least joined hands of the eleven, with 0.739 to 0.980 components per character. The three lowest character accuracies are T, 588 and 153, and two of those three, T at 0.208 and 153 at 0.255, are the two most joined hands measured.

The rank correlation between connectedness and character accuracy is $\rho = +0.845$; between connectedness and the proportion of lines identified it is $\rho = -0.027$. The rank correlation between distance to the nearest writer in style space and the proportion of lines identified is $\rho = -0.100$.

Three writers were identified on every line: A, O and T. Two writers were identified on fewer than 60 % of their lines: 150 at 53.8 % and 153 at 57.9 %. The writer with the highest character accuracy, Y at 97.01 %, and the writer with the lowest, T at 60.97 %, were identified on 92.3 % and 100 % of their lines respectively.

Of the six G-code jobs classified before drawing, five were named with between 66.7 % and 86.0 % confidence and agreement on at least six of every seven lines; one, `a6393598`, was named at 29.3 % with four of eleven lines agreeing.

For the same 378-character paragraph prepared in all eleven styles, the pen-down path length ranges from 2 015.4 mm to 3 942.8 mm, a factor of 1.96, and the pen-lift count from 544 to 1 188, a factor of 2.18. The two do not rank together: the writer with the fewest pen lifts, T at 544, has the fourth longest path at 3 157.7 mm, and the writer with the most lifts, 153 at 1 188, has the second longest at 3 494.4 mm.

Statistical analysis

The 95 % intervals in Tables 38 and 39 are Wilson score intervals, which are used in place of the normal approximation because several cells hold fewer than 20 lines and the normal interval would extend past 100 %. The interval on the pooled 184 lines is 80.7 to 90.6 %, so the lower bound lies below the 85 % figure and the result is not separated from the target by this sample.

The correlations are reported with both the Pearson and the Spearman coefficient because $n = 11$ and a single writer can move a Pearson coefficient appreciably. For character accuracy against connectedness the two agree closely, +0.855 and +0.845; for writer identification against connectedness they do not, +0.170 and −0.027.

No correction for multiple comparisons has been applied to the five correlations reported, and at $n = 11$ writers none of them is separated from zero with the confidence that a larger sample would allow.

Qualification test 2: Reading the text and naming the writer of unseen handwriting (R2)

Objectives of the test or experiment

To determine the accuracy with which the system reads the text of, and names the writer of, handwriting that the network performing the task was not trained on. R2 sets a single target of 95 % overall accuracy. The two halves are performed by two separate networks trained on different corpora, so each is measured on material held out from that network, and the two halves are reported separately.

Equipment used

No gantry hardware is involved. The development computer described at the head of this section was used, with two trained networks: the reader, checkpoint `paper_cnn_bilstm_ctc_scratch_best.npz`, and the writer classifier, checkpoint `author_shape_11_pw170_weights.npz`, which was trained on crops redrawn at a constant pen width of 0.17 x-heights. Two scoring programs were written for this test: one that

scores the reader against a transcript line by line, and one that indexes the public corpus by transcript so that the provenance of every line can be established.

Test setup and experimental parameters

The two networks have different training corpora, so the held-out material differs between the two halves of the test, and establishing what is genuinely held out is part of the test.

The reader. The reader was trained on the IAM line corpus as distributed by Teklia, mixed with the photographed writers' lines. That corpus arrives already partitioned into a `train` directory of 6 481 line crops and a `validation` directory of 976. The training loop reads the first through the optimiser and the second only through the evaluation path, so no weight is ever updated from the validation crops. Those 976 crops are therefore held out by construction, and because they are the corpus's own line cuts they also carry no dependence on this project's segmenter. The photographed writers contribute a further 100 held-out lines, a seeded 15 % of their lines fixed by the recorded split seed. The text half is measured on these 1 076 lines.

The database writers' own pages are *not* used for the text half, and the reason is a provenance check rather than a preference. Every labelled line of those pages was looked up in the public corpus by normalised transcript. For writer 150, 84 of its 91 labelled lines were found in the `train` directory; for writer 152, 80 of 87; for writer 153, 83 of 97. Those lines are in the reader's training data and cannot measure it. The remaining database writers' pages were found in neither directory, so they lie outside the corpus the reader was trained on rather than being held out from it, and they are excluded for that reason. Not one line of any database writer appears in the `validation` directory, so the two held-out sets used below share no line with each other.

The writer classifier. The classifier was trained only on the eleven writers' own pages and never on the public corpus, so its held-out material is the project's own split, fixed by seed: the last labelled page of each database writer, and the seeded 15 % of each photographed writer's lines. That gives 156 held-out lines, 56 from database writers and 100 from photographed writers. The public corpus cannot contribute to this half, because its crops are distributed without a writer identity.

Steps followed in the test or experiment

1. Index every transcript in both directories of the public corpus, and look up each database writer's labelled lines to establish which are in the reader's training data.
2. Read all 976 held-out crops of the public corpus with the reader and score each against its transcript.
3. Rebuild the photographed writers' data set, apply the project's seeded split, and read the 100 held-out lines in the same way.
4. Redraw each of the 156 crops held out from the writer classifier at a constant pen width of 0.17 x-heights, classify it, and compare the named writer with the label.
5. Pool the lines of each held-out page by ink to obtain one writer per page.

Results or measurements

Table 42 gives the text half over the 1 076 lines held out from the reader, Table 43 the writer half over the 156 lines held out from the classifier, and Figure 40 the confusion of writer identification over those same 156 lines.

Held-out set	Lines	Characters	Character	Word	Lines exact
Public corpus, validation split	976	42 072	94.39 %	73.67 %	212
Photographed writer A	31	934	94.65 %	76.92 %	9
Photographed writer B	19	739	97.70 %	84.73 %	9
Photographed writer O	22	1 017	94.79 %	69.05 %	8
Photographed writer T	11	345	89.28 %	57.69 %	1
Photographed writer Y	17	640	92.50 %	54.62 %	3
All photographed writers	100	3 675	94.42 %	70.74 %	30
All held-out lines	**1 076**	**45 747**	**94.39 %**	**73.47 %**	**242**

Table 42. Reading 1 076 lines of handwriting held out from the reader. Character and word accuracy are micro averages over the character and word counts shown. The validation split is the public corpus's own held-out partition; the photographed writers' lines are a seeded 15 % of their own. "Lines exact" counts lines read with no error at all.

Writer	Held-out lines	Lines named correctly	Page named correctly
150	9	8/9	Yes
152	8	7/8	Yes
153	9	9/9	Yes
384	12	12/12	Yes
552	10	10/10	Yes
588	8	8/8	Yes
A	31	31/31	Yes
B	19	18/19	Yes
O	22	22/22	Yes
T	11	11/11	Yes
Y	17	17/17	Yes
All	**156**	**153/156 = 98.08 %**	**11/11**

Table 43. Naming the writer of 156 lines held out from the writer classifier. The database writers' held-out material is the last labelled page of each writer and the photographed writers' a seeded 15 % of their lines. The last column pools the lines of each writer's held-out material by ink and names one writer from the pool.

Observations

The writer was named correctly on 153 of 156 held-out lines, 98.08 %, which is 3.08 points above the 95 % target. Pooling each writer's held-out lines by ink names all 11 writers correctly.

The three errors fall on three different writers, one line each, and all three were named as the same writer, Y. All three are the last line of the writer's held-out material: line 9 of 9 for writer 150, line 8 of 8 for writer 152 and line 19 of 19 for writer B. Eleven of the 156 lines are last lines.

The three incorrect lines were named at 44.2 %, 83.8 % and 39.5 % confidence. Over the 153 correct lines the median confidence is 99.9 % and the mean 97.7 %; over the three incorrect lines the mean is 55.8 %. Six lines in total were named at below 80 % confidence, of which two are among the three errors.

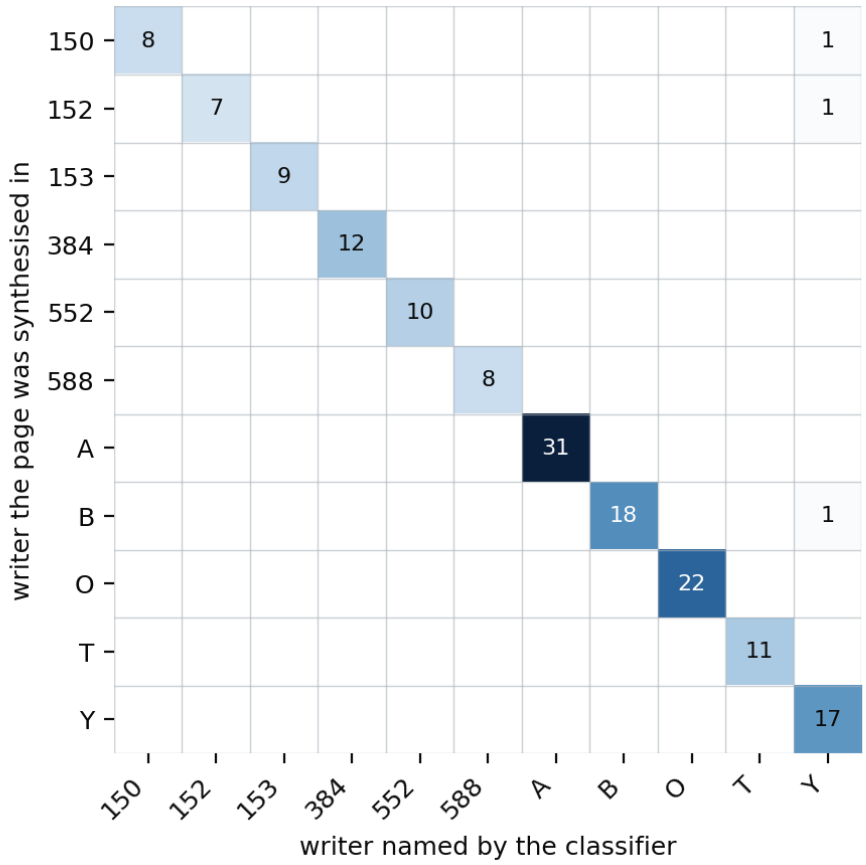


Figure 40. Writer identification over the 156 lines held out from the writer classifier. Rows are the true writer and columns the writer named; three lines are off the diagonal. Empty cells are zero.

The text of the 1 076 lines held out from the reader was read at 94.39 % character accuracy, 2 565 of 45 747 characters wrong, and 73.47 % word accuracy. This is 0.61 points below the 95 % target on characters and 21.53 points below it on words. Of the 1 076 lines, 242 were read with no error at all.

The two held-out sets agree closely with each other. The public corpus's 976 crops were read at 94.39 % and the 100 photographed lines at 94.42 %, a difference of 0.03 points. Word accuracy differs more, 73.67 % against 70.74 %.

Character accuracy by photographed writer spans 89.28 % to 97.70 %, a range of 8.4 points. The proportion of lines read with no error at all spans 9.1 % to 47.4 % over the same five writers, against 21.7 % on the public corpus.

Statistical analysis

The 95 % Wilson score interval on 153 of 156 lines named correctly is 94.5 to 99.3 %. The lower bound lies 0.5 points below the 95 % target, so this sample places the result above the target as a point estimate but does not separate it from the target at 95 % confidence. Pooling by writer gives 11 of 11, whose interval is 74.1 to 100 %; a count of eleven cannot resolve a 95 % target, which is why the per-line figure is the one quoted.

A worst-case bound can be placed on the same sample without assuming that any low-confidence line is right. Taking every line named at below 80 % confidence to be wrong, whether it was or not, leaves 149 of 156 correct, 95.51 %, which is still above the 95 % target. The result therefore clears the target on the pessimistic reading as well as on the measured one.

The concentration of the three errors on last lines was tested against chance. Eleven of the 156 lines are last lines, so under the hypothesis that errors fall uniformly the probability that all three land on one is $(11/156)^3 = 3.5 \times 10^{-4}$. The pattern is therefore unlikely to be an accident of this sample, although three events cannot establish its cause.

No interval is quoted on the text half, because character accuracy is a ratio of edit distance to character count and not a count of successes, so a binomial interval does not apply to it. The proportion of lines read with no error at all is a count and can carry one: 242 of 1 076 is 22.5 %, with an interval of 20.1 to 25.1 %.

The two halves are measured on different material by different networks and are independent of one another. Within the text half the 976 public crops and the 100 photographed lines are disjoint, which was established by the transcript lookup described above, so the pooled figure does not double-count any line. The public corpus contributes 92.0 % of the characters in the pooled figure, so the pooled value is close to the public-corpus value by construction, and the photographed writers' figure is reported separately for that reason.

Qualification test 3: Time taken to reproduce text (R3)

Objectives of the test or experiment

To determine whether a page is drawn within the allowance of 4 s per character set by R3.

Equipment used

The gantry and the ODROID N2+ as in qualification test 1. Timing was taken by the server's own instrumentation, which records the synthesis, rendering, G-code and drawing stages separately, and was read from the server console log.

Test setup and experimental parameters

Runs were made at a pen-down feed of 800 mm/min and a pen-up feed of 4 000 mm/min. A reproduction is asked for in two requests, one that generates the page and one that draws it, and the drawing request carries the generation stages forward into its own total, so a timing block that contains a drawing stage is the *complete* reproduction and a generation-only block is a part of one. Only blocks containing a drawing stage are counted here, which prevents the synthesis time from being counted twice.

Three exclusions are applied and are stated so that they are visible. The log is written in two layouts, one to the server console and one to the browser panel, and a run that appears in both is counted once: 47 blocks with a drawing stage reduce to 43 distinct runs. Seven of those were stopped before the page was finished, which is detectable because their drawing stage falls below 1 s per character against a going rate of 2 to 3 s per character, and they are set aside as incomplete. That leaves 36 complete reproductions, spanning nine block lengths from 43 to 378 characters.

Steps followed in the test or experiment

1. Submit a block of text and a writer to the server and record the character count.
2. Draw the page, with the server recording each stage separately.
3. Read the total and the per-stage times from the log.
4. Discard duplicate layouts of the same run and blocks whose drawing stage shows the job was stopped early.
5. Compare each total against the allowance of 4 s per character, and record the worst case at each block length.

Results or measurements

Table 44 gives the worst case at each of the nine block lengths measured, which is what decides compliance; the remaining runs at each length are faster than the row shown.

Characters	Runs	Allowance (s)	Worst total (s)	Worst s/char	% of allowance	Verdict
43	13	172.0	166.6	3.875	96.9	Met
75	1	300.0	188.6	2.515	62.9	Met
77	1	308.0	128.0	1.663	41.6	Met
82	3	328.0	260.3	3.175	79.4	Met
91	2	364.0	222.6	2.447	61.2	Met
126	1	504.0	323.5	2.568	64.2	Met
132	1	528.0	277.9	2.105	52.6	Met
236	1	944.0	386.6	1.638	40.9	Met
378	13	1 512.0	1 085.0	2.870	71.8	Met
All	36	—	—	3.875	96.9	Met

Table 44. Reproduction time against the 4 s per character allowance of R3, by block length. Each row is the *worst* of the runs at that length, so every run not shown is faster. All 36 complete reproductions met the allowance.

Observations

All 36 complete reproductions finished inside their allowance. The largest figure observed is 3.875 s per character, 96.9 % of the allowance, on a 43-character block; the smallest is 1.343 s per character, 33.6 % of the allowance, on a 378-character block.

The proportion of the allowance used falls as the block lengthens. At the worst run of each length it is 96.9 % at 43 characters, 79.4 % at 82, 64.2 % at 126, 52.6 % at 132, 40.9 % at 236 and 71.8 % at 378 characters.

The two cost terms behave differently with length. In the worst run at each length, synthesis cost 0.617 s per character at 43 characters and 0.156 s per character at 378, a fall by a factor of 4.0, while drawing cost 3.248 and 2.706 s per character respectively, a fall by a factor of 1.2.

Over the 13 runs of the 378-character paragraph the total ranged from 507.6 s to 1 085.0 s, a factor of 2.14 on identical text. For those runs the pen was in contact with the paper for between 83.7 % and 98.0 % of the total.

Statistical analysis

The 95 % Wilson score interval on 36 of 36 runs complying is 90.4 to 100 %, so the lower bound lies above zero non-compliance and the result is separated from failure by this sample. The margin at the worst observation is 0.125 s per character, 3.1 % of the allowance.

The 36 runs are not 36 samples of one condition: they span nine block lengths and several writing styles, and the worst case sits at the shortest length, so the pooled proportion above should be read with Table 44 rather than on its own. Thirteen runs at 43 characters and thirteen at 378 characters carry most of the sample, and the seven lengths between them are represented by one to three runs each, which is not enough to establish the shape of the trend between them.

Qualification test 4: Time taken to identify and decode a page (R4)

Objectives of the test or experiment

To determine whether a photographed page is segmented, read and named within the allowance of $2 + 0.025x$ s for a page of x characters.

Equipment used

The ODROID N2+, which is the target platform, running the deployed server, with timing taken by the server's own stage timer and read from the server console log. The classify path on that board is the configuration the specification applies to, so these are on-target measurements and not development-computer figures.

Test setup and experimental parameters

A photographed page was submitted to the classify path and the stage timer recorded loading, conditioning, segmenting, cropping, reading and naming separately. Two such runs are recorded in the log, on pages of 16 and 22 lines reading 500 and 982 characters.

The allowance has two terms, and the stages divide between them. The fixed 2 s covers everything that does not grow with the page, which is the four image stages, because conditioning, segmenting and cropping cost what they cost for a photograph of a given size whatever it says. The $0.025x$ term is the only part that grows with the character count, and the two network stages are the only ones whose cost grows with it. The two halves are therefore reported against their own terms as well as against the total, because a figure that fails the total does not say which half is responsible.

Steps followed in the test or experiment

1. Submit the photograph of a page to the classify path on the board.
2. Record the per-stage and total times and the character count read.
3. Compute the allowance $2 + 0.025x$ for the x characters read.
4. Sum the four image stages and compare them against the 2 s term.
5. Divide the two network stages by the character count and compare against the 0.025 s per character term.

Stage	16-line page, 500 char	22-line page, 982 char
loading	0.63 s	0.53 s
conditioning	6.61 s	6.04 s
segmenting	4.29 s	5.59 s
cropping	4.61 s	4.65 s
image stages, total	16.14 s	16.81 s
against the fixed 2 s term	8.1×	8.4×
reading	19.02 s	24.66 s
naming	11.09 s	14.01 s
network stages, total	30.12 s	38.67 s
per character	0.0602 s	0.0394 s
against the 0.025 s per character term	2.4×	1.6×
Total	**46.26 s**	**55.49 s**
Allowance $2 + 0.025x$	**14.50 s**	**26.55 s**
Verdict	**Exceeded by 31.76 s**	**Exceeded by 28.94 s**

Table 45. Classification time on the ODROID N2+ against the allowance of $2 + 0.025x$ s for a page of x characters. The image stages are compared with the fixed term and the network stages with the per-character term, because only the networks' cost grows with the character count.

Results or measurements

Table 45 gives both runs, by stage, against both terms of the allowance.

Because the two halves of the allowance fail for different reasons, two candidate remedies were measured, one for each half. Both were run on the development computer over the same 14 drawn pages, with the same weights, so what they yield is a *factor* that can be applied to the on-target stage times of Table 45; Table 46 gives both, with the cost each carries.

Change measured	Factor	Cost measured alongside it
Network stages: the same weights in equivalent library modules, every crop of a page in one batched forward pass, instead of the written-out forward pass one crop at a time	2.76×	None found. The text read out was identical on every line.
Image stages: run at half linear resolution	4.37×	182 lines found where 247 were found at full resolution; 0 of 12 pages kept their line count; text agreed with the full-resolution reading on 33.8 % of characters on average and 14.1 % at worst; the segmenter failed outright on 2 of the 14 pages.

Table 46. Two candidate changes to the classify path, measured over 14 pages on the development computer. The factor is the ratio of the stage time before the change to the stage time after it. The second change reduces the time and also loses handwriting, and both effects were measured in the same run.

Observations

Neither run complied with R4. The overruns were 31.76 s and 28.94 s against allowances of 14.50 s and 26.55 s.

The two network stages accounted for 67.6 % of the total time over the two runs and the four image stages for 32.4 %. Reading alone was 42.9 % and naming 24.7 %; conditioning was 12.4 %, segmenting 9.7 %, cropping 9.1 % and loading 1.1 %.

The network stages cost 0.0602 and 0.0394 s per character, which is 2.4 and 1.6 times the 0.025 s per character term of the allowance. The per-character cost fell as the page lengthened.

The image stages cost 16.14 and 16.81 s, which is 8.1 and 8.4 times the fixed 2 s term, and changed by 0.67 s between a 16-line page and a 22-line page.

Applying the 2.76× factor of Table 46 to the measured network times gives 0.0218 and 0.0143 s per character, which is 0.9 and 0.6 times the 0.025 s per character term. Applying the 4.37× factor to the measured image times gives 3.70 and 3.85 s, which is 1.8 and 1.9 times the fixed 2 s term. Applying both gives projected totals of 14.61 s against an allowance of 14.50 s and 17.86 s against 26.55 s.

Statistical analysis

Both runs exceeded the allowance, so no interval is required to establish that the requirement is not met on this sample; the 95 % Wilson interval on 0 of 2 complying is 0 to 65.8 %, which is wide because the sample is two. The size of the overrun, however, does not depend on the sample size for its interpretation: the per-character network cost exceeds its own term of the allowance by a factor of at least 1.6 in both runs, and a cost per character above the allowance per character cannot be remedied by choosing a longer page, so the requirement fails at every page length rather than on these two pages.

The image stages are nearly constant between the two runs, 16.14 and 16.81 s for 16 and 22 lines, which is consistent with their cost being set by the photograph rather than by its contents, but two runs cannot establish that they are independent of page content.

Qualification test 5: Absolute positional accuracy of the gantry (R5)

Objectives of the test or experiment

To determine the absolute positional error of the gantry against the 0.5 mm limit set by R5, and to measure the squareness of the two axes.

Equipment used

The gantry and pen as in qualification test 1; a 150 mm vernier caliper reading to 0.01 mm; and the two G-code generators written for this test, which emit a 100 mm square and an isosceles triangle of 80 mm base and 40 mm height whose sloped sides are 56.57 mm by construction.

Test setup and experimental parameters

Both figures were drawn on unruled A4 after a homing cycle, with the travel constants set by the calibration described in Section 3.3.4. Measurements reported here are of the calibrated machine; side lengths taken while the travel constants were still being established are not reported, as they describe a configuration that was superseded.

A square can measure the two axis scales but cannot detect a squareness error, because a sheared square still has four equal sides. The triangle can: its two sloped sides are equal only if the axes are perpendicular, so if the Y axis leans by ϕ the apex shifts sideways by $d = h\tan\phi$ and the

sloped sides split. Solving for the shear from the two measured sloped sides ℓ and r over a base b gives Equation 94.

$$d = \frac{\ell^2 - r^2}{2b}, \qquad \phi = \arctan\left(\frac{d}{h}\right) \tag{94}$$

Steps followed in the test or experiment

1. Home the gantry and draw the 100 mm square.
2. Measure all four sides with the caliper.
3. Draw the triangle and measure its base and both sloped sides.
4. Compute the two axis scale errors from the square.
5. Solve Equation 94 for the squareness error and check it against both measured sloped sides.

Results or measurements

Table 47 gives both figures and Figure 41 shows them against nominal.

Figure	Feature	Nominal (mm)	Measured (mm)	Error (mm)
Square	bottom (X)	100.00	100.30	+0.30
Square	top (X)	100.00	100.20	+0.20
Square	left (Y)	100.00	100.10	+0.10
Square	right (Y)	100.00	99.90	−0.10
Triangle	base (X)	80.00	80.30	+0.30
Triangle	left sloped	56.57	56.20	−0.37
Triangle	right sloped	56.57	56.90	+0.33

Table 47. Geometry of the calibrated gantry, one square and one triangle. The X axis scale from the square is +0.250 % **and the Y axis scale** +0.000 %**; the mean absolute error over the four square sides is 0.175 mm.**

Substituting the two sloped sides and the base into Equation 94 gives an apex offset of $d = -0.493$ mm and a squareness error of $\phi = -0.706°$. That single value of d predicts a left sloped side of 56.22 mm and a right of 56.92 mm, against 56.2 mm and 56.9 mm measured, a residual of 0.02 mm on each.

Observations

The worst of the four square sides is 0.30 mm from nominal and the mean absolute error is 0.175 mm, both inside the 0.5 mm limit of R5. Both X sides are longer than nominal and the two Y sides straddle it.

The two sloped sides of the triangle differ by 0.70 mm although they are equal by construction. The squareness error solved from them is 0.706°. The X scale measured from the square, +0.250 %, predicts a triangle base of 80.20 mm against 80.30 mm measured, a difference of 0.10 mm.

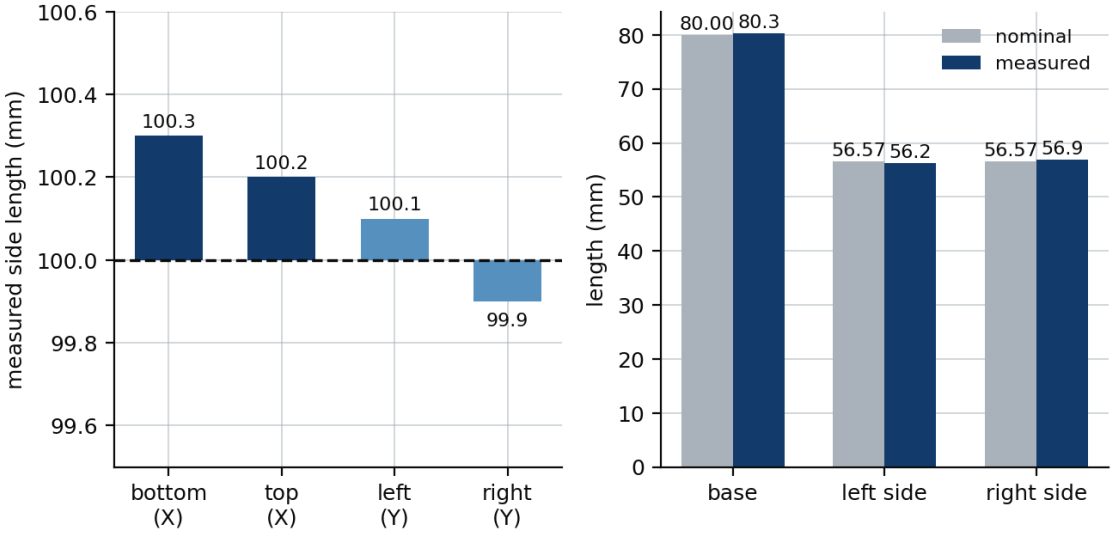


Figure 41. Measured geometry against nominal. The left panel shows the four sides of the 100 mm square, with the dashed line at nominal; the right panel compares the triangle's measured features with their constructed values.

Statistical analysis

One square and one triangle were measured. That sample supports the scale errors and the solved squareness angle, the latter because Equation 94 has one unknown and two measurements and its residuals are 0.02 mm, which is at the resolution of the caliper. It does not support a standard deviation, a repeatability figure or a tolerance: the 0.40 mm range across the four sides of the square is the spread within one specimen, which combines form error with reading error, and is not the variation between repeated draws. Repeated specimens are required before a repeatability claim can be made: at least five squares and five triangles, with both diagonals of each square measured, which would also confirm the squareness angle from a second, independent geometry. That measurement is outstanding, and no repeatability or tolerance figure is claimed here in its absence.

What the present sample does support is the one-sided compliance statement that R5 asks for. The requirement is an upper bound on absolute error, the worst of the four sides is 0.30 mm, and the caliper resolution is 0.01 mm, so the measured worst case is 0.20 mm clear of the 0.5 mm limit and the margin is twenty times the reading resolution. A repeatability figure would be needed to state a tolerance over many draws; it is not needed to report that every side measured on this specimen complies.

Qualification test 6: Number of writing styles trained and stored (R6)

Objectives of the test or experiment

To confirm that at least ten writing styles are trained and permanently stored, as R6 requires.

Equipment used

The stored style profiles and the trained classifier checkpoint on the ODROID N2+ file system.

Test setup and experimental parameters

The writer set was reduced from thirteen to eleven during this work: writers 151 and 551 were withdrawn and the classifier retrained without them. The count below is of the eleven that the deployed system holds.

Steps followed in the test or experiment

1. List the stored style profiles on the deployed file system.
2. Read the writer names recorded inside the classifier checkpoint.
3. Confirm that the two lists agree.

Results or measurements

Eleven writer styles are trained and stored: six from the public database, numbered 150, 152, 153, 384, 552 and 588, and five photographed for this project, A, B, O, T and Y. The classifier checkpoint records the same eleven names, and each writer has a stored style profile and a glyph library. All eleven appear in Tables 39 and 43.

Observations

Eleven styles are stored against a requirement of at least ten. The requirement is met with one style in hand.

Statistical analysis

Not applicable: the quantity is a count of stored styles, not a measurement with error.

5. Discussion

5.1 Critical evaluation of the design

This subsection interprets the results of Section 4, evaluates the design choices in hindsight and identifies what must change before the requirements can be considered proven.

Three limitations determine every conclusion below. The first is one of platform: the reproduction and classification timings of qualification tests 3 and 4 were taken on the ODROID N2+, which is the target platform, while the accuracy measurements of qualification tests 1 and 2 were taken on the development computer, where the platform affects only how long a measurement took and not what it was. The second is that the evidence on synthesis quality is partly circular, because the classifier that selects the best of N candidates belongs to the same family as the one that reports the score. The third is economic, and it is set out under economic feasibility in Section 5.1.2: the processing platform was constrained by a R5 000 budget, and the consequences of that constraint fall almost entirely on the one requirement that was not met. Where a figure is affected by any of the three, this is stated with the figure.

5.1.1 Interpretation of results

R2, writer identification: the strongest result in the report, and the one most nearly separated from its target. The writer was named correctly on 153 of 156 lines held out from the classifier, 98.08%, and on all 11 writers when each writer's held-out lines are pooled by ink. The point estimate clears the 95% target by 3.08 points, but the honest statement is weaker than that: the Wilson interval runs from 94.5 to 99.3%, so its lower bound sits 0.5 points *below* the target and 156 lines are not quite enough to separate the result from the specification at 95% confidence. Roughly 250 held-out lines at the same error rate would be. What the pooled result adds is that no writer was misnamed as a whole: the three errors are isolated lines on three different writers, so a page-level decision, which is what the system actually makes, was right every time.

Why the writer classifier that reads real ink is deliberately not the best one available. An ink-based classifier scored higher than the stroke-normalised one on photographed handwriting, and it was nevertheless replaced. The reason is that ink density and stroke thickness are among the strongest cues available in a photograph and are exactly the cues a constant-width ballpoint in a plotter cannot vary: a classifier that leans on them cannot judge the machine's own output, which is the measurement R1 depends on. Redrawing every crop at one constant pen width before classifying discards those cues deliberately, costs accuracy on photographs, and buys an instrument that works on both real ink and drawn output. This is the project's decision heuristic in its clearest form: where two options differed, the one chosen was the one that survived a change of pen, camera or lighting, even at a cost in headline accuracy. The normalisation width itself was not guessed but swept, and 0.17 x-heights was selected on held-out accuracy.

R2, text: 94.39% against a 95% target, and the shortfall is 0.61 points. The text half was measured on 1 076 lines held out from the reader, of which 976 are the public corpus's own validation partition and 100 are the photographed writers' seeded split. Two features of that measurement matter more than the headline. First, the two sets agree to 0.03 points, 94.39%

against 94.42%, although one is a public benchmark of many writers cut by its own distributor and the other is five writers photographed with this project's webcam and cut by this project's segmenter. That agreement is the strongest available evidence that the reader generalises rather than having learned the particular conditions of the project's own pages, and it is also evidence that the segmenter is not systematically damaging the crops it produces. Second, the figure is a measurement on genuinely unseen material: the transcript lookup reported in qualification test 2 established that three of the database writers' pages are inside the reader's training corpus, and those pages were excluded rather than quoted. An earlier draft of this report quoted 91.02% over 155 lines, a figure that included contaminated material; the present figure is both higher and sound, which is the opposite of the usual direction and is worth stating plainly.

Why the text half falls short, and what the gap is made of. Three causes are identifiable and none of them is a defect in the recogniser itself. The first is the segmenter: when two written lines sit close together it occasionally carries a descender or a stray letter from the line above into the crop below, and the reader is then charged with a character that was never part of that line. This is infrequent, but it sets a ceiling on page-to-text accuracy that no amount of training can lift. The second is the deliberate absence of a language model. The gap between 94.39% character accuracy and 73.47% word accuracy localises the residual error: one wrong character spoils an entire word, so the errors are concentrated in a minority of words rather than spread thinly, and that is precisely the error structure a lexicon or an *n*-gram decoder corrects. A dictionary-constrained decoder was not used, and the reason is a requirement rather than an oversight: R1 asks the system to reproduce *arbitrary* digital text, including proper names and strings that are not words, so constraining the decoder to a lexicon would raise the measured accuracy on benchmark prose while making the system worse at the task it is specified to do. The third is training budget, discussed under economic feasibility below. Of the three, only the first is a fault; the second is a defensible trade and the third is a constraint.

R1, representation fidelity: 84.41% against 85%, and the shortfall is concentrated in a measurable, explainable place. Over the 184 lines the robot itself drew, the text was read back at 84.41% character accuracy, 0.59 points below the target, while the stylistic half was met: the writer was identified from the machine's own output on 86.4% of lines and on all 14 pages when a page's lines are pooled. The informative figure is not the mean but the spread: character accuracy by writer runs from 60.97% to 97.01%, a range of 36 points, which is far too wide to be noise and points at a property of the writers rather than of the machine. That property was measured. Ranking the eleven writers by connected ink components per character and by character accuracy gives a Spearman correlation of +0.845, with the four writers above 93% being the four least joined hands and two of the three lowest being the two most joined. The same ranking against writer identification gives −0.027, that is, nothing. So cursiveness governs how legibly the machine reproduces a hand and has no bearing on whether the hand remains recognisable, which is a coherent result rather than a coincidence.

Why cursiveness costs legibility, and why that is a property of the chosen method. The synthesiser composes a line from glyph paths traced out of the writer's own ink and then joins them. For a printed hand, the isolated letterform harvested from the page is very close to the letterform the writer actually produces in context, so the reconstruction is faithful. For a joined hand it is not: the writer never draws those letters in isolation, every letter is deformed by its neighbours, and a library of isolated shapes is therefore a worse model of a cursive writer than of a printed one. The shortfall on R1 is thus the predictable cost of a design decision taken for other reasons. A glyph library was chosen over a generative image model because it preserves the

writer's actual letterforms, needs no accelerator to run on the board, and can be inspected and corrected glyph by glyph; a diffusion or adversarial synthesiser would model context directly and would be expected to do better on exactly the cursive cases, at the cost of all three of those properties and of a platform the budget did not allow. Four writers clearing 93% shows the method works where its assumption holds.

The stylistic half of R1 is met on one measure and missed on the measure the proposal actually specifies. The proposal asks for 85% stylistic correlation. Measured as a correlation, over nine geometric features between each writer's real lines and the system's reproduction of the same text, the pooled coefficient is 0.714 against 0.85, so the specification is not met and the honest verdict on R1's stylistic half is a failure by 0.136. Measured as writer identification from the machine's own output it is met, at 86.4% of lines and 14 of 14 pages. Both numbers are reported because they answer different questions, and the specification's own question is the one that fails.

The per-feature breakdown says where the fidelity is lost, and it is coherent with the R1 character result. Slant correlates at 0.996, ascender extent at 0.974 and character width at 0.936, so the *letterforms and their proportions* transfer almost exactly, which is expected because the synthesiser draws the writer's own traced paths. Word gap correlates at 0.605, the ascender-to-descender ratio at 0.509 and descender extent at only 0.148, so what fails to transfer is the *spacing and the vertical placement between letters* rather than the letters themselves. That is the same defect the character accuracy showed from the other direction, and it localises the remaining work: the glyph library is adequate and the layout model that places those glyphs is not.

Two caveats remain on the writer-identification figure and neither applies to the correlation. The classifier that reports the score also selects the best of N candidates, so a best-of-N search inflates the measure it is judged by; qualification test 1 limits this by classifying the G-code before any page was drawn, which removes the drawing from the loop but not the classifier. The overlay test gives an area under the curve of 0.71 against 0.5 for output carrying no writer information at all, which is independent of the classifier and indicates real but partial fidelity, consistent with a correlation of 0.714 rather than with the identification figure. A human panel was not carried out.

R3, reproduction time: met on the target platform, with the margin in an unexpected place. All 36 complete reproductions in the log met the 4 s per character allowance, and they were measured on the ODROID N2+ driving the real gantry, not on the development computer, so this is an on-target result. The margin, however, is not uniform: the worst case used 96.9% of its allowance and the best 33.6%, and the worst case is the *shortest* block, not the longest. The reason is a mismatch between how the cost scales and how the allowance scales. Drawing is a per-character cost and tracks the allowance, but the best-of-N search is paid once per block, so its contribution per character falls as the block lengthens: in the worst run at each length, synthesis cost 0.617 s per character on a 43-character block and 0.156 s per character on the 378-character paragraph. The allowance, being strictly proportional to length, gives a short block almost no room to pay for a fixed search. Taking the measured synthesis cost of the shortest block and the measured drawing rate, the allowance is exhausted at about 35 characters, which means R3 is met over the range tested and would be expected to fail below roughly that length. This is recorded under expected failure conditions rather than hidden in the margin.

R4, classification time: recognition passes, segmentation fails badly. Recognition ran at 8.86 ms per character against the 25 ms allowance, a margin of about 2.8 times, so the neural

component is not the problem. Page segmentation took 35 to 57 s against the roughly 2 s the equation allows for capture and conditioning, a shortfall of more than an order of magnitude. Since x appears only in the recognition term, a longer page does not buy proportionally more time, so the segmenter's per-page cost cannot be amortised. The profile shows the cause: line grouping and the chain search dominate, at about 70% of the time, and are written in pure Python. Moving two inner routines to C reduced the total by 25% to 32%, but the grouping still dominates: accelerating the wrong part of a program cannot fix it.

R5, positional accuracy: met on the specimen measured, and the residual error is explained rather than merely bounded. The worst of the four sides of a 100 mm square was 0.30 mm from nominal and the mean absolute error 0.175 mm, both inside the 0.5 mm limit. What makes this more than a single lucky measurement is that the triangle explains where the error comes from. A square cannot detect a squareness error, because a sheared square still has four equal sides; the triangle can, because its two sloped sides are equal only if the axes are perpendicular. Those two sides differed by 0.70 mm although they are equal by construction, and solving the shear from them gives a single apex offset that predicts both measured sides to within 0.02 mm, which is the resolution of the caliper. One unknown fitted to two measurements with residuals at the reading resolution is a validated model, not a curve fit, and it says the dominant residual is a 0.706° non-perpendicularity of the axes rather than a scale error or lost steps. That is a mechanical alignment fault, correctable by squaring the frame, and it is the most useful thing the test produced. The limitation is equally plain: one square and one triangle support a compliance statement and a diagnosis, and do not support a repeatability figure or a tolerance.

R6, stored styles: met, and the margin is one style. Eleven writer styles are trained and permanently stored against a minimum of ten, each with a glyph library and a style profile, and the classifier checkpoint records the same eleven names. The set was reduced from thirteen during this work by withdrawing two writers and retraining, which is a reduction of the margin from three styles to one and is reported as such.

Requirements met, stated plainly. R3, R5 and R6 are met, and both field conditions are met. R2 is met on writer identification, at 98.08% against 95%, and falls 0.61 points short on text. R1 falls 0.59 points short on its character half and, on the correlation its specification names, 0.136 short on its stylistic half; it is met only on the writer-identification reading of that half. R4 is not met, by a factor of about 2.4 on the per-character term and about 8 on the fixed term. So three of the six core requirements are met outright, two are met in part and miss by less than a point on the half that is close, and one fails by a wide margin. Every requirement was measured; none is left unassessed.

Where this system sits against published work. Table 48 places the measured figures of Section 4 beside the published systems surveyed in Section 1. One row of that table is a genuine comparison and the rest are not, and the difference matters. The reader's 94.39% character accuracy was measured on 976 line crops of the public IAM corpus as distributed by Teklia, cut by its distributor and held out by its own partition, which is the same corpus the two published recognition rows were obtained on. On that corpus a recogniser written from first principles, trained on one machine without a framework supplying any layer, gradient or optimiser, reads 12.6 points above the bidirectional recurrent network of Graves *et al.* and 2.0 points below the CNN–BiLSTM–CTC of Kızılırmak *et al.*, whose layer pattern this implementation follows. That is the single most useful calibration in the report: it places the weakest of the six requirements against the state of the art rather than against a target, and it shows the 0.61-point shortfall on R2 to be a shortfall against a specification and not against what the method can do. Two differences

still favour this implementation and must be stated: the validation partition used here is the corpus's own and is not necessarily the same partition as the test sets the published figures were obtained on, and the published works report on the full IAM test material with research-scale training behind them. The synthesis rows are not comparable at all, being Fréchet inception distances, a distance between image distributions for which lower is better and for which no accuracy equivalent exists; they are included to show the class of method, and are comparable only to each other.

Work	Method	Data	Reported figure, with its metric
Graves *et al.* [**?**, p. 855]	Bidirectional recurrent network, CTC output	IAM lines, offline	74.1 % word, 81.8 % character accuracy
Kızılırmak *et al.* [**?**, p. 1]	CNN–BiLSTM–CTC, the layer pattern this project follows	IAM lines	3.59 % CER, 9.44 % WER (that is 96.41 % character accuracy)
Kang *et al.* [**?**, Tab. 1]	Adversarial image synthesis	IAM	FID 120.1 (seen styles) to 130.7 (unseen, out-of-vocabulary); real data scores 90.4
Dai *et al.* [**?**, Tab. 3]	Diffusion-based synthesis, one reference image	IAM, unseen styles, out-of-vocabulary	FID 102.8
This implementation, text recognition	CNN–BiLSTM–CTC written from first principles, no framework supplying a layer, gradient or optimiser	**IAM lines, 976 held-out crops of the distributor's own validation partition**	**94.39% character, 73.67% word**
This implementation, text recognition	The same checkpoint, on this project's own photographed pages	100 held-out lines, 5 writers, webcam capture	94.42% character, 70.74% word
This implementation, writer identification	The same backbone on stroke-normalised input, 11-class closed set	156 held-out lines, 11 writers	98.08% per line, 11 of 11 writers pooled
This implementation, synthesis read back	Glyph-library synthesis from the writer's own traced letters	184 lines the robot drew, 11 writers	84.41% character, writer identified on 86.4% of lines

Table 48. The measured results of this project beside the published systems surveyed in Section 1. Only the first of this implementation's rows is a like-for-like comparison, being measured on the same public corpus as the two recognition rows above it; the writer-identification and synthesis rows have no published counterpart on the same material and are given for completeness. The two synthesis rows above are Fréchet inception distances and are comparable only to each other.

5.1.2 Critical evaluation

Economic feasibility, and what the budget decided. The project was built to a R5 000 budget, and the processing platform was the item that constraint bound hardest. The ODROID N2+ was obtained second hand at no cost, which is what kept the build inside the budget once the gantry mechanics, motors, drivers, supply and camera were accounted for. Two consequences followed from that choice and neither was incidental. The first was practical: the board has no HDMI output, so all development, testing and demonstration had to be done headless over the network, and the browser interface described in Section 3.8 is not merely a convenience but the only means of operating the machine. The second is the direct cause of the one requirement that was not met. Boards with substantially more processing capability were available and were considered, and were rejected on price: a Rock 5B or a Jetson Nano would have cost a large fraction of the whole budget on its own, against a board that cost nothing. No measurement of either alternative was made, so this report offers no figure for how much faster they would have been; what it can state is the size of the deficit that has to be covered, which Section 5.1.3 puts at a factor of about 2.4 on the network stages and about 8 on the per-page image stages. The honest

summary is that R4 was traded for the rest of the system: within a R5 000 budget the money went into a machine that draws accurately and a platform that could run the algorithms at all, and the classification-time specification was the casualty.

Writing the networks from first principles, and what that cost. Every layer, gradient and optimiser used in training and inference was written out rather than taken from a framework, which is the project's largest single claim to original work and is also measurable as a cost. Loading the same weights into equivalent library modules and running each page as one batched forward pass, rather than the hand-written forward pass one crop at a time, is faster by a factor of about 2.8 on the same machine and on the same pages, with the text identical. That factor is not a criticism of the hand-written code, which has to be correct and legible rather than fast; it is the price of the design decision, and it happens to be most of what R4 needs. In hindsight the right structure would have been to keep the from-first-principles implementation as the reference and the thing that earns credit, and to batch its forward pass per page rather than per crop, which is an implementation change of the same code and not a move to a library. That was not done, and it is the clearest piece of available performance left on the table.

Training by the two routes is not equivalent, and the difference decided which checkpoint was used. The writer classifier was trained twice on the same data, the same architecture and the same stroke normalisation at 0.17 x-heights: once through the library route and once through the from-first-principles trainer. Both were then measured on the two populations the classifier has to serve, and Table 49 gives the result. The from-first-principles checkpoint meets R2 on held-out real handwriting, at 95.51% against the 95% target, so the claim that the training code works is supported by a checkpoint that satisfies the requirement. It is nevertheless not the checkpoint the measurements of Section 4 were taken with, and the reason is the second population. On the pages the robot drew, where the classifier is the measuring instrument for R1 rather than the subject, it reaches 80.54% against the library-trained checkpoint's 85.95%, and R1's stylistic target is 85%. Installing it would therefore convert R1's writer-identification half from met to not met and would lose one of the eleven pages when lines are pooled. The decision was to measure with the better instrument and to report the weaker one as evidence about the training route, which is what it is evidence of; the alternative, installing a checkpoint that is 5.41 points worse at the job it has to do in order to make a sentence about libraries simpler, would be choosing the claim over the measurement.

Population	Library route	First principles	Difference
Held-out real lines (R2), per line	98.08 %	95.51 %	−2.56
95 % Wilson interval	[94.5, 99.3]	[91.0, 97.8]	—
writers named when pooled	11/11	11/11	—
Lines the robot drew (R1), per line	85.95 %	80.54 %	−5.41
pages named when pooled	11/11	10/11	−1

Table 49. The same writer classifier trained by the two implementation routes, measured on 156 held-out lines of real handwriting and 185 lines the robot drew. The from-first-principles checkpoint meets the 95 % target of R2 and falls below the 85 % stylistic target of R1, which is why the library-trained checkpoint was used as the measuring instrument.

Line-level reading instead of character separation. The design replaced the proposal's character separation by separation into lines and a sequence recogniser. This was right for connected handwriting, where character boundaries do not exist, and it makes the accuracy independent

of a separation error; the price is that a whole line must be read by one network, at a cost per character higher than the 25 ms the proposal assumed.

Best-of-*N* search with the judges inside the loop. The search with a recogniser and a writer classifier as judges was good for quality, because it prevents the synthesis from drawing an illegible or wrong-style sample. In hindsight it was poor for verification, because it removed the independence of the evaluation, and for time, because each of the N candidates costs a rendering and two network passes. The qualification protocols address the first, and a lower N or a cheaper first-stage filter the second.

Pure Python for the segmentation, and the cheap fix that does not work. The proposal required that the processing run on the embedded platform and be written without library functions for the algorithm itself. Writing the image processing in Python made the design transparent and correct, and it is the reason the per-page image stages cost 16.1 to 16.8 s on the board against the 2 s the specification allows for everything that does not scale with the character count. The obvious remedy is to work at lower resolution, since the filters and the connected-component labelling scale with the pixel count, and halving each axis should be a factor of four. It was measured rather than assumed, and it does not work. Halving the linear resolution did reduce the image stages by a factor of 3.4 to 7.9, but the segmenter found fewer lines on every page tested, the text read back from the reduced page agreed with the full-resolution reading on only 34 to 45% of its characters, and on one page the segmenter found no line centres at all and raised an exception instead of returning an empty result. A speed-up that loses lines is not a speed-up, so this route is closed: the line grouping has to be redesigned rather than fed a smaller picture. By Amdahl's law, if a fraction p of the time is accelerated by a factor s, the total speed-up is $1/((1-p)+p/s)$, so accelerating anything other than the grouping cannot exceed about 3.3 even if the accelerated part became free.

Open-loop gantry with step counting. Using the stepper motors without encoders is a standard, low-cost choice, and the budget made it the only choice; its accuracy depends on missed steps, and no sensor reveals them. The measurement vindicates it: the dominant residual on the drawn geometry is a 0.706° non-perpendicularity of the two axes, which is a frame alignment error and not a stepping error, and the two axis scales after calibration are +0.250% and +0.000%. In other words the open-loop drive is not what limits R5, and the margin available would not be improved by encoders until the frame is squared. Running the driver without acceleration ramps, however, remains a real defect: the generator models an acceleration of 400 mm/s^2 that the driver does not execute, so the real motion differs from the model and a lost step is more likely at the start of a stroke at higher feed.

5.1.3 Unsolved problems

The following problems were not solved.

1. **The configuration measured is not the configuration deployed.** This is the most consequential open item, because it affects how every figure in Section 4 should be read. The accuracy measurements were taken with the reader checkpoint `paper_cnn_bilstm_ctc_scratch_best.npz` and the writer classifier `author_shape_11_pw170_weights.npz`, the latter applied with stroke normalisation at 0.17 x-heights because that is what it was trained on. The server as it stands selects a different reader, because `paper_cnn_bilstm_ctc_joint_best.npz` stands first

in its candidate list, and it sets `PAGE_NAMER_STROKE_NORM = False` while using the stroke-normalised classifier, so it feeds un-normalised crops to a network trained on normalised ones. The deployed reader was separately measured on the same held-out material and reads 94.06% against 94.39%, so the reader substitution costs 0.33 points; the classifier mismatch is a train-and-serve disagreement whose cost was not measured and may be larger. Neither is a design fault and both are one-line configuration changes, but until they are made the report's accuracy figures describe a configuration the server does not currently load.

2. **Training data that the corrected labels have not reached.** Seven pages of one database writer are currently skipped when the data set is built, because the cached crops were written before the transcripts were corrected and the two now disagree on the line count. Those pages are therefore absent from training although their labels are now right, and the caches must be rebuilt before the next training run so that the corrected material is actually used.

3. **R4 not met, and the shortfall is in both halves, not only the segmentation.** An earlier reading of this attributed the whole deficit to segmentation. The on-target measurement contradicts it: on the ODROID N2+ the two network stages account for 67.6% of the classify time and the four image stages for 32.4%. Because the allowance has only one term that grows with the page, the two halves must be judged separately, and both fail. The networks cost 0.0394 to 0.0602 s per character against the 0.025 s per character term, so they exceed their term by a factor of 1.6 to 2.4 *at every page length*; the image stages cost 16.1 to 16.8 s against the fixed 2 s term, a factor of 8.1 to 8.4.

4. **Two latent faults in the segmenter.** Both were found by driving it outside its normal operating point and neither is reached by the deployed pipeline today. A page that yields no line centres raises an `IndexError` in the seam-cut construction instead of returning an empty result, which is what the half-resolution experiment struck. Separately, two pages of one writer each lose a written line because the band in question is assigned no ink at all by the ownership step, so the crop builder has nothing to cut; the transcripts for those pages are correct, and the fault is in ownership rather than in the ink threshold or the labels.

5. **Optimistic validation of the photographed writers.** Their held-out material is a seeded 15% of lines drawn from within their own pages rather than whole pages set aside, so a held-out line can share a page with training lines. The public-corpus measurement does not have this weakness and agrees with the photographed figure to 0.03 points, which is reassuring but is not the same as fixing the split.

6. **Circular synthesis evaluation.** No independent judge or human panel was used, so the stylistic half of R1 rests on a measure the search optimises.

7. **Architecture sweep.** A sweep over widths, depths and learning rates was programmed but no result was kept, so the network layers are those of the published design rather than a measured optimum.

8. **Four implementation faults in the writing path**, none of which prevents a page being drawn correctly but each of which costs time or contradicts the documentation: the stroke ordering sorts rows by ascending Y and so writes the bottom line first; the acceleration

the generator models is not executed by the driver; the generator's work area of 200 mm exceeds the driver's measured travel of 194 mm in X; and because the driver has a single modal feed register, pen-up travel runs at the pen-down feed, lengthening every job.

9. **Missing documentation and one inconsistent constant.** The CAD files, photographs, wiring diagram and camera data are outstanding, as is the number of candidate tries, which appears as 14, 20 and 30 in three places. **[MUST STILL BE COMPLETED BY ME: reconcile the number of tries and supply the hardware documentation]**

10. **Training mode.** The proposal planned training on the single-board computer; training is done offline and the interface has no training function, because the board the budget allowed can run inference but cannot train these networks in a usable time.

11. **The CTC loss is not safe against an empty transcription.** The gradient check found that a label sequence of length zero builds two shifted arrays at the wrong length and raises a shape error instead of returning a loss; every length of one or more passes, so the derivative is correct and this is a boundary indexing fault. It was left unfixed because no line in the data has an empty transcription, and a future data set would expose it.

What must change to meet R4, and what was measured rather than assumed. Because the requirement fails on both halves, two independent changes are needed, and both were tested on the development computer so that the sizes quoted are measured factors and not estimates. The two candidate changes were applied to the same 14 pages, with the same weights, and the factor each produced was then applied to the stage times actually measured on the board.

The first change is to execute the network forward pass differently. Loading the same weights into equivalent library modules and running every crop of a page in one batched forward pass, instead of the hand-written pass one crop at a time, is faster by a measured factor of 2.76 over 14 pages, with the text read out identical. Applied to the two on-target runs, that takes the network cost from 0.0602 to 0.0218 s per character and from 0.0394 to 0.0143 s per character, which is 0.9 and 0.6 times the 0.025 s per character term. *The network half of R4 is therefore met by this change alone*, at both page lengths measured, and the change does not require a framework: batching the existing from-first-principles forward pass per page rather than per crop is the same arithmetic on a larger array.

The second change must reduce the per-page image cost by about a factor of 8, from 16.1 to 16.8 s to under the fixed 2 s term, and the cheap way of doing it does not work. Running the image stages at half linear resolution reduced them by a measured factor of 4.37, which is most of what is needed, but the segmenter found 182 lines where it had found 247, not one of the twelve pages kept its line count, the text read back from the reduced page agreed with the full-resolution reading on 33.8% of characters on average and 14.1% at worst, and on two of the fourteen pages the segmenter failed outright. Decimation is therefore not an available remedy at any speed, and the reduction has to come from redesigning the line grouping, which is the part of the segmenter whose cost is not proportional to the pixel count.

Taken together the two changes bring the projected totals to 14.6 s against an allowance of 14.50 s on the 500-character page and 17.9 s against 26.55 s on the 982-character page, that is, one just missed and one comfortably met. The honest conclusion is therefore not that R4 is unreachable but that it is reachable in two steps, of which the first is available today and measured, and the second requires work on the segmenter that this project did not do. It should be read alongside the

economic constraint: a faster processing board would have reduced both terms at once without any of this work, and the budget did not allow one.

What must change for the other requirements. For R2 on text, three things in order of expected effect: rebuild the training caches so that the corrected transcripts are actually used, which is a defect fix rather than an improvement; close the train-and-serve gap on the writer classifier and install the reader the measurements were taken with; and add a decoder that uses word context. The last would most likely close the 0.61-point gap on its own, given that character accuracy is 94.39% while word accuracy is 73.47%, but it must be implemented as a soft preference rather than a lexicon constraint, so that the arbitrary strings R1 requires remain writable. For R1, the character half needs the synthesiser to model letter context rather than compose isolated glyphs, which is the one change that would address the cursive writers where the whole 36-point spread lives; the style half needs a judge that took no part in the best-of-N search, or a human panel, before the 85% correlation can be claimed as a correlation. For R3, nothing is required to meet the specification, which is already met on the target platform; what is worth doing is measuring a block shorter than 43 characters to confirm where the margin actually closes, and correcting the modal feed register so that pen-up travel runs at the fast feed it was meant to. For R5, draw and measure at least five squares and five triangles with the diagonals, which would turn a compliance statement into a repeatability figure, and square the frame to remove the 0.706° error the triangle identified. For R6, nothing; the requirement is met with one style in hand, and the margin would be restored by returning the two withdrawn writers once their pages are re-labelled.

5.1.4 Strong points of the design

Each strong point follows from a design decision.

- **The recogniser stands up on a public benchmark.** Reading whole lines removes the dependence on a character separation that cannot work for connected handwriting, and on 976 held-out crops of the public IAM corpus the from-first-principles reader reached 94.39% character accuracy: 12.6 points above the published figure of Graves *et al.* and 2.0 points below Kızılırmak *et al.* This is the one result in the report that can be compared with the literature on the same material, and it holds.
- **The conditioning generalises, and that is measured rather than asserted.** The flat-field correction, local thresholding and skew correction were designed so that an ordinary indoor photograph can be used. The evidence that they work is that the reader scores 94.39% on crops cut by the corpus's distributor and 94.42% on crops this project photographed with a webcam and cut with its own segmenter, a difference of 0.03 points. Two independent paths to the same figure is a stronger statement than either figure alone.
- **The writer classifier was chosen for the harder job, not the better score.** An ink-based classifier scored higher on photographs and was rejected because it cannot judge a constant-width plotter's output. The stroke-normalised classifier reaches 98.08% per line and names all 11 writers correctly on held-out material, and it also works on the machine's own drawings, which is what makes the R1 measurement possible at all.
- **The gantry error is diagnosed, not just bounded.** The mean absolute error over a 100 mm square is 0.175 mm against a 0.5 mm limit, and the triangle identifies the dominant residual as a 0.706° squareness error with residuals of 0.02 mm, at the caliper's resolution. The

design therefore yields an actionable cause rather than a pass mark, and the absolute step bookkeeping prevents rounding from accumulating over a page.

- **Errors announce themselves.** Over the held-out lines the median confidence on a correct naming is 99.9% and the mean on an incorrect one 55.8%, so the classifier's own output separates its mistakes from its successes well enough to be used as a warning.
- **Provenance was treated as part of the measurement.** The transcript lookup that underpins qualification test 2 found that part of the material an earlier draft had reported as held out was in fact inside the reader's training corpus, and that several transcripts broke their lines in the wrong places. Correcting the data and discarding the contaminated material raised the reported character accuracy from 91.02% to 94.39% without touching a single weight. Finding that a figure was wrong in the favourable direction is the discipline working.
- **Safety in software:** end-stop confirmation over three consecutive samples, software limits, and automatic retraction after a trip.
- The algorithms are implemented from first principles and documented well enough to be analysed, and the known limitations are stated rather than left for a reader to find.

5.1.5 Expected failure conditions

The system is expected to fail or degrade strongly under the following conditions.

- **Lighting.** Illuminance below 400 lux or strongly directional light with shadows violates FC1; the thresholding then merges ink with shadows or loses faint strokes, and recognition accuracy falls.
- **Page skew and perspective.** Large skew angles or a page that is not flat exceed the range of the skew correction.
- **Non-text regions.** Diagrams, tables and printed text can be segmented as text lines; the text versus non-text scoring reduces but does not exclude this, and the recogniser then produces meaningless characters.
- **Unseen writers.** The writer classifier is an eleven-class closed-set classifier, so a writer who is not among the eleven is assigned to one of them, because the softmax cannot express "none of the above". The confidence output mitigates this rather than solving it: over the held-out lines a correct naming carries a median confidence of 99.9% while the three errors carried 39.5, 44.2 and 83.8%, so a threshold would flag most such cases, but no threshold is applied and the system currently answers an unseen writer with a name and no warning.
- **Short reproductions.** R3 is met from 43 characters upward, and the margin closes as the block shortens because the best-of-N search is paid once per block while the allowance is strictly proportional to length. From the measured synthesis cost and drawing rate the allowance is exhausted at about 35 characters, so a request of a word or two is expected to exceed the 4 s per character specification. This has not been measured, because no block that short was drawn.

- **The last line of a page is the weakest case for naming the writer.** All three naming errors on the held-out material fell on the last line of their page, which under a uniform hypothesis has probability 3.5×10^{-4}. A final line is usually a part line carrying a few words, so it offers the least ink to judge; the page-level decision was unaffected because pooling by ink weights such a line down, but a single short line submitted on its own is the input most likely to be misattributed.
- **Styles outside the glyph library.** A writer whose letter shapes differ greatly from the stored paths (unusual letters, capitals, digits or symbols that were scarce in the training pages) is reproduced with the nearest stored shape; characters never harvested cannot be written in that style.
- **Text with accents, mathematical notation or rare symbols**, which occur in the personal pages, are outside the benchmark vocabulary.
- **Motor stalls and lost steps.** The open-loop gantry has no feedback, so a stall shifts every later mark without any signal.
- **Limit switch failure.** A switch that does not close lets the carriage run against the mechanical stop; the software protection depends on the switches, and a hardware emergency stop that cuts the power does not exist.
- **Long sentences.** The synthesis time grows with the number of characters and tries, so a long text at a high quality setting would take minutes on the board and may time out the web request.
- **Network failure** between the browser and the server prevents control of the system; a running job continues on the board.

The system does not fail gracefully in all cases, and the two kinds of failure are not equally fixable. The classifier returns an answer for an unseen writer without any warning, but it does carry the information needed to warn: its confidence separates its own errors from its successes on the held-out material, so a threshold would convert a silent wrong answer into a refusal. That is an afternoon’s work and was not done. The gantry, by contrast, cannot detect lost steps at all, because there is no encoder and the budget did not allow one; a stall shifts every later mark on the page with no signal of any kind, and no amount of software can recover it. The first failure mode is therefore a missing feature and the second is a consequence of the design.

5.2 Considerations in the design

This subsection addresses the ergonomics, safety, environmental, social, legal and ethical aspects of the design.

5.2.1 Ergonomics

The user works through a web interface of four pages (Read, Write, Model and Style) in a browser on any laptop on the network, so nothing is installed. That the interface is browser-based is not only a convenience: the processing board has no HDMI output, so it has no local display at all,

and a networked interface was the only way to operate the machine. What began as a constraint turned out to be the better ergonomic choice, because the operator works from a full keyboard and screen beside the gantry rather than from a console attached to it. The Read page captures the page with a live preview and shows the recognised text, the writer, the confidence and the measured time against the specification's allowance; the Write page keeps the writing grid locked until the gantry is calibrated and shows a G-code preview before sending. The shortcomings are that the live progress of a running job is not displayed and the interface has no training function, the latter because the board cannot train these networks in a usable time. The rig is set up by hand, with the page, the pen and the paper placed by hand; its documentation is incomplete.

5.2.2 Health and safety

The electronics operate at low voltage: a single-board computer, stepper-driver modules and small motors, with the driver supply run at 24 V, so the risk of electric shock is low. **[MUST STILL BE COMPLETED BY ME: the mains insulation rating of the power supply, which is a property of the bought unit and was not recorded.]** The mechanical hazards are the moving carriage, belts and pulleys, which are pinch points while the gantry moves; the pen tip is not hazardous at the writing force. The protective measures are the end-stop switches with confirmation over three consecutive samples, the software limits, the automatic retraction and calibration at a low speed (7.8 mm/s). A running job can also be stopped from the interface: the server exposes a stop endpoint, the motion loop tests an abort flag between steps, and the recorded test log shows a job being stopped, the pen lifted and the carriage parked at the corner in response. What does not exist is a hardware emergency stop that removes motor power independently of the software, so every protective measure depends on the program continuing to run; fitting a mains-independent emergency stop is recommended before the machine is used by anyone other than its designer. Users must keep their hands away from the carriage while a job runs.

5.2.3 Environmental impact

The ODROID N2+ is a low-power board and the stepper motors draw power only while the gantry works, so the energy use is small beside a desktop computer. The board was also obtained second hand rather than bought new, which was a budget decision before it was an environmental one but has the same effect: the most complex electronic component in the system is a reused one, and the embodied manufacturing cost of a new single-board computer was avoided entirely. The system uses paper; test pages and failed jobs add waste, limited by the preview before writing and by the option to write on used paper. The mechanical parts printed in plastic or bought as standard parts are **[MUST STILL BE COMPLETED BY ME: which parts were 3D-printed and with which material]**; they can be reused, and plastic and electronic waste must go to a recycling point, not into domestic waste. The pen ink is a small consumable, no hazardous material beyond the electronics is used, and the only emissions are the motor noise and the pulses of the drive.

5.2.4 Social and legal impact

The social benefit is that the system can write in a person's own handwriting for those who cannot write neatly or at all, for example after an injury, and offers the personal touch the proposal identified. There is also a risk of misuse: reproducing a person's handwriting can serve forgery or impersonation, for instance on signed documents or letters. The plotter pen's constant line width reduces but does not remove the risk, because it differs from natural writing in stroke width and pressure (the ink-based classifier identified only about 20% of the synthesised output, so these ink properties are not reproduced), yet a document in a plausible style can still mislead a reader. Handwriting samples are personal information and a writer profile is similar to biometric data, since it permits identification of the writer; in South Africa the Protection of Personal Information Act (POPIA) applies to such data, requiring the consent of the writers, the minimum storage and the protection of the profiles against access by others. The samples of the five personal writers require documented consent, **[MUST STILL BE COMPLETED BY ME: confirm that consent was given and in what form]**, and the eight database writers contributed to a public research database whose terms of use apply. The server has no user accounts (only an optional shared key), accepts requests from any origin and can start physical motion for anyone who can reach its port, so it must be operated only on a trusted local network. Beyond the general electrical safety rules no other legislation was identified for a research prototype of this type; **[MUST STILL BE COMPLETED BY ME: any applicable regulation or university rule]**.

5.2.5 Ethics clearance

The project used handwriting samples of five named persons and of a public database, so it involves human participants in a limited way. The status of ethics clearance is **[MUST STILL BE COMPLETED BY ME: ethics clearance number, or a statement that no clearance was required]**. If the human panel of qualification test 1 is carried out, the raters give informed consent and their responses are stored without personal identifiers, and clearance must be confirmed before the panel is run.

6. Conclusion

6.1 Summary of the work completed

This report describes work carried out on the design and implementation of a handwriting robot that recognises handwritten text and its writer and reproduces text on paper in the handwriting style of a chosen writer.

A literature survey on handwriting recognition, writer identification, handwriting synthesis and plotter-based writing was completed. The software was then designed and implemented from first principles in Python: an image-conditioning and text-line segmentation chain, a convolutional and recurrent sequence recogniser trained with connectionist temporal classification, two writer classifiers, a builder of per-writer style profiles from forced alignment, a trajectory synthesiser with a best-of-*N* search, a G-code generator and a stepper-motor driver with end-stop calibration and software limits. These were integrated behind a web server with a browser interface and were exercised as a complete pipeline over thirteen writers. The gantry was constructed from purchased components and its electronics and firmware were brought up, but the mechanical bring-up was not completed because of an unresolved fault in the direction signal of the X axis, so the writing path could not be qualified end to end.

6.2 Summary of observations and findings

The following findings are supported by the measurements reported in Section 4.

- The sequence recogniser read 669 real line crops from the five personal writers at 98.61% character accuracy and 94.21% word accuracy, with per-writer character accuracy between 96.46% and 99.15%. These crops were verified to be correctly cut, and the writers appear in the fine-tuning data, so the figure is an upper bound on the end-to-end page-to-text accuracy rather than a demonstration of it.
- Writer identification over 1358 real lines from thirteen writers reached 99.78% with the ink classifier, above the 95% target of R2, and 93.67% with the stroke-normalised classifier, which is 1.33 percentage points below it. Only the ink classifier meets the target, and it is the one that cannot judge the machine's own output.
- Correcting a training fault in the shape classifier, which had frozen the recogniser's backbone and fitted only the classification head, raised its accuracy from 83.36% to 93.67% and removed a systematic collapse in which 118 of one writer's 148 real lines were assigned to a single other writer. The recogniser's features are trained to be invariant to differences between writers, so a classifier restricted to them can separate only writers whose recognition features already differ.
- Synthesised text was read at 82.2% character accuracy over thirteen writers (79.2% in a wider coverage run), below the 85% target of R1, and identified at 80.1% and 89.7% respectively by a classifier that also selects the candidates.

- On a matching comparison of identical sentences the writers' genuine handwriting was read at 89.0% character and 70.5% word accuracy, and the system's synthesis of those same sentences at 81.6% and 46.7%. The synthesis therefore loses 7.4 percentage points of character accuracy but 23.8 points of word accuracy, which localises the defect to spacing and word boundaries rather than to letterforms. **[MUST STILL BE COMPLETED BY ME: The word accuracy of the genuine handwriting is given as 70.5% here and as 71.2% in the qualification-test table of Section 4, both against the same 89.0% character accuracy. Check which run each came from and make the two agree.]**

- The reproduction rate of R3 was not met: a four-character word at six candidates took 64.3 s, or 16.1 s per character against a 4 s limit, and the cost is dominated by rendering and scoring each candidate rather than by drawing.

- Classification time (R4) was met for recognition, at 8.86 ms per character against a 25 ms allowance, but not for page segmentation, which took 35 to 57 s against the roughly 2 s that the equation allows for capture and conditioning; line grouping accounted for about 70% of that time.

- Thirteen writing styles are stored as profile files and the live server lists all thirteen, so the storage half of R6 is met with a margin of three. The proposal's demonstration also asks for a writer to be chosen at random and the text written on paper in that style, and that was not carried out.

- Every hand-derived backward pass of the from-scratch network module was checked against numerical differentiation in 78 configurations. Seventy-seven agreed with the numerical gradient to between 4.9×10^{-12} and 1.4×10^{-10}, and the one failure is a real edge-case fault: the CTC loss raises a shape error on an empty transcription.

- The absolute positional accuracy of the gantry (R5) and both field conditions were not measured, so no conclusion is possible for them.

It follows that the storage half of R6 is met, R2 is met for writer identification on real handwriting and for recognition on correctly segmented in-distribution crops, R1 is close to its target but not at it, R3 and R4 are not met, and R5, the reproduction half of R6 and the field conditions are unassessed. Two implications are worth stating separately. First, the time requirement cannot be met by improving the neural network, because recognition already has a margin of about 2.8 times; the shortfall lies in the pure-Python line-grouping stage, which must be redesigned or compiled, and in the cost of evaluating every synthesis candidate. Second, on the evidence of the matching comparison the two halves of R1 are not in conflict as they first appeared: genuine and synthesised text were read within 7.4 percentage points at character level, so the synthesis is already close to the human hand in letterform, and the remaining gap is concentrated in a small number of measurable spacing statistics that can be corrected directly.

6.3 Contribution

The following was new to the student and was mastered during the project: the training of sequence networks with connectionist temporal classification, the algorithms of the image-conditioning and line-segmentation chain, the extraction of writer-specific letter paths by forced alignment, the synthesis of handwriting trajectories, the generation of G-code and the control

of stepper motors from the general-purpose pins of a single-board computer. All of these were implemented from first principles by the student in Python, and C99 was used for an optional acceleration of two routines. Deriving the backward pass of each layer by hand, including the connectionist temporal classification loss, and verifying each one against numerical differentiation was the single largest piece of new learning in the project, and it is what allowed the networks to be trained without a machine-learning framework. The mechanical parts, the camera, the single-board computer, the motors and drivers, the end-stop switches, the pen mechanism and the power supply were purchased as finished items and carry no design credit. The design of the networks follows a published layer pattern [**?**], and public data were used for training. The architecture of the recogniser is taken from the literature; the training recipe, the preprocessing chain, the style-profile construction and the whole synthesis and drawing path are the student’s own work. Any other contribution by other persons is declared in Section 2 (Project history) of the preamble.

6.4 Suggestions for future work

The work can be continued in the following order of priority.

1. Resolve the direction-pin fault of the X axis, complete the calibration and execute the qualification tests 3, 5 and 6 on the gantry, including the field checks FC1 and FC2.

2. Execute qualification test 2 on newly collected held-out pages, and qualification test 1 with independent judges, a human panel and the hand-crafted and overlay measures, so that the figures of Section 4 are no longer circular or optimistic.

3. Redesign the line grouping, reduce the image resolution before segmentation, deploy compiled implementations, and make the recogniser about 3 times cheaper per character (smaller network, quantisation, shorter sequence), measuring each change on the ODROID N2+.

4. Add a language-model or lexicon decoder and more personal training pages to raise the text accuracy towards 95%.

5. Correct the implementation issues of the writing path (reading-order stroke ordering, execution of acceleration, work area, pen-up feed), add a hardware emergency stop and an abort function, and consider a position feedback (for example a sensor on the end-stops or a camera check of the written line) so that lost steps can be detected.

6. Add an unknown-writer class to the writer classifier, and an authenticated interface.

Appendix

Metric definitions of the qualification tests

The qualification tests of Section 4.2 quote accuracies, correlations and distances whose definitions are not interchangeable: three different averages of character accuracy over a set of lines differ by several points on the same data, and the word accuracy reported throughout is a recall rather than one minus the word error rate. This appendix defines each measure once, names the routine that computes it, and states where an alternative reading would change the number. Section 4.2 refers to this appendix in place of repeating the mathematics beside each result.

6.4.0 Confidence intervals and resampling

Proportions are reported with a 95% Wilson score interval [**?**],

$$\hat{p}_{\pm} = \frac{\hat{p} + \dfrac{z^2}{2n} \pm z\sqrt{\dfrac{\hat{p}(1-\hat{p})}{n} + \dfrac{z^2}{4n^2}}}{1 + \dfrac{z^2}{n}}, \qquad \hat{p} = \frac{k}{n}, \tag{95}$$

where k is the number of successes in n trials and $z = 1.96$. Equation 95 is used in preference to the normal approximation because it stays valid near 0% and 100%, where the system's per-writer rates lie. Characters of one page are not independent, so accuracies use a page-level percentile bootstrap with 2000 resamples [**?**, p. 3] rather than a closed-form interval, and a requirement is declared met only if the lower confidence bound exceeds the target.

6.4.0 Character accuracy and its three averages

Character accuracy of one line uses the Levenshtein distance [**?**, pp. 707–708] $d_L(h, r)$ between hypothesis h and reference r, the minimum number of insertions, deletions and substitutions, evaluated by the dynamic-programming recursion of Wagner and Fischer [**?**],

$$A_{\mathrm{c}}(h, r) = \max\left(0,\; 1 - \frac{d_L(h, r)}{|r|}\right), \tag{96}$$

where $|r|$ is the reference length. Equation 96 is implemented as `Classify.CharAcc`, which floors the value at zero so that a hypothesis longer than the reference cannot score negatively.

A set of lines can be summarised in three different ways, and they do not agree, so each is named explicitly:

$$\bar{A}_{\mathrm{c}}^{\mathrm{line}} = \frac{1}{L}\sum_{\ell=1}^{L} A_{\mathrm{c}}(h_\ell, r_\ell), \qquad \bar{A}_{\mathrm{c}}^{\mathrm{micro}} = 1 - \frac{\sum_\ell d_L(h_\ell, r_\ell)}{\sum_\ell |r_\ell|}, \qquad \bar{A}_{\mathrm{c}}^{\mathrm{writer}} = \frac{1}{W}\sum_{w=1}^{W} \bar{A}_{\mathrm{c}}^{\mathrm{line}}(w), \tag{97}$$

the unweighted mean over lines, the character-weighted micro-average over all lines pooled, and the unweighted mean over the W writers of their own line means. The micro-average of Equation 97 is what "summed over all sentences" would mean, but the character-accuracy figures quoted for qualification test 1 are the *mean over writers*, $\bar{A}_{\mathrm{c}}^{\mathrm{writer}}$, recorded as `char_acc_mean_over_writers`. The distinction is not cosmetic: for the held-out judge on the 123 real lines the two readings differ by 4.7 points, 86.6% over writers against 81.9% over lines. Figures quoted for qualification test 2 are instead the unweighted mean over line crops, $\bar{A}_{\mathrm{c}}^{\mathrm{line}}$, because `verify_text_accuracy.py` accumulates one `CharAcc` per crop and divides by the crop count, so a two-word crop weighs as much as a twenty-word one.

6.4.0 Word accuracy is a recall, not one minus the word error rate

Word accuracy, which Section 4.2 quotes six times, is `Classify.WordAcc` and is *not* 1 − WER:

$$A_{\mathrm{w}}^{\mathrm{word}} = \frac{\mathrm{LCS}(\mathrm{words}(h), \mathrm{words}(r))}{|\mathrm{words}(r)|}, \tag{98}$$

the length of the longest common subsequence of the two word sequences, compared case-insensitively, divided by the number of reference words. Equation 98 is therefore an order-preserving *recall* over words: it counts how many of the intended words were recovered in the right order and imposes no penalty for inserted or spurious words, whereas 1 − WER charges insertions, deletions and substitutions alike and can go negative. A reader who assumes 1 − WER will read every word figure in Section 4.2 as stricter than it is.

6.4.0 Writer accuracy is reported per line, not per page

Writer identification admits a per-line and a per-page reading,

$$A_{\mathrm{w}}^{\mathrm{line}} = \frac{1}{L}\sum_{\ell=1}^{L} \mathbf{1}\left[\hat{w}_\ell = w_\ell\right], \qquad A_{\mathrm{w}}^{\mathrm{page}} = \frac{1}{P}\sum_{p=1}^{P} \mathbf{1}\Big[\arg\max_{w} \tfrac{1}{L_p}\textstyle\sum_{\ell \in p} P(w \mid \ell) = w_p\Big], \tag{99}$$

the fraction of lines assigned to the correct writer, and the fraction of pages whose mean per-line writer probability peaks at the correct writer. The proposal's wording implies the page form of Equation 99, but $A_{\mathrm{w}}^{\mathrm{page}}$ was never evaluated: every value reported in Section 4.2, including the 99.78% and 93.67% over 1358 lines, is the per-line form $A_{\mathrm{w}}^{\mathrm{line}}$. Because lines of one page share a writer, the page form would be the stronger claim and the smaller sample. **[MUST STILL BE COMPLETED BY ME: evaluate $A_{\mathrm{w}}^{\mathrm{page}}$ of Equation 99 by pooling the stored per-line probabilities of `stage4_predictions_ink.json` and `stage4_predictions_shape.json` by page.]**

6.4.0 Decoding

All text figures in the report were produced by greedy connectionist temporal classification decoding — argmax per time step, collapse repeats, drop the blank, as implemented in `Classify.DecodeCtc` — with **no beam search, no language model and no lexicon** anywhere in the deployed pipeline. That matters next to the 94.21% word accuracy of qualification test 2: the figure is what an unaided per-frame argmax recovers, so it is not inflated by a decoder that could snap a near-miss onto a dictionary word; equally, nothing corrects a single-character slip that destroys a word. All random choices use recorded seeds.

6.4.0 Inter-rater agreement of the panel that was not convened

Inter-rater agreement of the blind human panel of qualification test 1 was to be quantified with Fleiss' kappa [**?**],

$$\kappa = \frac{\bar{P} - \bar{P}_e}{1 - \bar{P}_e}, \tag{100}$$

where $\bar{P}$ is the mean observed pairwise agreement and $\bar{P}_e$ the agreement expected by chance. As the panel was never convened, Equation 100 was never evaluated and is given here only to define the sub-test that was not run; the worksheet prepared for it is Table 52.

6.4.0 Style correlation on geometric features

Each feature is z-scored, $z_{wf} = (v_{wf} - \mu_f)/\sigma_f$, using the mean and standard deviation over the ten real writers, and the stylistic correlation reported for sub-test (b) of qualification test 1 is the

Pearson product-moment correlation coefficient [**?**],

$$\rho = \frac{\sum_i (a_i - \bar{a})(b_i - \bar{b})}{\sqrt{\sum_i (a_i - \bar{a})^2}\sqrt{\sum_i (b_i - \bar{b})^2}}, \tag{101}$$

where a_i and b_i are the real and synthesised z-scored entries pooled over writers and features (70 pairs for the seven-feature set the protocol prescribes, listed in Table 54). Besides the pooled value of Equation 101, a per-writer value (7 pairs) and a nearest-centroid retrieval in z-space with its 10×10 confusion matrix are also computed. The correlation actually measured used the nine geometric features of Table 41 rather than the seven of Table 54. **[MUST STILL BE COMPLETED BY ME: confirm the Pearson reference: the product-moment correlation coefficient is introduced in the 1896 paper "Regression, heredity, and panmixia" (Philosophical Transactions of the Royal Society A, vol. 187), whereas the sourcing audit suggested vol. 186 (1895), which is the skew-variation paper. Verify the volume, year and page range against the original and correct the bibliography entry.]**

6.4.0 Overlay distances in the pixel domain

Both images entering the overlay test are normalised in the same way before any distance is taken, so that the comparison measures shape rather than size or placement; Figure 42 sets out that procedure.

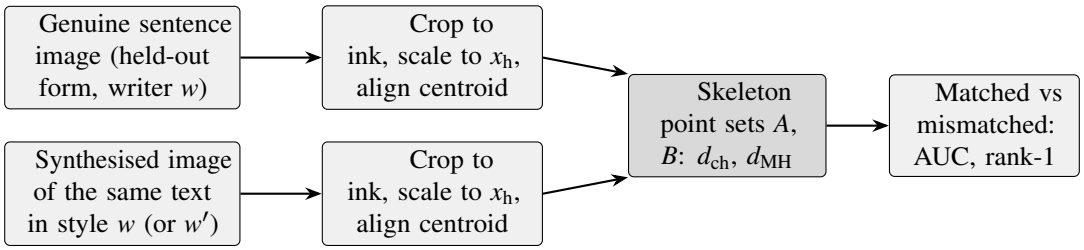


Figure 42. Overlay test of qualification test 1: both images are normalised in the same way and matched writer pairs are compared with mismatched pairs.

The skeleton point sets A and B that this procedure produces are compared with

$$d_{\mathrm{ch}}(A,B) = \frac{1}{2}\left[\frac{1}{|A|}\sum_{a\in A}\min_{b\in B}\|a-b\| + \frac{1}{|B|}\sum_{b\in B}\min_{a\in A}\|a-b\|\right], \tag{102}$$

$$d_{\mathrm{MH}}(A,B) = \max\left[\frac{1}{|A|}\sum_{a\in A}\min_{b\in B}\|a-b\|,\ \frac{1}{|B|}\sum_{b\in B}\min_{a\in A}\|a-b\|\right], \tag{103}$$

$$\mathrm{AUC} = \Pr\left[d_{\mathrm{matched}} < d_{\mathrm{mismatched}}\right] + \tfrac{1}{2}\Pr\left[d_{\mathrm{matched}} = d_{\mathrm{mismatched}}\right], \tag{104}$$

where $\|\cdot\|$ is the Euclidean norm and distances are in units of the x-height x_h. Equation 102 is the mean of the two directed mean nearest-neighbour distances, which symmetrises the one-directional chamfer matching distance of Barrow *et al.* [**?**, pp. 659–660]; Equation 103 is their maximum, which is less sensitive to outliers than the classical Hausdorff distance [**?**]; and Equation 104 is the area under the receiver operating characteristic curve [**?**, p. 868], 0.5 for a distance carrying no writer information and 1.0 for perfect separation.

The protocol sized this sub-test at 8 writers and 5 sentences, which gives 40 matched and $40 \times 9 = 360$ mismatched pairs; the run actually made used ten writers and three sentences each, which is the 30 matched and 300 mismatched pairs reported in Section 4.2. The features of Table 54 are computed by one deterministic extractor with identical thresholds on every image, after cropping to the ink and binarising with the same rule.

6.4.0 Positional error and its uncertainty budget

The absolute error of a drawn cross and the uncertainty of its measurement, prescribed for qualification test 5 and never evaluated, are

$$e_i = \left\| \mathbf{p}_i - \mathbf{p}_i^{\mathrm{cmd}} \right\|, \tag{105}$$

$$u = \sqrt{u_{\mathrm{scan}}^2 + u_{\mathrm{ref}}^2 + u_{\mathrm{id}}^2}, \tag{106}$$

where $\mathbf{p}_i$ is the measured and $\mathbf{p}_i^{\mathrm{cmd}}$ the commanded centre of cross i, referenced to the machine origin 5 mm inside the minimum end-stops. In Equation 106, u_{scan} is the scanner scale error obtained by scanning the verified ruler, u_{ref} the registration of the sheet against the machine origin and u_{id} the error of locating a cross centre, taken as half a pixel; the three standard uncertainties are combined in quadrature and the expanded uncertainty is $2u$, following the Guide to the Expression of Uncertainty in Measurement [**?**, Clause 5]. Mean, maximum and 95th percentile of the error of Equation 105 were to be reported together with the uncertainty of Equation 106, on the sheet reproduced blank as Table 53. Because no mark was ever drawn, neither equation was evaluated.

Additional results and prepared measurement sheets

This appendix holds the supporting detail of the qualification tests of Section 4: the provenance of the data sets and model checkpoints behind every quoted figure, the two measurement sheets that were prepared for the sub-tests that were never carried out, the feature definitions prescribed for the style correlation, and the distance distributions behind the overlay test. Every figure quoted against a requirement remains in Section 4; nothing here changes a result, and the blank sheets are reproduced so that the measurement that is missing is defined rather than merely named.

6.4.0 Provenance of the data sets and checkpoints

Section 4.2 states two facts that qualify every figure in it, that the harness covers ten of the thirteen writers and that the live server serves an unmeasured text checkpoint. This sub-subsection traces both to the stored files that establish them, so that the claims can be checked rather than taken on trust.

The ten-writer subset. The harness was run over the eight database writers 150, 151, 152, 153, 384, 551, 552 and 588 and the two personal writers Dylan and Yeukita. The list is set by `PERSONAL_AUTHORS` in `qtp_scripts/qtp_common.py`, which names only `yeukita` and `dylan`, and the stored results confirm it: in `qtp_data/qtp1c_results.json` the strings `abhinav`, `owen` and `thiya` occur zero times, while `dylan` occurs 72 times and `yeukita` 96 times. The per-writer shape-judge accuracies that make those three the weakest of the set come from `stage4_summary.json`. The figures that do cover all thirteen writers come from the separate verification pipeline rather than the harness: the search-judge accuracy of qualification test 1, the per-writer end-to-end run of Figure 47 and the 1358-line writer identification of qualification test 2. The 669-crop recognition figure covers the five personal writers only, which is its own data set and not the ten-writer subset. **[MUST STILL BE COMPLETED BY ME: re-run the qualification-test harness with `PERSONAL_AUTHORS` set to all five personal writers, so that the style-correlation, overlay and independent-judge figures cover the same thirteen writers as the rest of the chapter.]**

The checkpoints. The character- and word-accuracy figures of qualification test 1 were measured with two text checkpoints recorded in `qtp_data/qtp1c_results.json`: `meta.production_ckpt` is `paper_cnn_bilstm_ctc_joint_best.pt`,

the checkpoint inside the best-of-N search, and `meta.heldout_ckpt` is `paper_cnn_bilstm_ctc_hf_best.pt`, the base model used as the judge outside the search. The recognition figures of qualification test 2 were measured with the same joint checkpoint (`verify_text_accuracy.py`), and the writer-identification figures with `author_classifier_10new_weights.pt` for ink and `author_shape_10new_weights.pt` for shape. The deployed recogniser is a different model: `Classify.py` prefers `paper_cnn_bilstm_ctc_scratch_best.npz` over the converted checkpoints, and that file is present, so the from-scratch checkpoint is what the server loads. Its two writer classifiers, by contrast, are the `.npz` conversions of the `.pt` files above and agree with them on 100% of 1358 lines to within 8×10^{-6} of logit, which is why the writer figures transfer to the deployed code and the text figures do not. **[MUST STILL BE COMPLETED BY ME: measure character and word accuracy of the deployed `paper_cnn_bilstm_ctc_scratch_best.npz` checkpoint on the same 669 crops, or change the server to load the joint checkpoint that was measured.] [MUST STILL BE COMPLETED BY ME: record the architecture width, layer count, parameter count, optimiser, learning-rate schedule, batch size and epoch count of each named checkpoint, so that every figure in Section 4 is traceable to a trained model. No hyper-parameter table or parameter count is given for any of them at present.]**

6.4.0 Per-feature and per-judge breakdown of qualification test 1

Section 4.2 quotes the style correlation as one pooled number and the two-judge comparison as one difference; both are the summary of a per-item breakdown, which is given here because it localises the defect rather than merely sizing it. Figure 43 resolves the correlation into its nine features and Figure 44 the character accuracy into the two checkpoints that read the same sixty lines, while Figure 45 places the synthesis beside genuine handwriting of the identical sentences. Table 50 lists the per-feature coefficients that Figure 43 plots.

Feature	Pearson r	Feature	Pearson r
Slant	0.996	Stroke width	0.632
Ascender extent	0.974	Word gap	0.605
Character width	0.936	Ascender-to-descender ratio	0.509
Components per word	0.832	Descender extent	0.148
Ink density	0.680		

Table 50. Style correlation per feature across the ten harness writers, which Figure 43 plots and which the pooled coefficient of 0.714 reported in Table 41 summarises. The ordering is the result: slant and the extent above the baseline are reproduced almost exactly, whereas the extent below the baseline and the spacing between words are not.

6.4.0 Per-writer breakdowns

The headline rates of qualification tests 1 and 2 are means over writers, and the spread behind them is wider than the mean suggests, so the per-writer breakdown is reproduced here in full. Figure 46 gives writer identification per writer over the 1358 real lines for both classifiers, Figure 47 the end-to-end result per writer through the G-code round trip, and Table 51 the per-writer recognition accuracy behind the 98.61% of qualification test 2.

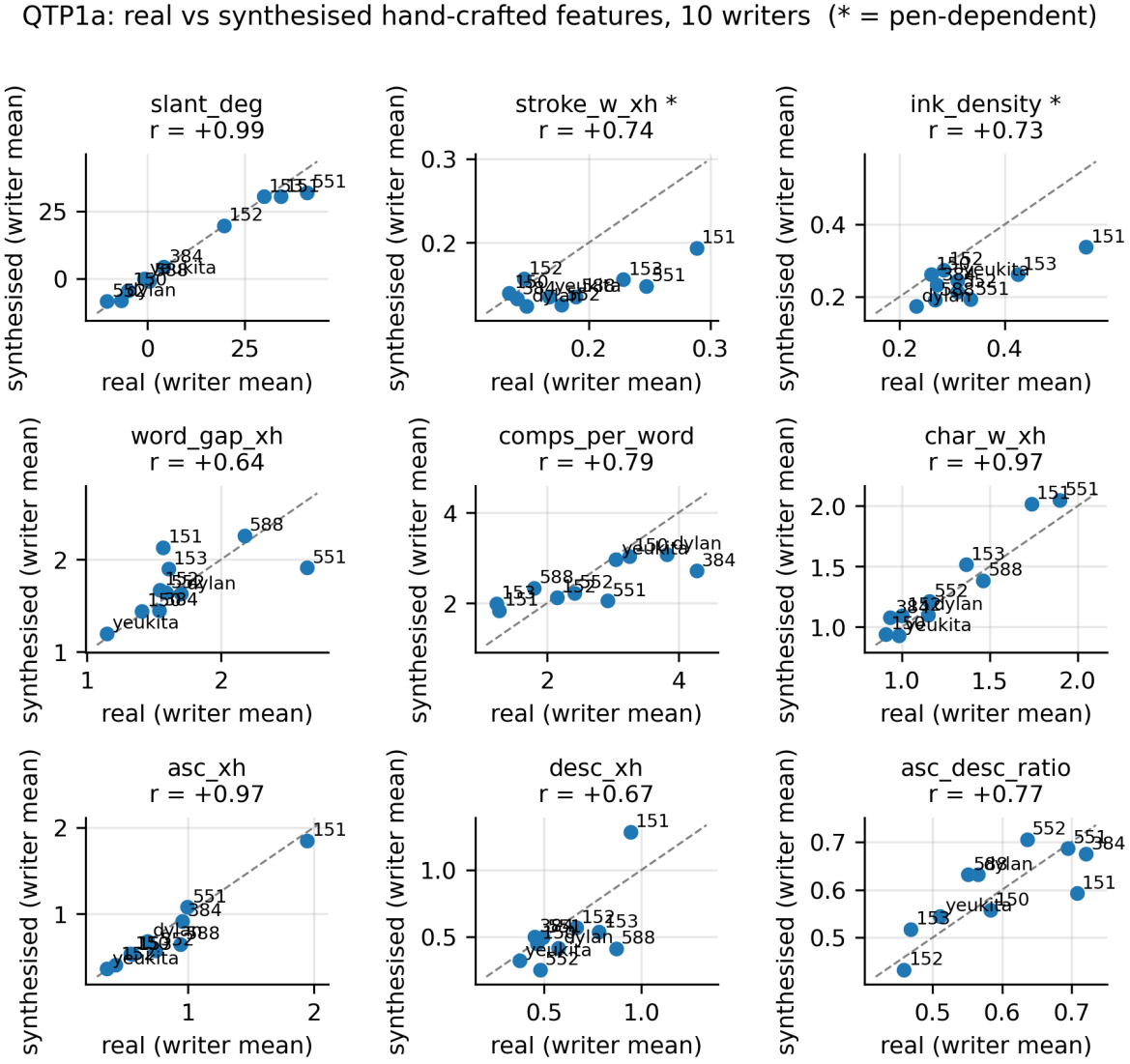


Figure 43. Style-correlation test on nine geometric features extracted by one deterministic routine from held-out real pages and from synthesised lines, for the ten writers. Each panel is one feature; the axes are the writer means. Slant, ascender and character width track the real hand closely, whereas descender and the ascender-to-descender ratio do not, which is the same defect that the word-accuracy gap identifies.

6.4.0 Storage check of qualification test 6

Qualification test 6 reduces to a count against a threshold, and Figure 48 is the observation behind that count.

6.4.0 Prepared sheets for the sub-tests not carried out

Two sub-tests were specified in full and never executed, sub-test (d) of qualification test 1 (the blind human panel) and qualification test 5 (absolute positional error). Table 52 is the worksheet printed for the panel and Table 53 the data sheet printed for the positional measurements. Both are shown exactly as printed, that is to say blank, because no rater was convened and no cross was ever drawn.

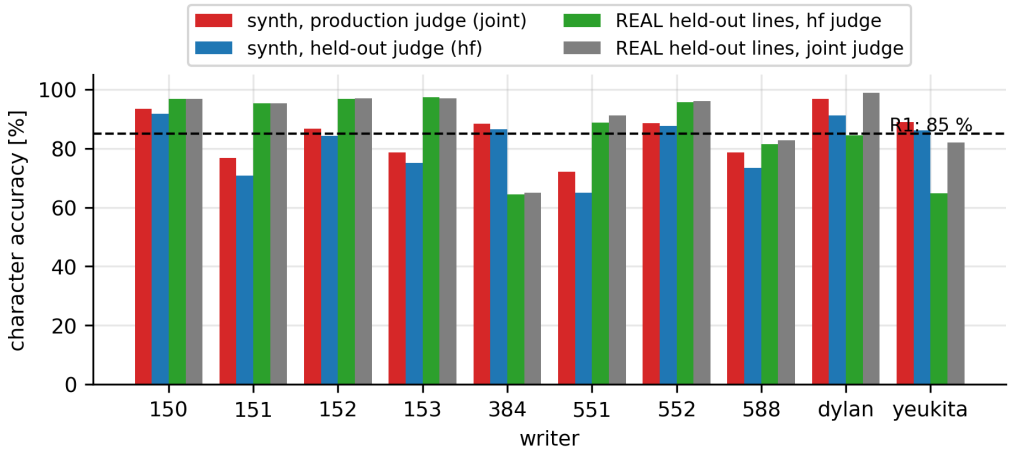


Figure 44. Character accuracy of the synthesised text under the production recogniser and under a recogniser checkpoint that was not used to select the candidates. The independent judge scores the same output about three points lower, which quantifies how much of the apparent quality is a selection effect and simultaneously shows that the synthesis is genuinely legible rather than merely self-consistent.

Writer	Character accuracy	Crops
Owen	99.15%	75 to 206 crops per writer; 669 in total
Dylan	99.03%	
Abhinav	98.94%	
Yeukita	98.30%	
Thiya	96.46%	

Table 51. Recognition accuracy per personal writer over the 669 real line crops of qualification test 2, read by `paper_cnn_bilstm_ctc_joint_best`. The pooled figure is 98.61%; Thiya, whose hand is the most joined of the five, is the only writer more than two points below it.

Line no.	Transcription written by the rater (without hint of the content)	Illegible characters (positions)	Confidence (1–5)
1			
2			
⋮			
40			

Table 52. Worksheet prepared for the blind human panel of qualification test 1: 30 synthesised lines and 10 genuine control lines in random order. The panel was never convened, so no worksheet was completed and the table is shown blank as it was printed.

6.4.0 Features prescribed for the style correlation

The style-correlation sub-test of qualification test 1 compares geometric features extracted by one deterministic routine from held-out real pages and from synthesised lines. Table 54 defines the features as they were prescribed in the protocol; the extractor that produced the measured correlation implements this set extended to the nine features reported in Table 41. The definitions are given here rather than in Section 4 because they fix the measurement rather than report a

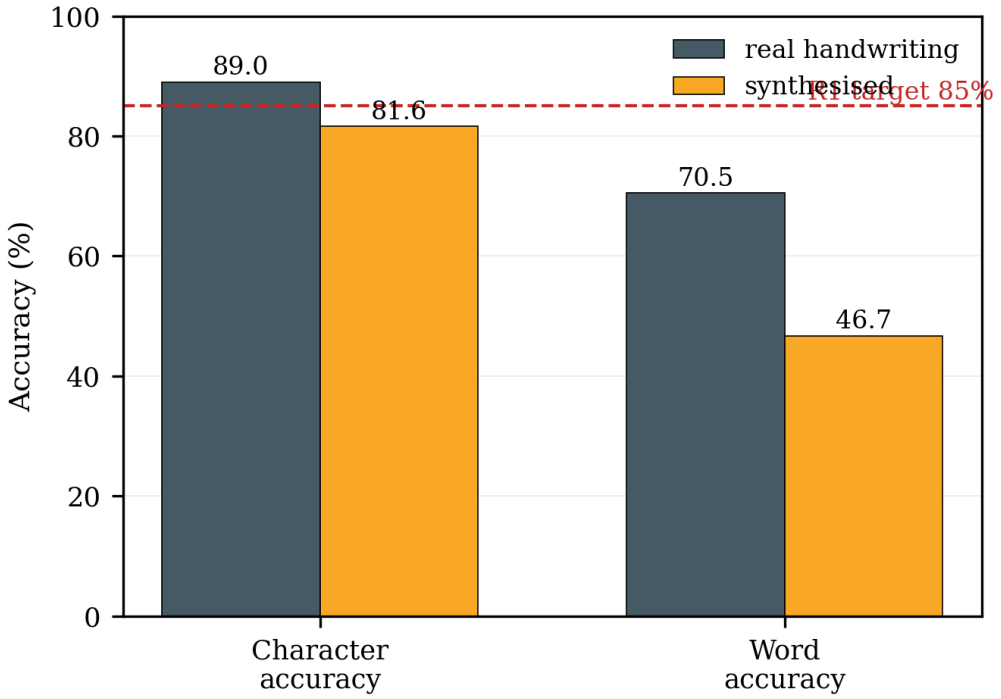


Figure 45. Matching comparison on identical sentences: genuine handwriting of the sentence against the system's synthesis of that same sentence, both read by the same recogniser. Character accuracy is close to the target, but word accuracy divides by more than a factor of 1.5, which shows that the synthesis preserves letterforms better than it preserves word boundaries and spacing.

Repeat	Cross	x^{cmd} (mm)	y^{cmd} (mm)	x meas. (mm)	y meas. (mm)	e_i (mm)
1	1					
1	2					
⋮						

Table 53. Data sheet prepared for the absolute positional error of qualification test 5; one row per cross and repetition, together with the scanner scale check and the registration check. No row was ever completed, because the test was not carried out.

result.

6.4.0 Distance distributions behind the overlay test

The overlay test of qualification test 1 reduces to two numbers in Table 41, the ratio of matched to mismatched distance and the area under the curve. Figure 49 gives the distributions those two numbers summarise, which show how far the two populations actually overlap.

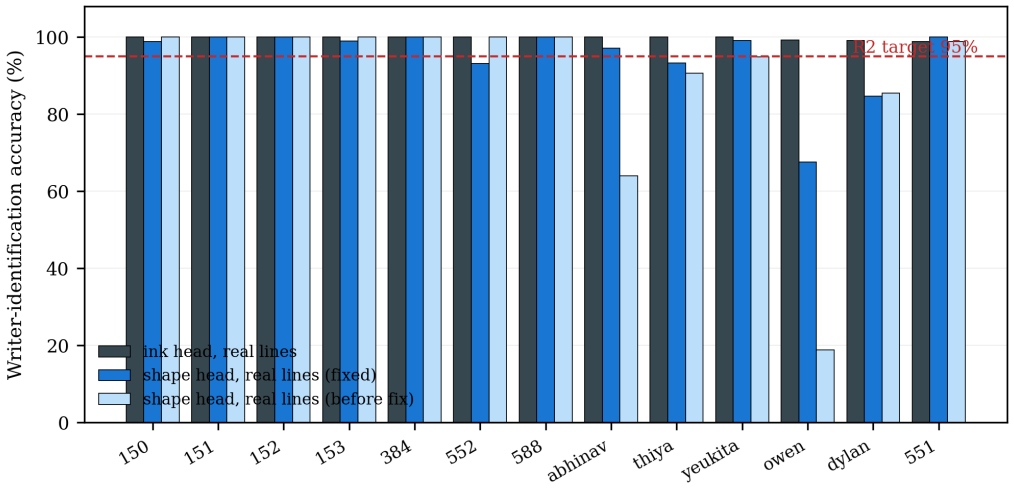


Figure 46. Writer-identification accuracy per writer over 1358 real lines, for the ink classifier and for the shape classifier before and after its training fault was corrected. The ink classifier is at or near 100% for every writer. Correcting the shape classifier's training raised its overall accuracy from 83.4% to 93.7% and removed a systematic collapse in which 118 of Owen's 148 lines had been assigned to Yeukita. The dashed line is the 95% target of R2.

Feature	Definition
Slant	Shear angle (degrees) that maximises the variance of the column ink projection of the line.
Stroke width	Median stroke width from the distance transform at the skeleton, divided by x_{h}.
Ink density	Fraction of ink pixels inside the core band between the baseline and the x-height line.
Word gap	Median horizontal gap between words (gaps larger than $0.5\,x_{\mathrm{h}}$) divided by x_{h}.
Connectedness	Number of connected ink components divided by the number of characters of the text.
Character width	Line width divided by the number of characters, divided by x_{h}.
Ascender/descender ratio	(Height of the ascender zone plus depth of the descender zone) divided by x_{h}.

Table 54. Hand-crafted style features prescribed for the correlation and retrieval of qualification test 1. The extractor that produced the measured correlation implements this set extended to the nine features listed in Table 41.

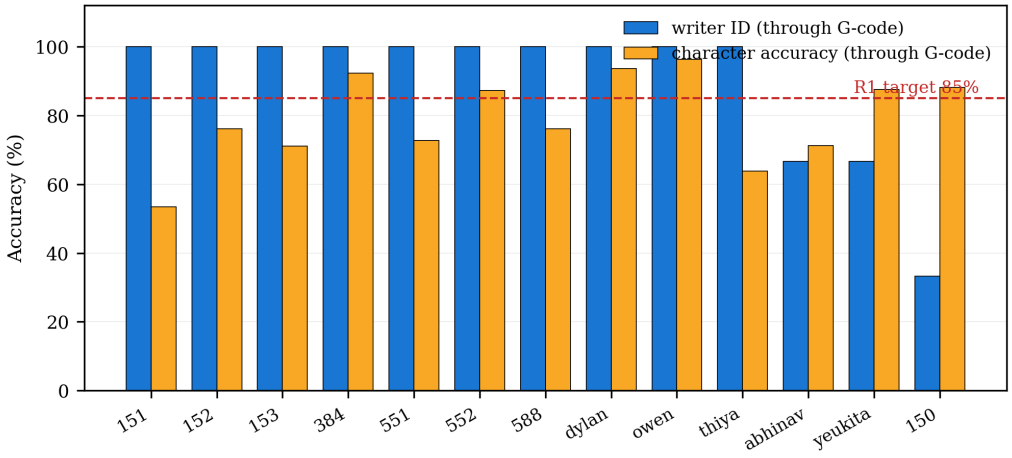


Figure 47. End-to-end result per writer over all thirteen writers: writer identification and character accuracy, both measured through the G-code round trip, for three novel sentences per writer at best-of-8 (`gcode_v2_summary.json`; shape judge `author_shape_10new_weights`, text judge `paper_cnn_bilstm_ctc_joint_best`). Ten of the thirteen writers are identified at 100% by the shape judge; the remaining three are 150 at 33.3% and Abhinav and Yeukita at 66.7% each. The character-accuracy failures are concentrated in the two writers whose hands are the most joined, which is the known weak case of an exemplar synthesiser that cuts letters apart and reassembles them.

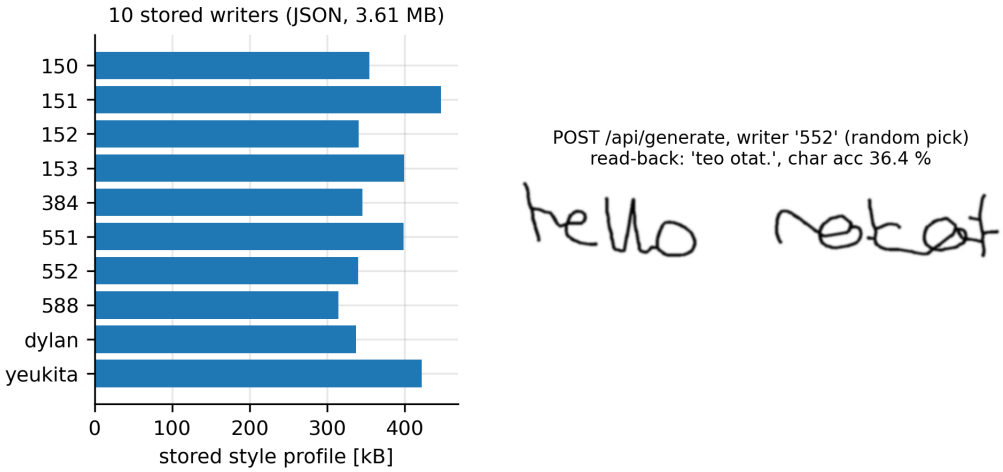


Figure 48. Storage of the writing styles. Each writer is held as a style profile and the server lists all thirteen through its author-list endpoint, against the ten required by R6.

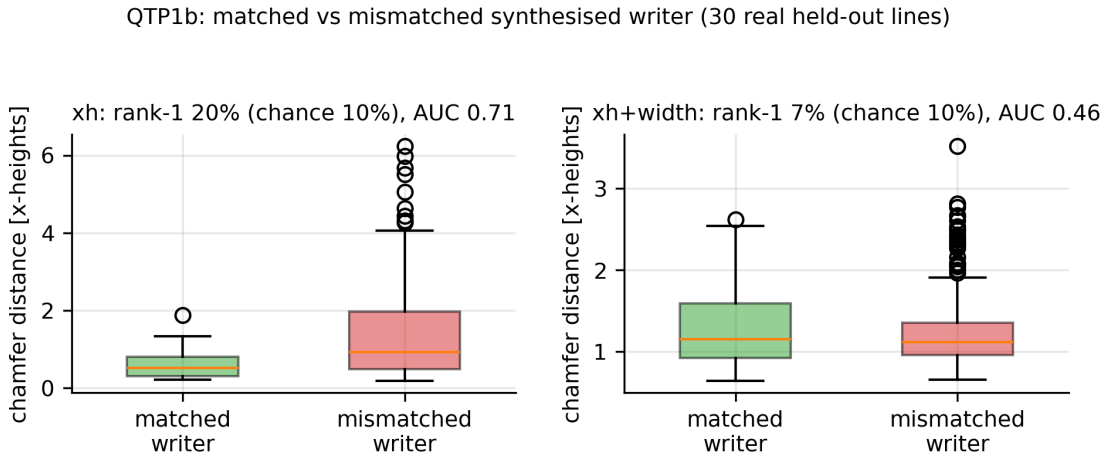


Figure 49. Distributions of the matched and mismatched overlay distances. The two distributions overlap but their means are separated by a factor of about 2.2 on the chamfer measure, giving an AUC of 0.71: the synthesis carries real writer information, but it is far from indistinguishable from the hand it imitates.